



हां भाई पढ़ ली डीएसए कर ली ...
TXT 57.67KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also with explanation

LLD (Low Level Design) — Complete Notes from Transcript

1. Introduction: What is LLD?

LLD = Low Level Design

LLD is the process of building a **complete application** around one or more **DSA concepts**. It's not just solving isolated algorithm problems — it's about designing the **skeleton/structure** of a real-world application.

DSA vs LLD

Aspect	DSA	LLD
Focus	Isolated problems (search, sort, etc.)	Complete application design
Scope	Single algorithm	Objects, relationships, scalability, security
Example	Binary Search on an array	Designing Ola/Uber backend
Output	Algorithm/function	Class diagrams, object interactions, architecture

Key Line: *"If DSA is the brain of an application, LLD is the skeleton."*

2. Story: Anurag vs Maurya (Quick Ride Example)

The Setup

Two friends — **Anurag** (knows DSA, no LLD) and **Maurya** (knows both DSA and LLD) — join a ride-booking company called **Quick Ride** (like Ola/Uber).

Anurag's Approach (DSA-only)

Manager asks: "Build Quick Ride. Find problems and solutions."

Anurag jumps straight to algorithms:

Problem 1: Route from Source to Destination

- Model the city as a **Graph** — intersections = nodes, roads = edges
- Use **Dijkstra's Algorithm** to find shortest path

Problem 2: Assign Nearest Rider to User

- Draw concentric circles around user
- Riders R1, R2, R3, R4 at various distances
- Map each user to a **Priority Queue (Min Heap)** of nearby riders
- Pop from heap → get closest rider

Result: Anurag solves both problems and goes to manager.

Manager's Response:

"This is just an algorithm. Where is the application?"

- What are the **objects/entities**?
- What are the **relationships** between them?
- How will **data security** be maintained?
- How to integrate **notifications** and **payment gateway**?
- How will it **scale to millions of users**?

Anurag failed because he jumped straight to DSA.

Maurya's Approach (LLD-first)

Maurya thinks:

Step 1: Identify Objects/Entities

- User (books ride)
- Rider (drives)
- Location
- Notification
- Payment

Step 2: Define Relationships

- User ↔ Rider (via ride)
- Rider ↔ Location
- User ↔ Payment
- User ↔ Notification

Step 3: Data Security

- User's phone number shouldn't be visible to rider after ride completes
- Sensitive data handling

Step 4: Scalability

- How to sustain millions of users
- Code should not break under load

Step 5: THEN Apply DSA

- Now solve route-finding and rider-mapping using DSA

Key Insight: LLD comes first, DSA comes later.

3. What LLD Focuses On

LLD focuses on **3 pillars**:

3.1 Scalability

- Can the application handle **millions of users**?
- How easily can we add new features?
- How quickly can we scale?

3.2 Maintainability

- Is the code easy to maintain?
- Adding a new feature shouldn't break 4 old features
- Code should be **easily debuggable**
- Minimum bugs

3.3 Reusability

- Code should be **highly reusable**
- **Plug-and-play model** — take a module, drop it in another app, it works
- Avoid **tight coupling**

Example of Reusability:

- **Notification** and **Payment** are application-independent
- Same notification/payment code works in Zomato, Swiggy, Amazon, Blinkit
- **Rider Mapping Algorithm** is generic:
 - Ola/Uber: assign nearest driver
 - Zomato/Swiggy: assign nearest delivery boy
 - Amazon/Blinkit: assign nearest delivery partner
 - Same algorithm, different applications

Tightly Coupled: Code fixed to one application — bad.

Loosely Coupled: Code reusable across applications — good.

4. What is NOT LLD: HLD vs LLD vs DSA

HLD = High Level Design

Focuses on:

- **Tech Stack** — Java Spring Boot? Node.js? Python?
- **Database** — SQL vs NoSQL vs Hybrid
- **Server Scaling** — how servers expand/contract with traffic
- **Cost Optimization** — minimize server costs, scale down when users drop

- **System Architecture Diagram**

HLD interviews (System Design): Almost **zero code**. Pure architecture.

LLD

- Focuses on **code structure**
- Class diagrams, object relationships
- Actual code implementation

DSA

- A **tool** used by LLD to solve problems
- Algorithms and data structures

Comparison Table

Aspect	HLD	LLD	DSA
Focus	System architecture	Code structure	Algorithms
Code	~Zero	Full code	Functions
Scope	Servers, DB, tech stack	Classes, objects, relationships	Problems
Example	"Use SQL + AWS + Load Balancer"	"User class has Rider object"	"Use Dijkstra"
Output	Architecture diagram	Class diagram + code	Algorithm

All three together build an application.

5. Code Examples (Java & Python)

The transcript doesn't contain explicit code, but based on the concepts discussed, here are implementations of the key ideas:

5.1 Rider Mapping using Priority Queue (Min Heap)

Concept: Assign nearest rider to user using a min heap.

Java

java

```
import java.util.PriorityQueue;

class Rider implements Comparable<Rider> {
    int id;
    double distance; // distance from user

    Rider(int id, double distance) {
        this.id = id;
        this.distance = distance;
    }

    // Min heap based on distance
    @Override
    public int compareTo(Rider other) {
        return Double.compare(this.distance, other.distance);
    }
}

public class RiderMapping {
    public static void main(String[] args) {
        PriorityQueue<Rider> minHeap = new PriorityQueue<>();

        // Add nearby riders with distances
        minHeap.add(new Rider(1, 2.5));
        minHeap.add(new Rider(2, 1.2));
        minHeap.add(new Rider(3, 3.8));
        minHeap.add(new Rider(4, 0.9));

        // Pop closest rider
        Rider closest = minHeap.poll();
        System.out.println("Closest Rider ID: " + closest.id +
                           ", Distance: " + closest.distance);
    }
}
```

Explanation:

- PriorityQueue in Java implements a **min heap** by default
- Comparable interface sorts riders by distance
- poll() returns the rider with smallest distance

- Time complexity: **$O(\log n)$** for insert and poll

Python

python

```
import heapq

class Rider:
    def __init__(self, id, distance):
        self.id = id
        self.distance = distance

    def __lt__(self, other):
        return self.distance < other.distance

    def __repr__(self):
        return f"Rider(id={self.id}, distance={self.distance})"

def assign_nearest_rider(riders):
    min_heap = []
    for rider in riders:
        heapq.heappush(min_heap, rider)
    return heapq.heappop(min_heap)

# Example
riders = [
    Rider(1, 2.5),
    Rider(2, 1.2),
    Rider(3, 3.8),
    Rider(4, 0.9)
]

closest = assign_nearest_rider(riders)
print(f"Closest Rider ID: {closest.id}, Distance: {closest.distance}")
```

Explanation:

- `heapq` is Python's min heap implementation
- `__lt__` defines comparison for heap ordering
- `heappush` and `heappop` maintain heap property
- Time complexity: **$O(\log n)$**

5.2 Route Finding using Dijkstra's Algorithm

Concept: Find shortest path from source to destination in a city graph.

Java

java

```
import java.util.*;

public class Dijkstra {
    static class Edge {
        int target, weight;
        Edge(int target, int weight) {
            this.target = target;
            this.weight = weight;
        }
    }

    public static int[] shortestPath(List<List<Edge>> graph, int source) {
        int n = graph.size();
        int[] dist = new int[n];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[source] = 0;

        PriorityQueue<int[]> pq = new PriorityQueue<>(
            Comparator.comparingInt(a -> a[1])
        );
        pq.add(new int[]{source, 0});

        while (!pq.isEmpty()) {
            int[] curr = pq.poll();
            int node = curr[0], d = curr[1];

            if (d > dist[node]) continue;

            for (Edge edge : graph.get(node)) {
                int newDist = dist[node] + edge.weight;
                if (newDist < dist[edge.target]) {
                    dist[edge.target] = newDist;
                    pq.add(new int[]{edge.target, newDist});
                }
            }
        }

        return dist;
    }

    public static void main(String[] args) {
```

```

    int n = 4;
    List<List<Edge>> graph = new ArrayList<>();
    for (int i = 0; i < n; i++) graph.add(new ArrayList<>());

    graph.get(0).add(new Edge(1, 4));
    graph.get(0).add(new Edge(2, 1));
    graph.get(2).add(new Edge(1, 2));
    graph.get(1).add(new Edge(3, 1));
    graph.get(2).add(new Edge(3, 5));

    int[] dist = shortestPath(graph, 0);
    System.out.println("Shortest distances from node 0: " +
        Arrays.toString(dist));
}
}

```

Explanation:

- Graph represented as adjacency list
- `dist[]` stores shortest distance from source
- Priority queue picks node with minimum distance
- Relaxation: if new path is shorter, update
- Time complexity: $O((V + E) \log V)$

Python

python

```

import heapq

def dijkstra(graph, source):
    n = len(graph)
    dist = [float('inf')] * n
    dist[source] = 0

    pq = [(0, source)] # (distance, node)

    while pq:
        d, node = heapq.heappop(pq)

        if d > dist[node]:
            continue

        for neighbor, weight in graph[node]:
            new_dist = dist[node] + weight
            if new_dist < dist[neighbor]:

```

```

        dist[neighbor] = new_dist
        heapq.heappush(pq, (new_dist, neighbor))

    return dist

# Example graph: adjacency list
graph = {
    0: [(1, 4), (2, 1)],
    1: [(3, 1)],
    2: [(1, 2), (3, 5)],
    3: []
}

dist = dijkstra(graph, 0)
print("Shortest distances from node 0:", dist)

```

Explanation:

- `heapq` used as min priority queue
- Tuple `(distance, node)` — heap orders by distance
- Same relaxation logic as Java
- Time complexity: $O((V + E) \log V)$

5.3 LLD Example: Ride Booking Classes

Concept: Design classes for a ride-booking system.

Java

```

java

import java.util.*;

// Location class
class Location {
    double lat, lng;
    Location(double lat, double lng) {
        this.lat = lat;
        this.lng = lng;
    }
    double distanceTo(Location other) {
        // Haversine or Euclidean
        return Math.sqrt(Math.pow(this.lat - other.lat, 2) +
                           Math.pow(this.lng - other.lng, 2));
    }
}

```

```
    }  
}  
  
// User class  
class User {  
    int id;  
    String name;  
    String phone;  
    Location location;  
  
    User(int id, String name, String phone, Location location) {  
        this.id = id;  
        this.name = name;  
        this.phone = phone;  
        this.location = location;  
    }  
}  
  
// Rider class  
class Rider {  
    int id;  
    String name;  
    String phone;  
    Location location;  
    boolean available;  
  
    Rider(int id, String name, String phone, Location location) {  
        this.id = id;  
        this.name = name;  
        this.phone = phone;  
        this.location = location;  
        this.available = true;  
    }  
}  
  
// Ride class  
class Ride {  
    int id;  
    User user;  
    Rider rider;  
    Location source, destination;  
    double fare;  
    String status; // REQUESTED, ONGOING, COMPLETED  
  
    Ride(int id, User user, Location source, Location destination) {  
        this.id = id;  
        this.user = user;  
        this.source = source;  
    }  
}
```

```

        this.destination = destination;
        this.status = "REQUESTED";
    }
}

// RideService - core LLD Logic
class RideService {
    List<Rider> riders = new ArrayList<>();
    Map<Integer, Ride> rides = new HashMap<>();

    void addRider(Rider rider) {
        riders.add(rider);
    }

    Rider findNearestRider(Location userLocation) {
        PriorityQueue<Rider> pq = new PriorityQueue<>(
            Comparator.comparingDouble(r -> r.location.distanceTo(userLocation))
        );
        for (Rider r : riders) {
            if (r.available) pq.add(r);
        }
        return pq.isEmpty() ? null : pq.poll();
    }

    Ride bookRide(User user, Location destination) {
        Rider rider = findNearestRider(user.location);
        if (rider == null) {
            System.out.println("No riders available");
            return null;
        }
        rider.available = false;
        Ride ride = new Ride(rides.size() + 1, user, user.location, destination);
        ride.rider = rider;
        ride.status = "ONGOING";
        rides.put(ride.id, ride);
        System.out.println("Ride booked with rider: " + rider.name);
        return ride;
    }

    void completeRide(int rideId) {
        Ride ride = rides.get(rideId);
        if (ride != null) {
            ride.status = "COMPLETED";
            ride.rider.available = true;
            // Hide phone numbers after ride
            System.out.println("Ride " + rideId + " completed.");
        }
    }
}

```



```

    }

    public class Main {
        public static void main(String[] args) {
            RideService service = new RideService();

            service.addRider(new Rider(1, "Ravi", "111", new Location(10.0, 20.0)));
            service.addRider(new Rider(2, "Sita", "222", new Location(11.0, 21.0)));
            service.addRider(new Rider(3, "Ram", "333", new Location(15.0, 25.0)));

            User user = new User(1, "Anurag", "999", new Location(10.5, 20.5));

            Ride ride = service.bookRide(user, new Location(12.0, 22.0));
            if (ride != null) {
                service.completeRide(ride.id);
            }
        }
    }
}

```

Explanation of LLD Concepts:

- **Entities:** User, Rider, Ride, Location
- **Relationships:** Ride links User ↔ Rider
- **Encapsulation:** Phone numbers hidden after completion
- **Reusability:** `findNearestRider` is generic (works for food delivery too)
- **Scalability:** Can add more riders; priority queue handles selection
- **Maintainability:** Adding new features (payment, notification) easy

Python

python

```

import heapq
import math

class Location:
    def __init__(self, lat, lng):
        self.lat = lat
        self.lng = lng

    def distance_to(self, other):
        return math.sqrt((self.lat - other.lat)**2 +
                        (self.lng - other.lng)**2)

class User:

```

```
def __init__(self, id, name, phone, location):
    self.id = id
    self.name = name
    self.phone = phone
    self.location = location

class Rider:
    def __init__(self, id, name, phone, location):
        self.id = id
        self.name = name
        self.phone = phone
        self.location = location
        self.available = True

class Ride:
    def __init__(self, id, user, source, destination):
        self.id = id
        self.user = user
        self.rider = None
        self.source = source
        self.destination = destination
        self.status = "REQUESTED"

class RideService:
    def __init__(self):
        self.riders = []
        self.rides = {}

    def add_rider(self, rider):
        self.riders.append(rider)

    def find_nearest_rider(self, user_location):
        available = [r for r in self.riders if r.available]
        if not available:
            return None
        # Min heap by distance
        heap = [(r.location.distance_to(user_location), r) for r in available]
        heapq.heapify(heap)
        return heapq.heappop(heap)[1]

    def book_ride(self, user, destination):
        rider = self.find_nearest_rider(user.location)
        if rider is None:
            print("No riders available")
            return None
        rider.available = False
        ride = Ride(len(self.rides) + 1, user, user.location, destination)
        ride.rider = rider
```

```

ride.status = "ONGOING"
self.rides[ride.id] = ride
print(f"Ride booked with rider: {rider.name}")
return ride

def complete Ride(self, ride_id):
ride = self.rides.get(ride_id)
if ride:
ride.status = "COMPLETED"
ride.rider.available = True
print(f"Ride {ride_id} completed.")

# Example
service = RideService()
service.add_rider(Rider(1, "Ravi", "111", Location(10.0, 20.0)))
service.add_rider(Rider(2, "Sita", "222", Location(11.0, 21.0)))
service.add_rider(Rider(3, "Ram", "333", Location(15.0, 25.0)))

user = User(1, "Anurag", "999", Location(10.5, 20.5))
ride = service.book_ride(user, Location(12.0, 22.0))
if ride:
service.complete_ride(ride.id)

```

Explanation:

- Same LLD structure as Java
- `heapq` for nearest rider
- `RideService` orchestrates all operations
- Phone privacy handled in `complete_ride`

6. Key Takeaways

1. LLD = Building complete applications around DSA concepts
2. DSA is the brain; LLD is the skeleton
3. LLD focuses on: Scalability, Maintainability, Reusability
4. HLD focuses on: Tech stack, Database, Server scaling, Cost
5. DSA focuses on: Algorithms and data structures
6. Order of thinking: LLD first → then DSA
7. Avoid tight coupling — make code plug-and-play
8. Prerequisites: C++ (Java source also provided in original course)

7. Next Steps (from transcript)

- Next video: **Pillars of OOPs**
- Even if you know OOPs, don't skip — you'll learn something new
- Rohit's OOPs playlist attached for in-depth learning

8. Summary Diagram

text

APPLICATION BUILDING
HLD (High Level Design)
- Tech Stack, Database, Server Scaling
- Cost Optimization, System Architecture
- ~Zero Code
LLD (Low Level Design)
- Objects, Entities, Relationships
- Scalability, Maintainability, Reusability
- Class Diagrams, Full Code
DSA (Data Structures & Algorithms)
- Algorithms (Dijkstra, Priority Queue)
- Used BY LLD to solve problems
- The "Brain" of the application

Final Quote: *"If DSA is the brain of an application, LLD is the skeleton."*



हां जी करें स्टार्ट। सो वेलकम बै...
TXT 126.81KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also with explanation

OOP Concepts & The Four Pillars — Complete Notes (Lecture 2)

Part 1: Why OOP? — History of Programming

1.1 Machine Language (Binary)

- Directly interacts with CPU using 0s and 1s
- **Problems:**
 - Highly prone to errors (one bit change breaks everything)
 - Tedious process
 - Not scalable
 - Impossible to build large applications

1.2 Assembly Language

- Introduced **mnemonics** (English keywords)
- Example: `MOV A, 61H` — move value 61H into register A
- **Problems:**
 - Still tightly coupled to hardware
 - Prone to errors (missing instructions, register mismatch)
 - Not scalable — change hardware → change code
 - No proper loops/methods

1.3 Procedural Programming (C, Pascal, etc.)

- Introduced **functions, loops, if-else blocks, switch statements**
- Code = **set of instructions** ("Do this, then do this")
- Recipe-book style programming
- **Problems:**
 - Couldn't model real-world interactions
 - No data security
 - Not highly scalable or reusable
 - Not suitable for enterprise/complex applications

1.4 Object-Oriented Programming (OOP)

- **Core Idea:** *"The real world works with objects that interact with each other — programming should work the same way."*
- Model real-world objects as **classes** and **objects**
- Objects interact via **methods**

Part 2: Why OOP Over Procedural? (Car & Owner Example)

Real-World Observation

- Real world = **objects** (mic, laptop, camera, car)
- Objects **interact** with each other

What is an Object?

An object has **two things**:

1. **Characteristics (Attributes/Fields)** — unique identifiers
2. **Behavior (Methods/Functions)** — actions it performs

Car Example

Characteristics	Behavior
brand	start()
model	stop()
isEngineOn	shiftGear()
wheels	accelerate()
	brake()

Procedural Approach (Problem)

Representing a Car:

text

```
string brand;
string model;
bool isEngineOn;
start();
stop();
shiftGear();
```

Representing Owner owning Car:

text

```
string ownerName;
void drive(string brand, string model) {
    start();
    shiftGear();
    accelerate();
}
```

Problems:

1. **No real-world modeling** — Owner and Car aren't separate objects
2. **Not scalable** — Add another car → duplicate all variables (brand2 , model2 , isEngineOn2)
3. **Not reusable** — drive() must pass car details manually
4. **Complex** — A simple "Owner owns Car" becomes a mess of methods calling methods

OOP Approach (Solution)

text

```
class Car {
    string brand, model;
    bool isEngineOn;
    void start() { ... }
    void shiftGear(int g) { ... }
    void accelerate() { ... }
    void brake() { ... }
    void stop() { ... }
}

class Owner {
    string name;
```

```
Car car;    // Owner HAS-A Car
void drive() {
    car.start();
    car.shiftGear(1);
    car.accelerate();
}
}
```

Benefits:

- Real-world modeling: `owner.car.start()` = "Owner owns Car"
- Highly scalable: Create multiple `car` objects easily
- Highly reusable: `car` class reusable across apps
- Clean interaction: Objects talk to each other via methods

Key Insight: *"In OOP code, nothing else happens — a bunch of objects interact with each other. One object calls another's method, or passes another object as a parameter."*

Part 3: The Four Pillars of OOP

1. Abstraction
2. Encapsulation
3. Inheritance (covered in Part 2 of video)
4. Polymorphism (covered in Part 2 of video)

Pillar 1: Abstraction

Real-World Analogy

- Driving a car: You use **steering, clutch, gear, accelerator, brake** — but you don't need to know how the engine works internally
- Watching TV: You use a **remote** — you don't need to know internal wiring
- Using a laptop: You use the **screen/keyboard** — you don't need to know hardware

Key Idea: Real-world objects are **abstracted**. They hide unnecessary details from the client and expose only what's needed.

Definition

Abstraction hides unnecessary details from a client and showcases only what is necessary.

Data Hiding vs Data Security

- **Abstraction = Data Hiding** → You *could* see it (open the hood), but you don't *need* to
- **Encapsulation = Data Security** → You *cannot* access it, and you *shouldn't*

Code Example: Abstraction with Abstract Class

C++ (from transcript)

cpp

```
#include <iostream>
using namespace std;

// Abstract class – the "interface"
class Car {
public:
    virtual void startEngine() = 0;
    virtual void shiftGear(int gear) = 0;
    virtual void accelerate() = 0;
    virtual void brake() = 0;
    virtual void stopEngine() = 0;
    virtual ~Car() {}
};

// Concrete implementation
class SportsCar : public Car {
public:
    string brand;
    string model;
    bool isEngineOn;
    int currentSpeed;
    int currentGear;

    SportsCar(string b, string m) {
        brand = b;
        model = m;
        isEngineOn = false;
        currentSpeed = 0;
        currentGear = 0;
    }
}
```

```
void startEngine() override {
    isEngineOn = true;
    cout << brand << " " << model << " : Engine starts with a roar!" << endl;
}

void shiftGear(int gear) override {
    if (!isEngineOn) {
        cout << "Engine is off. Cannot shift gear." << endl;
        return;
    }
    currentGear = gear;
    cout << "Shifted to gear " << currentGear << endl;
}

void accelerate() override {
    if (!isEngineOn) {
        cout << "Engine is off. Cannot accelerate." << endl;
        return;
    }
    currentSpeed += 20;
    cout << "Accelerating to " << currentSpeed << " km/h" << endl;
}

void brake() override {
    currentSpeed -= 20;
    if (currentSpeed < 0) currentSpeed = 0;
    cout << "Braking. Speed is " << currentSpeed << " km/h" << endl;
}

void stopEngine() override {
    isEngineOn = false;
    currentGear = 0;
    currentSpeed = 0;
    cout << "Engine turned off." << endl;
}
};

int main() {
    Car* myCar = new SportsCar("Ford", "Mustang");
    myCar->startEngine();
    myCar->shiftGear(1);
    myCar->accelerate();
    myCar->shiftGear(2);
    myCar->accelerate();
    myCar->brake();
    myCar->stopEngine();
    delete myCar;
}
```

```
        return 0;  
    }  
}
```

Java Equivalent

java

```
// Abstract class – the "interface"  
abstract class Car {  
    abstract void startEngine();  
    abstract void shiftGear(int gear);  
    abstract void accelerate();  
    abstract void brake();  
    abstract void stopEngine();  
}  
  
// Concrete implementation  
class SportsCar extends Car {  
    String brand, model;  
    boolean isEngineOn;  
    int currentSpeed, currentGear;  
  
    SportsCar(String brand, String model) {  
        this.brand = brand;  
        this.model = model;  
        this.isEngineOn = false;  
        this.currentSpeed = 0;  
        this.currentGear = 0;  
    }  
  
    @Override  
    void startEngine() {  
        isEngineOn = true;  
        System.out.println(brand + " " + model + " : Engine starts with a roar!");  
    }  
  
    @Override  
    void shiftGear(int gear) {  
        if (!isEngineOn) {  
            System.out.println("Engine is off. Cannot shift gear.");  
            return;  
        }  
        currentGear = gear;  
        System.out.println("Shifted to gear " + currentGear);  
    }  
  
    @Override  
    void accelerate() {
```

```

        if (!isEngineOn) {
            System.out.println("Engine is off. Cannot accelerate.");
            return;
        }
        currentSpeed += 20;
        System.out.println("Accelerating to " + currentSpeed + " km/h");
    }

    @Override
    void brake() {
        currentSpeed -= 20;
        if (currentSpeed < 0) currentSpeed = 0;
        System.out.println("Braking. Speed is " + currentSpeed + " km/h");
    }

    @Override
    void stopEngine() {
        isEngineOn = false;
        currentGear = 0;
        currentSpeed = 0;
        System.out.println("Engine turned off.");
    }
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new SportsCar("Ford", "Mustang");
        myCar.startEngine();
        myCar.shiftGear(1);
        myCar.accelerate();
        myCar.shiftGear(2);
        myCar.accelerate();
        myCar.brake();
        myCar.stopEngine();
    }
}

```

Python Equivalent

python

```
from abc import ABC, abstractmethod
```

Abstract class – the "interface"

```

class Car(ABC):
    @abstractmethod
    def start_engine(self): pass

```

```
@abstractmethod
def shift_gear(self, gear): pass

@abstractmethod
def accelerate(self): pass

@abstractmethod
def brake(self): pass

@abstractmethod
def stop_engine(self): pass

# Concrete implementation
class SportsCar(Car):
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model
        self.is_engine_on = False
        self.current_speed = 0
        self.current_gear = 0

    def start_engine(self):
        self.is_engine_on = True
        print(f"{self.brand} {self.model} : Engine starts with a roar!")

    def shift_gear(self, gear):
        if not self.is_engine_on:
            print("Engine is off. Cannot shift gear.")
            return
        self.current_gear = gear
        print(f"Shifted to gear {self.current_gear}")

    def accelerate(self):
        if not self.is_engine_on:
            print("Engine is off. Cannot accelerate.")
            return
        self.current_speed += 20
        print(f"Accelerating to {self.current_speed} km/h")

    def brake(self):
        self.current_speed -= 20
        if self.current_speed < 0:
            self.current_speed = 0
        print(f"Braking. Speed is {self.current_speed} km/h")

    def stop_engine(self):
        self.is_engine_on = False
```

```
        self.current_gear = 0
        self.current_speed = 0
        print("Engine turned off.")

# Usage
my_car = SportsCar("Ford", "Mustang")
my_car.start_engine()
my_car.shift_gear(1)
my_car.accelerate()
my_car.shift_gear(2)
my_car.accelerate()
my_car.brake()
my_car.stop_engine()
```

Output:

text

```
Ford Mustang : Engine starts with a roar!
Shifted to gear 1
Accelerating to 20 km/h
Shifted to gear 2
Accelerating to 40 km/h
Braking. Speed is 20 km/h
Engine turned off.
```

Code Explanation

- Car **is abstract** — cannot be instantiated directly
- virtual / abstract **methods** — declared but not defined; child must implement
- SportsCar **extends** Car — provides actual implementations
- **Client (main)** uses only Car interface methods — doesn't know internal implementation
- **This is Abstraction** — hiding "how it works" behind a clean interface

Bonus Insight

Even programming languages themselves are an abstraction:

- You write `if`, `for`, `while` (English keywords)
- Compiler converts to machine code

- You don't need to know binary conversion

Pillar 2: Encapsulation

Real-World Analogy

- **Capsule** — medicine wrapped in a protective layer
- **Car** — all characteristics and behaviors inside one "box" (the car body)
- **Odometer** — you can't directly change km driven; it increases only as you drive
- **Current Speed** — can't set to 500 directly; must accelerate gradually

Definition

Encapsulation = Binding data (characteristics) and methods (behavior) together into a single unit (class), AND providing data security via access modifiers.

Two Parts of Encapsulation

Part 1: Binding Data + Behavior

- All characteristics and methods live inside one class
- Class = blueprint of object

Part 2: Data Security

- Some data should NOT be directly accessible from outside
- Use **access modifiers**: `public` , `private` , `protected`

Access Modifiers (C++/Java)

Modifier	Same Class	Child Class	Outside
public	✓	✓	✓
private	✓	✗	✗
protected	✓	✓	✗

- `public` — accessible from anywhere (e.g., AC temperature — user can set freely)
- `private` — accessible only within the class (e.g., odometer, current speed)
- `protected` — accessible within class and child classes (covered in Inheritance)

Abstraction vs Encapsulation

Aspect	Abstraction	Encapsulation
Focus	Data Hiding	Data Security
Meaning	You <i>don't need</i> to see it	You <i>cannot</i> see it
Example	Engine internals (can open hood)	Odometer (can't directly change)
Mechanism	Abstract classes, interfaces	Access modifiers + getters/setters

Code Example: Encapsulation

C++ (from transcript)

cpp

```
#include <iostream>
using namespace std;

class SportsCar {
private:    // Data security – cannot be accessed directly
    string brand;
    string model;
    bool isEngineOn;
    int currentSpeed;
    int currentGear;
    string tyre;

public:    // Behaviors – accessible from outside
    SportsCar(string b, string m) {
        brand = b;
        model = m;
        isEngineOn = false;
        currentSpeed = 0;
        currentGear = 0;
        tyre = "MRF";
    }
```



```
// Getter for currentSpeed – read-only
int getCurrentSpeed() {
    return currentSpeed;
}

// Getter + Setter for tyre
string getTyre() {
    return tyre;
}

void setTyre(string t) {
    // Validation can be added here
    if (t.empty()) {
        cout << "Invalid tyre name!" << endl;
        return;
    }
    tyre = t;
}

void startEngine() {
    isEngineOn = true;
    cout << brand << " " << model << " : Engine starts with a roar!" << endl;
}

void shiftGear(int gear) {
    if (!isEngineOn) {
        cout << "Engine is off. Cannot shift gear." << endl;
        return;
    }
    currentGear = gear;
    cout << "Shifted to gear " << currentGear << endl;
}

void accelerate() {
    if (!isEngineOn) {
        cout << "Engine is off. Cannot accelerate." << endl;
        return;
    }
    currentSpeed += 20;
    cout << "Accelerating to " << currentSpeed << " km/h" << endl;
}

void brake() {
    currentSpeed -= 20;
    if (currentSpeed < 0) currentSpeed = 0;
    cout << "Braking. Speed is " << currentSpeed << " km/h" << endl;
}
```

```

void stopEngine() {
    isEngineOn = false;
    currentGear = 0;
    currentSpeed = 0;
    cout << "Engine turned off." << endl;
}
};

int main() {
    SportsCar myCar("Ford", "Mustang");

    // myCar.currentSpeed = 500; // ❌ ERROR: private member
    // myCar.currentSpeed = 500; // Not allowed

    myCar.startEngine();
    myCar.shiftGear(1);
    myCar.accelerate();
    myCar.shiftGear(2);
    myCar.accelerate();
    myCar.brake();
    myCar.stopEngine();

    cout << "Current Speed: " << myCar.getCurrentSpeed() << " km/h" << endl;

    myCar.setTyre("CEAT");
    cout << "Tyre: " << myCar.getTyre() << endl;

    return 0;
}

```

Java Equivalent

java

```

class SportsCar {
    // Private data – data security
    private String brand;
    private String model;
    private boolean isEngineOn;
    private int currentSpeed;
    private int currentGear;
    private String tyre;

    public SportsCar(String brand, String model) {
        this.brand = brand;
        this.model = model;
        this.isEngineOn = false;
        this.currentSpeed = 0;
    }
}

```

```
        this.currentGear = 0;
        this.tyre = "MRF";
    }

    // Getter – read-only access
    public int getCurrentSpeed() {
        return currentSpeed;
    }

    // Getter + Setter with validation
    public String getTyre() {
        return tyre;
    }

    public void setTyre(String t) {
        if (t == null || t.isEmpty()) {
            System.out.println("Invalid tyre name!");
            return;
        }
        this.tyre = t;
    }

    public void startEngine() {
        isEngineOn = true;
        System.out.println(brand + " " + model + " : Engine starts with a roar!");
    }

    public void shiftGear(int gear) {
        if (!isEngineOn) {
            System.out.println("Engine is off. Cannot shift gear.");
            return;
        }
        currentGear = gear;
        System.out.println("Shifted to gear " + currentGear);
    }

    public void accelerate() {
        if (!isEngineOn) {
            System.out.println("Engine is off. Cannot accelerate.");
            return;
        }
        currentSpeed += 20;
        System.out.println("Accelerating to " + currentSpeed + " km/h");
    }

    public void brake() {
        currentSpeed -= 20;
        if (currentSpeed < 0) currentSpeed = 0;
    }
}
```

```

        System.out.println("Braking. Speed is " + currentSpeed + " km/h");
    }

    public void stopEngine() {
        isEngineOn = false;
        currentGear = 0;
        currentSpeed = 0;
        System.out.println("Engine turned off.");
    }
}

public class Main {
    public static void main(String[] args) {
        SportsCar myCar = new SportsCar("Ford", "Mustang");

        // myCar.currentSpeed = 500; // ❌ Compile error: private

        myCar.startEngine();
        myCar.shiftGear(1);
        myCar.accelerate();
        myCar.shiftGear(2);
        myCar.accelerate();
        myCar.brake();
        myCar.stopEngine();

        System.out.println("Current Speed: " + myCar.getCurrentSpeed() + " km/h");

        myCar.setTyre("CEAT");
        System.out.println("Tyre: " + myCar.getTyre());
    }
}

```

Python Equivalent

python

```

class SportsCar:
    def __init__(self, brand, model):
        # Private attributes (name mangling with __)
        self.__brand = brand
        self.__model = model
        self.__is_engine_on = False
        self.__current_speed = 0
        self.__current_gear = 0
        self.__tyre = "MRF"

    # Getter – read-only
    def get_current_speed(self):

```

```

        return self.__current_speed

# Getter + Setter with validation
def get_tyre(self):
    return self.__tyre

def set_tyre(self, t):
    if not t:
        print("Invalid tyre name!")
        return
    self.__tyre = t

def start_engine(self):
    self.__is_engine_on = True
    print(f"{self.__brand} {self.__model} : Engine starts with a roar!")

def shift_gear(self, gear):
    if not self.__is_engine_on:
        print("Engine is off. Cannot shift gear.")
        return
    self.__current_gear = gear
    print(f"Shifted to gear {self.__current_gear}")

def accelerate(self):
    if not self.__is_engine_on:
        print("Engine is off. Cannot accelerate.")
        return
    self.__current_speed += 20
    print(f"Accelerating to {self.__current_speed} km/h")

def brake(self):
    self.__current_speed -= 20
    if self.__current_speed < 0:
        self.__current_speed = 0
    print(f"Braking. Speed is {self.__current_speed} km/h")

def stop_engine(self):
    self.__is_engine_on = False
    self.__current_gear = 0
    self.__current_speed = 0
    print("Engine turned off.")

# Usage
my_car = SportsCar("Ford", "Mustang")

# my_car.__current_speed = 500 # ❌ Name mangling prevents direct access
# my_car.current_speed = 500   # This creates a NEW attribute, not the private one

```

```
my_car.start_engine()
my_car.shift_gear(1)
my_car.accelerate()
my_car.shift_gear(2)
my_car.accelerate()
my_car.brake()
my_car.stop_engine()

print(f"Current Speed: {my_car.get_current_speed()} km/h")

my_car.set_tyre("CEAT")
print(f"Tyre: {my_car.get_tyre()}")
```

Output:

text

```
Ford Mustang : Engine starts with a roar!
Shifted to gear 1
Accelerating to 20 km/h
Shifted to gear 2
Accelerating to 40 km/h
Braking. Speed is 20 km/h
Engine turned off.
Current Speed: 0 km/h
Tyre: CEAT
```

Code Explanation

Why private?

- `currentSpeed = 500` directly would be unrealistic and dangerous
- Odometer, current speed, current gear are **sensitive** — must be controlled

Why getters/setters?

- **Getter** (`getCurrentSpeed`) — allows reading without modifying
- **Setter** (`setTyre`) — allows modification with **validation** (control remains with class)
- Even if you provide both getter & setter, **you still keep control** (can add validation, logging, etc.)

Python's approach:

- `__variable` triggers **name mangling** (`_ClassName__variable`)
- Not truly private (can still access via mangled name), but discourages direct access
- Convention: `_variable` = "protected" (by convention only)

Summary: All Concepts Compared

Concept	Focus	Mechanism	Example
Abstraction	Data Hiding	Abstract classes/interfaces	Car interface → SportsCar
Encapsulation	Data Security	Access modifiers + getters/setters	Private <code>currentSpeed</code>
Inheritance	Reusability	<code>extends</code> / <code>:</code>	SportsCar extends Car (next video)
Polymorphism	Flexibility	Overloading/Overriding	Next video

Key Takeaways

1. OOP exists because procedural programming couldn't model real-world complexity
2. **Object = Characteristics + Behavior**
3. **OOP code = Objects interacting with each other**
4. **Abstraction** = hide unnecessary details (data hiding)
5. **Encapsulation** = bind data + methods, and secure data (access modifiers)
6. **Abstraction ≠ Encapsulation** — one is about *hiding*, the other about *securing*
7. **Getters/Setters** give controlled access with validation
8. **Programming languages themselves are an abstraction** over machine code

What's Next (from transcript)

- **Inheritance** — covered in Part 2
- **Polymorphism** — covered in Part 2

- These complete the **4 Pillars of OOP**



हां जी। तो करें स्टार्ट। सो वेलक...
TXT 110.16KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also with explanation

OOP Pillars: Inheritance & Polymorphism — Complete Notes (Lecture 3)

Recap

In the previous lecture, we covered:

1. **Abstraction** — hiding unnecessary details
2. **Encapsulation** — binding data + behavior, data security

Today: **Inheritance** and **Polymorphism** — the remaining two pillars.

Part 1: Inheritance

1.1 What is Inheritance?

In real life, objects are **related** to each other — usually in a **parent-child relationship**.

Definition:

Inheritance is an OOP mechanism where a **child class** acquires the **properties (characteristics) and behaviors (methods)** of a **parent class**, and can additionally define its own specific features.

1.2 Real-World Example: Car Hierarchy

text

```
Car (Parent)
/      \
```


ManualCar

ElectricCar

(Child)

(Child)

Entity	Type	Common Features	Specific Features
Car	Parent	brand, model, isEngineOn, currentSpeed, startEngine(), stopEngine(), accelerate(), brake()	—
ManualCar	Child	inherits all above	currentGear, shiftGear()
ElectricCar	Child	inherits all above	batteryLevel, chargeBattery()

Key Insight:

- Common features live in the **parent** (Car)
- Specific features live in the **child** (ManualCar , ElectricCar)
- Children **don't need to redefine** common features — they inherit them

1.3 Access Modifiers in Inheritance

Modifier	Same Class	Child Class	Outside
public	✓	✓	✓
protected	✓	✓	✗
private	✓	✗	✗

protected is specifically designed for inheritance — accessible to child classes but not outside.

1.4 Inheritance Types (Public vs Private vs Protected)

When inheriting, the access modifier affects how parent members appear in the child:

Parent Member	Public Inherit	Protected Inherit	Private Inherit
public	public	protected	private

Parent Member	Public Inherit	Protected Inherit	Private Inherit
protected	protected	protected	private
private	✗ (not accessible)	✗	✗

Practical Rule: In 99% of real LLD/system design, we use **public inheritance**.

Private/protected inheritance is rare and breaks the spirit of inheritance.

1.5 Code Example: Inheritance

C++ (from transcript)

cpp

```
#include <iostream>
using namespace std;

// Parent class
class Car {
protected:
    string brand;
    string model;
    bool isEngineOn;
    int currentSpeed;

public:
    Car(string b, string m) {
        brand = b;
        model = m;
        isEngineOn = false;
        currentSpeed = 0;
    }

    void startEngine() {
        isEngineOn = true;
        cout << brand << " " << model << " : Engine starts!" << endl;
    }

    void stopEngine() {
        isEngineOn = false;
        currentSpeed = 0;
        cout << brand << " " << model << " : Engine turned off." << endl;
    }

    void accelerate() {
```

```
        if (!isEngineOn) {
            cout << "Engine is off. Cannot accelerate." << endl;
            return;
        }
        currentSpeed += 20;
        cout << "Accelerating to " << currentSpeed << " km/h" << endl;
    }

    void brake() {
        currentSpeed -= 20;
        if (currentSpeed < 0) currentSpeed = 0;
        cout << "Braking. Speed: " << currentSpeed << " km/h" << endl;
    }
};

// Child class 1: ManualCar
class ManualCar : public Car {
private:
    int currentGear;

public:
    ManualCar(string b, string m) : Car(b, m) {
        currentGear = 0;
    }

    void shiftGear(int gear) {
        if (!isEngineOn) {
            cout << "Engine is off. Cannot shift gear." << endl;
            return;
        }
        currentGear = gear;
        cout << "Shifted to gear " << currentGear << endl;
    }
};

// Child class 2: ElectricCar
class ElectricCar : public Car {
private:
    int batteryLevel;

public:
    ElectricCar(string b, string m) : Car(b, m) {
        batteryLevel = 100;
    }

    void chargeBattery() {
        batteryLevel = 100;
        cout << "Battery fully charged." << endl;
    }
};
```

```

    }
};

int main() {
    ManualCar manual("Suzuki", "WagonR");
    manual.startEngine();
    manual.shiftGear(1);
    manual.accelerate();
    manual.brake();
    manual.stopEngine();

    cout << "---" << endl;

    ElectricCar electric("Tesla", "Model S");
    electric.startEngine();
    electric.accelerate();
    electric.chargeBattery();
    electric.brake();
    electric.stopEngine();

    return 0;
}

```

Java Equivalent

java

```

// Parent class
class Car {
    protected String brand;
    protected String model;
    protected boolean isEngineOn;
    protected int currentSpeed;

    public Car(String brand, String model) {
        this.brand = brand;
        this.model = model;
        this.isEngineOn = false;
        this.currentSpeed = 0;
    }

    public void startEngine() {
        isEngineOn = true;
        System.out.println(brand + " " + model + " : Engine starts!");
    }

    public void stopEngine() {
        isEngineOn = false;
    }
}

```

```
        currentSpeed = 0;
        System.out.println(brand + " " + model + " : Engine turned off.");
    }

    public void accelerate() {
        if (!isEngineOn) {
            System.out.println("Engine is off. Cannot accelerate.");
            return;
        }
        currentSpeed += 20;
        System.out.println("Accelerating to " + currentSpeed + " km/h");
    }

    public void brake() {
        currentSpeed -= 20;
        if (currentSpeed < 0) currentSpeed = 0;
        System.out.println("Braking. Speed: " + currentSpeed + " km/h");
    }
}

// Child class 1
class ManualCar extends Car {
    private int currentGear;

    public ManualCar(String brand, String model) {
        super(brand, model);
        this.currentGear = 0;
    }

    public void shiftGear(int gear) {
        if (!isEngineOn) {
            System.out.println("Engine is off. Cannot shift gear.");
            return;
        }
        currentGear = gear;
        System.out.println("Shifted to gear " + currentGear);
    }
}

// Child class 2
class ElectricCar extends Car {
    private int batteryLevel;

    public ElectricCar(String brand, String model) {
        super(brand, model);
        this.batteryLevel = 100;
    }
}
```

```

    public void chargeBattery() {
        batteryLevel = 100;
        System.out.println("Battery fully charged.");
    }
}

public class Main {
    public static void main(String[] args) {
        ManualCar manual = new ManualCar("Suzuki", "WagonR");
        manual.startEngine();
        manual.shiftGear(1);
        manual.accelerate();
        manual.brake();
        manual.stopEngine();

        System.out.println("---");

        ElectricCar electric = new ElectricCar("Tesla", "Model S");
        electric.startEngine();
        electric.accelerate();
        electric.chargeBattery();
        electric.brake();
        electric.stopEngine();
    }
}

```

Python Equivalent

python

Parent class

```

class Car:
    def __init__(self, brand, model):
        self._brand = brand          # protected by convention
        self._model = model
        self._is_engine_on = False
        self._current_speed = 0

    def start_engine(self):
        self._is_engine_on = True
        print(f"{self._brand} {self._model} : Engine starts!")

    def stop_engine(self):
        self._is_engine_on = False
        self._current_speed = 0
        print(f"{self._brand} {self._model} : Engine turned off.")

    def accelerate(self):

```

```
        if not self._is_engine_on:
            print("Engine is off. Cannot accelerate.")
            return
        self._current_speed += 20
        print(f"Accelerating to {self._current_speed} km/h")

    def brake(self):
        self._current_speed -= 20
        if self._current_speed < 0:
            self._current_speed = 0
        print(f"Braking. Speed: {self._current_speed} km/h")

# Child class 1
class ManualCar(Car):
    def __init__(self, brand, model):
        super().__init__(brand, model)
        self.__current_gear = 0

    def shift_gear(self, gear):
        if not self._is_engine_on:
            print("Engine is off. Cannot shift gear.")
            return
        self.__current_gear = gear
        print(f"Shifted to gear {self.__current_gear}")

# Child class 2
class ElectricCar(Car):
    def __init__(self, brand, model):
        super().__init__(brand, model)
        self.__battery_level = 100

    def charge_battery(self):
        self.__battery_level = 100
        print("Battery fully charged.")

# Usage
manual = ManualCar("Suzuki", "WagonR")
manual.start_engine()
manual.shift_gear(1)
manual.accelerate()
manual.brake()
manual.stop_engine()

print("---")
```

```

electric = ElectricCar("Tesla", "Model S")
electric.start_engine()
electric.accelerate()
electric.charge_battery()
electric.brake()
electric.stop_engine()

```

Code Explanation

- protected **members** in `Car` are accessible by `ManualCar` and `ElectricCar`
- Children inherit** `startEngine`, `stopEngine`, `accelerate`, `brake` — no need to redefine
- Children add their own** specific features: `shiftGear` (`Manual`) and `chargeBattery` (`Electric`)
- Constructor chaining:** child constructors call parent constructor (`super()` in Java/Python, `: Car(b, m)` in C++)
- `electric.shiftGear()` **would be an error** — `ElectricCar` doesn't have that method

Output:

text

```

Suzuki WagonR : Engine starts!
Shifted to gear 1
Accelerating to 20 km/h
Braking. Speed: 0 km/h
Suzuki WagonR : Engine turned off.
---
Tesla Model S : Engine starts!
Accelerating to 20 km/h
Battery fully charged.
Braking. Speed: 0 km/h
Tesla Model S : Engine turned off.

```

Part 2: Polymorphism

2.1 What is Polymorphism?

Etymology:

- Poly** = many

- **Morphism** = forms
- **Polymorphism** = "Same object, many forms"

2.2 Real-World Scenarios

Scenario 1: Different animals, same behavior

- Animal: `run()`
- Duck runs differently
- Human runs differently
- Tiger runs differently
- Same stimulus ("run"), different responses

Scenario 2: Same object, different situations

- Human: `run()`
- Tired human → runs slowly
- Scared human (tiger chasing) → runs fast
- Same object, same method, different parameter → different behavior

2.3 Two Types of Polymorphism in Programming

Type	Also Called	Mechanism	When Resolved
Dynamic	Method Overriding	Same method signature, different class	Runtime
Static	Method Overloading	Same method name, different parameters	Compile time

2.4 Dynamic Polymorphism (Method Overriding)

Definition:

A child class provides its own implementation of a method already defined in the parent class. The method signature (name, return type, parameters) must be **exactly the same**. Resolution happens at **runtime**.

Rules for Overriding:

- Same method name
- Same return type
- Same number and type of parameters
- Requires inheritance
- Uses `virtual` in C++, `@Override` in Java

2.5 Static Polymorphism (Method Overloading)

Definition:

Multiple methods in the **same class** with the **same name** but **different parameters** (different number or type of arguments). Resolution happens at **compile time**.

Rules for Overloading:

- Same method name
- Same return type (in C++; Java allows different but not recommended)
- Different number of parameters OR different parameter types
- No inheritance required

2.6 Code Example: Dynamic Polymorphism

C++ (from transcript)

cpp

```
#include <iostream>
using namespace std;

class Car {
protected:
    string brand, model;
    bool isEngineOn;
    int currentSpeed;

public:
    Car(string b, string m) : brand(b), model(m),
                             isEngineOn(false), currentSpeed(0) {}

    void startEngine() {
        isEngineOn = true;
        cout << brand << " " << model << " : Engine started." << endl;
    }
}
```

```
}

void stopEngine() {
    isEngineOn = false;
    currentSpeed = 0;
    cout << brand << " " << model << " : Engine turned off." << endl;
}

// Pure virtual – MUST be overridden
virtual void accelerate() = 0;
virtual void brake() = 0;
};

class ManualCar : public Car {
private:
    int currentGear;

public:
    ManualCar(string b, string m) : Car(b, m), currentGear(0) {}

    void shiftGear(int gear) {
        currentGear = gear;
        cout << "Shifted to gear " << currentGear << endl;
    }

    void accelerate() override {
        if (!isEngineOn) { cout << "Engine off." << endl; return; }
        currentSpeed += 20;
        cout << "Manual accelerating to " << currentSpeed << " km/h" << endl;
    }

    void brake() override {
        currentSpeed -= 20;
        if (currentSpeed < 0) currentSpeed = 0;
        cout << "Manual braking. Speed: " << currentSpeed << " km/h" << endl;
    }
};

class ElectricCar : public Car {
private:
    int batteryLevel;

public:
    ElectricCar(string b, string m) : Car(b, m), batteryLevel(100) {}

    void chargeBattery() {
        batteryLevel = 100;
        cout << "Battery charged." << endl;
    }
};
```

```

    }

    void accelerate() override {
        if (batteryLevel <= 0) { cout << "Battery empty." << endl; return; }
        batteryLevel -= 5;
        currentSpeed += 15;
        cout << "Electric accelerating to " << currentSpeed
              << " km/h. Battery: " << batteryLevel << "%" << endl;
    }

    void brake() override {
        currentSpeed -= 15;
        if (currentSpeed < 0) currentSpeed = 0;
        cout << "Regenerative braking. Speed: " << currentSpeed << " km/h" << endl;
    }
};

int main() {
    ManualCar manual("Suzuki", "WagonR");
    ElectricCar electric("Tesla", "Model S");

    manual.startEngine();
    manual.accelerate();
    manual.accelerate();
    manual.brake();
    manual.stopEngine();

    cout << "---" << endl;

    electric.startEngine();
    electric.accelerate();
    electric.accelerate();
    electric.brake();
    electric.stopEngine();

    return 0;
}

```

Java Equivalent

java

```

abstract class Car {
    protected String brand, model;
    protected boolean isEngineOn;
    protected int currentSpeed;

    public Car(String brand, String model) {

```

```
        this.brand = brand;
        this.model = model;
        this.isEngineOn = false;
        this.currentSpeed = 0;
    }

    public void startEngine() {
        isEngineOn = true;
        System.out.println(brand + " " + model + " : Engine started.");
    }

    public void stopEngine() {
        isEngineOn = false;
        currentSpeed = 0;
        System.out.println(brand + " " + model + " : Engine turned off.");
    }

    // Abstract methods – must be overridden
    public abstract void accelerate();
    public abstract void brake();
}

class ManualCar extends Car {
    private int currentGear;

    public ManualCar(String brand, String model) {
        super(brand, model);
        this.currentGear = 0;
    }

    public void shiftGear(int gear) {
        currentGear = gear;
        System.out.println("Shifted to gear " + currentGear);
    }

    @Override
    public void accelerate() {
        if (!isEngineOn) { System.out.println("Engine off."); return; }
        currentSpeed += 20;
        System.out.println("Manual accelerating to " + currentSpeed + " km/h");
    }

    @Override
    public void brake() {
        currentSpeed -= 20;
        if (currentSpeed < 0) currentSpeed = 0;
        System.out.println("Manual braking. Speed: " + currentSpeed + " km/h");
    }
}
```

```
}

class ElectricCar extends Car {
    private int batteryLevel;

    public ElectricCar(String brand, String model) {
        super(brand, model);
        this.batteryLevel = 100;
    }

    public void chargeBattery() {
        batteryLevel = 100;
        System.out.println("Battery charged.");
    }

    @Override
    public void accelerate() {
        if (batteryLevel <= 0) { System.out.println("Battery empty."); return; }
        batteryLevel -= 5;
        currentSpeed += 15;
        System.out.println("Electric accelerating to " + currentSpeed
            + " km/h. Battery: " + batteryLevel + "%");
    }

    @Override
    public void brake() {
        currentSpeed -= 15;
        if (currentSpeed < 0) currentSpeed = 0;
        System.out.println("Regenerative braking. Speed: " + currentSpeed + " km/h");
    }
}

public class Main {
    public static void main(String[] args) {
        ManualCar manual = new ManualCar("Suzuki", "WagonR");
        manual.startEngine();
        manual.accelerate();
        manual.accelerate();
        manual.brake();
        manual.stopEngine();

        System.out.println("---");

        ElectricCar electric = new ElectricCar("Tesla", "Model S");
        electric.startEngine();
        electric.accelerate();
        electric.accelerate();
        electric.brake();
    }
}
```

```

        electric.stopEngine();
    }
}

```

Python Equivalent

python

```

from abc import ABC, abstractmethod

class Car(ABC):
    def __init__(self, brand, model):
        self._brand = brand
        self._model = model
        self._is_engine_on = False
        self._current_speed = 0

    def start_engine(self):
        self._is_engine_on = True
        print(f"{self._brand} {self._model} : Engine started.")

    def stop_engine(self):
        self._is_engine_on = False
        self._current_speed = 0
        print(f"{self._brand} {self._model} : Engine turned off.")

    @abstractmethod
    def accelerate(self): pass

    @abstractmethod
    def brake(self): pass

class ManualCar(Car):
    def __init__(self, brand, model):
        super().__init__(brand, model)
        self.__current_gear = 0

    def shift_gear(self, gear):
        self.__current_gear = gear
        print(f"Shifted to gear {self.__current_gear}")

    def accelerate(self):
        if not self._is_engine_on:
            print("Engine off.")
            return
        self._current_speed += 20
        print(f"Manual accelerating to {self._current_speed} km/h")

```

```

def brake(self):
    self._current_speed -= 20
    if self._current_speed < 0:
        self._current_speed = 0
    print(f"Manual braking. Speed: {self._current_speed} km/h")

class ElectricCar(Car):
    def __init__(self, brand, model):
        super().__init__(brand, model)
        self.__battery_level = 100

    def charge_battery(self):
        self.__battery_level = 100
        print("Battery charged.")

    def accelerate(self):
        if self.__battery_level <= 0:
            print("Battery empty.")
            return
        self.__battery_level -= 5
        self._current_speed += 15
        print(f"Electric accelerating to {self._current_speed} km/h. "
              f"Battery: {self.__battery_level}%")

    def brake(self):
        self._current_speed -= 15
        if self._current_speed < 0:
            self._current_speed = 0
        print(f"Regenerative braking. Speed: {self._current_speed} km/h")

# Usage
manual = ManualCar("Suzuki", "WagonR")
manual.start_engine()
manual.accelerate()
manual.accelerate()
manual.brake()
manual.stop_engine()

print("---")

electric = ElectricCar("Tesla", "Model S")
electric.start_engine()
electric.accelerate()
electric.accelerate()

```



```
electric.brake()
electric.stop_engine()
```

Output:

text

```
Suzuki WagonR : Engine started.
Manual accelerating to 20 km/h
Manual accelerating to 40 km/h
Manual braking. Speed: 20 km/h
Suzuki WagonR : Engine turned off.
---
Tesla Model S : Engine started.
Electric accelerating to 15 km/h. Battery: 95%
Electric accelerating to 30 km/h. Battery: 90%
Regenerative braking. Speed: 15 km/h
Tesla Model S : Engine turned off.
```

Code Explanation

- `accelerate()` and `brake()` are **abstract** in `Car` — no body, must be overridden
- **Both children override** them with different implementations
- **Same method name, same signature** → **Dynamic Polymorphism (Overriding)**
- **ManualCar accelerates by 20, ElectricCar by 15** (and drains battery)
- **Braking differs:** normal vs regenerative
- **Runtime decision:** which `accelerate()` runs depends on the actual object

2.7 Code Example: Static Polymorphism (Method Overloading)

C++ (from transcript)

cpp

```
#include <iostream>
using namespace std;

class ManualCar {
private:
    string brand, model;
    bool isEngineOn;
```

```
int currentSpeed;
int currentGear;

public:
    ManualCar(string b, string m) : brand(b), model(m),
                                    isEngineOn(false),
                                    currentSpeed(0),
                                    currentGear(0) {}

    void startEngine() {
        isEngineOn = true;
        cout << brand << " " << model << " : Engine started." << endl;
    }

    void stopEngine() {
        isEngineOn = false;
        currentSpeed = 0;
        currentGear = 0;
        cout << "Engine turned off." << endl;
    }

    // Overload 1: no parameter
    void accelerate() {
        if (!isEngineOn) { cout << "Engine off." << endl; return; }
        currentSpeed += 20;
        cout << "Accelerating (default) to " << currentSpeed << " km/h" << endl;
    }

    // Overload 2: with speed parameter
    void accelerate(int speed) {
        if (!isEngineOn) { cout << "Engine off." << endl; return; }
        currentSpeed += speed;
        cout << "Accelerating by " << speed
              << " to " << currentSpeed << " km/h" << endl;
    }

    void brake() {
        currentSpeed -= 20;
        if (currentSpeed < 0) currentSpeed = 0;
        cout << "Braking. Speed: " << currentSpeed << " km/h" << endl;
    }

    void shiftGear(int gear) {
        currentGear = gear;
        cout << "Shifted to gear " << currentGear << endl;
    }
};
```

```

int main() {
    ManualCar car("Suzuki", "WagonR");
    car.startEngine();
    car.accelerate();           // calls overload 1
    car.accelerate(40);         // calls overload 2
    car.brake();
    car.stopEngine();
    return 0;
}

```

Java Equivalent

java

```

class ManualCar {
    private String brand, model;
    private boolean isEngineOn;
    private int currentSpeed;
    private int currentGear;

    public ManualCar(String brand, String model) {
        this.brand = brand;
        this.model = model;
        this.isEngineOn = false;
        this.currentSpeed = 0;
        this.currentGear = 0;
    }

    public void startEngine() {
        isEngineOn = true;
        System.out.println(brand + " " + model + " : Engine started.");
    }

    public void stopEngine() {
        isEngineOn = false;
        currentSpeed = 0;
        currentGear = 0;
        System.out.println("Engine turned off.");
    }

    // OverLoad 1
    public void accelerate() {
        if (!isEngineOn) { System.out.println("Engine off."); return; }
        currentSpeed += 20;
        System.out.println("Accelerating (default) to " + currentSpeed + " km/h");
    }

    // OverLoad 2

```

```

public void accelerate(int speed) {
    if (!isEngineOn) { System.out.println("Engine off."); return; }
    currentSpeed += speed;
    System.out.println("Accelerating by " + speed
        + " to " + currentSpeed + " km/h");
}

public void brake() {
    currentSpeed -= 20;
    if (currentSpeed < 0) currentSpeed = 0;
    System.out.println("Braking. Speed: " + currentSpeed + " km/h");
}

public void shiftGear(int gear) {
    currentGear = gear;
    System.out.println("Shifted to gear " + currentGear);
}
}

public class Main {
    public static void main(String[] args) {
        ManualCar car = new ManualCar("Suzuki", "WagonR");
        car.startEngine();
        car.accelerate();           // overload 1
        car.accelerate(40);        // overload 2
        car.brake();
        car.stopEngine();
    }
}

```

Python Equivalent

Python doesn't support traditional method overloading. We use **default arguments** or

*args :

python

```

class ManualCar:
    def __init__(self, brand, model):
        self.__brand = brand
        self.__model = model
        self.__is_engine_on = False
        self.__current_speed = 0
        self.__current_gear = 0

    def start_engine(self):
        self.__is_engine_on = True

```

```

    print(f"{self.__brand} {self.__model} : Engine started.")

def stop_engine(self):
    self.__is_engine_on = False
    self.__current_speed = 0
    self.__current_gear = 0
    print("Engine turned off.")

# Simulates overloading via default argument
def accelerate(self, speed=None):
    if not self.__is_engine_on:
        print("Engine off.")
        return
    if speed is None:
        self.__current_speed += 20
        print(f"Accelerating (default) to {self.__current_speed} km/h")
    else:
        self.__current_speed += speed
        print(f"Accelerating by {speed} to {self.__current_speed} km/h")

def brake(self):
    self.__current_speed -= 20
    if self.__current_speed < 0:
        self.__current_speed = 0
    print(f"Braking. Speed: {self.__current_speed} km/h")

def shift_gear(self, gear):
    self.__current_gear = gear
    print(f"Shifted to gear {self.__current_gear}")

car = ManualCar("Suzuki", "WagonR")
car.start_engine()
car.accelerate()           # default
car.accelerate(40)         # with parameter
car.brake()
car.stop_engine()

```

Output:

text

```

Suzuki WagonR : Engine started.
Accelerating (default) to 20 km/h
Accelerating by 40 to 60 km/h
Braking. Speed: 40 km/h
Engine turned off.

```

Code Explanation

- **Two methods with same name** `accelerate` but **different parameters**
- `accelerate()` → default 20 km/h
- `accelerate(40)` → adds 40 km/h
- **Resolution at compile time** based on arguments → **Static Polymorphism**
- **Python note:** No true overloading — use default args or `*args`

Part 3: Combined Example — All 4 Pillars in One Program

The transcript mentions a GitHub repo containing a complete example demonstrating all four pillars together. Here's a comprehensive version:

Java (Complete Example)

java

```
// ===== ABSTRACTION: abstract class Car =====
abstract class Car {
    // ===== ENCAPSULATION: protected data =====
    protected String brand;
    protected String model;
    protected boolean isEngineOn;
    protected int currentSpeed;

    public Car(String brand, String model) {
        this.brand = brand;
        this.model = model;
        this.isEngineOn = false;
        this.currentSpeed = 0;
    }

    public void startEngine() {
        isEngineOn = true;
        System.out.println(brand + " " + model + " : Engine started.");
    }

    public void stopEngine() {
        isEngineOn = false;
        currentSpeed = 0;
        System.out.println(brand + " " + model + " : Engine turned off.");
    }
}
```

```

// ===== STATIC POLYMORPHISM: overloaded accelerate =====
// ===== DYNAMIC POLYMORPHISM: virtual accelerate =====
public abstract void accelerate();
public abstract void accelerate(int speed);

// ===== DYNAMIC POLYMORPHISM: virtual brake =====
public abstract void brake();
}

// ===== INHERITANCE: ManualCar extends Car =====
class ManualCar extends Car {
    private int currentGear;

    public ManualCar(String brand, String model) {
        super(brand, model);
        this.currentGear = 0;
    }

    public void shiftGear(int gear) {
        currentGear = gear;
        System.out.println("Shifted to gear " + currentGear);
    }

    // ===== STATIC POLYMORPHISM: overLoading =====
    @Override
    public void accelerate() {
        if (!isEngineOn) { System.out.println("Engine off."); return; }
        currentSpeed += 20;
        System.out.println("Manual accelerating to " + currentSpeed + " km/h");
    }

    @Override
    public void accelerate(int speed) {
        if (!isEngineOn) { System.out.println("Engine off."); return; }
        currentSpeed += speed;
        System.out.println("Manual accelerating by " + speed
            + " to " + currentSpeed + " km/h");
    }

    @Override
    public void brake() {
        currentSpeed -= 20;
        if (currentSpeed < 0) currentSpeed = 0;
        System.out.println("Manual braking. Speed: " + currentSpeed + " km/h");
    }
}

```

```
// ===== INHERITANCE: ElectricCar extends Car =====
class ElectricCar extends Car {
    private int batteryLevel;

    public ElectricCar(String brand, String model) {
        super(brand, model);
        this.batteryLevel = 100;
    }

    public void chargeBattery() {
        batteryLevel = 100;
        System.out.println("Battery charged.");
    }

    @Override
    public void accelerate() {
        if (batteryLevel <= 0) { System.out.println("Battery empty."); return; }
        batteryLevel -= 5;
        currentSpeed += 15;
        System.out.println("Electric accelerating to " + currentSpeed
            + " km/h. Battery: " + batteryLevel + "%");
    }

    @Override
    public void accelerate(int speed) {
        if (batteryLevel <= 0) { System.out.println("Battery empty."); return; }
        batteryLevel -= 5;
        currentSpeed += speed - 5; // electric is slightly slower
        System.out.println("Electric accelerating by " + (speed - 5)
            + " to " + currentSpeed + " km/h. Battery: "
            + batteryLevel + "%");
    }

    @Override
    public void brake() {
        currentSpeed -= 15;
        if (currentSpeed < 0) currentSpeed = 0;
        System.out.println("Regenerative braking. Speed: " + currentSpeed + " km/h");
    }
}

public class Main {
    public static void main(String[] args) {
        ManualCar manual = new ManualCar("Suzuki", "WagonR");
        ElectricCar electric = new ElectricCar("Tesla", "Model S");

        manual.startEngine();
        manual.accelerate(); // overLoad 1
    }
}
```



```
manual.accelerate(40);    // overLoad 2
manual.brake();
manual.stopEngine();

System.out.println("---");

electric.startEngine();
electric.accelerate();
electric.accelerate(40);
electric.brake();
electric.stopEngine();
}
}
```

Output:

text

```
Suzuki WagonR : Engine started.
Manual accelerating to 20 km/h
Manual accelerating by 40 to 60 km/h
Manual braking. Speed: 40 km/h
Suzuki WagonR : Engine turned off.
---
Tesla Model S : Engine started.
Electric accelerating to 15 km/h. Battery: 95%
Electric accelerating by 35 to 50 km/h. Battery: 90%
Regenerative braking. Speed: 35 km/h
Tesla Model S : Engine turned off.
```

How All 4 Pillars Appear

Pillar	Where It Appears
Abstraction	Car abstract class with abstract methods — client only knows "what", not "how"
Encapsulation	protected data members; specific data like currentGear , batteryLevel are private
Inheritance	ManualCar and ElectricCar extend Car
Polymorphism (Dynamic)	accelerate() and brake() overridden in children

Pillar	Where It Appears
Polymorphism (Static)	Two <code>accelerate()</code> overloads in each class

Part 4: Comparison Summary

Overloading vs Overriding

Feature	Method Overloading	Method Overriding
Type	Static Polymorphism	Dynamic Polymorphism
Where	Same class	Parent & child class
Method name	Same	Same
Parameters	Different	Same
Return type	Same (C++)	Same
Inheritance	Not required	Required
Resolution	Compile-time	Runtime
Keyword	—	<code>virtual / override</code> (C++), <code>@Override</code> (Java)

All 4 Pillars Summary

Pillar	Meaning	Mechanism
Abstraction	Hide unnecessary details	Abstract classes / interfaces
Encapsulation	Bind data + methods; secure data	Access modifiers + getters/setters
Inheritance	Acquire parent's properties	<code>extends</code> / <code>:</code>
Polymorphism	Many forms	Overloading (static) + Overriding (dynamic)

Part 5: Homework (from transcript)

1. What is Operator Overloading in C++?

- Answer: Giving special meaning to operators (+ , - , * , etc.) for user-defined types.

2. Why don't Java and Python support operator overloading?

- **Java:** Designers chose simplicity; operator overloading can lead to confusing code. Java allows only string concatenation with + .
- **Python:** Actually **does** support operator overloading via dunder methods (__add__ , __sub__ , etc.) — but the question likely means "traditional operator overloading" as in C++.

Python Operator Overloading Example (Bonus)

python

```
class Vector:
    def __init__(self, x, y):
        self.x = x        self.y = y

    def __add__(self, other):    # Overloads '+'
        return Vector(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"({self.x}, {self.y})"

v1 = Vector(2, 3)
v2 = Vector(4, 5)
print(v1 + v2)    # (6, 8)
```

Key Takeaways

1. **Inheritance** = child acquires parent's features + adds its own
2. **protected** = accessible to children, not outside
3. **Public inheritance** is 99% of real-world usage
4. **Polymorphism** = "many forms" — same interface, different behavior
5. **Dynamic Polymorphism** = method overriding (runtime, needs inheritance)

6. **Static Polymorphism** = method overloading (compile-time, same class)
7. **All 4 pillars work together** in real applications
8. **A single well-designed example** (like Car hierarchy) can demonstrate all 4 pillars
9. **Python doesn't have true method overloading** — use default args
10. **Operator overloading** is C++-specific (Python uses dunder methods)

What's Next (from transcript)

- The complete source code is available on the instructor's GitHub repository
- Next videos will build on these OOP foundations for deeper LLD concepts



हां भाई करें स्टार्ट। सो वेलकम ...
TXT 172.25KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also with explanation

UML Diagrams — Complete Notes (Lecture 4)

Part 1: What Are UML Diagrams?

The Problem

You have an application idea in your mind. You need to communicate it to a friend/team. Two options:

1. **Write huge paragraphs** — boring, non-intuitive, hard to understand
2. **Draw diagrams** — intuitive, visual, easy to grasp

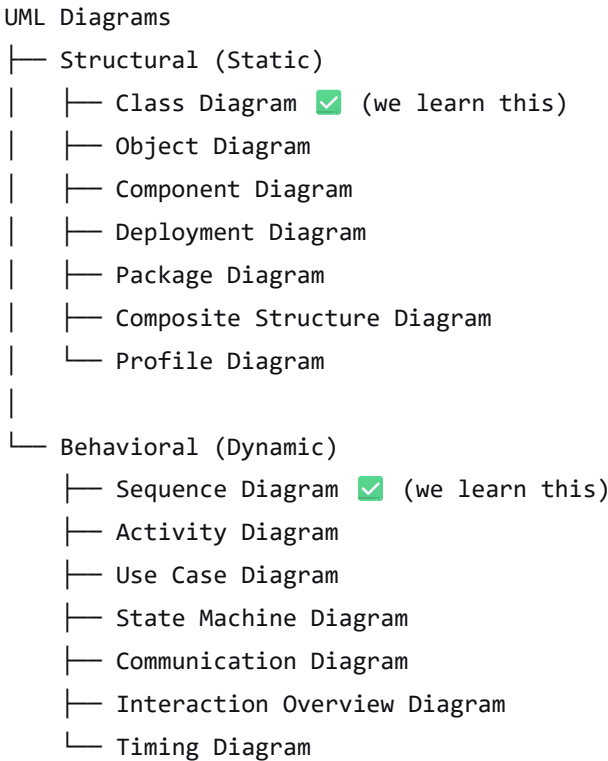
UML (Unified Modeling Language) Diagrams = expressing your application idea diagrammatically:

- What **components/objects/entities** exist
- How they **interact** with each other
- How they're **connected**

Part 2: Types of UML Diagrams

UML diagrams are divided into **two categories**:

text



Structural vs Behavioral

Aspect	Structural (Static)	Behavioral (Dynamic)
Shows	Application structure	Application interactions
Focus	Components & their connections	How components talk to each other
Example	Class Diagram	Sequence Diagram

Key Insight: Of 14 UML diagrams, **only 2 matter for LLD**:

- 1. **Class Diagram** (Structural) — 99% of LLD interviews
- 2. **Sequence Diagram** (Behavioral) — for specific use cases

The other 12 are rarely used and very use-case specific.

Part 3: Class Diagram

3.1 What is a Class Diagram?

Shows:

- What **classes** exist in your application
- How those classes are **connected/associated** with each other

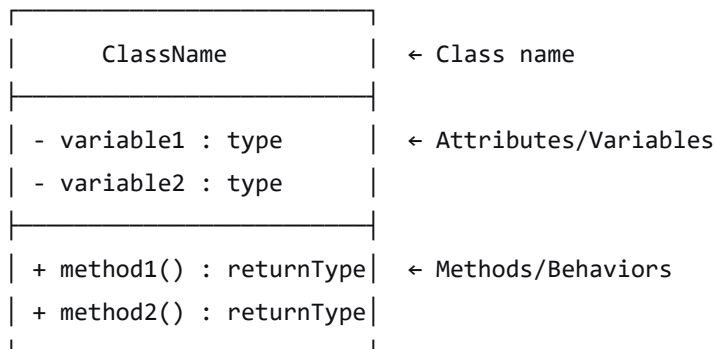
Two things to learn:

1. **Class structure** — how to represent a class
2. **Associations** — how to show connections between classes

3.2 Representing a Class

A class is drawn as a **rectangle divided into 3 parts**:

text



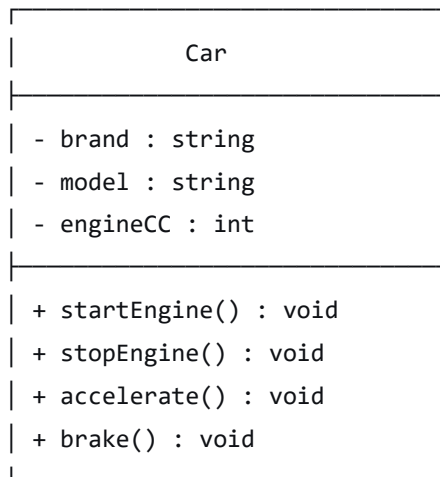
Note: In UML, the format is `name : type` (opposite of code).

3.3 Access Modifier Symbols

Modifier	Symbol
public	+
protected	#
private	-

Example — Car Class:

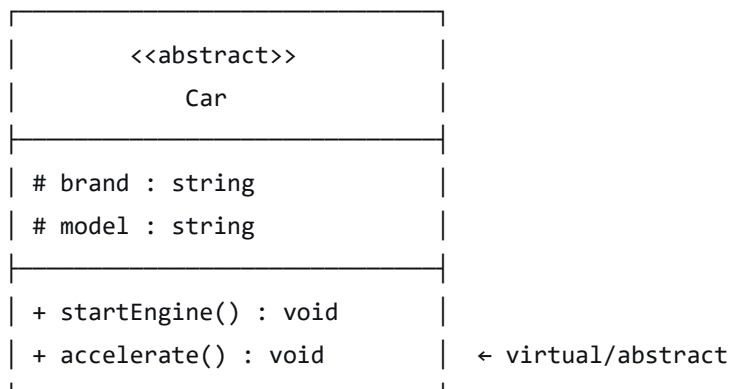
text



3.4 Abstract Class Representation

If a class is **abstract**, we write <<abstract>> above the class name:

text



If it's a **concrete class**, nothing is written above.

3.5 Class Associations

Two major types of associations:

text

Associations

- └─ Class-level Association
 - └─ Inheritance (IS-A relationship)
- └─ Object-level Association

- |— Simple Association (weakest)
- |— Aggregation (medium)
- |— Composition (strongest)

Key Insight: All three object-level associations are **combinedly called Composition** in programming because they're **represented the same way in code**.

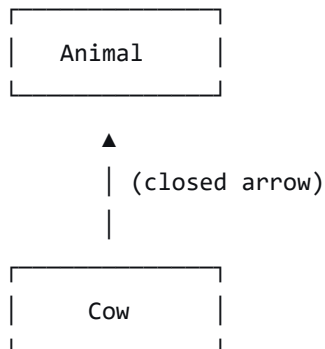
3.6 Inheritance (IS-A Relationship)

Inheritance shows an "IS-A" relationship:

- Cow IS-A Animal
- Tiger IS-A Animal
- ManualCar IS-A Car
- ElectricCar IS-A Car

UML Representation: A **closed (filled) arrow** pointing from child to parent.

text



Code Representation:

- **Java:** `class Cow extends Animal`
- **C++:** `class Cow : public Animal`
- **Python:** `class Cow(Animal)`

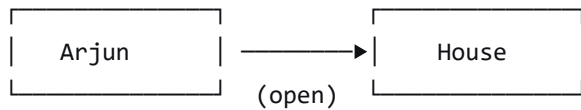
3.7 Object-Level Associations

(a) Simple Association (Weakest)

Real-world Example: Arjun lives in a House. Arjun HAS-A House.

UML Representation: An **open arrow** pointing from one class to another.

text



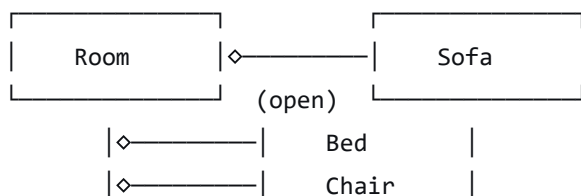
Relationship: Arjun HAS-A House (or Arjun LIVES-IN House)

(b) Aggregation (Medium Strength)

Real-world Example: A Room HAS-A Sofa, Bed, Chair. But Sofa/Bed/Chair can **exist independently** without the Room.

UML Representation: An **open (unfilled) diamond** on the container side, with a line to the contained class.

text



The diamond is on the **Room** side (the container/aggregator).

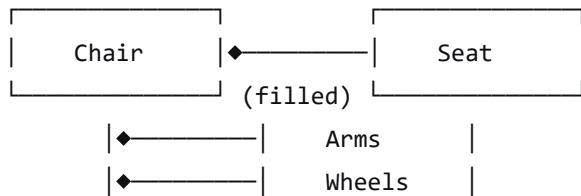
Key Point: Contained objects can exist independently.

(c) Composition (Strongest)

Real-world Example: A Chair HAS-A Seat, Arms, Wheels. But Seat/Arms/Wheels **cannot exist independently** without the Chair.

UML Representation: A **filled (closed) diamond** on the container side.

text



Key Point: Contained objects **cannot exist** without the container. If container is destroyed, parts are destroyed too.

3.8 Comparison of Object Associations

Type	Relationship	UML Symbol	Independence
Simple Association	HAS-A	Open arrow →	Fully independent
Aggregation	HAS-A (container)	Open diamond ◇	Can exist independently
Composition	HAS-A (strong)	Filled diamond ◆	Cannot exist independently

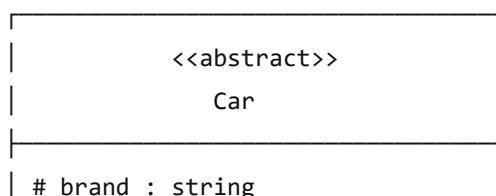
Important: In **code**, all three look the same — you store a reference of one class inside another:

```
java
class B {
    A a; // composition/aggregation/simple association – all look like this
}
```

The difference is **conceptual/subjective** — depends on how you design your application.

3.9 Complete Class Diagram Example — Car Hierarchy

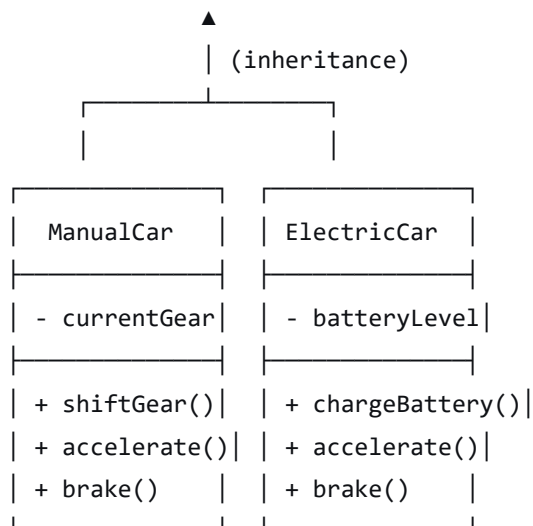
text



```

| # model : string |
| # isEngineOn : boolean |
| # currentSpeed : int |
|-----|
| + Car(brand, model) |
| + startEngine() : void |
| + stopEngine() : void |
| + accelerate() : void <<virtual>> |
| + brake() : void <<virtual>> |
|-----|

```



Class Diagram Interpretation:

- **Car** is abstract (has virtual methods `accelerate` , `brake`)
- **ManualCar** and **ElectricCar** inherit from **Car**
- **ManualCar** adds `currentGear` and `shiftGear()`
- **ElectricCar** adds `batteryLevel` and `chargeBattery()`

3.10 Zomato Example — Association Subjectivity

Scenario: Restaurant HAS-A Menu

Question: Is Menu **aggregation** or **composition**?

- **Aggregation view:** Menu can exist independently (a menu could be shown elsewhere)
- **Composition view:** Menu cannot exist without Restaurant (menu belongs to that restaurant)

Key Insight: This is **subjective** — depends on how you design your application. There's no single "correct" answer.

Part 4: Sequence Diagram

4.1 What is a Sequence Diagram?

A **behavioral (dynamic) diagram** that shows how **objects interact** with each other over **time**.

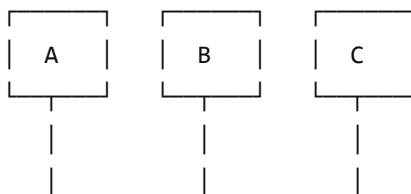
Key components:

1. **Objects** — represented as rectangles at the top
2. **Lifeline** — dashed vertical line showing object's existence
3. **Activation Bar** — rectangle on lifeline showing when object is active
4. **Messages** — arrows between lifelines

4.2 Representing Objects

In a sequence diagram, each object is drawn as a rectangle at the top:

text

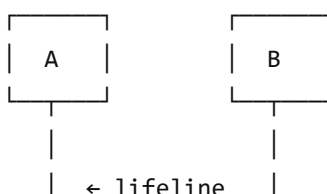


Note: Only the object name is shown — class structure isn't relevant here. You're showing **interactions**, not structure.

4.3 Lifeline

A **dashed vertical line** below each object showing **when the object exists** in the application.

text

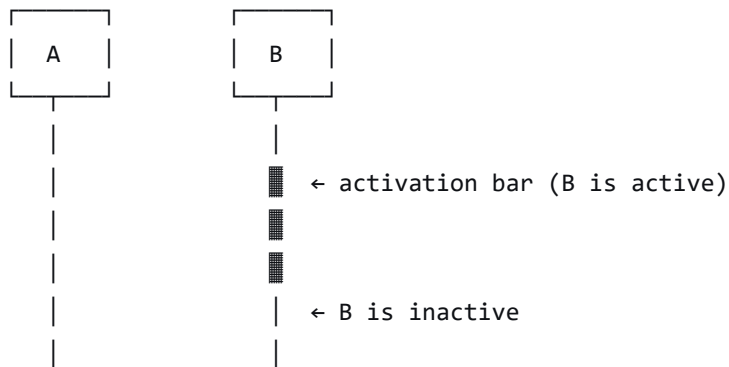


If an object is created mid-flow, its lifeline starts mid-diagram.

4.4 Activation Bar

A **rectangle on the lifeline** showing when the object is **active** (can send/receive messages).

text



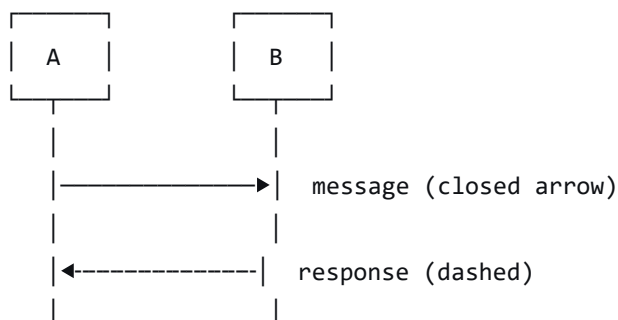
Rule: Objects can only send/receive messages **while their activation bar is active**.

4.5 Message Types

(a) Synchronous Message

- Sender **waits** for response
- Drawn as **closed arrow**
- Response drawn as **dashed arrow**

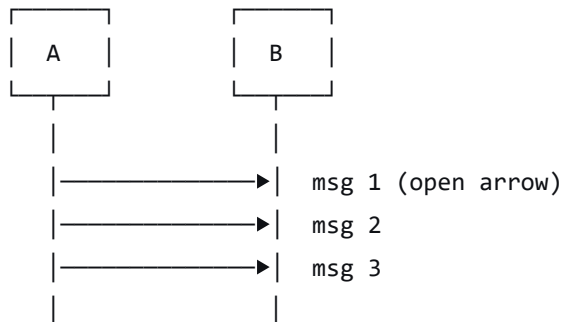
text



(b) Asynchronous Message

- Sender **doesn't wait** for response
- Drawn as **open arrow**
- Can send multiple messages quickly

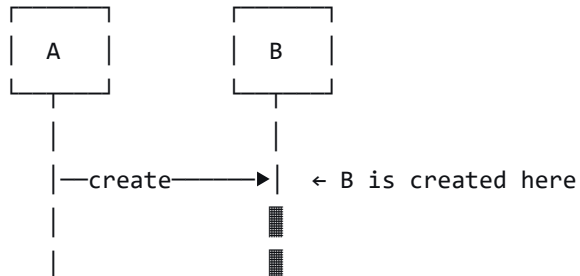
text



(c) Create Message

- Creates a **new object**
- Lifeline of new object starts at this point

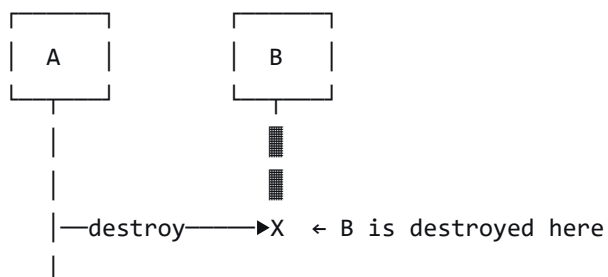
text



(d) Destroy Message

- **Destroys** an object
- Lifeline ends with an **X**

text



(e) Lost Message

- A message is sent but **never reaches** the target

text



(f) Found Message

- A message arrives at an object from an **unknown source**

text



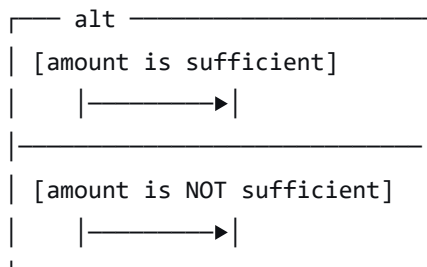
4.6 Combined Fragments: Alt, Opt, Loop

These are special notations for control flow:

Fragment	Meaning	Programming Equivalent
alt	Alternative	if-else
opt	Optional	if (no else)
loop	Iteration	for / while

Example — alt fragment:

text



Note: Usually, we only discuss the **happy flow** (everything succeeds), so alt/opt/loop are rarely used.

4.7 Complete Sequence Diagram Example — ATM Withdrawal

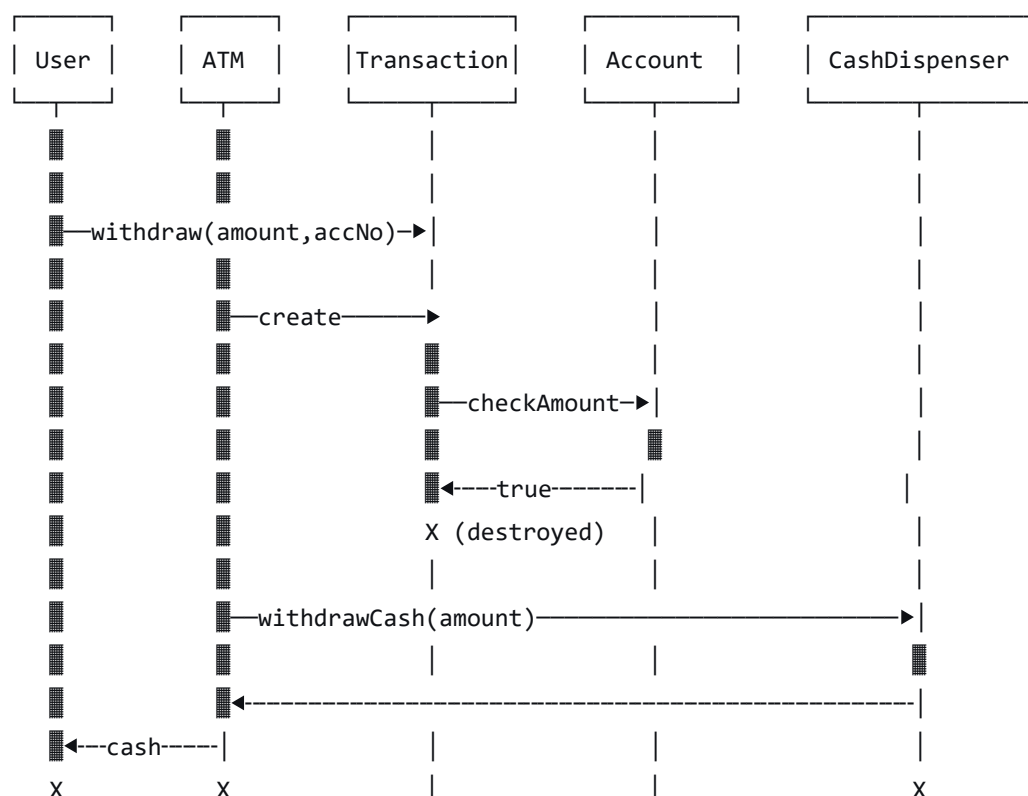
Use Case: User goes to ATM, enters account number and amount, transaction is verified, and cash is dispensed.

Step-by-step flow:

1. User sends `withdraw(amount, accountNumber)` to ATM
2. ATM **creates** a Transaction object
3. Transaction sends `checkAmount(amount)` to Account
4. Account verifies sufficient funds → returns `true`
5. Transaction is **destroyed** (work done)
6. ATM sends `withdrawCash(amount)` to CashDispenser
7. CashDispenser returns cash to ATM
8. ATM returns cash to User

Sequence Diagram:

text



Explanation:

- **User** is active from start to end (they need to receive cash)
- **ATM** is active throughout (orchestrates everything)

- **Transaction** is created mid-flow and destroyed after verification
- **Account** is activated only when `checkAmount` is called
- **CashDispenser** is activated only when cash needs to be dispensed

Part 5: Code Examples — Class Diagram Implementation

Here's the Car hierarchy from the class diagram, implemented in all three languages:

Java

java

```
// Abstract class – Car
abstract class Car {
    // Protected members (denoted with # in UML)
    protected String brand;
    protected String model;
    protected boolean isEngineOn;
    protected int currentSpeed;

    public Car(String brand, String model) {
        this.brand = brand;
        this.model = model;
        this.isEngineOn = false;
        this.currentSpeed = 0;
    }

    public void startEngine() {
        isEngineOn = true;
        System.out.println(brand + " " + model + " : Engine started.");
    }

    public void stopEngine() {
        isEngineOn = false;
        currentSpeed = 0;
        System.out.println(brand + " " + model + " : Engine turned off.");
    }

    // Abstract methods – must be overridden
    public abstract void accelerate();
    public abstract void brake();
}

// Child class – ManualCar
```

```
class ManualCar extends Car {
    private int currentGear;    // private member (denoted with - in UML)

    public ManualCar(String brand, String model) {
        super(brand, model);
        this.currentGear = 0;
    }

    public void shiftGear(int gear) {
        currentGear = gear;
        System.out.println("Shifted to gear " + currentGear);
    }

    @Override
    public void accelerate() {
        if (!isEngineOn) { System.out.println("Engine off."); return; }
        currentSpeed += 20;
        System.out.println("Manual accelerating to " + currentSpeed + " km/h");
    }

    @Override
    public void brake() {
        currentSpeed -= 20;
        if (currentSpeed < 0) currentSpeed = 0;
        System.out.println("Manual braking. Speed: " + currentSpeed + " km/h");
    }
}

// Child class - ElectricCar
class ElectricCar extends Car {
    private int batteryLevel;

    public ElectricCar(String brand, String model) {
        super(brand, model);
        this.batteryLevel = 100;
    }

    public void chargeBattery() {
        batteryLevel = 100;
        System.out.println("Battery fully charged.");
    }

    @Override
    public void accelerate() {
        if (batteryLevel <= 0) { System.out.println("Battery empty."); return; }
        batteryLevel -= 5;
        currentSpeed += 15;
        System.out.println("Electric accelerating to " + currentSpeed
```

```

        + " km/h. Battery: " + batteryLevel + "%");
    }

    @Override
    public void brake() {
        currentSpeed -= 15;
        if (currentSpeed < 0) currentSpeed = 0;
        System.out.println("Regenerative braking. Speed: " + currentSpeed + " km/h");
    }
}

public class Main {
    public static void main(String[] args) {
        ManualCar manual = new ManualCar("Suzuki", "WagonR");
        ElectricCar electric = new ElectricCar("Tesla", "Model S");

        manual.startEngine();
        manual.shiftGear(1);
        manual.accelerate();
        manual.brake();
        manual.stopEngine();

        System.out.println("---");

        electric.startEngine();
        electric.accelerate();
        electric.chargeBattery();
        electric.brake();
        electric.stopEngine();
    }
}

```

Python

python

```
from abc import ABC, abstractmethod
```

```

class Car(ABC):
    def __init__(self, brand, model):
        self._brand = brand                # protected (convention)
        self._model = model
        self._is_engine_on = False
        self._current_speed = 0

    def start_engine(self):
        self._is_engine_on = True

```

```

        print(f"{self._brand} {self._model} : Engine started.")

    def stop_engine(self):
        self._is_engine_on = False
        self._current_speed = 0
        print(f"{self._brand} {self._model} : Engine turned off.")

    @abstractmethod
    def accelerate(self): pass

    @abstractmethod
    def brake(self): pass


class ManualCar(Car):
    def __init__(self, brand, model):
        super().__init__(brand, model)
        self.__current_gear = 0          # private (name mangling)

    def shift_gear(self, gear):
        self.__current_gear = gear
        print(f"Shifted to gear {self.__current_gear}")

    def accelerate(self):
        if not self._is_engine_on:
            print("Engine off.")
            return
        self._current_speed += 20
        print(f"Manual accelerating to {self._current_speed} km/h")

    def brake(self):
        self._current_speed -= 20
        if self._current_speed < 0:
            self._current_speed = 0
        print(f"Manual braking. Speed: {self._current_speed} km/h")


class ElectricCar(Car):
    def __init__(self, brand, model):
        super().__init__(brand, model)
        self.__battery_level = 100

    def charge_battery(self):
        self.__battery_level = 100
        print("Battery fully charged.")

    def accelerate(self):
        if self.__battery_level <= 0:

```

```

        print("Battery empty.")
        return
    self.__battery_level -= 5
    self._current_speed += 15
    print(f"Electric accelerating to {self._current_speed} km/h. "
          f"Battery: {self.__battery_level}%")

def brake(self):
    self._current_speed -= 15
    if self._current_speed < 0:
        self._current_speed = 0
    print(f"Regenerative braking. Speed: {self._current_speed} km/h")

manual = ManualCar("Suzuki", "WagonR")
electric = ElectricCar("Tesla", "Model S")

manual.start_engine()
manual.shift_gear(1)
manual.accelerate()
manual.brake()
manual.stop_engine()

print("---")

electric.start_engine()
electric.accelerate()
electric.charge_battery()
electric.brake()
electric.stop_engine()

```

Part 6: Code Example — Associations (Simple, Aggregation, Composition)

Example: Room and Furniture (Aggregation)

```

java

// Aggregation: Furniture can exist independently of Room
class Furniture {
    String name;
    Furniture(String name) { this.name = name; }
}

```

```
class Room {  
    List<Furniture> furnitureList = new ArrayList<>();  
  
    void addFurniture(Furniture f) {  
        furnitureList.add(f);  
    }  
}  
  
// Usage  
Furniture sofa = new Furniture("Sofa");    // exists independently  
Room room = new Room();  
room.addFurniture(sofa);
```

Example: Chair Composition

```
java  
  
// Composition: Parts cannot exist without Chair  
class Seat { }  
class Arms { }  
class Wheels { }  
  
class Chair {  
    private Seat seat;  
    private Arms arms;  
    private Wheels wheels;  
  
    Chair() {  
        // Chair creates its own parts – parts die with Chair  
        this.seat = new Seat();  
        this.arms = new Arms();  
        this.wheels = new Wheels();  
    }  
}
```

Key Difference:

- **Aggregation:** Parts are passed **in** from outside (they exist independently)
- **Composition:** Parts are created **inside** the container (they die with it)

Part 7: Summary — UML Diagram Symbols

Class Diagram Symbols

Symbol	Meaning
+	public
-	private
#	protected
<<abstract>>	abstract class
Closed arrow ▲	Inheritance (IS-A)
Open arrow →	Simple association (HAS-A)
Open diamond ◇	Aggregation
Filled diamond ◆	Composition

Sequence Diagram Symbols

Symbol	Meaning
Rectangle	Object
Dashed vertical line	Lifeline
Rectangle on lifeline	Activation bar
Closed arrow →	Synchronous message
Open arrow →	Asynchronous message
Dashed arrow --►	Response
X on arrow	Lost message
X →	Found message
alt fragment	if-else
opt fragment	if

Symbol	Meaning
loop fragment	for/while

Key Takeaways

1. **UML diagrams** = visual way to communicate application design
2. **Only 2 UML diagrams matter for LLD**: Class Diagram & Sequence Diagram
3. **Class Diagram** shows classes + their associations
4. **Sequence Diagram** shows object interactions over time
5. **Inheritance** = IS-A relationship (closed arrow)
6. **Composition** (all 3 types) = HAS-A relationship (open arrow / diamond)
7. **In code**, all 3 association types look the same (object reference inside class)
8. **Composition is more common than inheritance in LLD**
9. **Simple vs Aggregation vs Composition** is often **subjective** — depends on your design
10. **Sequence diagrams** use lifelines, activation bars, and message arrows
11. **alt/opt/loop** fragments represent if-else/if/loop
12. **Create/Destroy messages** show object lifecycle

Homework (from transcript)

Task: Draw a Class Diagram for the Car scenario:

- A `Car` class with 3-4 attributes and 3-4 methods
- Two child classes: `ManualCar` and `ElectricCar`
- Decide: Inheritance or Composition?
- If Composition — which type (simple/aggregation/composition)?
- `ManualCar` has `shiftGear()` (specific to manual)
- `ElectricCar` has `chargeBattery()` (specific to electric)

Practice drawing:

1. Car class with attributes and methods

2. Association type between Car and its children
3. Specific methods in each child

What's Next (from transcript)

Next lecture: **SOLID Design Principles** — very important, new concepts ahead!



हां जी। सो आज हम स्टार्ट कर...
TXT 160.68KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also with explanation

SOLID Design Principles — Complete Notes (Lecture 5, Part 1)

Part 1: Introduction — Why SOLID?

The Problem: Real-World Projects Have Thousands of Classes

Imagine a house where all wiring (electrical, internet, pipes) runs through a single point in a tangled mess. If one wire faults, finding and replacing it is a nightmare.

In programming:

- If classes are **tightly coupled** and messy, resolving bugs becomes extremely difficult
- Adding new features becomes risky
- New engineers can't understand the code

Problems Without Design Principles

Problem	Meaning
Maintainability Issue	Adding a new feature requires changing lots of code; new bugs introduced
Readability Issue	New engineers struggle to understand the codebase

Problem	Meaning
Debugging Issue	Bugs are hard to trace; debugging takes too much time

The Solution: SOLID Principles

Robert C. Martin (Uncle Bob) published these principles in 2000.

SOLID Acronym:

Letter	Principle	Abbreviation
S	Single Responsibility Principle	SRP
O	Open/Closed Principle	OCP
L	Liskov Substitution Principle	LSP
I	Interface Segregation Principle	ISP
D	Dependency Inversion Principle	DIP

Part 2: Single Responsibility Principle (SRP)

Definition

A class should have only one reason to change.

Or: A class should do only one thing.

Real-World Analogy: TV Remote

- A TV remote controls **only the TV**
- If it also controlled the fridge and AC, it would be complicated and hard to maintain

Why SRP?

When a class handles multiple responsibilities:

- Changing database logic → modify the class
- Changing invoice logic → modify the same class
- Changing price calculation → modify the same class

Multiple reasons to change = SRP violated.

Common Misconception

SRP does **NOT** mean "one method per class."

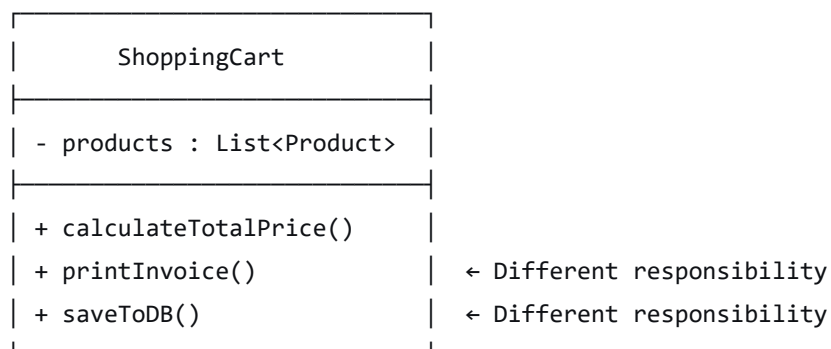
It means: **All methods in a class should serve the same single responsibility.**

A class can have many methods (including helper methods) as long as they all contribute to one job.

Example: Shopping Cart (SRP Violation)

Class Diagram (Bad):

text



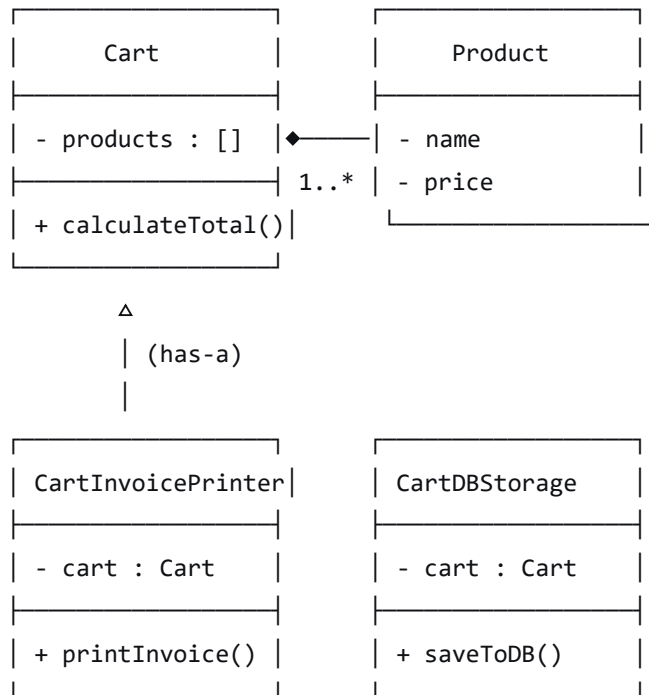
Problems: 3 responsibilities in one class:

1. Price calculation
2. Invoice printing
3. Database persistence

Refactored Design (SRP Followed)

Class Diagram (Good):

text



Each class now has **one responsibility**.

Code — SRP Violation

Java

```

java

import java.util.ArrayList;
import java.util.List;

class Product {
    String name;
    double price;

    Product(String name, double price) {
        this.name = name;
        this.price = price;
    }
}

// SRP VIOLATION - multiple responsibilities
class ShoppingCart {
    private List<Product> products = new ArrayList<>();

    public void addProduct(Product p) {
        products.add(p);
    }
}
  
```

```

public List<Product> getProducts() {
    return products;
}

// Responsibility 1: price calculation
public double calculateTotalPrice() {
    double total = 0;
    for (Product p : products) total += p.price;
    return total;
}

// Responsibility 2: invoice printing
public void printInvoice() {
    System.out.println("Invoice:");
    for (Product p : products) {
        System.out.println(p.name + " - Rs." + p.price);
    }
    System.out.println("Total: Rs." + calculateTotalPrice());
}

// Responsibility 3: DB persistence
public void saveToDB() {
    System.out.println("Saving shopping cart to database...");
}

}

public class Main {
    public static void main(String[] args) {
        ShoppingCart cart = new ShoppingCart();
        cart.addProduct(new Product("Laptop", 1500));
        cart.addProduct(new Product("Mouse", 50));

        cart.printInvoice();
        cart.saveToDB();
    }
}

```

Python

python

```

class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

```

```
# SRP VIOLATION
```

```
class ShoppingCart:
    def __init__(self):
        self.products = []

    def add_product(self, product):
        self.products.append(product)

    def calculate_total_price(self):
        return sum(p.price for p in self.products)

    def print_invoice(self):
        print("Invoice:")
        for p in self.products:
            print(f"{p.name} - Rs.{p.price}")
        print(f"Total: Rs.{self.calculate_total_price()}")

    def save_to_db(self):
        print("Saving shopping cart to database...")

cart = ShoppingCart()
cart.add_product(Product("Laptop", 1500))
cart.add_product(Product("Mouse", 50))
cart.print_invoice()
cart.save_to_db()
```

Code — SRP Followed

Java

```
java
```

```
import java.util.ArrayList;
import java.util.List;

class Product {
    String name;
    double price;
    Product(String name, double price) {
        this.name = name;
        this.price = price;
    }
}

// Single responsibility: manage cart products & calculate total
class ShoppingCart {
```

```
private List<Product> products = new ArrayList<>();

public void addProduct(Product p) { products.add(p); }
public List<Product> getProducts() { return products; }

public double calculateTotalPrice() {
    double total = 0;
    for (Product p : products) total += p.price;
    return total;
}
}

// Single responsibility: print invoice
class CartInvoicePrinter {
    private ShoppingCart cart;

    public CartInvoicePrinter(ShoppingCart cart) {
        this.cart = cart;
    }

    public void printInvoice() {
        System.out.println("Invoice:");
        for (Product p : cart.getProducts()) {
            System.out.println(p.name + " - Rs." + p.price);
        }
        System.out.println("Total: Rs." + cart.calculateTotalPrice());
    }
}

// Single responsibility: persist to DB
class CartDBStorage {
    private ShoppingCart cart;

    public CartDBStorage(ShoppingCart cart) {
        this.cart = cart;
    }

    public void saveToDB() {
        System.out.println("Saving shopping cart to database...");
    }
}

public class Main {
    public static void main(String[] args) {
        ShoppingCart cart = new ShoppingCart();
        cart.addProduct(new Product("Laptop", 1500));
        cart.addProduct(new Product("Mouse", 50));
    }
}
```

```
    CartInvoicePrinter printer = new CartInvoicePrinter(cart);
    printer.printInvoice();

    CartDBStorage storage = new CartDBStorage(cart);
    storage.saveToDB();
}
}
```

Python

python

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

class ShoppingCart:
    def __init__(self):
        self.products = []

    def add_product(self, product):
        self.products.append(product)

    def calculate_total_price(self):
        return sum(p.price for p in self.products)

class CartInvoicePrinter:
    def __init__(self, cart):
        self.cart = cart

    def print_invoice(self):
        print("Invoice:")
        for p in self.cart.products:
            print(f"{p.name} - Rs.{p.price}")
        print(f"Total: Rs.{self.cart.calculate_total_price()}")

class CartDBStorage:
    def __init__(self, cart):
        self.cart = cart

    def save_to_db(self):
        print("Saving shopping cart to database...")
```



```
cart = ShoppingCart()  
cart.add_product(Product("Laptop", 1500))  
cart.add_product(Product("Mouse", 50))  
  
CartInvoicePrinter(cart).print_invoice()  
CartDBStorage(cart).save_to_db()
```

Benefit: Now if you change DB logic, only `CartDBStorage` changes. Invoice logic → `CartInvoicePrinter` . Price logic → `ShoppingCart` .

Part 3: Open/Closed Principle (OCP)

Definition

A class should be open for extension but closed for modification.

- **Extension** = adding new features
- **Modification** = changing existing code

Goal: Add new features **without modifying existing classes**.

Real-World Analogy

A power strip with multiple sockets — you can plug in new devices (extension) without rewiring the strip (modification).

Why is This Important?

Every time you modify existing, tested code, you risk introducing bugs. If you can add new features by only **adding new code** (not modifying), your existing code stays safe.

Example: Persistence Layer

Scenario: Initially, the app saves shopping carts to SQL. Now we also want MongoDB and File persistence.

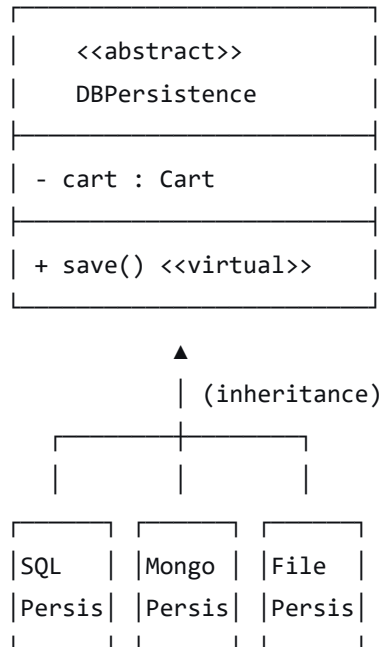
Bad Approach (OCP Violation): Add `saveToMongo()` and `saveToFile()` methods into the same `ShoppingCartStorage` class.

This **modifies** existing code every time we add a new persistence type.

Good Approach: Use abstraction + inheritance + polymorphism.

Class Diagram (OCP Followed)

text



Code — OCP Violation

Java

java

```

class ShoppingCartStorage {
    private ShoppingCart cart;

    public ShoppingCartStorage(ShoppingCart cart) {
        this.cart = cart;
    }

    // Adding new persistence → modify this class (BAD)
    public void saveToSQL() {
        System.out.println("Saving cart to SQL database...");
    }

    public void saveToMongo() {
        System.out.println("Saving cart to MongoDB...");
    }

    public void saveToFile() {

```

```

        System.out.println("Saving cart to file...");
    }
}

```

Code — OCP Followed

Java

```
java
```

```

import java.util.*;

class Product {
    String name; double price;
    Product(String name, double price) { this.name = name; this.price = price; }
}

class ShoppingCart {
    private List<Product> products = new ArrayList<>();
    public void addProduct(Product p) { products.add(p); }
    public List<Product> getProducts() { return products; }
    public double calculateTotalPrice() {
        double total = 0;
        for (Product p : products) total += p.price;
        return total;
    }
}

class CartInvoicePrinter {
    private ShoppingCart cart;
    public CartInvoicePrinter(ShoppingCart cart) { this.cart = cart; }
    public void printInvoice() {
        System.out.println("Invoice:");
        for (Product p : cart.getProducts())
            System.out.println(p.name + " - Rs." + p.price);
        System.out.println("Total: Rs." + cart.calculateTotalPrice());
    }
}

// Abstract persistence – closed for modification
abstract class DBPersistence {
    protected ShoppingCart cart;
    public DBPersistence(ShoppingCart cart) { this.cart = cart; }
    public abstract void save();
}

// Extensions – open for extension

```

```

class SQLPersistence extends DBPersistence {
    public SQLPersistence(ShoppingCart cart) { super(cart); }
    @Override
    public void save() {
        System.out.println("Saving shopping cart to SQL database...");
    }
}

class MongoPersistence extends DBPersistence {
    public MongoPersistence(ShoppingCart cart) { super(cart); }
    @Override
    public void save() {
        System.out.println("Saving shopping cart to MongoDB...");
    }
}

class FilePersistence extends DBPersistence {
    public FilePersistence(ShoppingCart cart) { super(cart); }
    @Override
    public void save() {
        System.out.println("Saving shopping cart to file...");
    }
}

public class Main {
    public static void main(String[] args) {
        ShoppingCart cart = new ShoppingCart();
        cart.addProduct(new Product("Laptop", 1500));
        cart.addProduct(new Product("Mouse", 50));

        new CartInvoicePrinter(cart).printInvoice();

        DBPersistence sql = new SQLPersistence(cart);
        DBPersistence mongo = new MongoPersistence(cart);
        DBPersistence file = new FilePersistence(cart);

        sql.save();
        mongo.save();
        file.save();
    }
}

```

Python

python

```
from abc import ABC, abstractmethod
```

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

class ShoppingCart:
    def __init__(self):
        self.products = []

    def add_product(self, product):
        self.products.append(product)

    def calculate_total_price(self):
        return sum(p.price for p in self.products)

class CartInvoicePrinter:
    def __init__(self, cart):
        self.cart = cart

    def print_invoice(self):
        print("Invoice:")
        for p in self.cart.products:
            print(f"{p.name} - Rs.{p.price}")
        print(f"Total: Rs.{self.cart.calculate_total_price()}")

class DBPersistence(ABC):
    def __init__(self, cart):
        self.cart = cart

    @abstractmethod
    def save(self): pass

class SQLPersistence(DBPersistence):
    def save(self):
        print("Saving shopping cart to SQL database...")

class MongoPersistence(DBPersistence):
    def save(self):
        print("Saving shopping cart to MongoDB...")

class FilePersistence(DBPersistence):
    def save(self):
```

```
print("Saving shopping cart to file...")

cart = ShoppingCart()
cart.add_product(Product("Laptop", 1500))
cart.add_product(Product("Mouse", 50))

CartInvoicePrinter(cart).print_invoice()

persistences = [SQLPersistence(cart), MongoPersistence(cart), FilePersistence(cart)]
for p in persistences:
    p.save()
```

Benefit: Adding Cassandra persistence → just create a new class extending DBPersistence . **No existing class modified.**

Practical Note

OCP is an **idealistic** principle. 100% adherence is impractical, but you should follow it as much as possible. A pragmatic approach is better than dogma.

Part 4: Liskov Substitution Principle (LSP)

Definition

Subclasses should be substitutable for their base classes.

If a client expects a base class, you should be able to substitute a derived class without breaking the code.

Real-World Analogy

If a function accepts a `car` , it should work correctly if you pass a `ManualCar` or `ElectricCar` — no surprises.

Fundamental Rule

Subclasses must **extend** (not **narrow**) the behavior of the base class. They must not remove or restrict inherited capabilities.

Example: Bank Accounts

Scenario: Account (abstract) → SavingsAccount , CurrentAccount , FixedDepositAccount

Problem: Fixed Deposit cannot be withdrawn from before maturity, so it can't actually implement `withdraw()` .

Naive (bad) fix: Override `withdraw()` in `FixedDepositAccount` to throw an exception.

Why it's bad: Client code iterating over `Account[]` and calling `withdraw()` on each would crash unexpectedly for FD accounts. Client didn't sign up for exceptions! **LSP violated.**

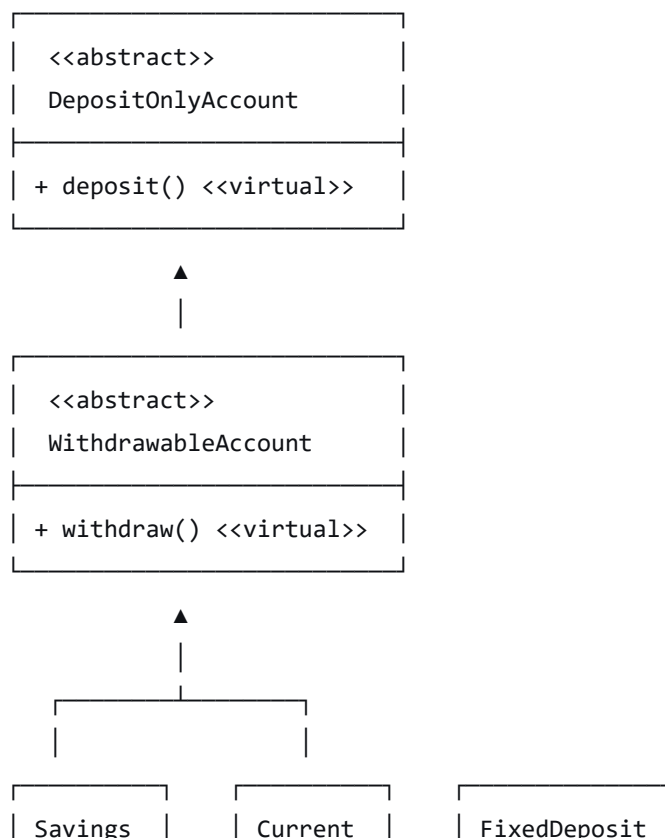
Bad Attempt 2 (Also Wrong): Client-Side Type Checking

Adding `if (account instanceof FixedDepositAccount)` in client code:

- Makes client **tightly coupled** to subtypes
- Breaks **OCP** (client changes every time a new account type comes)
- Defeats the purpose of **abstraction**

Good Solution: Split the Hierarchy

text



(extends DepositOnly)

Now:

- SavingsAccount and CurrentAccount extend WithdrawableAccount (which extends DepositOnlyAccount)
- FixedDepositAccount extends DepositOnlyAccount directly
- Client has **two separate lists**: one for withdrawable, one for deposit-only
- No instanceof checks. No exceptions. LSP respected.

Code — LSP Violation

Java

java

```
abstract class Account {
    protected double balance;
    public Account(double balance) { this.balance = balance; }
    public abstract void deposit(double amount);
    public abstract void withdraw(double amount);
}

class SavingsAccount extends Account {
    public SavingsAccount(double balance) { super(balance); }
    public void deposit(double amount) { balance += amount; System.out.println("Deposited " + amount); }
    public void withdraw(double amount) {
        if (amount > balance) { System.out.println("Insufficient funds"); return; }
        balance -= amount;
        System.out.println("Withdrawn " + amount);
    }
}

class CurrentAccount extends Account {
    public CurrentAccount(double balance) { super(balance); }
    public void deposit(double amount) { balance += amount; System.out.println("Deposited " + amount); }
    public void withdraw(double amount) {
        if (amount > balance) { System.out.println("Insufficient funds"); return; }
        balance -= amount;
        System.out.println("Withdrawn " + amount);
    }
}
```



```
// VIOLATES LSP – cannot withdraw, throws exception
class FixedDepositAccount extends Account {
    public FixedDepositAccount(double balance) { super(balance); }
    public void deposit(double amount) { balance += amount; System.out.println("Deposited
" + amount); }
    public void withdraw(double amount) {
        throw new UnsupportedOperationException("Withdrawal not allowed in FD account");
    }
}

// Client expects any Account to support both deposit and withdraw
class BankClient {
    public void processTransactions(List<Account> accounts) {
        for (Account acc : accounts) {
            acc.deposit(100);
            try {
                acc.withdraw(50); // CRASHES on FixedDepositAccount
            } catch (Exception e) {
                System.out.println("Exception: " + e.getMessage());
            }
        }
    }
}
```

Code — LSP Followed (Correct Design)

Java

```
java

// Abstract base – deposit only
abstract class DepositOnlyAccount {
    protected double balance;
    public DepositOnlyAccount(double balance) { this.balance = balance; }
    public abstract void deposit(double amount);
}

// Abstract extension – supports withdrawal
abstract class WithdrawableAccount extends DepositOnlyAccount {
    public WithdrawableAccount(double balance) { super(balance); }
    public abstract void withdraw(double amount);
}

class SavingsAccount extends WithdrawableAccount {
    public SavingsAccount(double balance) { super(balance); }
    public void deposit(double amount) {
```

```

        balance += amount;
        System.out.println("Savings: Deposited " + amount);
    }
    public void withdraw(double amount) {
        if (amount > balance) { System.out.println("Insufficient funds"); return; }
        balance -= amount;
        System.out.println("Savings: Withdrawn " + amount);
    }
}

class CurrentAccount extends WithdrawableAccount {
    public CurrentAccount(double balance) { super(balance); }
    public void deposit(double amount) {
        balance += amount;
        System.out.println("Current: Deposited " + amount);
    }
    public void withdraw(double amount) {
        if (amount > balance) { System.out.println("Insufficient funds"); return; }
        balance -= amount;
        System.out.println("Current: Withdrawn " + amount);
    }
}

class FixedDepositAccount extends DepositOnlyAccount {
    public FixedDepositAccount(double balance) { super(balance); }
    public void deposit(double amount) {
        balance += amount;
        System.out.println("FD: Deposited " + amount);
    }
    // No withdraw method – no LSP violation!
}

// Client maintains TWO separate Lists
class BankClient {
    private List<WithdrawableAccount> withdrawableAccounts = new ArrayList<>();
    private List<DepositOnlyAccount> depositOnlyAccounts = new ArrayList<>();

    public void addWithdrawable(WithdrawableAccount a) { withdrawableAccounts.add(a); }
    public void addDepositOnly(DepositOnlyAccount a) { depositOnlyAccounts.add(a); }

    public void processTransactions() {
        for (WithdrawableAccount acc : withdrawableAccounts) {
            acc.deposit(100);
            acc.withdraw(50);
        }
        for (DepositOnlyAccount acc : depositOnlyAccounts) {
            acc.deposit(100);
            // No withdraw call – safe!
        }
    }
}

```

```

    }
}

public class Main {
    public static void main(String[] args) {
        BankClient client = new BankClient();
        client.addWithdrawable(new SavingsAccount(1000));
        client.addWithdrawable(new CurrentAccount(2000));
        client.addDepositOnly(new FixedDepositAccount(5000));
        client.processTransactions();
    }
}

```

Python

python

```

from abc import ABC, abstractmethod

class DepositOnlyAccount(ABC):
    def __init__(self, balance):
        self.balance = balance

    @abstractmethod
    def deposit(self, amount): pass

class WithdrawableAccount(DepositOnlyAccount):
    @abstractmethod
    def withdraw(self, amount): pass

class SavingsAccount(WithdrawableAccount):
    def deposit(self, amount):
        self.balance += amount
        print(f"Savings: Deposited {amount}")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient funds")
            return
        self.balance -= amount
        print(f"Savings: Withdrawn {amount}")

class CurrentAccount(WithdrawableAccount):
    def deposit(self, amount):

```

```

        self.balance += amount
        print(f"Current: Deposited {amount}")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient funds")
            return
        self.balance -= amount
        print(f"Current: Withdrawn {amount}")

class FixedDepositAccount(DepositOnlyAccount):
    def deposit(self, amount):
        self.balance += amount
        print(f"FD: Deposited {amount}")
    # No withdraw - correct!

class BankClient:
    def __init__(self):
        self.withdrawable_accounts = []
        self.deposit_only_accounts = []

    def add_withdrawable(self, account):
        self.withdrawable_accounts.append(account)

    def add_deposit_only(self, account):
        self.deposit_only_accounts.append(account)

    def process_transactions(self):
        for acc in self.withdrawable_accounts:
            acc.deposit(100)
            acc.withdraw(50)
        for acc in self.deposit_only_accounts:
            acc.deposit(100)

client = BankClient()
client.add_withdrawable(SavingsAccount(1000))
client.add_withdrawable(CurrentAccount(2000))
client.add_deposit_only(FixedDepositAccount(5000))
client.process_transactions()

```

Key Insight: By restructuring the hierarchy, we avoided exceptions and type checks. Every subclass of `WithdrawableAccount` can be safely substituted where a `WithdrawableAccount` is expected.

Part 5: Summary So Far

Principle	Core Idea	Solution Mechanism
SRP	One class, one responsibility	Split classes by responsibility
OCP	Open for extension, closed for modification	Abstraction + inheritance + polymorphism
LSP	Subclass must be substitutable for base class	Restructure hierarchy; don't narrow behavior

What's Coming (from transcript)

Next lecture will cover:

- I — Interface Segregation Principle (ISP)
- D — Dependency Inversion Principle (DIP)

Common Themes Across All SOLID Principles

1. **Use abstraction** to decouple components
2. **Prefer composition** over deep inheritance trees
3. **Design for change** — make future modifications easy
4. **Keep clients decoupled** from implementations
5. **Avoid exceptions in overrides** as a workaround for bad design

Quick Memory Aid

- **SRP** = "Do one thing well"
- **OCP** = "Extend, don't modify"
- **LSP** = "Substitute safely"
- **ISP** = "Small, focused interfaces" (next)
- **DIP** = "Depend on abstractions, not concretions" (next)

Key Takeaways

1. **SOLID principles exist** to make code maintainable, readable, and bug-free
2. **SRP**: A class has one reason to change (one responsibility, not one method)
3. **OCP**: Extend behavior without modifying existing classes — use abstraction
4. **LSP**: Subclasses must extend (not narrow) base class behavior; don't throw exceptions in overrides as workaround
5. **Fixing LSP violations** often requires splitting hierarchies to avoid inappropriate methods
6. **Real-world pragmatism**: SOLID is idealistic; follow as much as practical
7. **Composition is heavily used** in LLD to satisfy SOLID principles



हां जी करें स्टार्ट। सो वेलकम बै...
TXT 195.62KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also with explanation

SOLID Design Principles — Complete Notes (Lecture 5, Part 2)

Part 1: Recap — The Liskov Substitution Principle (LSP) Guidelines

Last lecture covered **S** (SRP), **O** (OCP), and **L** (LSP). The transcript continues with LSP guidelines because it's the most commonly broken principle.

The Core Idea of LSP

A client expecting a **parent class** should work correctly when given a **child class** without breaking.

LSP Guidelines — Three Categories

text

```
LSP Guidelines
├─ Signature Rules
│   ├── Method Argument Rule
│   ├── Return Type Rule
│   └─ Exception Rule
│
├─ Property Rules
│   ├── Class Invariant
│   └─ History Constraint
│
└─ Method Rules
    ├── Pre-condition
    └─ Post-condition
```

Terms to Remember

Term	Meaning
Broad	Parent class or any ancestor
Narrow	Child class or any descendant

Example: `Animal` is **broad**; `Dog` is **narrow** (`Dog` extends `Animal`).

1.1 Signature Rule

The method signature consists of:

- Method name
- Argument types
- Return type

(a) Method Argument Rule (Contravariance)

When overriding, the child's argument must be **same or broader** than the parent's.

Parent Argument	Child Argument	Allowed?
String	String	✅ (Same)
String	Object	✅ (Broader)

Parent Argument	Child Argument	Allowed?
String	Integer	✗ (Unrelated/Worse)
String	int	✗ (Unrelated)

Why? The client calls the method with the argument type promised by the parent (contract). If the child expects a different type, the client's call may fail.

Example (Bad):

```
java

class Parent {
    void solve(String s) { System.out.println("Parent: " + s); }
}

class Child extends Parent {
    // @Override // <- This would be a compile error
    void solve(int i) { System.out.println("Child: " + i); }
}
```

Java Code — Method Argument Rule

```
java

// Parent with String argument
class Parent {
    public void print(String s) {
        System.out.println("Parent: " + s);
    }
}

// Correct: same argument type
class Child extends Parent {
    @Override
    public void print(String s) {
        System.out.println("Child: " + s);
    }
}

// Correct: broader argument type (Object is parent of String)
class ChildBroader extends Parent {
    @Override
    public void print(String s) { // Must still accept String
        System.out.println("ChildBroader: " + s);
    }
}
```



```

    }
}

// WRONG – this is overloading, not overriding
class ChildWrong extends Parent {
    // Compile error if you try @Override here
    public void print(int i) {
        System.out.println("ChildWrong: " + i);
    }
}

public class Main {
    public static void main(String[] args) {
        Parent p = new Child();
        p.print("Hello");    // Works – calls Child.print
    }
}

```

Python Code — Method Argument Rule

python

```

class Parent:
    def print_msg(self, s: str):
        print(f"Parent: {s}")

class Child(Parent):
    def print_msg(self, s: str):    # Same signature
        print(f"Child: {s}")

# Usage
p = Child()
p.print_msg("Hello")    # Child: Hello

```

Note: Python doesn't enforce signature constraints at compile time — you must respect the principle conceptually.

(b) Return Type Rule (Covariance)

When overriding, the child's return type must be **same or narrower** than the parent's.

Parent Return	Child Return	Allowed?
Animal	Animal	✓ (Same)

Parent Return	Child Return	Allowed?
Animal	Dog	✓ (Narrower)
Animal	Organism	✗ (Broader)

Why? The client expects the parent's return type. If the child returns a **narrower** type, it's still compatible. If the child returns a **broader** type, the client's variable can't hold it.

Example — Covariance in Action:

```
java

class Animal {
    void describe() { System.out.println("Animal"); }
}

class Dog extends Animal {
    @Override
    void describe() { System.out.println("Dog"); }
}

class Parent {
    public Animal getAnimal() {
        return new Animal();
    }
}

class Child extends Parent {
    @Override
    public Dog getAnimal() { // Narrower return type – ALLOWED (covariance)
        return new Dog();
    }
}

class Client {
    public void use(Parent p) {
        Animal a = p.getAnimal(); // Works whether Parent or Child
        a.describe();
    }
}
```

Java Code — Return Type Rule

```
java
```

```

class Animal { void describe() { System.out.println("Animal"); } }
class Dog extends Animal { void describe() { System.out.println("Dog"); } }

class Parent {
    public Animal getAnimal() { return new Animal(); }
}

class Child extends Parent {
    @Override
    public Dog getAnimal() { return new Dog(); } // Covariance – allowed
}

// This would be illegal:
// class BadChild extends Parent {
//     @Override
//     public Object getAnimal() { return new Object(); } // Broader – compile error
// }

public class Main {
    public static void main(String[] args) {
        Parent p = new Child();
        Animal a = p.getAnimal(); // Client expects Animal – gets Dog (still an Animal)
        a.describe();
    }
}

```

Python Code — Return Type Rule

python

```

class Animal:
    def describe(self): print("Animal")

class Dog(Animal):
    def describe(self): print("Dog")

class Parent:
    def get_animal(self) -> Animal:
        return Animal()

class Child(Parent):
    def get_animal(self) -> Dog: # Narrower – allowed
        return Dog()

p = Child()

```

```
a = p.get_animal()
a.describe()    # Dog
```

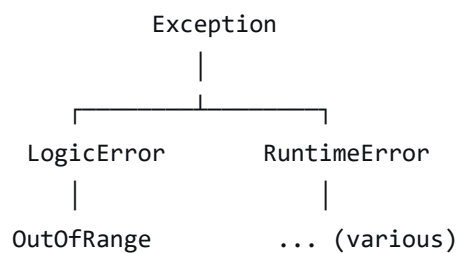
Term: Covariance — Child returns a **sub-type** of the parent's return type.

(c) Exception Rule

When overriding, the child's thrown exceptions must be **same or narrower** than the parent's. It cannot throw **broader** exceptions.

Exception Hierarchy:

text



Rule:

- Parent throws `LogicError`
- Child can throw `LogicError` ✓ (same)
- Child can throw `OutOfRangeException` ✓ (narrower — subclass of `LogicError`)
- Child CANNOT throw `RuntimeError` ✗ (unrelated)
- Child CANNOT throw `Exception` ✗ (broader)

Why? The client's `catch` block is written based on the parent's contract. If the child throws a broader exception, the client can't catch it.

Java Code — Exception Rule

java

```
class LogicError extends Exception {}
class OutOfRange extends LogicError {}
class RuntimeError extends Exception {}

class Parent {
    public void getValue() throws LogicError {
        throw new LogicError();
    }
}
```

```

    }

    // CORRECT
    class Child1 extends Parent {
        @Override
        public void getValue() throws OutOfRange {    // Narrower
            throw new OutOfRange();
        }
    }

    // WRONG – compile error
    // class Child2 extends Parent {
    //     @Override
    //     public void getValue() throws RuntimeException {    // Unrelated
    //         throw new RuntimeException();
    //     }
    // }

    class Client {
        public void use(Parent p) {
            try {
                p.getValue();
            } catch (LogicError e) {
                System.out.println("Logic error handled: " + e.getMessage());
            }
        }
    }

    public class Main {
        public static void main(String[] args) {
            Client c = new Client();
            c.use(new Parent());
            c.use(new Child1());    // Works – OutOfRange caught by LogicError
        }
    }

```

Python Code — Exception Rule

python

```

class LogicError(Exception): pass
class OutOfRange(LogicError): pass
class RuntimeError(Exception): pass

class Parent:
    def get_value(self):
        raise LogicError("parent error")

```

```

class Child1(Parent):
    def get_value(self):
        raise OutOfRange("narrower error")    # OK

# class Child2(Parent):
#     def get_value(self):
#         raise RuntimeError("unrelated")    # BAD - violates LSP

def client(p):
    try:
        p.get_value()
    except LogicError as e:
        print(f"Handled: {e}")

client(Parent())
client(Child1())

```

1.2 Property Rules

(a) Class Invariant

A rule that must **always be true** for a class. Children must not break the parent's invariants.

Example: BankAccount invariant — *"Balance can never be negative."*

java

```

class BankAccount {
    // INVARIANT: balance must always be >= 0
    protected double balance;

    public BankAccount(double initial) {
        if (initial < 0) throw new IllegalArgumentException("Balance cannot be negativ
e");
        this.balance = initial;
    }

    public void withdraw(double amount) {
        if (balance - amount < 0)
            throw new RuntimeException("Insufficient funds");
    }
}

```

```

        balance -= amount;
        System.out.println("Withdrawn: " + amount + ", Balance: " + balance);
    }
}

// BAD: breaks the invariant
class CheatAccount extends BankAccount {
    public CheatAccount(double initial) { super(initial); }

    @Override
    public void withdraw(double amount) {
        balance -= amount; // No check - balance goes negative!
        System.out.println("Cheat: Withdrawn " + amount + ", Balance: " + balance);
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount real = new BankAccount(100);
        real.withdraw(50); // OK

        BankAccount cheat = new CheatAccount(100);
        cheat.withdraw(200); // Breaks invariant - balance = -100!
    }
}

```

Python Code — Class Invariant

python

```

class BankAccount:
    def __init__(self, balance):
        if balance < 0:
            raise ValueError("Balance cannot be negative")
        self.balance = balance

    def withdraw(self, amount):
        if self.balance - amount < 0:
            raise RuntimeError("Insufficient funds")
        self.balance -= amount
        print(f"Withdrawn {amount}, Balance: {self.balance}")

class CheatAccount(BankAccount):
    def withdraw(self, amount):
        self.balance -= amount # VIOLATES INVARIANT
        print(f"Cheat: Withdrawn {amount}, Balance: {self.balance}")

```

```

cheat = CheatAccount(100)
cheat.withdraw(200)    # Balance: -100 (invariant broken)

```

(b) History Constraint

Once a parent declares a behavior/state as permanent, children cannot change that history.

Example: Parent Account says "*Withdrawal must always be allowed.*" Child FixedDepositAccount overrides withdraw() to throw an exception — **history constraint violated.**

Related concept: Immutable classes/methods — a parent can mark methods as `final` (C++) / `final` (Java) / conceptually immutable in Python to prevent overriding. Children must not override these.

```

java

class BankAccount {
    // HISTORY CONSTRAINT: withdrawal must always be allowed
    public void withdraw(double amount) {
        System.out.println("Withdrawn " + amount);
    }
}

class FixedDepositAccount extends BankAccount {
    @Override
    public void withdraw(double amount) {
        // VIOLATES HISTORY CONSTRAINT
        throw new UnsupportedOperationException("Cannot withdraw from FD");
    }
}

```

Immutable method (Java):

```

java

class Parent {
    public final void criticalMethod() {
        System.out.println("Cannot be overridden");
    }
}

```

Python — conceptual immutability (no true enforcement):

python

```
class Parent:
    def critical_method(self):
        print("Cannot be overridden")

class Child(Parent):
    # Violates LSP – child should not override critical_method
    def critical_method(self):
        print("Overridden (bad)")
```

1.3 Method Rules

(a) Pre-condition Rule

A child's pre-condition must be **same or weaker** than the parent's — never stronger.

Why? The client checks the parent's pre-condition. If the child demands **stricter** inputs, the client's valid input may fail.

Parent Pre-condition	Child Pre-condition	Allowed?
$x \geq 0$	$x \geq 0$	✓ Same
$x \geq 0$	$x \geq -5$	✓ Weaker
$x \geq 0$	$x \geq 5$	✗ Stronger

Example — Password Creation:

java

```
class User {
    // PRE-CONDITION: password.length() >= 8
    public void createPassword(String password) {
        if (password.length() < 8)
            throw new IllegalArgumentException("Password must be at least 8 chars");
        System.out.println("User password created");
    }
}

class AdminUser extends User {
    // Weaker pre-condition – ALLOWED
    @Override
```

```

public void createPassword(String password) {
    if (password.length() < 6)
        throw new IllegalArgumentException("Password must be at least 6 chars");
    System.out.println("Admin password created");
}
}

```

Python Code — Pre-condition Rule

```

python

class User:
    def create_password(self, password: str):
        if len(password) < 8:
            raise ValueError("Password must be at least 8 chars")
        print("User password created")

class AdminUser(User):
    def create_password(self, password: str):
        # Weaker pre-condition – allowed
        if len(password) < 6:
            raise ValueError("Password must be at least 6 chars")
        print("Admin password created")

# Client expects User's contract: password >= 8
def client(u: User):
    u.create_password("abcdefgh") # 8 chars – valid for both

client(User())
client(AdminUser()) # Works fine

```

(b) Post-condition Rule

A child's post-condition must be **same or stronger** than the parent's — never weaker.

Why? The client expects certain guarantees after the method runs. If the child provides **fewer** guarantees, the client's assumption may fail.

Parent Post-condition	Child Post-condition	Allowed?
speed decreases	speed decreases	✔ Same
speed decreases	speed decreases AND charge increases	✔ Stronger

Parent Post-condition	Child Post-condition	Allowed?
speed decreases	(nothing guaranteed)	✗ Weaker

Example — Car Braking:

java

```
class Car {
    protected int speed = 100;
    protected int charge = 0;

    // POST-CONDITION: after brake(), speed should decrease
    public void brake() {
        speed -= 20;
        System.out.println("Car braked, speed: " + speed);
    }
}

class HybridCar extends Car {
    // Stronger post-condition — ALLOWED
    @Override
    public void brake() {
        speed -= 20;           // Speed decreases (parent's guarantee)
        charge += 10;         // Additional stronger guarantee
        System.out.println("Hybrid braked, speed: " + speed + ", charge: " + charge);
    }
}
```

Violation: If a child's `brake()` **didn't** decrease speed, it would weaken the post-condition.

Python Code — Post-condition Rule

python

```
class Car:
    def __init__(self):
        self.speed = 100
        self.charge = 0

    def brake(self):
        # POST-CONDITION: speed decreases
        self.speed -= 20
        print(f"Car braked, speed: {self.speed}")
```

```

class HybridCar(Car):
    def brake(self):
        # Stronger post-condition
        self.speed -= 20
        self.charge += 10
        print(f"Hybrid braked, speed: {self.speed}, charge: {self.charge}")

# Client expects speed to decrease after brake()
c = HybridCar()
c.brake() # Speed decreased – post-condition satisfied

```

Part 2: Interface Segregation Principle (ISP)

Definition

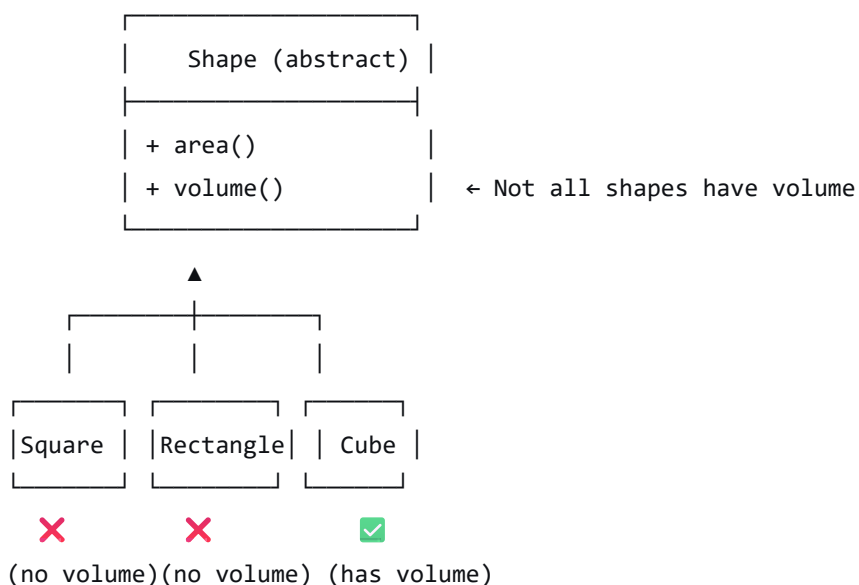
Many client-specific interfaces are better than one general-purpose interface.

Clients should not be forced to implement methods they don't need.

Real-World Example: Shapes

Violation Scenario:

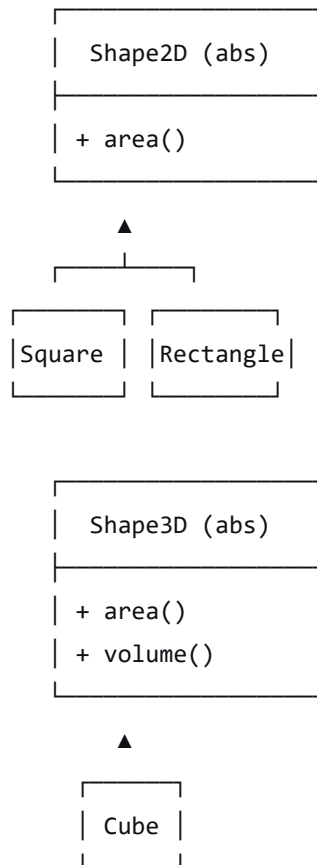
text



Problem: Square and Rectangle are **forced** to implement `volume()` which throws exceptions — a clear violation of ISP and LSP.

Corrected Design (ISP Applied)

text



Java Code — ISP Violation

java

```

abstract class Shape {
    public abstract double area();
    public abstract double volume();
}

class Square extends Shape {
    private double side;
    public Square(double side) { this.side = side; }

    @Override
    public double area() { return side * side; }

    @Override

```

```
        public double volume() {
            throw new UnsupportedOperationException("Volume not applicable for 2D shape");
        }
    }

    class Rectangle extends Shape {
        private double l, w;
        public Rectangle(double l, double w) { this.l = l; this.w = w; }

        @Override
        public double area() { return l * w; }

        @Override
        public double volume() {
            throw new UnsupportedOperationException("Volume not applicable for 2D shape");
        }
    }

    class Cube extends Shape {
        private double side;
        public Cube(double side) { this.side = side; }

        @Override
        public double area() { return 6 * side * side; }

        @Override
        public double volume() { return side * side * side; }
    }
```

Java Code — ISP Followed

java

```
// Interface for all 2D shapes
abstract class Shape2D {
    public abstract double area();
}

// Interface for all 3D shapes
abstract class Shape3D {
    public abstract double area();
    public abstract double volume();
}

class Square extends Shape2D {
    private double side;
    public Square(double side) { this.side = side; }
```

```

    @Override
    public double area() { return side * side; }
}

class Rectangle extends Shape2D {
    private double l, w;
    public Rectangle(double l, double w) { this.l = l; this.w = w; }
    @Override
    public double area() { return l * w; }
}

class Cube extends Shape3D {
    private double side;
    public Cube(double side) { this.side = side; }
    @Override
    public double area() { return 6 * side * side; }
    @Override
    public double volume() { return side * side * side; }
}

public class Main {
    public static void main(String[] args) {
        Shape2D square = new Square(5);
        Shape2D rect = new Rectangle(4, 6);
        Shape3D cube = new Cube(3);

        System.out.println("Square area: " + square.area());
        System.out.println("Rectangle area: " + rect.area());
        System.out.println("Cube area: " + cube.area());
        System.out.println("Cube volume: " + cube.volume());
    }
}

```

Python Code — ISP Followed

```
python
```

```
from abc import ABC, abstractmethod
```

```
class Shape2D(ABC):
    @abstractmethod
    def area(self): pass

```

```
class Shape3D(ABC):
    @abstractmethod

```

```
def area(self): pass

@abstractmethod
def volume(self): pass

class Square(Shape2D):
    def __init__(self, side): self.side = side
    def area(self): return self.side ** 2

class Rectangle(Shape2D):
    def __init__(self, l, w): self.l = l; self.w = w
    def area(self): return self.l * self.w

class Cube(Shape3D):
    def __init__(self, side): self.side = side
    def area(self): return 6 * self.side ** 2
    def volume(self): return self.side ** 3

# Usage
shapes_2d = [Square(5), Rectangle(4, 6)]
for s in shapes_2d:
    print(f"Area: {s.area()}")

cube = Cube(3)
print(f"Cube area: {cube.area()}, volume: {cube.volume()}")
```

Benefit: No class is forced to implement methods it doesn't need. ISP is satisfied.

Part 3: Dependency Inversion Principle (DIP)

Definition

High-level modules should not depend on low-level modules. Both should depend on abstractions.

Abstractions should not depend on details. Details should depend on abstractions.

Key Terms

Term	Meaning	Example
High-level module	Business logic	Application, UserService
Low-level module	Talks to system/external	Database, File System, API
Abstraction	Interface or abstract class	Database interface

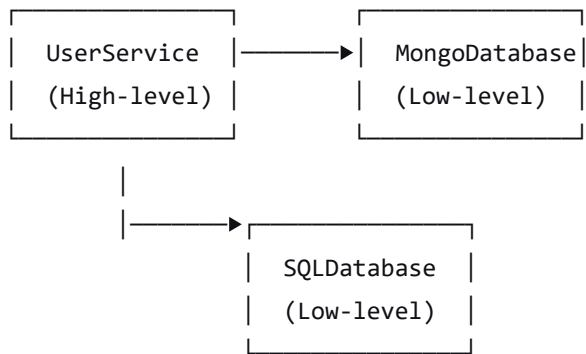
Real-World Analogy

In a company:

- **CEO (high-level)** shouldn't directly manage **developers (low-level)**
- There's a **manager (abstraction)** in between
- CEO → Manager → Developers
- If developers change, the CEO doesn't need to know

Violation Scenario

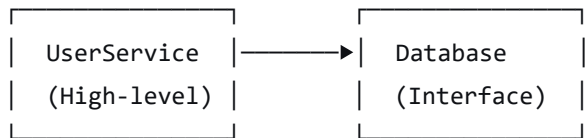
text

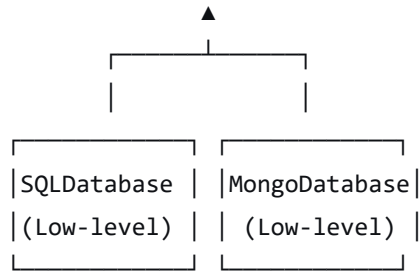


UserService **directly** depends on MongoDBDatabase and SQLiteDatabase . Changing the database requires modifying UserService — OCP is broken.

Corrected Design (DIP Applied)

text





`UserService` depends only on the `Database` **abstraction**. Both databases implement that abstraction.

Java Code — DIP Violation

java

```

class MongoDBatabase {
    public void saveToMongo(String user) {
        System.out.println("Saving " + user + " to MongoDB");
    }
}

class SQLDatabase {
    public void saveToSQL(String user) {
        System.out.println("Saving " + user + " to SQL");
    }
}

// UserService directly depends on BOTH databases (BAD)
class UserService {
    private MongoDBatabase mongo = new MongoDBatabase();
    private SQLDatabase sql = new SQLDatabase();

    public void storeUserToMongo(String user) {
        mongo.saveToMongo(user);
    }

    public void storeUserToSQL(String user) {
        sql.saveToSQL(user);
    }
}
  
```

Problems:

- Adding a new database → modify `UserService` (OCP violation)
- `UserService` tightly coupled to specific DB implementations

Java Code — DIP Followed

java

```
// Abstraction – both high-level and low-level depend on this
abstract class Database {
    public abstract void save(String user);
}

// Low-level modules implement the abstraction
class SQLiteDatabase extends Database {
    @Override
    public void save(String user) {
        System.out.println("Saving " + user + " to SQL");
    }
}

class MongoDBDatabase extends Database {
    @Override
    public void save(String user) {
        System.out.println("Saving " + user + " to MongoDB");
    }
}

class CassandraDatabase extends Database {
    @Override
    public void save(String user) {
        System.out.println("Saving " + user + " to Cassandra");
    }
}

// High-Level module depends only on abstraction
class UserService {
    private Database db;

    // Dependency Injection – Database passed in
    public UserService(Database db) {
        this.db = db;
    }

    public void storeUser(String user) {
        db.save(user);
    }
}

public class Main {
    public static void main(String[] args) {
        UserService sqlService = new UserService(new SQLiteDatabase());
    }
}
```

```

sqlService.storeUser("Alice");

UserService mongoService = new UserService(new MongoDBatabase());
mongoService.storeUser("Bob");

UserService cassandraService = new UserService(new CassandraDatabase());
cassandraService.storeUser("Carol");
}
}

```

Python Code — DIP Followed

python

```

from abc import ABC, abstractmethod

class Database(ABC):
    @abstractmethod
    def save(self, user: str): pass

class SQLiteDatabase(Database):
    def save(self, user):
        print(f"Saving {user} to SQL")

class MongoDBatabase(Database):
    def save(self, user):
        print(f"Saving {user} to MongoDB")

class CassandraDatabase(Database):
    def save(self, user):
        print(f"Saving {user} to Cassandra")

class UserService:
    def __init__(self, db: Database):
        self.db = db    # Dependency injection

    def store_user(self, user):
        self.db.save(user)

# Usage
UserService(SQLDatabase()).store_user("Alice")

```

```
UserService(MongoDatabase()).store_user("Bob")
UserService(CassandraDatabase()).store_user("Carol")
```

Benefits:

- Adding new DB → create new subclass; `UserService` untouched
- OCP is preserved
- High-level and low-level modules depend on `Database` abstraction

Key Insight

If Open/Closed Principle is the target, Dependency Inversion Principle is the solution.

Part 4: Why SOLID Despite Real-World Complexity?

Objection

"OOP maps real-world objects, but real-world objects have many responsibilities. Why SRP?"

Resolution

Real-world objects are complex, but **in a given scenario**, only some behaviors matter:

Application	User's responsibility
Ola/Uber	Book ride, cancel ride, pay
Zomato/Swiggy	Order food, cancel order, pay
LinkedIn	Post, connect, message

So in each application, the `User` class has a **single responsibility** suited to that application.

Important Caveat

SOLID principles are **principles**, not **laws**.

- 100% adherence is nearly impossible in real projects
- Sometimes business logic forces a **trade-off** (like time vs space in DSA)
- Aim to follow them as much as possible — your code becomes **cleaner, more extensible, more scalable**

Master Summary Table

Principle	Statement	Key Solution	Violation Signal
SRP	One class = one responsibility	Split classes by responsibility	Multiple reasons to change
OCP	Open for extension, closed for modification	Abstraction + inheritance + polymorphism	Modifying existing code for new features
LSP	Subclass substitutable for base class	Restructure hierarchy; don't narrow behavior	Exceptions in overrides; strict pre-conditions
ISP	Many small interfaces > one big interface	Segregate interfaces by client needs	Classes forced to implement unused methods
DIP	Depend on abstractions, not concretions	Introduce interfaces between high and low modules	High-level module imports low-level concrete class

LSP Detailed Reference (Sub-Rules)

Rule	Requirement	Example
Method Argument	Same or broader	Parent: <code>String</code> → Child: <code>String</code> or <code>Object</code>
Return Type	Same or narrower (covariance)	Parent: <code>Animal</code> → Child: <code>Dog</code>
Exception	Same or narrower	Parent: <code>LogicError</code> → Child: <code>OutOfRangeException</code>

Rule	Requirement	Example
Class Invariant	Must not break	If parent says "balance ≥ 0 ", child must too
History Constraint	Must not change past behavior	If parent allowed withdraw, child must too
Pre-condition	Same or weaker	Parent: $len \geq 8 \rightarrow$ Child: $len \geq 6$
Post-condition	Same or stronger	Parent: "speed decreases" $\rightarrow$ Child: "speed decreases AND charge increases"

Common Anti-Patterns to Avoid (LSP Violations)

1. **Throwing exceptions in overridden methods** — like `FixedDepositAccount.withdraw()` throwing `UnsupportedOperationException`
2. **Empty overrides** — overriding a method but doing nothing (silently breaking contract)
3. **Hard-coded values** — child always returns a fixed value
4. **Client-side type checking** — using `instanceof` to workaround LSP violations (breaks OCP)
5. **Stricter pre-conditions in child** — requiring more specific inputs
6. **Weaker post-conditions in child** — providing less guarantee

Key Takeaways

1. **LSP sub-rules** cover method signatures, properties, and method contracts
2. **ISP** says: don't force classes to implement methods they don't need — segregate interfaces
3. **DIP** says: high and low-level modules should both depend on abstractions, not each other
4. **DIP is the solution to OCP** — abstractions enable extension without modification

5. **Real-world objects are complex**, but each application only cares about a subset of their behaviors
6. **SOLID is idealistic** — aim for it, but be pragmatic; business logic may require trade-offs
7. **Composition and abstraction** are the most common mechanisms to satisfy SOLID



हां जी तो फिर करें स्टार्ट। सो वे...
TXT 125.08KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also with explanation

LLD Case Study: Document Editor — Complete Notes (Lecture 6)

Part 1: Problem Statement

Build a **Document Editor** (like Google Docs) with the following requirements:

Core Features:

- User can add **text** to the document
- User can insert **images** (by path)
- User can add formatting elements: **new line, tab, fonts, bold, italic, etc.**
- Document can be **rendered** (displayed as output)
- Document can be **saved to file** or **database**

Critical Requirement: The design must be **scalable** — adding new element types (video, tables, etc.) should be easy.

Part 2: Two Approaches to LLD Problems

Approach	Description	When to Use
Top-Down	Start with the biggest/topmost object, then create smaller objects inside	Good for certain problems
Bottom-Up	Start with small objects, build them up, then compose into larger objects	Most common in LLD interviews

This case study uses **Bottom-Up**.

Part 3: Bad Design (Everything in One Class)

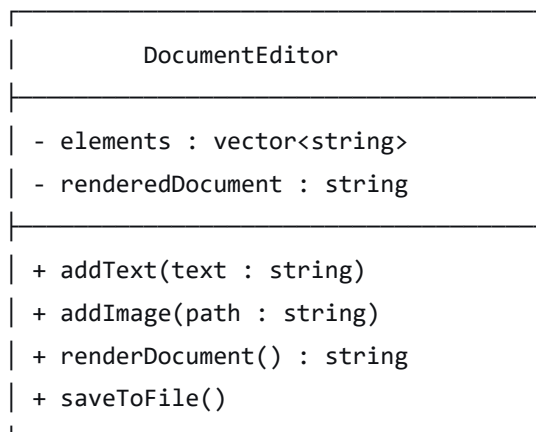
The Naive Approach

Put everything into a single `DocumentEditor` class:

- A list of elements (strings for text and image paths)
- `addText()` , `addImage()` , `renderDocument()` , `saveToFile()`

Class Diagram (Bad)

text



Why Is This Bad?

1. **Breaks SRP** — Handles adding, rendering, saving all in one class

2. **Breaks OCP** — Adding video/table/formatting requires modifying this class
3. **No abstraction** — Text and images are stored as strings (hacky detection using `.png` / `.jpg` suffix)

C++ Code (Bad Design)

cpp

```
#include <iostream>
#include <vector>
#include <fstream>
using namespace std;

class DocumentEditor {
private:
    vector<string> documentElements;
    string renderedDocument;

public:
    void addText(string text) {
        documentElements.push_back(text);
    }

    void addImage(string imagePath) {
        documentElements.push_back(imagePath);
    }

    string renderDocument() {
        if (renderedDocument.empty()) {
            string result;
            for (auto element : documentElements) {
                if (element.size() > 4 &&
                    (element.substr(element.size() - 4) == ".jpg" ||
                     element.substr(element.size() - 4) == ".png")) {
                    result += "[Image: " + element + "]\n";
                } else {
                    result += element + "\n";
                }
            }
            renderedDocument = result;
        }
        return renderedDocument;
    }

    void saveToFile() {
        ofstream file("document.txt");
        if (file.is_open()) {
```

```

        file << renderDocument();
        file.close();
        cout << "Document saved to file." << endl;
    } else {
        cout << "Unable to open file." << endl;
    }
}
};

int main() {
    DocumentEditor editor;
    editor.addText("Hello, world!");
    editor.addImage("picture.jpg");
    editor.addText("This is a document editor.");
    cout << editor.renderDocument() << endl;
    editor.saveToFile();
    return 0;
}

```

Java Code (Bad Design)

java

```

import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

class DocumentEditor {
    private List<String> documentElements = new ArrayList<>();
    private String renderedDocument = "";

    public void addText(String text) {
        documentElements.add(text);
    }

    public void addImage(String imagePath) {
        documentElements.add(imagePath);
    }

    public String renderDocument() {
        if (renderedDocument.isEmpty()) {
            StringBuilder result = new StringBuilder();
            for (String element : documentElements) {
                if (element.length() > 4 &&
                    (element.endsWith(".jpg") || element.endsWith(".png"))) {
                    result.append("[Image: ").append(element).append("]\n");
                }
            }
            renderedDocument = result.toString();
        }
    }
}

```

```

        } else {
            result.append(element).append("\n");
        }
    }
    renderedDocument = result.toString();
}
return renderedDocument;
}

public void saveToFile() {
    try (FileWriter fw = new FileWriter("document.txt")) {
        fw.write(renderDocument());
        System.out.println("Document saved to file.");
    } catch (IOException e) {
        System.out.println("Unable to open file: " + e.getMessage());
    }
}

}

public class Main {
    public static void main(String[] args) {
        DocumentEditor editor = new DocumentEditor();
        editor.addText("Hello, world!");
        editor.addImage("picture.jpg");
        editor.addText("This is a document editor.");
        System.out.println(editor.renderDocument());
        editor.saveToFile();
    }
}

```

Python Code (Bad Design)

python

```

class DocumentEditor:
    def __init__(self):
        self.document_elements = []
        self.rendered_document = ""

    def add_text(self, text):
        self.document_elements.append(text)

    def add_image(self, image_path):
        self.document_elements.append(image_path)

    def render_document(self):
        if not self.rendered_document:

```

```
        result = ""
        for element in self.document_elements:
            if element.endswith((' .jpg', ' .png')):
                result += f"[Image: {element}]\n"
            else:
                result += element + "\n"
        self.rendered_document = result
        return self.rendered_document

def save_to_file(self):
    try:
        with open("document.txt", "w") as f:
            f.write(self.render_document())
        print("Document saved to file.")
    except IOError as e:
        print(f"Unable to open file: {e}")

editor = DocumentEditor()
editor.add_text("Hello, world!")
editor.add_image("picture.jpg")
editor.add_text("This is a document editor.")
print(editor.render_document())
editor.save_to_file()
```

Problems Identified

Principle	Status
SRP	✗ Broken (multiple responsibilities)
OCP	✗ Broken (modification needed for new element types)
LSP	N/A
ISP	N/A
DIP	N/A

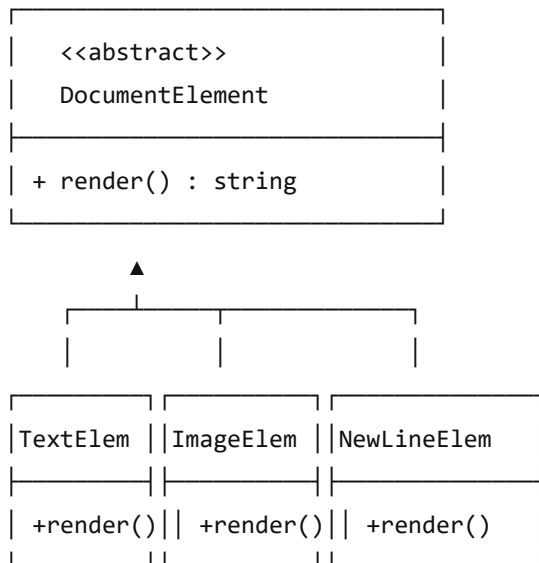
Part 4: Improved Design — Applying SOLID

Step 1: Abstract Document Elements

Create an abstract `DocumentElement` with `render()` .

Class Diagram:

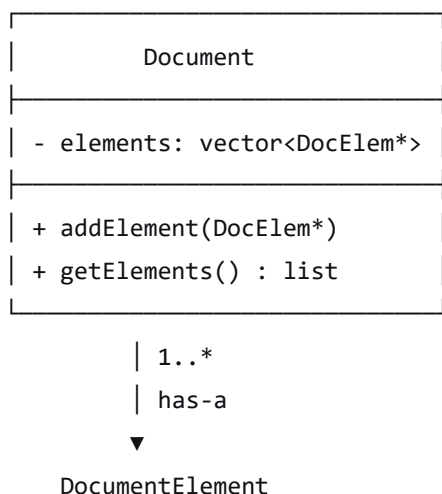
text



Step 2: Document Class

Now the `Document` class holds a list of `DocumentElement` s (not raw strings). It only handles CRUD on elements — not rendering (initially).

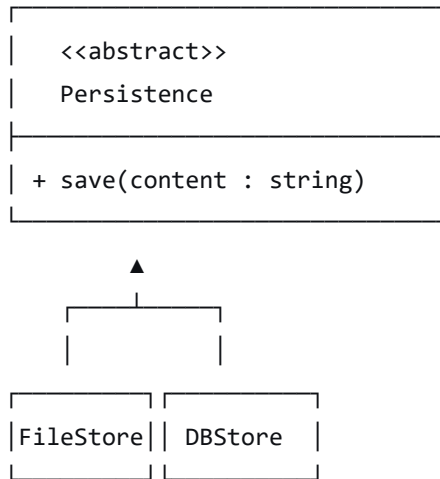
text



Step 3: Persistence Hierarchy

Create an abstract `Persistence` class with `save()` . Subclasses: `FileStorage` , `DBStorage` .

text

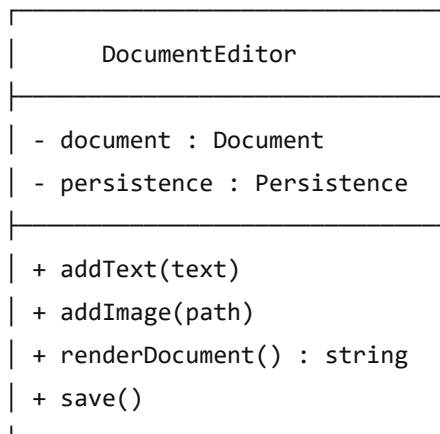


Step 4: DocumentEditor

Now the `DocumentEditor` **delegates** everything:

- `addText()` → delegates to `Document.addElement(TextElement)`
- `addImage()` → delegates to `Document.addElement(ImageElement)`
- `renderDocument()` → delegates to `Document.render()` (or a `DocumentRenderer`)
- `save()` → delegates to `Persistence.save()`

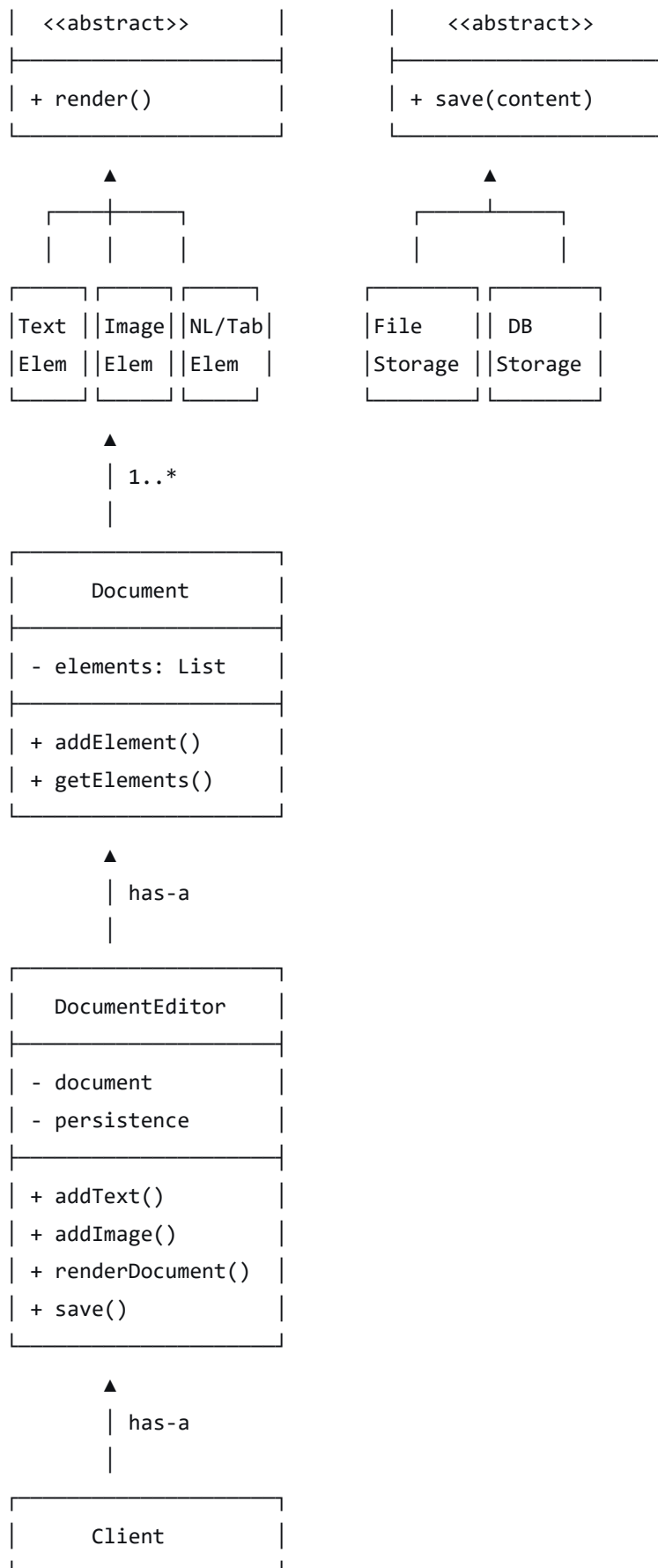
text



Complete Class Diagram

text





Part 5: Java Implementation (Improved Design)

java

```
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

// ===== ABSTRACT DOCUMENT ELEMENT =====
abstract class DocumentElement {
    public abstract String render();
}

class TextElement extends DocumentElement {
    private String text;
    public TextElement(String text) { this.text = text; }
    @Override
    public String render() { return text; }
}

class ImageElement extends DocumentElement {
    private String imagePath;
    public ImageElement(String path) { this.imagePath = path; }
    @Override
    public String render() { return "[Image: " + imagePath + "];" }
}

class NewLineElement extends DocumentElement {
    @Override
    public String render() { return "\n"; }
}

class TabSpaceElement extends DocumentElement {
    @Override
    public String render() { return "\t"; }
}

// ===== DOCUMENT (MODEL) =====
class Document {
    private List<DocumentElement> elements = new ArrayList<>();

    public void addElement(DocumentElement element) {
        elements.add(element);
    }

    public List<DocumentElement> getElements() {
        return elements;
    }
}
```

```
// ===== PERSISTENCE HIERARCHY =====
abstract class Persistence {
    public abstract void save(String content);
}

class FileStorage extends Persistence {
    @Override
    public void save(String content) {
        try (FileWriter fw = new FileWriter("document.txt")) {
            fw.write(content);
            System.out.println("Document saved to file.");
        } catch (IOException e) {
            System.out.println("Unable to open file: " + e.getMessage());
        }
    }
}

class DBStorage extends Persistence {
    @Override
    public void save(String content) {
        System.out.println("Saving document to database...");
        // Real implementation would use JDBC
    }
}

// ===== DOCUMENT RENDERER =====
class DocumentRenderer {
    private Document document;

    public DocumentRenderer(Document document) {
        this.document = document;
    }

    public String render() {
        StringBuilder result = new StringBuilder();
        for (DocumentElement element : document.getElements()) {
            result.append(element.render()).append("\n");
        }
        return result.toString();
    }
}

// ===== DOCUMENT EDITOR (delegates everything) =====
class DocumentEditor {
    private Document document;
    private Persistence persistence;
    private DocumentRenderer renderer;
```

```
private String renderedDocument = "";

public DocumentEditor(Document document, Persistence persistence) {
    this.document = document;
    this.persistence = persistence;
    this.renderer = new DocumentRenderer(document);
}

public void addText(String text) {
    document.addElement(new TextElement(text));
}

public void addImage(String imagePath) {
    document.addElement(new ImageElement(imagePath));
}

public void addNewLine() {
    document.addElement(new NewLineElement());
}

public void addTabSpace() {
    document.addElement(new TabSpaceElement());
}

public String renderDocument() {
    if (renderedDocument.isEmpty()) {
        renderedDocument = renderer.render();
    }
    return renderedDocument;
}

public void save() {
    persistence.save(renderDocument());
}
}

// ===== CLIENT =====
public class Main {
    public static void main(String[] args) {
        Document doc = new Document();
        Persistence fileStorage = new FileStorage(); // Could be DBStorage
        DocumentEditor editor = new DocumentEditor(doc, fileStorage);

        editor.addText("Hello, world!");
        editor.addNewLine();
        editor.addText("This is a real-world document editor example.");
        editor.addNewLine();
        editor.addTabSpace();
    }
}
```

```

        editor.addText("Indented text after a tab space.");
        editor.addNewLine();
        editor.addImage("picture.jpg");

        System.out.println(editor.renderDocument());
        editor.save();
    }
}

```

Part 6: Python Implementation (Improved Design)

python

```

from abc import ABC, abstractmethod

# ===== ABSTRACT DOCUMENT ELEMENT =====
class DocumentElement(ABC):
    @abstractmethod
    def render(self) -> str: pass

class TextElement(DocumentElement):
    def __init__(self, text): self.text = text
    def render(self): return self.text

class ImageElement(DocumentElement):
    def __init__(self, image_path): self.image_path = image_path
    def render(self): return f"[Image: {self.image_path}]"

class NewLineElement(DocumentElement):
    def render(self): return "\n"

class TabSpaceElement(DocumentElement):
    def render(self): return "\t"

# ===== DOCUMENT =====
class Document:
    def __init__(self):
        self._elements = []

```

```

def add_element(self, element: DocumentElement):
    self._elements.append(element)

def get_elements(self):
    return self._elements

# ===== PERSISTENCE =====
class Persistence(ABC):
    @abstractmethod
    def save(self, content: str): pass

class FileStorage(Persistence):
    def save(self, content):
        try:
            with open("document.txt", "w") as f:
                f.write(content)
            print("Document saved to file.")
        except IOError as e:
            print(f"Unable to open file: {e}")

class DBStorage(Persistence):
    def save(self, content):
        print("Saving document to database...")
        # Real implementation would connect to a DB

# ===== DOCUMENT RENDERER =====
class DocumentRenderer:
    def __init__(self, document: Document):
        self.document = document

    def render(self) -> str:
        return "\n".join(el.render() for el in self.document.get_elements())

# ===== DOCUMENT EDITOR =====
class DocumentEditor:
    def __init__(self, document: Document, persistence: Persistence):
        self.document = document
        self.persistence = persistence
        self.renderer = DocumentRenderer(document)
        self.rendered_document = ""

    def add_text(self, text):
        self.document.add_element(TextElement(text))

```

```

def add_image(self, path):
    self.document.add_element(ImageElement(path))

def add_new_line(self):
    self.document.add_element(NewLineElement())

def add_tab_space(self):
    self.document.add_element(TabSpaceElement())

def render_document(self):
    if not self.rendered_document:
        self.rendered_document = self.renderer.render()
    return self.rendered_document

def save(self):
    self.persistence.save(self.render_document())

# ===== CLIENT =====
if __name__ == "__main__":
    doc = Document()
    persistence = FileStorage() # Could be DBStorage
    editor = DocumentEditor(doc, persistence)

    editor.add_text("Hello, world!")
    editor.add_new_line()
    editor.add_text("This is a real-world document editor example.")
    editor.add_new_line()
    editor.add_tab_space()
    editor.add_text("Indented text after a tab space.")
    editor.add_new_line()
    editor.add_image("picture.jpg")

    print(editor.render_document())
    editor.save()

```

Output:

text

Hello, world!

This is a real-world document editor example.

Indented text after a tab space.

[Image: picture.jpg]

Document saved to file.

Part 7: Principle Analysis

How the Improved Design Satisfies Each SOLID Principle

Principle	How It's Satisfied
SRP	Each class has one reason to change: - <code>DocumentElement</code> → render an element - <code>Document</code> → hold elements - <code>DocumentRenderer</code> → render a document - <code>Persistence</code> → save content - <code>DocumentEditor</code> → talk to client
OCP	Adding a new element (e.g., <code>VideoElement</code>, <code>BoldElement</code>) requires only a new subclass — no modification to existing classes
LSP	Any subclass of <code>DocumentElement</code> can be substituted in <code>Document</code>'s list without breaking render
ISP	<code>DocumentElement</code> has just one method <code>render()</code> ; <code>Persistence</code> has just one method <code>save()</code> — no class is forced to implement unused methods
DIP	<code>Document</code> depends on the <code>DocumentElement</code> abstraction (not concrete text/image). <code>DocumentEditor</code> depends on <code>Persistence</code> abstraction (not on File/DB directly)

Part 8: The "More Than One Reason to Know" Issue

The Counter-Question

Even after the improved design, a counter-question may arise in an interview:

"The `DocumentEditor` still **knows** about all subclasses (`TextElement` , `ImageElement` , `Persistence`). If any of those change, `DocumentEditor` must change."

Discussion

- **SRP (Strict):** `DocumentEditor` has **one responsibility** — talk to client.
- **SRP (Loose):** It has **knowledge** of multiple dependencies.
- This is a **gray area** — SOLID principles are **principles, not laws**.
- In practice, a **small amount of coupling** in the façade/orchestrator class is acceptable.

Part 9: Principle of Least Knowledge (Law of Demeter)

What It Says

Talk only to your immediate friends.

Don't chain method calls to "friends of friends."

Violation Example

```
java

// VIOLATION: DocumentRenderer talks to Document, then talks to elements of Document
class DocumentRenderer {
    Document doc;
    public String render() {
        // Gets elements from Document (friend),
        // then calls render on each element (friend-of-friend)
        return doc.getElements().stream()
            .map(DocumentElement::render)
            .collect(Collectors.joining("\n"));
    }
}
```

Here, `DocumentRenderer` knows about `Document` **and** `DocumentElement` .

Resolution (Trade-off)

You cannot fully escape this — someone must iterate over elements to render them.

Move the `render()` method **into** `Document` :

java

```
class Document {
    private List<DocumentElement> elements = new ArrayList<>();

    public void addElement(DocumentElement el) { elements.add(el); }

    public String render() {
        StringBuilder sb = new StringBuilder();
        for (DocumentElement el : elements) sb.append(el.render()).append("\n");
        return sb.toString();
    }
}
```

But then `Document` has two responsibilities (element storage + rendering) — **SRP and LoD conflict here**. The transcript resolves this by choosing to keep the `DocumentRenderer` separate and accepting the LoD trade-off. This is a real-world engineering decision.

Part 10: Final Architecture Summary

Design Decisions

1. **Abstract** `DocumentElement` — extensible for text, image, newline, tab, video, table...
2. `Document` — holds list of `DocumentElement` s (CRUD)
3. `DocumentRenderer` — renders the document (delegation for rendering)
4. `Persistence` — abstract class for `FileStorage` and `DBStorage`
5. `DocumentEditor` — client-facing façade; delegates everything
6. **Client** — creates all objects and orchestrates the workflow

Responsibilities Table

Class	Responsibility
<code>DocumentElement</code> (abstract)	Define what a "renderable element" looks like
<code>TextElement</code> , <code>ImageElement</code> , etc.	Render their own content
<code>Document</code>	Store & provide access to elements

Class	Responsibility
DocumentRenderer	Render the entire document
Persistence (abstract)	Define save contract
FileStorage , DBStorage	Implement actual saving
DocumentEditor	Orchestrate add/render/save for the client
Client	Use the editor

Key Takeaways

1. **Bottom-Up approach** is preferred in LLD interviews
2. **Start with a bad design** — understand its flaws — then improve iteratively
3. **Identify entities** (Document, Element, Persistence) and their relationships
4. **Use abstraction** (DocumentElement , Persistence) to enable **OCP**
5. **Delegate responsibilities** — the DocumentEditor becomes a thin façade
6. **Class count grows** but each is small, focused, and easy to test
7. **SOLID principles are trade-offs, not absolutes** — always discuss with the interviewer
8. **Principle of Least Knowledge (LoD) vs SRP** can conflict — pick what matters most
9. **No design is perfect** — each change introduces new trade-offs
10. **In interviews**, the discussion around trade-offs is often more important than the final design

Comparison: Before vs After

Aspect	Bad Design	Improved Design
Classes	1	10+
SRP	✗	✓
OCP	✗ (must modify for new element)	✓ (just add subclass)

Aspect	Bad Design	Improved Design
LSP	N/A	✓
ISP	N/A	✓ (small interfaces)
DIP	N/A	✓ (depends on abstractions)
Extensibility	Hard	Easy
Testability	Hard	Easy

This response is AI-generated, for reference only.

 हां जी, करें स्टार्ट। सो, वेलकम ...
TXT 86.56KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also with explanation

Strategy Design Pattern — Complete Notes (Lecture 7)

Part 1: Introduction to Design Patterns

What Are Design Patterns?

Design patterns are **reusable solutions to common problems** in software design. When developers face the same problem repeatedly, they identify a **pattern** for solving it — and document it as a best practice.

Key Insight: *"Change is the only constant."* Applications evolve, new features arrive, and your design must accommodate them with **minimal code changes**.

Why Design Patterns?

Goal	Meaning
Flexibility	New features integrate with minimal code changes
Maintainability	Code is easier to debug and extend

Goal	Meaning
Reusability	Solutions can be applied across projects
Separation of Concerns	Changing parts are separated from stable parts

The Core Idea Behind All Design Patterns

Every application has:

- **Stable parts** — code that doesn't change often
- **Dynamic parts** — code that changes frequently

All design patterns say: *Extract the changing part from the stable part, and put it in a separate class or hierarchy.*

Officially, there are 23 design patterns. All of them apply this idea differently.

Part 2: The Problem — Robot Simulation

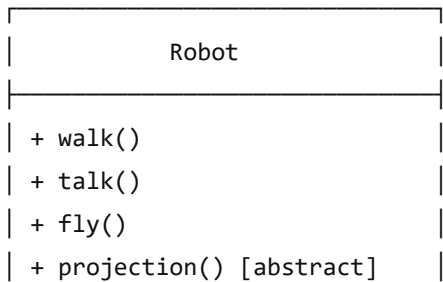
Problem Statement

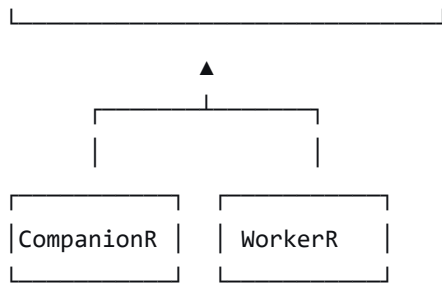
Design an application that simulates **robots**. Different robots:

- Walk differently (or can't walk)
- Talk differently (or can't talk)
- Fly differently (or can't fly)
- Look different (projection)

Initial Design — Inheritance-Based

text

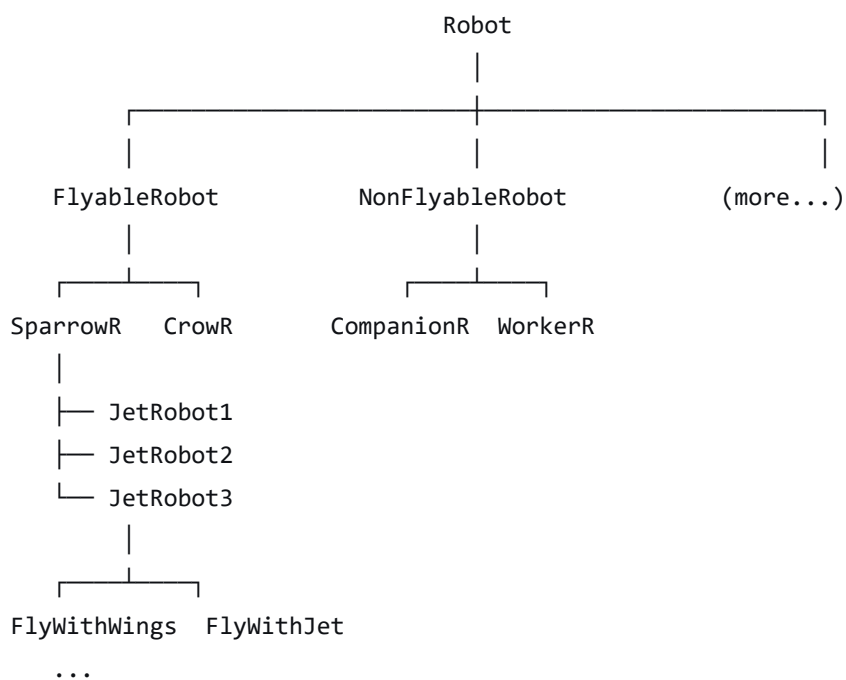




Problems Emerge

As new robots are added:

text



Problems Identified

1. **DRY Principle Violated** — `fly()` code duplicated across every flying robot
2. **Hierarchy Explosion** — Every combination (walk/talk/fly × wing/jet × ...) requires new classes
3. **OCF Violated** — Adding a new behavior modifies existing classes
4. **Reusability Broken** — Behavior code can't be easily reused

The Famous Line

"The solution to inheritance is not more inheritance."

Part 3: Strategy Design Pattern

Definition

The Strategy Pattern defines a family of algorithms, encapsulates each one in separate classes, and makes them interchangeable at runtime.

Core Idea

- **Extract** changing behaviors into **separate interface hierarchies**
- **Compose** these behaviors into the client class (instead of inheriting them)
- **Delegate** calls to the behavior objects

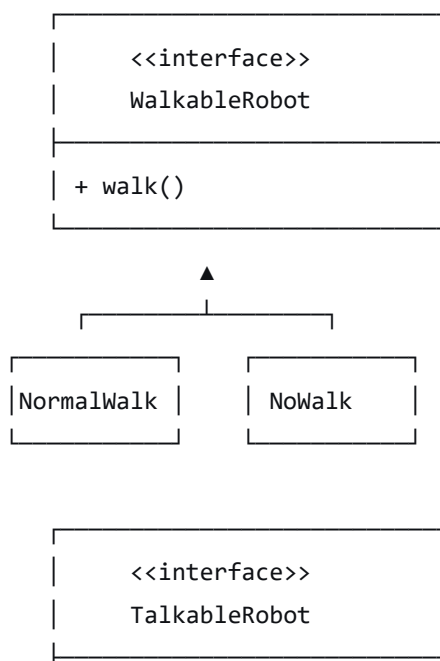
The Famous Conclusion

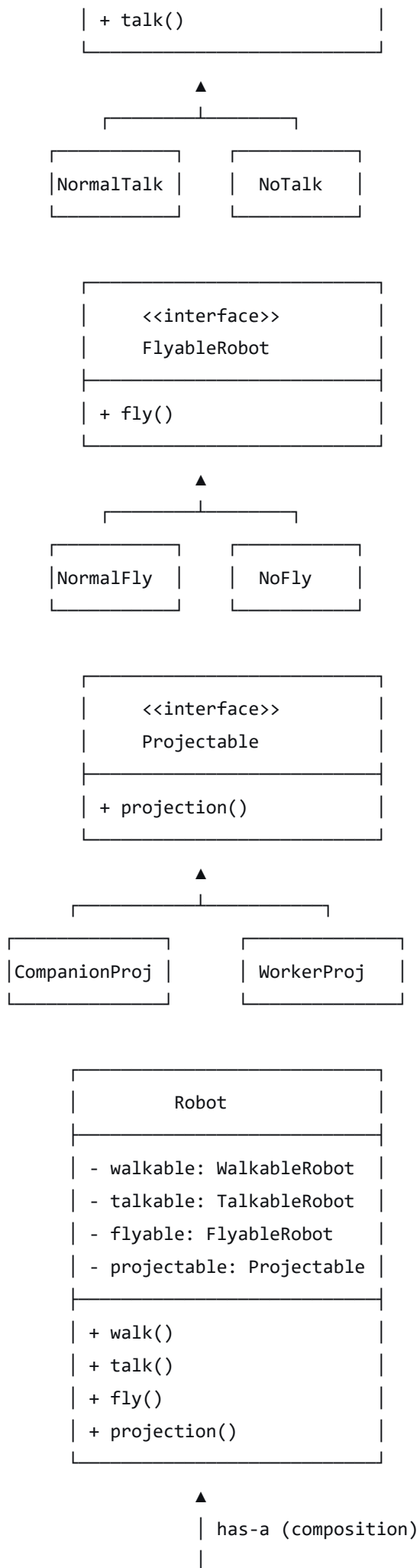
Favor composition over inheritance.

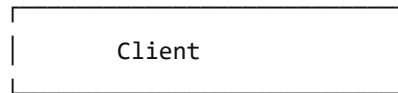
Part 4: Strategy Pattern — Solution for Robots

Class Diagram

text







Client = Robot (which uses strategies). Strategies = Walkable, Talkable, Flyable, Projectable.

Part 5: Java Implementation

Strategy Interfaces

java

```
// Strategy 1: Walkable
interface WalkableRobot {
    void walk();
}

class NormalWalk implements WalkableRobot {
    @Override
    public void walk() {
        System.out.println("Walking normally...");
    }
}

class NoWalk implements WalkableRobot {
    @Override
    public void walk() {
        System.out.println("Cannot walk.");
    }
}

// Strategy 2: Talkable
interface TalkableRobot {
    void talk();
}

class NormalTalk implements TalkableRobot {
    @Override
    public void talk() {
        System.out.println("Talking normally...");
    }
}
```



```
class NoTalk implements TalkableRobot {
    @Override
    public void talk() {
        System.out.println("Cannot talk.");
    }
}

// Strategy 3: Flyable
interface FlyableRobot {
    void fly();
}

class NormalFly implements FlyableRobot {
    @Override
    public void fly() {
        System.out.println("Flying normally...");
    }
}

class NoFly implements FlyableRobot {
    @Override
    public void fly() {
        System.out.println("Cannot fly.");
    }
}

// Strategy 4: Projectable
interface Projectable {
    void projection();
}

class CompanionProjection implements Projectable {
    @Override
    public void projection() {
        System.out.println("Displaying companion robot projection.");
    }
}

class WorkerProjection implements Projectable {
    @Override
    public void projection() {
        System.out.println("Displaying worker robot projection.");
    }
}
```

Robot Client Class

```
java
```

```
class Robot {
    private WalkableRobot walkable;
    private TalkableRobot talkable;
    private FlyableRobot flyable;
    private Projectable projectable;

    public Robot(WalkableRobot walkable, TalkableRobot talkable,
                FlyableRobot flyable, Projectable projectable) {
        this.walkable = walkable;
        this.talkable = talkable;
        this.flyable = flyable;
        this.projectable = projectable;
    }

    // Delegation methods – Robot itself does nothing
    public void walk() { walkable.walk(); }
    public void talk() { talkable.talk(); }
    public void fly() { flyable.fly(); }
    public void projection() { projectable.projection(); }
}
```

Client Usage

```
java
```

```
public class Main {
    public static void main(String[] args) {
        // Companion robot: walks, talks, doesn't fly
        Robot companionRobot = new Robot(
            new NormalWalk(),
            new NormalTalk(),
            new NoFly(),
            new CompanionProjection()
        );

        // Worker robot: doesn't walk, doesn't talk, but flies
        Robot workerRobot = new Robot(
            new NoWalk(),
            new NoTalk(),
            new NormalFly(),
            new WorkerProjection()
        );

        System.out.println("=== Companion Robot ===");
        companionRobot.walk();
    }
}
```

```

        companionRobot.talk();
        companionRobot.fly();
        companionRobot.projection();

        System.out.println("\n=== Worker Robot ===");
        workerRobot.walk();
        workerRobot.talk();
        workerRobot.fly();
        workerRobot.projection();
    }
}

```

Output:

text

```

=== Companion Robot ===
Walking normally...
Talking normally...
Cannot fly.
Displaying companion robot projection.

=== Worker Robot ===
Cannot walk.
Cannot talk.
Flying normally...
Displaying worker robot projection.

```

Part 6: Python Implementation

python

```

from abc import ABC, abstractmethod

# ===== STRATEGIES =====
class WalkableRobot(ABC):
    @abstractmethod
    def walk(self): pass

class NormalWalk(WalkableRobot):
    def walk(self): print("Walking normally...")

class NoWalk(WalkableRobot):
    def walk(self): print("Cannot walk.")

```

```
class TalkableRobot(ABC):
    @abstractmethod
    def talk(self): pass

class NormalTalk(TalkableRobot):
    def talk(self): print("Talking normally...")

class NoTalk(TalkableRobot):
    def talk(self): print("Cannot talk.")

class FlyableRobot(ABC):
    @abstractmethod
    def fly(self): pass

class NormalFly(FlyableRobot):
    def fly(self): print("Flying normally...")

class NoFly(FlyableRobot):
    def fly(self): print("Cannot fly.")

class Projectable(ABC):
    @abstractmethod
    def projection(self): pass

class CompanionProjection(Projectable):
    def projection(self): print("Displaying companion robot projection.")

class WorkerProjection(Projectable):
    def projection(self): print("Displaying worker robot projection.")

# ===== CLIENT (Robot) =====
class Robot:
    def __init__(self, walkable: WalkableRobot, talkable: TalkableRobot,
                  flyable: FlyableRobot, projectable: Projectable):
        self._walkable = walkable
        self._talkable = talkable
        self._flyable = flyable
        self._projectable = projectable

    def walk(self):
        self._walkable.walk()

    def talk(self):
        self._talkable.talk()
```

```

def fly(self):
    self._flyable.fly()

def projection(self):
    self._projectable.projection()

# ===== USAGE =====
if __name__ == "__main__":
    companion = Robot(NormalWalk(), NormalTalk(), NoFly(), CompanionProjection())
    worker = Robot(NoWalk(), NoTalk(), NormalFly(), WorkerProjection())

    print("=== Companion Robot ===")
    companion.walk()
    companion.talk()
    companion.fly()
    companion.projection()

    print("\n=== Worker Robot ===")
    worker.walk()
    worker.talk()
    worker.fly()
    worker.projection()

```

Output:

text

```

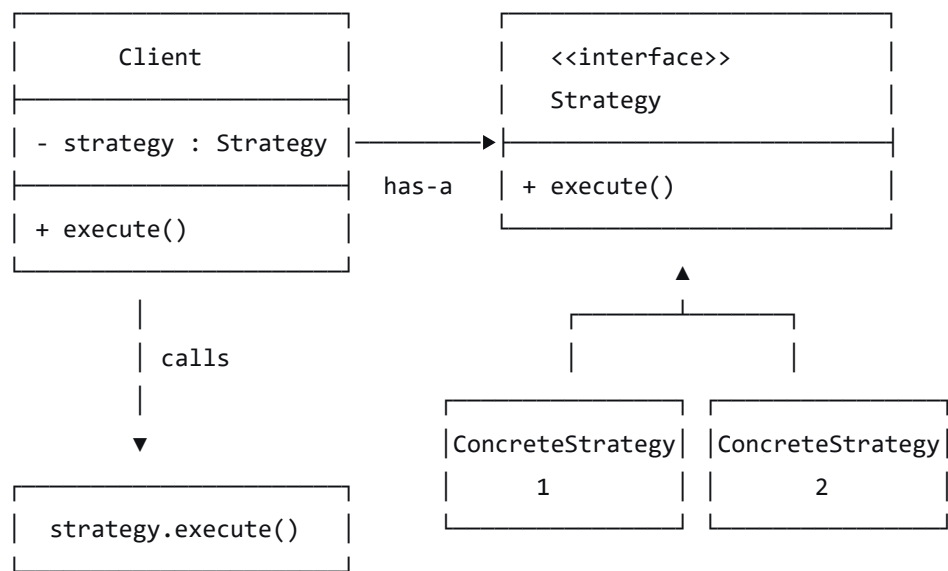
=== Companion Robot ===
Walking normally...
Talking normally...
Cannot fly.
Displaying companion robot projection.

=== Worker Robot ===
Cannot walk.
Cannot talk.
Flying normally...
Displaying worker robot projection.

```

Part 7: Standard UML Diagram of Strategy Pattern

text



Key relationships:

- Client **HAS-A** Strategy (composition)
- ConcreteStrategies **implement** the Strategy interface
- Client **delegates** to Strategy via `strategy.execute()`

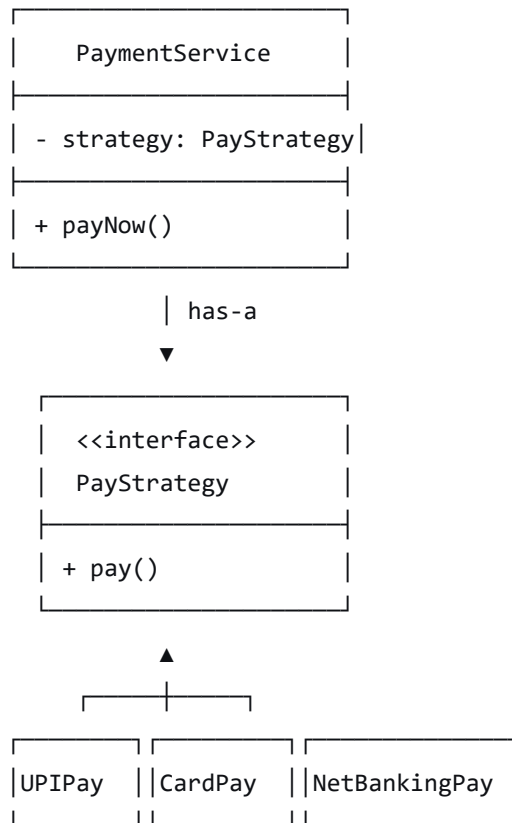
Strategy Pattern Terms

Term	Meaning	Example
Client	Uses strategies	Robot class
Strategy	Interface for algorithms	Walkable , Talkable , Flyable , Projectable
Concrete Strategy	Actual algorithm implementations	NormalWalk , NoWalk , NormalFly , etc.

Part 8: Real-World Examples

Example 1: Payment System

text



Java Code:

java

```

interface PayStrategy {
    void pay(int amount);
}

class UIPay implements PayStrategy {
    public void pay(int amount) {
        System.out.println("Paid ₹" + amount + " via UPI");
    }
}

class CardPay implements PayStrategy {
    public void pay(int amount) {
        System.out.println("Paid ₹" + amount + " via Credit/Debit Card");
    }
}

class NetBankingPay implements PayStrategy {
    public void pay(int amount) {
        System.out.println("Paid ₹" + amount + " via Net Banking");
    }
}
  
```

```

class PaymentService {
    private PayStrategy strategy;

    public PaymentService(PayStrategy strategy) {
        this.strategy = strategy;
    }

    public void payNow(int amount) {
        strategy.pay(amount);
    }
}

public class Main {
    public static void main(String[] args) {
        PaymentService payment = new PaymentService(new UPIPay());
        payment.payNow(500);

        payment = new PaymentService(new CardPay());
        payment.payNow(1000);
    }
}

```

Python Code:

python

```

from abc import ABC, abstractmethod

class PayStrategy(ABC):
    @abstractmethod
    def pay(self, amount): pass

class UPIPay(PayStrategy):
    def pay(self, amount): print(f"Paid ₹{amount} via UPI")

class CardPay(PayStrategy):
    def pay(self, amount): print(f"Paid ₹{amount} via Card")

class NetBankingPay(PayStrategy):
    def pay(self, amount): print(f"Paid ₹{amount} via Net Banking")

class PaymentService:
    def __init__(self, strategy: PayStrategy):
        self.strategy = strategy

    def pay_now(self, amount):
        self.strategy.pay(amount)

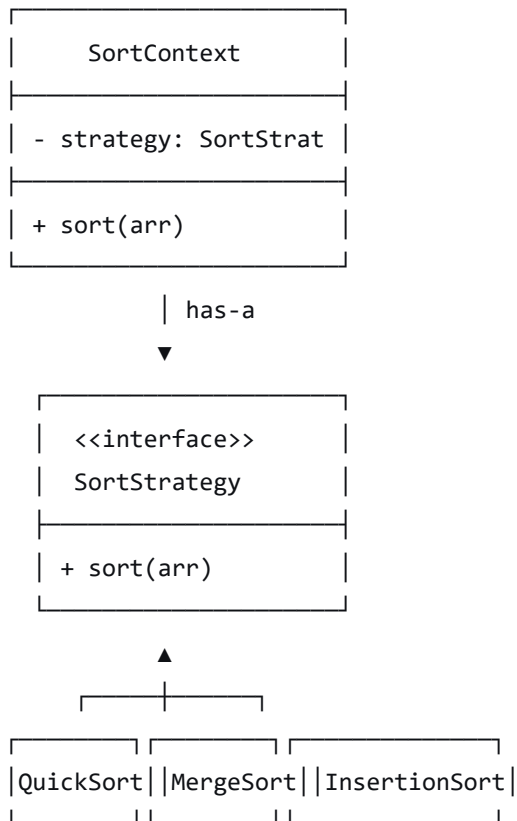
```



```
PaymentService(UPIPay()).pay_now(500)
PaymentService(CardPay()).pay_now(1000)
```

Example 2: Sorting Algorithms

text



Example 3: Text Editor Formatting

- Bold, Italic, Underline — each is a strategy

Example 4: Compression

- ZIP, RAR, 7z — different compression algorithms

Example 5: Authentication

- OAuth, JWT, Basic Auth — interchangeable strategies

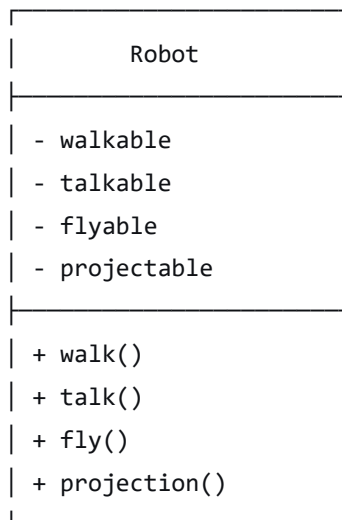
Part 9: Can We Remove ALL Inheritance?

The transcript ends with a great question: *"Is there still inheritance in the design?"*

Answer: Yes — the `Robot` class hierarchy for `Projection` still uses inheritance.

Fix: Make `Projectable` a strategy too!

text



Now `Robot` is a **single concrete class** with no child classes. Every behavior (walk/talk/fly/projection) is composed as a strategy.

Final Client Code:

java

```
Robot robot = new Robot(
    new NormalWalk(),
    new NormalTalk(),
    new NormalFly(),
    new CompanionProjection() // strategy, not subclass
);
robot.walk();
robot.talk();
robot.fly();
robot.projection();
```

Result: 0% inheritance — 100% composition.

Part 10: Key Concepts Summary

DRY Principle

Don't Repeat Yourself — Every piece of knowledge should have a single, unambiguous representation.

Strategy pattern eliminates duplicate `fly()` , `walk()` , etc. code.

Open/Closed Principle (OCP)

Classes should be open for extension but closed for modification.

New behaviors = new strategy classes. **Robot class never changes.**

Favor Composition Over Inheritance

- **Inheritance** = "IS-A" (fixed at compile time)
- **Composition** = "HAS-A" (flexible at runtime)

Runtime Flexibility

Strategies can be swapped at runtime:

```
java
```

```
Robot r = new Robot(new NormalWalk(), new NormalTalk(), new NoFly(), new CompanionProject  
ion());  
// Later...  
r = new Robot(new NoWalk(), new NoTalk(), new NormalFly(), new WorkerProjection());
```

Part 11: When to Use Strategy Pattern

✅ Use When:

- Multiple algorithms exist for the same task
- Algorithm needs to be chosen at **runtime**
- You want to avoid a large conditional (`if-else` / `switch`) chain
- You want to isolate algorithm implementation from the client

- You want to remove duplicated behavior code

✗ Avoid When:

- Only one algorithm exists (over-engineering)
- Algorithms never change
- Behavior is fundamental to the class's identity

Part 12: Client Delegation ("Dumb Object")

The `Robot` class in the improved design becomes a **"dumb object"**:

- It doesn't implement behavior
- It **delegates** all work to strategy objects
- It only knows **what to call**, not **how it works**

java

```
public void walk() { walkable.walk(); } // DeLegate!  
public void talk() { talkable.talk(); } // DeLegate!  
public void fly() { flyable.fly(); } // DeLegate!
```

This is a common theme in **many** design patterns.

Key Takeaways

1. **Design patterns** are reusable solutions to common problems
2. **All patterns** separate changing parts from stable parts
3. **Strategy Pattern** = "family of algorithms in separate classes, interchangeable at runtime"
4. **Favor composition over inheritance** — this is the heart of Strategy Pattern
5. **Client becomes a "dumb object"** that delegates to strategies
6. **Eliminates DRY violations** — no duplicated behavior code
7. **Satisfies OCP** — adding new behavior = new strategy class only
8. **Runtime swapping** of strategies is possible

- 9. **Real-world uses:** payment methods, sorting algorithms, compression, authentication, text formatting
- 10. **Strategy is one of the most-used design patterns** — likely appears in any LLD interview problem

Comparison: Before vs After

Aspect	Inheritance-Based	Strategy Pattern
New behavior	Add new subclass (hierarchy explodes)	Add new strategy class
Duplicated code	Yes (DRY violated)	No
Runtime swap	Impossible	Possible
Complexity	Combinatorial explosion	Linear growth
OCP	Violated	Satisfied
Robot class changes	Yes (when behaviors change)	Never

 हां जी, करें स्टार्ट। सो, वेलकम ...
TXT 86.56KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also with explanation

Strategy Design Pattern — Complete Notes

Part 1: Introduction to Design Patterns

What Are Design Patterns?

Design patterns are **reusable solutions to common software design problems**. When developers face the same problem repeatedly across different projects, they identify a **pattern** for solving it and document it as a best practice.

Key Insight: *"Change is the only constant."* Applications evolve, new features arrive, and your design must accommodate them with **minimal code changes**.

Why Design Patterns?

Goal	Meaning
Flexibility	New features integrate with minimal code changes
Maintainability	Code is easier to debug and extend
Reusability	Solutions can be applied across projects
Separation of Concerns	Changing parts separated from stable parts

The Core Idea Behind All Design Patterns

Every application has:

- **Stable parts** — code that doesn't change often
- **Dynamic parts** — code that changes frequently

All design patterns say: *Extract the changing part from the stable part, and put it in a separate class or hierarchy.*

Officially, there are 23 design patterns. All of them apply this idea differently.

Part 2: The Problem — Robot Simulation

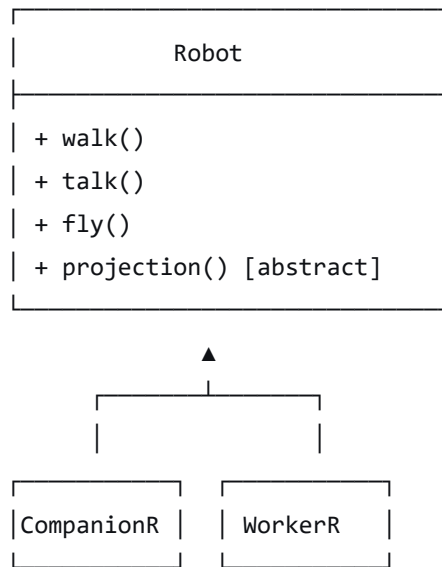
Problem Statement

Design an application that simulates **robots**. Different robots:

- Walk differently (or can't walk)
- Talk differently (or can't talk)
- Fly differently (or can't fly)
- Look different (projection)

Initial Design — Inheritance-Based

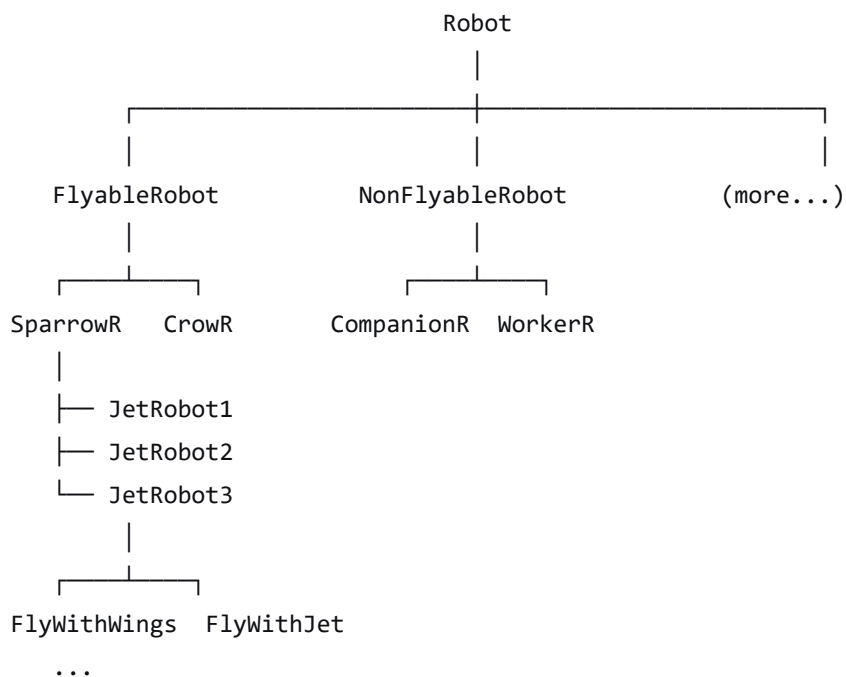
text



Problems Emerge

As new robots are added:

text



Problems Identified

1. **DRY Principle Violated** — `fly()` code duplicated across every flying robot

2. **Hierarchy Explosion** — Every combination (walk/talk/fly × wing/jet × ...) requires new classes
3. **OCF Violated** — Adding a new behavior modifies existing classes
4. **Reusability Broken** — Behavior code can't be easily reused

The Famous Line

"The solution to inheritance is not more inheritance."

Part 3: Strategy Design Pattern

Definition

The Strategy Pattern defines a family of algorithms, encapsulates each one in separate classes, and makes them interchangeable at runtime.

Core Idea

- **Extract** changing behaviors into **separate interface hierarchies**
- **Compose** these behaviors into the client class (instead of inheriting them)
- **Delegate** calls to the behavior objects

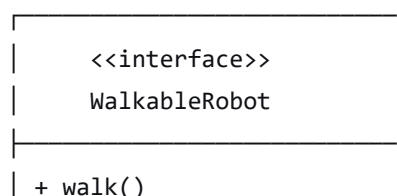
The Famous Conclusion

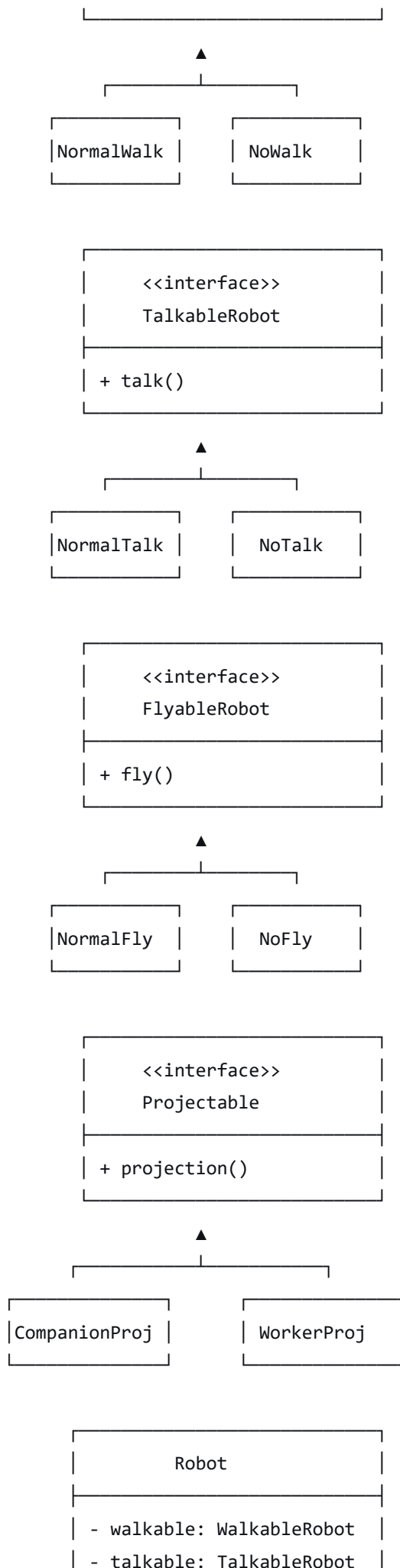
Favor composition over inheritance.

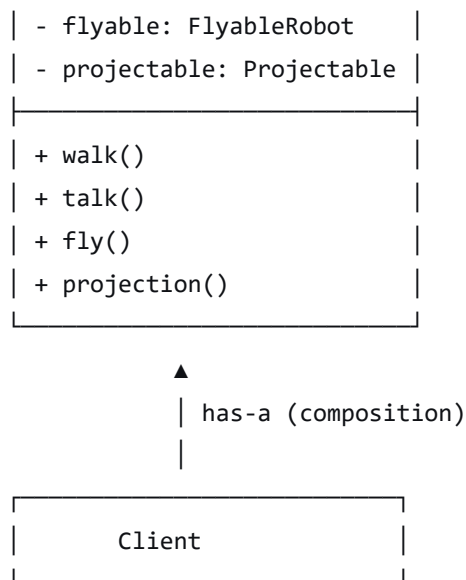
Part 4: Strategy Pattern — Solution for Robots

Class Diagram

text







Client = Robot (which uses strategies). Strategies = Walkable, Talkable, Flyable, Projectable.

Part 5: Java Implementation

Strategy Interfaces

java

```

// Strategy 1: Walkable
interface WalkableRobot {
    void walk();
}

class NormalWalk implements WalkableRobot {
    @Override
    public void walk() {
        System.out.println("Walking normally...");
    }
}

class NoWalk implements WalkableRobot {
    @Override
    public void walk() {
        System.out.println("Cannot walk.");
    }
}
  
```

// Strategy 2: Talkable

```
interface TalkableRobot {
    void talk();
}

class NormalTalk implements TalkableRobot {
    @Override
    public void talk() {
        System.out.println("Talking normally...");
    }
}

class NoTalk implements TalkableRobot {
    @Override
    public void talk() {
        System.out.println("Cannot talk.");
    }
}

// Strategy 3: Flyable
interface FlyableRobot {
    void fly();
}

class NormalFly implements FlyableRobot {
    @Override
    public void fly() {
        System.out.println("Flying normally...");
    }
}

class NoFly implements FlyableRobot {
    @Override
    public void fly() {
        System.out.println("Cannot fly.");
    }
}

// Strategy 4: Projectable
interface Projectable {
    void projection();
}

class CompanionProjection implements Projectable {
    @Override
    public void projection() {
        System.out.println("Displaying companion robot projection.");
    }
}
```

```

class WorkerProjection implements Projectable {
    @Override
    public void projection() {
        System.out.println("Displaying worker robot projection.");
    }
}

```

Robot Client Class

java

```

class Robot {
    private WalkableRobot walkable;
    private TalkableRobot talkable;
    private FlyableRobot flyable;
    private Projectable projectable;

    public Robot(WalkableRobot walkable, TalkableRobot talkable,
                FlyableRobot flyable, Projectable projectable) {
        this.walkable = walkable;
        this.talkable = talkable;
        this.flyable = flyable;
        this.projectable = projectable;
    }

    // Delegation methods – Robot itself does nothing
    public void walk() { walkable.walk(); }
    public void talk() { talkable.talk(); }
    public void fly() { flyable.fly(); }
    public void projection() { projectable.projection(); }
}

```

Client Usage

java

```

public class Main {
    public static void main(String[] args) {
        // Companion robot: walks, talks, doesn't fly
        Robot companionRobot = new Robot(
            new NormalWalk(),
            new NormalTalk(),
            new NoFly(),
            new CompanionProjection()
        );
    }
}

```

```

    );

    // Worker robot: doesn't walk, doesn't talk, but flies
    Robot workerRobot = new Robot(
        new NoWalk(),
        new NoTalk(),
        new NormalFly(),
        new WorkerProjection()
    );

    System.out.println("=== Companion Robot ===");
    companionRobot.walk();
    companionRobot.talk();
    companionRobot.fly();
    companionRobot.projection();

    System.out.println("\n=== Worker Robot ===");
    workerRobot.walk();
    workerRobot.talk();
    workerRobot.fly();
    workerRobot.projection();
}
}

```

Output:

text

```

=== Companion Robot ===
Walking normally...
Talking normally...
Cannot fly.
Displaying companion robot projection.

=== Worker Robot ===
Cannot walk.
Cannot talk.
Flying normally...
Displaying worker robot projection.

```

Part 6: Python Implementation

python

```
from abc import ABC, abstractmethod

# ===== STRATEGIES =====

class WalkableRobot(ABC):
    @abstractmethod
    def walk(self): pass

class NormalWalk(WalkableRobot):
    def walk(self): print("Walking normally...")

class NoWalk(WalkableRobot):
    def walk(self): print("Cannot walk.")

class TalkableRobot(ABC):
    @abstractmethod
    def talk(self): pass

class NormalTalk(TalkableRobot):
    def talk(self): print("Talking normally...")

class NoTalk(TalkableRobot):
    def talk(self): print("Cannot talk.")

class FlyableRobot(ABC):
    @abstractmethod
    def fly(self): pass

class NormalFly(FlyableRobot):
    def fly(self): print("Flying normally...")

class NoFly(FlyableRobot):
    def fly(self): print("Cannot fly.")

class Projectable(ABC):
    @abstractmethod
    def projection(self): pass

class CompanionProjection(Projectable):
    def projection(self): print("Displaying companion robot projection.")

class WorkerProjection(Projectable):
    def projection(self): print("Displaying worker robot projection.")

# ===== CLIENT (Robot) =====
```

```

class Robot:
    def __init__(self, walkable: WalkableRobot, talkable: TalkableRobot,
                 flyable: FlyableRobot, projectable: Projectable):
        self._walkable = walkable
        self._talkable = talkable
        self._flyable = flyable
        self._projectable = projectable

    def walk(self):
        self._walkable.walk()

    def talk(self):
        self._talkable.talk()

    def fly(self):
        self._flyable.fly()

    def projection(self):
        self._projectable.projection()

# ===== USAGE =====
if __name__ == "__main__":
    companion = Robot(NormalWalk(), NormalTalk(), NoFly(), CompanionProjection())
    worker = Robot(NoWalk(), NoTalk(), NormalFly(), WorkerProjection())

    print("=== Companion Robot ===")
    companion.walk()
    companion.talk()
    companion.fly()
    companion.projection()

    print("\n=== Worker Robot ===")
    worker.walk()
    worker.talk()
    worker.fly()
    worker.projection()

```

Output:

text

```

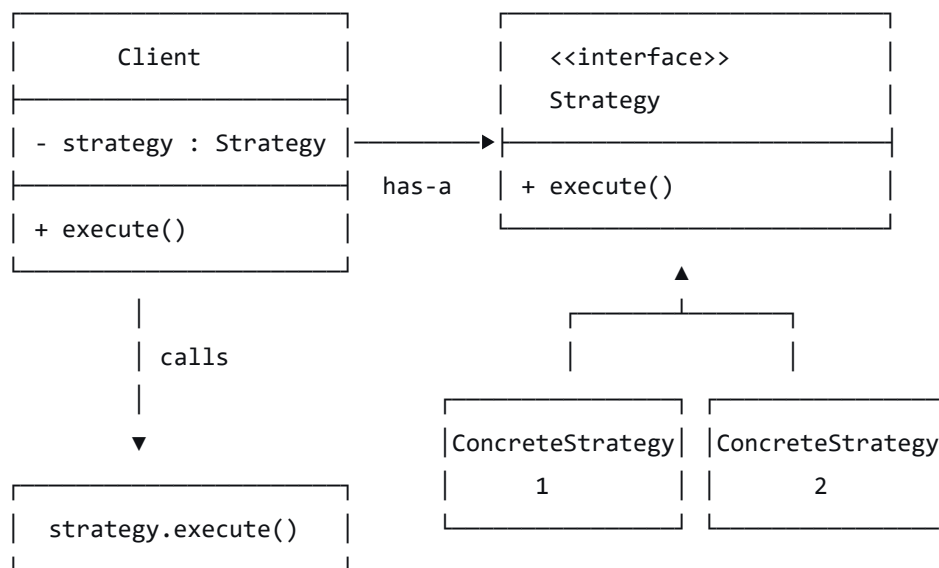
=== Companion Robot ===
Walking normally...
Talking normally...
Cannot fly.
Displaying companion robot projection.

```

```
=== Worker Robot ===  
Cannot walk.  
Cannot talk.  
Flying normally...  
Displaying worker robot projection.
```

Part 7: Standard UML Diagram of Strategy Pattern

text



Key relationships:

- Client **HAS-A** Strategy (composition)
- ConcreteStrategies **implement** the Strategy interface
- Client **delegates** to Strategy via `strategy.execute()`

Strategy Pattern Terms

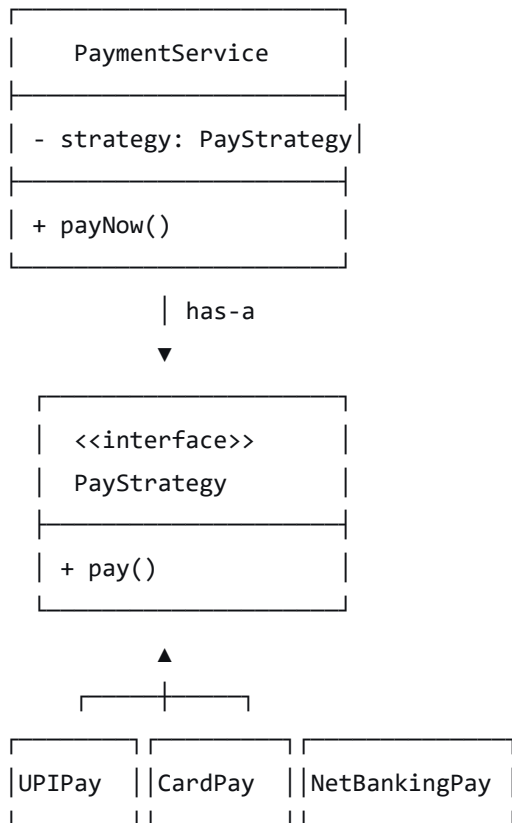
Term	Meaning	Example
Client	Uses strategies	Robot class
Strategy	Interface for algorithms	Walkable , Talkable , Flyable , Projectable

Term	Meaning	Example
Concrete Strategy	Actual algorithm implementations	NormalWalk , NoWalk , NormalFly , etc.

Part 8: Real-World Examples

Example 1: Payment System

text



Java Code:

java

```
interface PayStrategy {
    void pay(int amount);
}

class UPIPay implements PayStrategy {
    public void pay(int amount) {
        System.out.println("Paid ₹" + amount + " via UPI");
    }
}
```

```

    }
}

class CardPay implements PayStrategy {
    public void pay(int amount) {
        System.out.println("Paid ₹" + amount + " via Credit/Debit Card");
    }
}

class NetBankingPay implements PayStrategy {
    public void pay(int amount) {
        System.out.println("Paid ₹" + amount + " via Net Banking");
    }
}

class PaymentService {
    private PayStrategy strategy;

    public PaymentService(PayStrategy strategy) {
        this.strategy = strategy;
    }

    public void payNow(int amount) {
        strategy.pay(amount);
    }
}

public class Main {
    public static void main(String[] args) {
        PaymentService payment = new PaymentService(new UPIPay());
        payment.payNow(500);

        payment = new PaymentService(new CardPay());
        payment.payNow(1000);
    }
}

```

Python Code:

```

python

from abc import ABC, abstractmethod

class PayStrategy(ABC):
    @abstractmethod
    def pay(self, amount): pass

class UPIPay(PayStrategy):

```

```

def pay(self, amount): print(f"Paid ₹{amount} via UPI")

class CardPay(PayStrategy):
    def pay(self, amount): print(f"Paid ₹{amount} via Card")

class NetBankingPay(PayStrategy):
    def pay(self, amount): print(f"Paid ₹{amount} via Net Banking")

class PaymentService:
    def __init__(self, strategy: PayStrategy):
        self.strategy = strategy

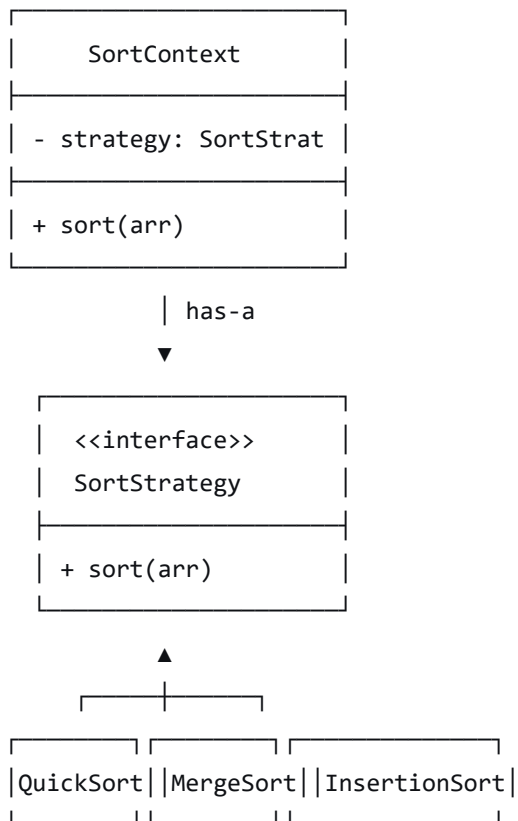
    def pay_now(self, amount):
        self.strategy.pay(amount)

PaymentService(UPIPay()).pay_now(500)
PaymentService(CardPay()).pay_now(1000)

```

Example 2: Sorting Algorithms

text



Example 3: Text Editor Formatting

- Bold, Italic, Underline — each is a strategy

Example 4: Compression

- ZIP, RAR, 7z — different compression algorithms

Example 5: Authentication

- OAuth, JWT, Basic Auth — interchangeable strategies

Part 9: Can We Remove ALL Inheritance?

The transcript ends with a great question: *"Is there still inheritance in the design?"*

Answer: Yes — the `Robot` class hierarchy for `Projection` still uses inheritance.

Fix: Make `Projectable` a strategy too!

text

Robot
- walkable
- talkable
- flyable
- projectable
+ walk()
+ talk()
+ fly()
+ projection()

Now `Robot` is a **single concrete class** with no child classes. Every behavior (walk/talk/fly/projection) is composed as a strategy.

Final Client Code:

java

```
Robot robot = new Robot(  
    new NormalWalk(),
```

```

        new NormalTalk(),
        new NormalFly(),
        new CompanionProjection() // strategy, not subclass
    );
    robot.walk();
    robot.talk();
    robot.fly();
    robot.projection();

```

Result: 0% inheritance — 100% composition.

Part 10: Key Concepts Summary

DRY Principle

Don't Repeat Yourself — Every piece of knowledge should have a single, unambiguous representation.

Strategy pattern eliminates duplicate `fly()`, `walk()`, etc. code.

Open/Closed Principle (OCP)

Classes should be open for extension but closed for modification.

New behaviors = new strategy classes. **Robot class never changes.**

Favor Composition Over Inheritance

- **Inheritance** = "IS-A" (fixed at compile time)
- **Composition** = "HAS-A" (flexible at runtime)

Runtime Flexibility

Strategies can be swapped at runtime:

```
java
```

```

Robot r = new Robot(new NormalWalk(), new NormalTalk(), new NoFly(), new CompanionProject
ion());

```

```
// Later...
```

```
r = new Robot(new NoWalk(), new NoTalk(), new NormalFly(), new WorkerProjection());
```

Part 11: When to Use Strategy Pattern

✅ Use When:

- Multiple algorithms exist for the same task
- Algorithm needs to be chosen at **runtime**
- You want to avoid a large conditional (`if-else` / `switch`) chain
- You want to isolate algorithm implementation from the client
- You want to remove duplicated behavior code

❌ Avoid When:

- Only one algorithm exists (over-engineering)
- Algorithms never change
- Behavior is fundamental to the class's identity

Part 12: Client Delegation ("Dumb Object")

The `Robot` class in the improved design becomes a "**dumb object**":

- It doesn't implement behavior
- It **delegates** all work to strategy objects
- It only knows **what to call**, not **how it works**

```
java
```

```
public void walk() { walkable.walk(); } // Delegate!  
public void talk() { talkable.talk(); } // Delegate!  
public void fly() { flyable.fly(); } // Delegate!
```

This is a common theme in **many** design patterns.

Key Takeaways

1. **Design patterns** are reusable solutions to common problems
2. **All patterns** separate changing parts from stable parts
3. **Strategy Pattern** = "family of algorithms in separate classes, interchangeable at runtime"
4. **Favor composition over inheritance** — this is the heart of Strategy Pattern
5. **Client becomes a "dumb object"** that delegates to strategies
6. **Eliminates DRY violations** — no duplicated behavior code
7. **Satisfies OCP** — adding new behavior = new strategy class only
8. **Runtime swapping** of strategies is possible
9. **Real-world uses:** payment methods, sorting algorithms, compression, authentication, text formatting
10. **Strategy is one of the most-used design patterns** — likely appears in any LLD interview problem

Comparison: Before vs After

Aspect	Inheritance-Based	Strategy Pattern
New behavior	Add new subclass (hierarchy explodes)	Add new strategy class
Duplicated code	Yes (DRY violated)	No
Runtime swap	Impossible	Possible
Complexity	Combinatorial explosion	Linear growth
OCP	Violated	Satisfied
Robot class changes	Yes (when behaviors change)	Never



हां जी, करें स्टार्ट। सो, हेलो को...
TXT 88.93KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python

also with explanation

Factory Design Pattern — Complete Notes

Part 1: Introduction to Factory Pattern

Real-World Analogy

A **factory** in the real world is a place where **products are manufactured**. Similarly, in LLD, a **factory** is a class responsible for **creating objects**.

Why Do We Need a Factory?

Core Problem: In Strategy Pattern, we assumed objects (`Talkable` , `Walkable` , `Flyable`) were already created somewhere. But **someone must create them** — using the `new` keyword.

The Goal: Separate business logic from object creation logic.

Logic Type	Meaning	Example
Business Logic	Core application logic	Sending notification, processing order
Object Creation Logic	Deciding which object to create	<code>new EmailNotification()</code> VS <code>new SMSNotification()</code>

Mixing them makes code complex. Factory Pattern decouples them.

How Factory Pattern Works

1. **Client** asks the factory for an object
2. **Factory** creates and returns the object
3. Client doesn't know **how** the object is created — just that it gets one

Key Insight: *"Keep the client decoupled from object creation."*

Part 2: Three Types of Factory

text

Factory Patterns

- └ Simple Factory (not officially a pattern – more a principle)
- └ Factory Method (proper design pattern)
- └ Abstract Factory (extended Factory Method)

Note: Simple Factory is a **principle**, not an official design pattern. Factory Method and Abstract Factory are proper patterns.

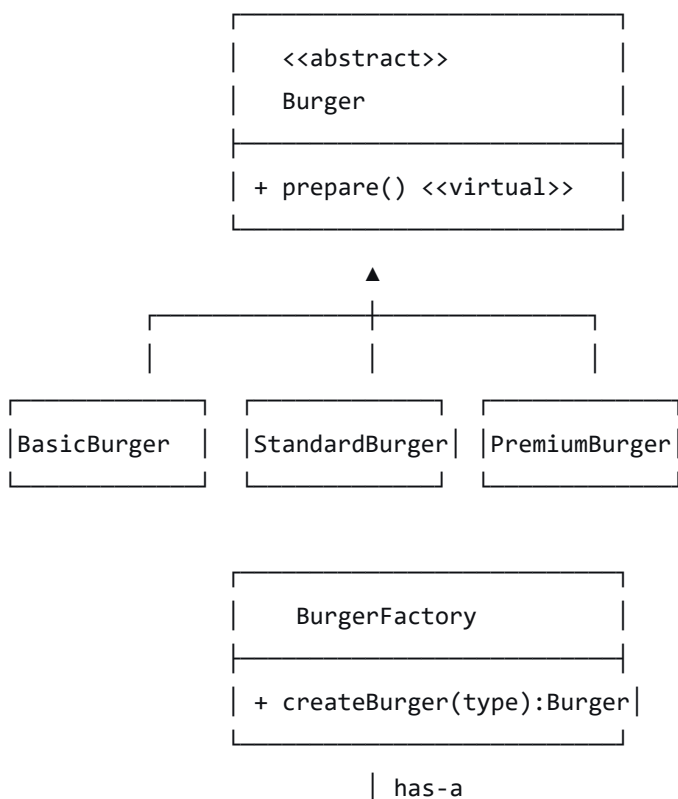
Part 3: Simple Factory

Problem Statement

A burger shop sells **different types of burgers**. A user chooses a type, and the shop prepares it. The shop shouldn't care **how** the burger is created — just that it gets a burger object.

Class Diagram

text





Burger (Product)

Java Code — Simple Factory

java

```
// ===== PRODUCT =====  
abstract class Burger {  
    public abstract void prepare();  
}  
  
class BasicBurger extends Burger {  
    @Override  
    public void prepare() {  
        System.out.println("Preparing Basic Burger with bun, patty, and ketchup!");  
    }  
}  
  
class StandardBurger extends Burger {  
    @Override  
    public void prepare() {  
        System.out.println("Preparing Standard Burger with bun, patty, cheese, and lettuce!");  
    }  
}  
  
class PremiumBurger extends Burger {  
    @Override  
    public void prepare() {  
        System.out.println("Preparing Premium Burger with gourmet bun, premium patty, cheese, and secret sauce!");  
    }  
}  
  
// ===== SIMPLE FACTORY =====  
class BurgerFactory {  
    public Burger createBurger(String type) {  
        if (type.equalsIgnoreCase("basic")) {  
            return new BasicBurger();  
        } else if (type.equalsIgnoreCase("standard")) {  
            return new StandardBurger();  
        } else if (type.equalsIgnoreCase("premium")) {  
            return new PremiumBurger();  
        } else {  
            System.out.println("Invalid burger type!");  
            return null;  
        }  
    }  
}
```

```

    }
}

// ===== CLIENT =====
public class Main {
    public static void main(String[] args) {
        String type = "standard";

        BurgerFactory factory = new BurgerFactory();
        Burger burger = factory.createBurger(type);

        if (burger != null) {
            burger.prepare();
        }
    }
}

```

Output:

text

Preparing Standard Burger with bun, patty, cheese, and lettuce!

Python Code — Simple Factory

python

```
from abc import ABC, abstractmethod
```

```
class Burger(ABC):
    @abstractmethod
    def prepare(self): pass
```

```
class BasicBurger(Burger):
    def prepare(self):
        print("Preparing Basic Burger with bun, patty, and ketchup!")
```

```
class StandardBurger(Burger):
    def prepare(self):
        print("Preparing Standard Burger with bun, patty, cheese, and lettuce!")
```

```
class PremiumBurger(Burger):  
    def prepare(self):  
        print("Preparing Premium Burger with gourmet bun, premium patty, cheese, and secret sauce!")
```

```
class BurgerFactory:  
    def create_burger(self, burger_type):  
        if burger_type == "basic":  
            return BasicBurger()  
        elif burger_type == "standard":  
            return StandardBurger()  
        elif burger_type == "premium":  
            return PremiumBurger()  
        else:  
            print("Invalid burger type!")  
            return None
```

Usage

```
burger_type = "standard"  
factory = BurgerFactory()  
burger = factory.create_burger(burger_type)
```

```
if burger:  
    burger.prepare()
```

Simple Factory — Definition

A factory class that decides which concrete class to instantiate.

Note: Simple Factory is a design **principle**, not an official Gang-of-Four pattern.

Part 4: Factory Method Pattern

Motivation

Simple Factory has a limitation: **one factory creates everything**. What if we have **multiple franchises** (e.g., SinghBurger, KingBurger) that create burgers differently?

Solution: Make the factory itself **abstract**. Subclasses decide which product to create.

Concept

Simple Factory:

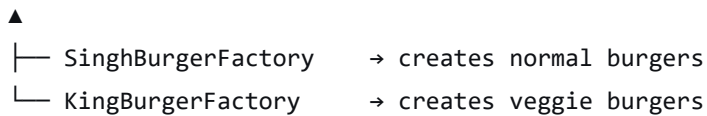
text

BurgerFactory → creates all burgers

Factory Method:

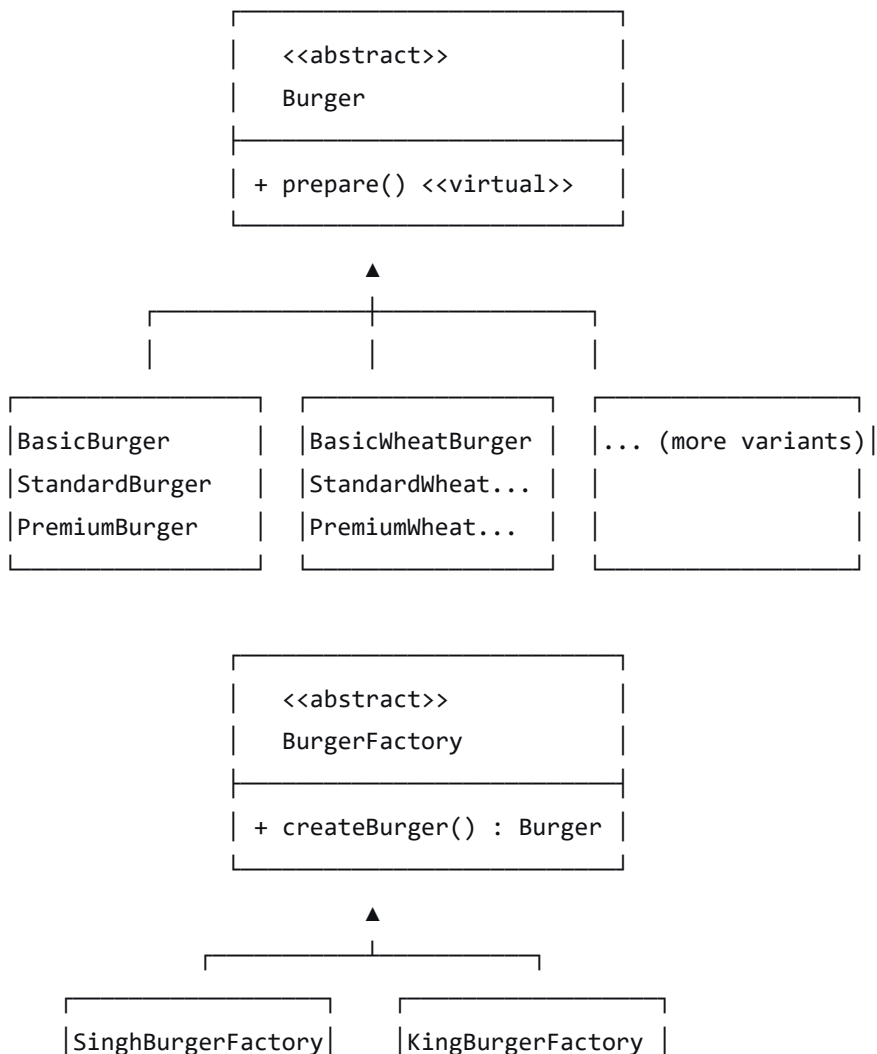
text

BurgerFactory (abstract)



Class Diagram

text





Java Code — Factory Method

java

```
// ===== PRODUCTS =====
abstract class Burger {
    public abstract void prepare();
}

// Normal burgers (Singh Burger)
class BasicBurger extends Burger {
    @Override
    public void prepare() {
        System.out.println("Preparing Basic Burger with bun, patty, and ketchup!");
    }
}

class StandardBurger extends Burger {
    @Override
    public void prepare() {
        System.out.println("Preparing Standard Burger with bun, patty, cheese, and lettuce!");
    }
}

class PremiumBurger extends Burger {
    @Override
    public void prepare() {
        System.out.println("Preparing Premium Burger with gourmet bun, premium patty, cheese, and secret sauce!");
    }
}

// Wheat burgers (King Burger)
class BasicWheatBurger extends Burger {
    @Override
    public void prepare() {
        System.out.println("Preparing Basic Wheat Burger with wheat bun, veggie patty, and ketchup!");
    }
}

class StandardWheatBurger extends Burger {
```

```
@Override
public void prepare() {
    System.out.println("Preparing Standard Wheat Burger with wheat bun, veggie patty,
cheese, and lettuce!");
}
}

class PremiumWheatBurger extends Burger {
    @Override
    public void prepare() {
        System.out.println("Preparing Premium Wheat Burger with wheat bun, veggie patty,
cheese, and secret sauce!");
    }
}

// ===== ABSTRACT FACTORY =====
abstract class BurgerFactory {
    public abstract Burger createBurger(String type);
}

// ===== CONCRETE FACTORIES =====
class SinghBurger extends BurgerFactory {
    @Override
    public Burger createBurger(String type) {
        if (type.equalsIgnoreCase("basic")) return new BasicBurger();
        if (type.equalsIgnoreCase("standard")) return new StandardBurger();
        if (type.equalsIgnoreCase("premium")) return new PremiumBurger();
        System.out.println("Invalid burger type!");
        return null;
    }
}

class KingBurger extends BurgerFactory {
    @Override
    public Burger createBurger(String type) {
        if (type.equalsIgnoreCase("basic")) return new BasicWheatBurger();
        if (type.equalsIgnoreCase("standard")) return new StandardWheatBurger();
        if (type.equalsIgnoreCase("premium")) return new PremiumWheatBurger();
        System.out.println("Invalid burger type!");
        return null;
    }
}

// ===== CLIENT =====
public class Main {
    public static void main(String[] args) {
        String type = "basic";
    }
}
```

```

    BurgerFactory factory = new KingBurger();
    Burger burger = factory.createBurger(type);

    if (burger != null) burger.prepare();
}
}

```

Output:

text

Preparing Basic Wheat Burger with wheat bun, veggie patty, and ketchup!

Python Code — Factory Method

python

```

from abc import ABC, abstractmethod

# ===== PRODUCTS =====
class Burger(ABC):
    @abstractmethod
    def prepare(self): pass

class BasicBurger(Burger):
    def prepare(self):
        print("Preparing Basic Burger with bun, patty, and ketchup!")

class StandardBurger(Burger):
    def prepare(self):
        print("Preparing Standard Burger with bun, patty, cheese, and lettuce!")

class PremiumBurger(Burger):
    def prepare(self):
        print("Preparing Premium Burger with gourmet bun, premium patty, cheese, and secret sauce!")

class BasicWheatBurger(Burger):
    def prepare(self):
        print("Preparing Basic Wheat Burger with wheat bun, veggie patty, and ketchup!")

```



```

class StandardWheatBurger(Burger):
    def prepare(self):
        print("Preparing Standard Wheat Burger with wheat bun, veggie patty, cheese, and lettuce!")

class PremiumWheatBurger(Burger):
    def prepare(self):
        print("Preparing Premium Wheat Burger with wheat bun, veggie patty, cheese, and secret sauce!")

# ===== ABSTRACT FACTORY =====
class BurgerFactory(ABC):
    @abstractmethod
    def create_burger(self, burger_type): pass

# ===== CONCRETE FACTORIES =====
class SinghBurger(BurgerFactory):
    def create_burger(self, burger_type):
        if burger_type == "basic": return BasicBurger()
        if burger_type == "standard": return StandardBurger()
        if burger_type == "premium": return PremiumBurger()
        print("Invalid burger type!")
        return None

class KingBurger(BurgerFactory):
    def create_burger(self, burger_type):
        if burger_type == "basic": return BasicWheatBurger()
        if burger_type == "standard": return StandardWheatBurger()
        if burger_type == "premium": return PremiumWheatBurger()
        print("Invalid burger type!")
        return None

# ===== CLIENT =====
burger_type = "basic"
factory = KingBurger()
burger = factory.create_burger(burger_type)
if burger:
    burger.prepare()

```

Factory Method — Definition

Define an interface for creating an object, but let subclasses decide which class to instantiate.

What's better than Simple Factory? The factory itself is **extensible** — new franchises add new factories without touching the abstract factory.

Part 5: Abstract Factory Pattern

Motivation

What if a franchise creates **multiple product families** — like **Burgers AND Garlic Bread**?

King Burger creates:

- Wheat Burgers (basic, standard, premium)
- Wheat Garlic Bread (basic, cheese)

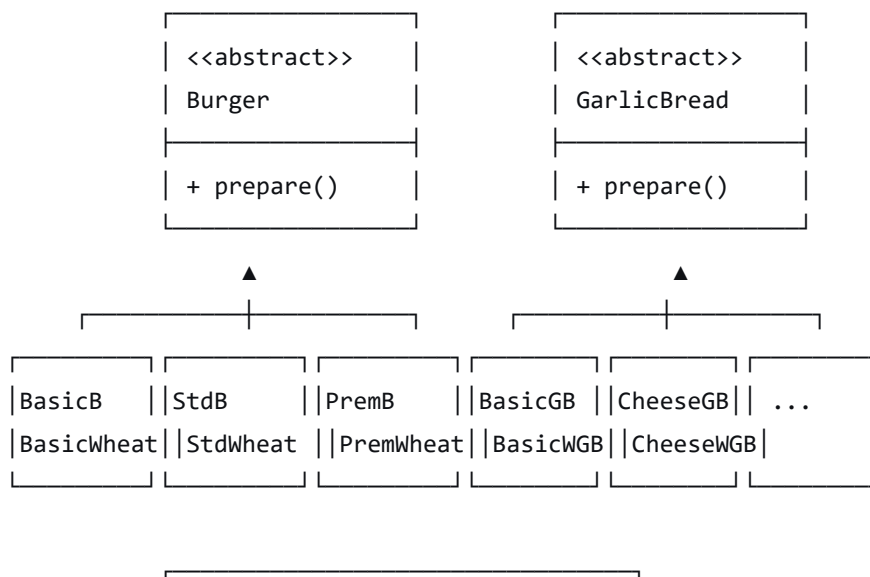
Singh Burger creates:

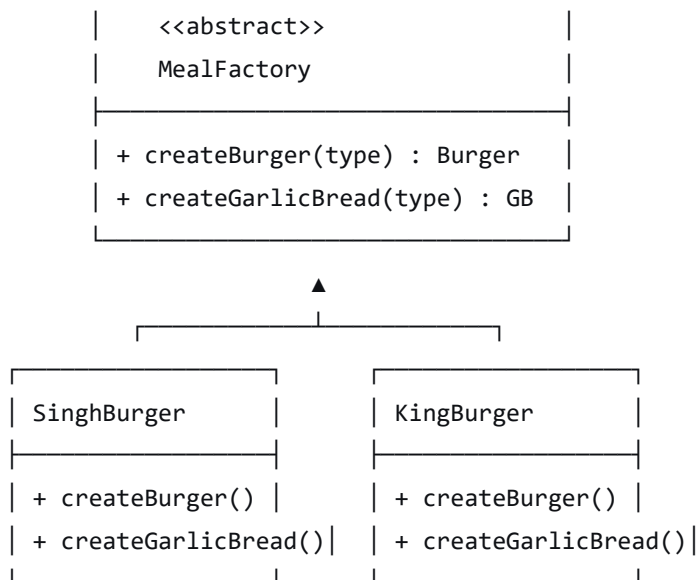
- Normal Burgers (basic, standard, premium)
- Normal Garlic Bread (basic, cheese)

This is **Abstract Factory** — creating **families of related products**.

Class Diagram

text





Java Code — Abstract Factory

java

```

// ===== PRODUCT FAMILY 1: BURGERS =====
abstract class Burger {
    public abstract void prepare();
}

class BasicBurger extends Burger {
    public void prepare() { System.out.println("Basic Burger"); }
}
class StandardBurger extends Burger {
    public void prepare() { System.out.println("Standard Burger"); }
}
class PremiumBurger extends Burger {
    public void prepare() { System.out.println("Premium Burger"); }
}
class BasicWheatBurger extends Burger {
    public void prepare() { System.out.println("Basic Wheat Burger"); }
}
class StandardWheatBurger extends Burger {
    public void prepare() { System.out.println("Standard Wheat Burger"); }
}
class PremiumWheatBurger extends Burger {
    public void prepare() { System.out.println("Premium Wheat Burger"); }
}

// ===== PRODUCT FAMILY 2: GARLIC BREADS =====
abstract class GarlicBread {
    public abstract void prepare();
}
  
```

```
class BasicGarlicBread extends GarlicBread {
    public void prepare() { System.out.println("Basic Garlic Bread"); }
}
class CheeseGarlicBread extends GarlicBread {
    public void prepare() { System.out.println("Cheese Garlic Bread"); }
}
class BasicWheatGarlicBread extends GarlicBread {
    public void prepare() { System.out.println("Basic Wheat Garlic Bread"); }
}
class CheeseWheatGarlicBread extends GarlicBread {
    public void prepare() { System.out.println("Cheese Wheat Garlic Bread"); }
}

// ===== ABSTRACT FACTORY =====
abstract class MealFactory {
    public abstract Burger createBurger(String type);
    public abstract GarlicBread createGarlicBread(String type);
}

// ===== CONCRETE FACTORIES =====
class SinghBurger extends MealFactory {
    @Override
    public Burger createBurger(String type) {
        if (type.equalsIgnoreCase("basic")) return new BasicBurger();
        if (type.equalsIgnoreCase("standard")) return new StandardBurger();
        if (type.equalsIgnoreCase("premium")) return new PremiumBurger();
        return null;
    }

    @Override
    public GarlicBread createGarlicBread(String type) {
        if (type.equalsIgnoreCase("basic")) return new BasicGarlicBread();
        if (type.equalsIgnoreCase("cheese")) return new CheeseGarlicBread();
        return null;
    }
}

class KingBurger extends MealFactory {
    @Override
    public Burger createBurger(String type) {
        if (type.equalsIgnoreCase("basic")) return new BasicWheatBurger();
        if (type.equalsIgnoreCase("standard")) return new StandardWheatBurger();
        if (type.equalsIgnoreCase("premium")) return new PremiumWheatBurger();
        return null;
    }

    @Override
```

```

    public GarlicBread createGarlicBread(String type) {
        if (type.equalsIgnoreCase("basic")) return new BasicWheatGarlicBread();
        if (type.equalsIgnoreCase("cheese")) return new CheeseWheatGarlicBread();
        return null;
    }
}

// ===== CLIENT =====
public class Main {
    public static void main(String[] args) {
        String burgerType = "basic";
        String garlicBreadType = "cheese";

        MealFactory factory = new KingBurger(); // Choose franchise at runtime

        Burger burger = factory.createBurger(burgerType);
        GarlicBread garlicBread = factory.createGarlicBread(garlicBreadType);

        if (burger != null) burger.prepare();
        if (garlicBread != null) garlicBread.prepare();
    }
}

```

Output:

text

```

Basic Wheat Burger
Cheese Wheat Garlic Bread

```

Python Code — Abstract Factory

```

python

from abc import ABC, abstractmethod

# ===== PRODUCT FAMILY 1: BURGERS =====
class Burger(ABC):
    @abstractmethod
    def prepare(self): pass

class BasicBurger(Burger):
    def prepare(self): print("Basic Burger")

class StandardBurger(Burger):

```

```

    def prepare(self): print("Standard Burger")

class PremiumBurger(Burger):
    def prepare(self): print("Premium Burger")

class BasicWheatBurger(Burger):
    def prepare(self): print("Basic Wheat Burger")

class StandardWheatBurger(Burger):
    def prepare(self): print("Standard Wheat Burger")

class PremiumWheatBurger(Burger):
    def prepare(self): print("Premium Wheat Burger")

# ===== PRODUCT FAMILY 2: GARLIC BREADS =====
class GarlicBread(ABC):
    @abstractmethod
    def prepare(self): pass

class BasicGarlicBread(GarlicBread):
    def prepare(self): print("Basic Garlic Bread")

class CheeseGarlicBread(GarlicBread):
    def prepare(self): print("Cheese Garlic Bread")

class BasicWheatGarlicBread(GarlicBread):
    def prepare(self): print("Basic Wheat Garlic Bread")

class CheeseWheatGarlicBread(GarlicBread):
    def prepare(self): print("Cheese Wheat Garlic Bread")

# ===== ABSTRACT FACTORY =====
class MealFactory(ABC):
    @abstractmethod
    def create_burger(self, burger_type): pass

    @abstractmethod
    def create_garlic_bread(self, garlic_bread_type): pass

# ===== CONCRETE FACTORIES =====
class SinghBurger(MealFactory):
    def create_burger(self, burger_type):
        if burger_type == "basic": return BasicBurger()
        if burger_type == "standard": return StandardBurger()
        if burger_type == "premium": return PremiumBurger()

```

```

        return None

    def create_garlic_bread(self, garlic_bread_type):
        if garlic_bread_type == "basic": return BasicGarlicBread()
        if garlic_bread_type == "cheese": return CheeseGarlicBread()
        return None

class KingBurger(MealFactory):
    def create_burger(self, burger_type):
        if burger_type == "basic": return BasicWheatBurger()
        if burger_type == "standard": return StandardWheatBurger()
        if burger_type == "premium": return PremiumWheatBurger()
        return None

    def create_garlic_bread(self, garlic_bread_type):
        if garlic_bread_type == "basic": return BasicWheatGarlicBread()
        if garlic_bread_type == "cheese": return CheeseWheatGarlicBread()
        return None

# ===== CLIENT =====
burger_type = "basic"
garlic_bread_type = "cheese"

factory = KingBurger() # Choose franchise

burger = factory.create_burger(burger_type)
garlic_bread = factory.create_garlic_bread(garlic_bread_type)

if burger: burger.prepare()
if garlic_bread: garlic_bread.prepare()

```

Abstract Factory — Definition

Provide an interface for creating families of related objects without specifying their concrete classes.

Key Insight: One factory creates multiple product families.

Part 6: Comparison — Simple vs Factory Method vs Abstract Factory

Aspect	Simple Factory	Factory Method	Abstract Factory
Is a pattern?	No (principle)	Yes	Yes
Factory Type	Concrete class	Abstract class	Abstract class
Subclasses?	No	Yes (each subclass is a factory)	Yes (each subclass is factory)
Product families	One product	One product per factory	Multiple product famili
Extensibility	Modify factory for new type	Add new factory subclass	Add new factory subclass
OCP	Violated (modify existing factory)	Satisfied	Satisfied

Quick Decision Guide

Question	Answer	Use
Do I need ONE product type?	Yes	Simple Factory or Factory Method
Do I need ONE product with MULTIPLE factories?	Yes	Factory Method
Do I need MULTIPLE product families?	Yes	Abstract Factory
Do I need RUNTIME algorithm swapping?	Yes	Strategy (not Factory)

Part 7: Practical Example — Notification System

Application

A notification system that sends **SMS, Push, or Email** notifications.

Using Simple Factory

java

```
abstract class Notification {
    public abstract void send(String message);
}

class SMSNotification extends Notification {
    public void send(String message) { System.out.println("SMS: " + message); }
}

class PushNotification extends Notification {
    public void send(String message) { System.out.println("Push: " + message); }
}

class EmailNotification extends Notification {
    public void send(String message) { System.out.println("Email: " + message); }
}

class NotificationFactory {
    public Notification createNotification(String type) {
        if (type.equalsIgnoreCase("sms")) return new SMSNotification();
        if (type.equalsIgnoreCase("push")) return new PushNotification();
        if (type.equalsIgnoreCase("email")) return new EmailNotification();
        return null;
    }
}

public class Main {
    public static void main(String[] args) {
        NotificationFactory factory = new NotificationFactory();
        Notification notif = factory.createNotification("email");
        notif.send("Hello from Notification Factory!");
    }
}
```

Using Strategy Pattern (Alternative)

java

```
interface NotificationStrategy {
    void send(String message);
}

class SMSStrategy implements NotificationStrategy {
    public void send(String message) { System.out.println("SMS: " + message); }
}
```

```
class NotificationService {
    private NotificationStrategy strategy;

    public NotificationService(NotificationStrategy strategy) {
        this.strategy = strategy;
    }

    public void notifyUser(String message) {
        strategy.send(message);
    }
}

public class Main {
    public static void main(String[] args) {
        NotificationService service = new NotificationService(new SMSStrategy());
        service.notifyUser("Hello!");
    }
}
```

Which Is Better? Factory or Strategy?

Aspect	Factory	Strategy
Intent	Object creation	Algorithm selection
Runtime swapping	No	Yes
Client knows	Type string	Strategy interface
Use Case	Creating objects	Changing behavior

Rule of Thumb:

- Need to vary the algorithm at runtime? → Strategy
- Need to separate object creation logic? → Factory
- Both might apply — the design is subjective!

Part 8: Key Takeaways

1. **Factory Pattern** separates object creation from business logic
2. **Simple Factory** = one concrete factory with conditional logic (not a formal pattern)

3. **Factory Method** = abstract factory + subclasses decide product type
4. **Abstract Factory** = one factory creates families of related products
5. **All factories** create objects; the difference is in the level of abstraction
6. **Factory vs Strategy** — different intents; both can solve the same problem
7. **Adding new product types** is easy with Factory Method/Abstract Factory (OCP satisfied)
8. **Factory Pattern** is one of the most-used patterns — 99% of LLD interviews include it
9. **Simple Factory** violates OCP (modifying factory for each new type)
10. **Runtime decision**: Which concrete factory to instantiate can be decided at runtime

Part 9: Common Pitfalls

Pitfall	Solution
Overusing Factory (over-engineering)	Only use when object creation is complex
Confusing Factory with Strategy	Remember intents: creation vs behavior
Making Simple Factory too large	Consider Factory Method for extensibility
Ignoring future product families	Use Abstract Factory if multiple families expected
Tight coupling in factory	Return the abstract type, not concrete

Part 10: Summary Table

Pattern	What It Does	When to Use
Simple Factory	One class decides which object to create	Small, fixed product set
Factory Method	Abstract factory; subclasses decide	Adding new products without modifying existing code

Pattern	What It Does	When to Use
Abstract Factory	Create families of related products	Multiple product families that must be consistent

Final Words

Factory Pattern is the **most-used** creational design pattern. In any LLD interview or real-world application, it will appear. Understanding **when** and **why** to apply each variant is what separates good design from great design.

Key Line: *"Factory = separate object creation logic from business logic."*



हां जी करें स्टार्ट। सो वेलकम बै...
TXT 87.27KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also with explanation

Singleton Design Pattern — Complete Notes

Part 1: Introduction to Singleton

What is Singleton?

A **Singleton** is a special class that allows **only one instance (object)** to be created. If you try to create another instance, it returns the **same first instance**.

Definition:

Singleton Design Pattern ensures that a class has only one instance and provides a global point of access to it.

Real-World Analogy

Think of a country's **government** — there's only one government at a time. Or a **company's CEO** — one CEO leads the company. Similarly, Singleton ensures one instance across the entire application.

Part 2: Why Do We Need Singleton?

The Problem

Normally, when you write `new MyClass()`, a new object is created **every time**. For some resources, this is wasteful or dangerous:

Resource	Why Only One Instance?
Logger	Multiple loggers → logs interleaved incorrectly
Database Connection	Creating connections is expensive
Configuration Manager	Single source of truth for config
Cache	Shared cache across the app
Thread Pool	One central thread pool

How Object Creation Works (Under the Hood)

When you write `A* a = new A();`:

1. **Heap memory reserved** — space allocated based on class size
2. **Constructor called** — initializes fields with default values
3. **Pointer assigned** — stack variable `a` points to heap object

Key Insight: The constructor is **public** by default, which lets anyone call `new A()` and create unlimited objects.

Solution: Make the constructor **private** so nobody outside can call `new`.

But then how do we create even one instance? We provide a **public static method** that handles creation.

Part 3: Building the Singleton — Step by Step

Step 1: Naive Class (Multiple Objects Allowed)

java

```
class Singleton {
    public Singleton() {
        System.out.println("Singleton constructor called. New object created.");
    }
}

public class Main {
    public static void main(String[] args) {
        Singleton s1 = new Singleton();
        Singleton s2 = new Singleton();
        System.out.println(s1 == s2); // false
    }
}
```

Output:

text

```
Singleton constructor called. New object created.
Singleton constructor called. New object created.
false
```

Problem: Two separate objects — the constructor is public.

Step 2: Make Constructor Private

java

```
class Singleton {
    private Singleton() {
        System.out.println("Singleton constructor called.");
    }
}

public class Main {
    public static void main(String[] args) {
        // Singleton s1 = new Singleton(); // COMPILER ERROR
    }
}
```

Problem: We can't create any object now. We need a controlled way.

Step 3: Add Static Method + Static Instance

java

```
class Singleton {
    private static Singleton instance; // holds the single instance

    private Singleton() {
        System.out.println("Singleton constructor called.");
    }

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}

public class Main {
    public static void main(String[] args) {
        Singleton s1 = Singleton.getInstance();
        Singleton s2 = Singleton.getInstance();
        System.out.println(s1 == s2); // true
    }
}
```

Output:

text

```
Singleton constructor called.
true
```

Explanation:

- `private static Singleton instance` — a static variable to hold the one instance
- `private Singleton()` — prevents `new` from outside
- `getInstance()` — public static method to return the single instance
- **First call:** `instance == null` → creates object → returns it
- **Later calls:** `instance != null` → returns existing object

Why static ?

- We need to call `getInstance()` without having an object.
- `static` methods belong to the **class**, not the object.

Part 4: Thread Safety — The Multi-Threading Problem

Problem

In multi-threaded environments, two threads can **both check** `instance == null` and create two objects.

text

Thread T1: checks `instance == null` → true → creates object A

Thread T2: checks `instance == null` → true → creates object B

Result: TWO singletons (violated!)

Solution 1: Mutex Lock (Thread-Safe But Expensive)

java

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

class Singleton {
    private static Singleton instance;
    private static final Lock mutex = new ReentrantLock();

    private Singleton() {}

    public static Singleton getInstance() {
        mutex.lock();
        if (instance == null) {
            instance = new Singleton();
        }
        mutex.unlock();
        return instance;
    }
}
```

Problem: Locking/unlocking on **every call** is expensive, even after the object exists.

Solution 2: Double-Checked Locking

java


```

class Singleton {
    private static Singleton instance;
    private static final Object mutex = new Object();

    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) {           // First check (no lock)
            synchronized (mutex) {
                if (instance == null) {   // Second check (with lock)
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}

```

Why double check?

- **First check (no lock):** Fast path — if instance exists, return immediately.
- **Second check (with lock):** Only one thread gets past the lock. Ensures no duplicate creation.

But wait — even with double-checking, **Java Memory Model** can allow reordering. Using `volatile` prevents that:

```

java

private static volatile Singleton instance;

```

Java Code — Thread-Safe Singleton

```

java

class Singleton {
    private static volatile Singleton instance;

    private Singleton() {
        System.out.println("Singleton constructed.");
    }

    public static Singleton getInstance() {
        if (instance == null) {

```

```

        synchronized (Singleton.class) {
            if (instance == null) {
                instance = new Singleton();
            }
        }
    }
    return instance;
}
}

public class Main {
    public static void main(String[] args) {
        Thread t1 = new Thread(() -> Singleton.getInstance());
        Thread t2 = new Thread(() -> Singleton.getInstance());
        t1.start();
        t2.start();
    }
}

```

Python Code — Thread-Safe Singleton

python

```

import threading

class Singleton:
    _instance = None
    _lock = threading.Lock()

    def __new__(cls):
        if cls._instance is None:
            with cls._lock:
                if cls._instance is None:
                    cls._instance = super().__new__(cls)
                    print("Singleton constructed.")
                return cls._instance

# Usage
s1 = Singleton()
s2 = Singleton()
print(s1 is s2) # True

```

Note: Python's GIL usually protects against race conditions, but using a lock is more robust.

Part 5: Eager Initialization (Simplest Thread-Safe Approach)

java

```
class Singleton {
    // Instance created at class-load time (before any thread runs)
    private static final Singleton instance = new Singleton();

    private Singleton() {
        System.out.println("Singleton constructed.");
    }

    public static Singleton getInstance() {
        return instance;
    }
}
```

Advantages:

- Thread-safe by default (class loading is thread-safe in Java)
- Simple — no locks, no null checks
- Fast `getInstance()`

Disadvantages:

- Object created even if never used → **wasted memory**
- If constructor is expensive (DB connection), it delays app startup

Python — Eager Initialization

python

```
class Singleton:
    _instance = None # Placeholder

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            print("Singleton constructed.")
        return cls._instance
```

Or, even simpler in Python:

```
class SingletonEager:
    def __new__(cls):
        if not hasattr(cls, "_instance"):
            cls._instance = super().__new__(cls)
        return cls._instance
```

Part 6: Full Comparison — Three Approaches

Approach	Thread-Safe?	Lazy?	Complexity	Memory Waste
Lazy (no lock)	✗	✓	Low	No
Mutex (every call)	✓	✓	Low	No (but slow)
Double-Checked Locking	✓	✓	Medium	No
Eager Initialization	✓	✗	Low	Yes (if unused)

Lazy = Object created only when first requested.
Eager = Object created at class load.

Part 7: Real-World Use Cases

1. Logging System

Why Singleton?

- One logger instance shared across the app
- Logs written in **order** (not interleaved by multiple loggers)
- Saves memory (no duplicate logger instances)

Example:

```
java
class Logger {
    private static Logger instance;
```

```
private Logger() {}  
public static Logger getInstance() {  
    if (instance == null) instance = new Logger();  
    return instance;  
}  
public void log(String msg) {  
    System.out.println("[LOG] " + msg);  
}  
}  
  
class PaymentService {  
    public void pay() {  
        Logger.getInstance().log("Payment processed");  
    }  
}  
  
class NotificationService {  
    public void notifyUser() {  
        Logger.getInstance().log("User notified");  
    }  
}
```

2. Database Connection

Why Singleton?

- DB connections are **expensive** to create
- One connection pool shared across the app
- Saves memory and connection overhead

3. Configuration Manager

Why Singleton?

- Single source of truth for API keys, URLs, etc.
- All services share the same config object
- Prevents inconsistency

4. Cache

Why Singleton?

- Shared cache for all services

- Prevents cache duplication

5. Thread Pool

Why Singleton?

- One central pool shared across the app
- Efficient resource utilization

Part 8: When NOT to Use Singleton

Situation	Why Avoid Singleton
Multiple instances needed	E.g., Game players — each player is unique
Testing/Unit Tests	Hard to mock/isolate; global state complicates tests
Thread safety overhead	Adds complexity to code
Tight coupling	Singleton becomes a global state → hard to refactor
Hidden dependencies	Classes silently depend on the singleton

Example — Game (DO NOT use Singleton):

```

java

class Player {
    private String name;
    // Each player is a separate object – Singleton is WRONG here
}
```

Part 9: Complete Code — All Approaches

Java — Complete Implementation

```

java
```

```
// ===== 1. LAZY (NOT THREAD-SAFE) =====
```

```
class LazySingleton {
    private static LazySingleton instance;
    private LazySingleton() {}
    public static LazySingleton getInstance() {
        if (instance == null) {
            instance = new LazySingleton();
        }
        return instance;
    }
}
```

```
// ===== 2. THREAD-SAFE (Double-Checked Locking) =====
```

```
class ThreadSafeSingleton {
    private static volatile ThreadSafeSingleton instance;
    private ThreadSafeSingleton() {}
    public static ThreadSafeSingleton getInstance() {
        if (instance == null) {
            synchronized (ThreadSafeSingleton.class) {
                if (instance == null) {
                    instance = new ThreadSafeSingleton();
                }
            }
        }
        return instance;
    }
}
```

```
// ===== 3. EAGER INITIALIZATION =====
```

```
class EagerSingleton {
    private static final EagerSingleton instance = new EagerSingleton();
    private EagerSingleton() {}
    public static EagerSingleton getInstance() {
        return instance;
    }
}
```

```
// ===== 4. BILL PUGH (Best Practice) =====
```

```
class BillPughSingleton {
    private BillPughSingleton() {}

    private static class SingletonHelper {
        private static final BillPughSingleton INSTANCE = new BillPughSingleton();
    }

    public static BillPughSingleton getInstance() {
        return SingletonHelper.INSTANCE;
    }
}
```

```

    }

    public class Main {
        public static void main(String[] args) {
            ThreadSafeSingleton s1 = ThreadSafeSingleton.getInstance();
            ThreadSafeSingleton s2 = ThreadSafeSingleton.getInstance();
            System.out.println("Thread-safe singleton: " + (s1 == s2)); // true

            EagerSingleton e1 = EagerSingleton.getInstance();
            EagerSingleton e2 = EagerSingleton.getInstance();
            System.out.println("Eager singleton: " + (e1 == e2)); // true

            BillPughSingleton b1 = BillPughSingleton.getInstance();
            BillPughSingleton b2 = BillPughSingleton.getInstance();
            System.out.println("Bill Pugh singleton: " + (b1 == b2)); // true
        }
    }
}

```

Python — Complete Implementation

python

```

import threading

# ===== 1. BASIC SINGLETON =====
class Singleton:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            print("Singleton created.")
        return cls._instance

# ===== 2. THREAD-SAFE SINGLETON =====
class ThreadSafeSingleton:
    _instance = None
    _lock = threading.Lock()

    def __new__(cls):
        if cls._instance is None:
            with cls._lock:
                if cls._instance is None:
                    cls._instance = super().__new__(cls)
                    print("Thread-safe Singleton created.")

```



```

        return cls._instance

# ===== 3. DECORATOR-BASED SINGLETON =====
def singleton(cls):
    instances = {}
    def get_instance(*args, **kwargs):
        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]
    return get_instance

@singleton
class DecoratedSingleton:
    def __init__(self):
        print("Decorated Singleton created.")

# ===== USAGE =====
if __name__ == "__main__":
    # Test 1: Basic
    s1 = Singleton()
    s2 = Singleton()
    print(f"Basic: s1 is s2 -> {s1 is s2}") # True

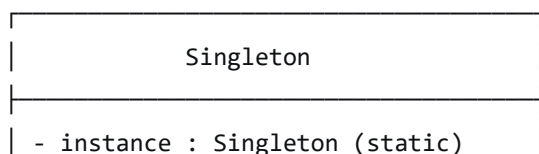
    # Test 2: Thread-safe
    t1 = threading.Thread(target=ThreadSafeSingleton)
    t2 = threading.Thread(target=ThreadSafeSingleton)
    t1.start(); t2.start()
    t1.join(); t2.join()

    # Test 3: Decorator
    d1 = DecoratedSingleton()
    d2 = DecoratedSingleton()
    print(f"Decorated: d1 is d2 -> {d1 is d2}") # True

```

Part 10: UML Diagram

text



```
| _____ |  
| - Singleton() <<private>> |  
| + getInstance() : Singleton (static) |  
|_____|
```

Key elements:

- private static instance — holds the single object
- private constructor — prevents external new
- public static getInstance() — global access point

Part 11: Common Pitfalls & Best Practices

Pitfall	Solution
Not thread-safe	Use double-checked locking or eager init
Not lazy	Use lazy initialization for expensive objects
Class loader issues	Use enum in Java (best practice)
Serialization creating new instance	Add readResolve() method
Reflection breaking singleton	Use enum or guard in constructor
Cloning creating new instance	Override clone() to return same instance

Java Enum Singleton (Best Practice)

```
java  
  
public enum SingletonEnum {  
    INSTANCE;  
  
    public void doSomething() {  
        System.out.println("Doing something...");  
    }  
}  
  
// Usage  
SingletonEnum.INSTANCE.doSomething();
```

Why enum?

- Thread-safe by JVM
- Handles serialization automatically
- Reflection cannot create a second instance

Part 12: Summary

Key Takeaways

1. **Singleton** = Class with **only one instance** globally
2. **Private constructor** = Prevents `new` from outside
3. **Static instance variable** = Holds the one object
4. **Static** `getInstance()` **method** = Global access point
5. **Thread safety** = Use double-checked locking or eager init
6. **Real-world uses**: Logging, DB connections, Config, Cache, Thread pools
7. **Avoid when**: Multiple instances needed, testing gets hard
8. **Best practice in Java**: Enum-based Singleton
9. **Best practice in Python**: `__new__` override or decorator

When to Use

✅ Use When:

- Only ONE instance must exist
- Need global point of access
- Object is expensive to create
- Sharing state (config, cache) across app

❌ Avoid When:

- Multiple instances are inherently needed
- Testing demands isolated instances
- Global state hurts maintainability

Final Note

"Singleton is easy to implement but easy to misuse. Use it when you truly need exactly one instance — not as a global variable workaround."

Singleton is one of the **simplest** and **most-used** design patterns. It appears in almost every real-world application — logging, database, configuration. But like all patterns, apply it **thoughtfully**, not reflexively.



हां जी, करें स्टार्ट। सो, वेलकम ...
TXT 188.28KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also with explanation

LLD Case Study: Food Delivery App (Swiggy/Zomato Clone) — Complete Notes

Part 1: Introduction — The LLD Interview Approach

What This Session Simulates

A complete LLD interview for designing a **Food Delivery Application** (like Zomato/Swiggy), which we'll call **Tomato**.

The Interview Flow

text

- | | |
|---------------------------|-------------------------------|
| 1. Problem Statement | → Understand what to build |
| 2. Requirements Gathering | → Functional & Non-Functional |
| 3. Happy Flow Discussion | → Step-by-step user journey |
| 4. Counter-Questioning | → Narrow down scope |
| 5. UML Diagram | → Class design |
| 6. Code Implementation | → Working code |

What Interviewers Actually Evaluate

Aspect	What They Look For
Thinking Process	Can you think through a full flow?
UML Design	Can you describe classes & relationships?
Clean Code	OOP, SOLID, Design Patterns
Object Relationships	How objects interact
Working Code	Structure that runs end-to-end

Key Insight: No one expects a **complete working application** in an hour. They expect a well-structured, extensible design.

Part 2: Requirements Gathering

Functional vs Non-Functional Requirements

Type	Meaning	Examples
Functional	Business logic, product behavior	Search restaurants, add to cart, checkout
Non-Functional	Quality attributes	Scalability, thread safety, security

Functional Requirements (Discussed with Interviewer)

1. User can search restaurants based on location
2. User can add items to cart
3. User can checkout and make payments
4. User should be notified once order is placed successfully

Non-Functional Requirements

- Scalability
- Extensibility (new payment methods, notification types)
- Loose coupling

Part 3: Happy Flow Discussion

Step-by-Step Flow

text

1. User arrives at platform
2. Sees multiple restaurants
3. Opens a restaurant → sees menu items
4. Adds items to cart
5. Places order (delivery OR pickup)
6. Makes payment
7. Receives notification (order placed)

Counter-Questions to Ask

Question	Purpose
"Is delivery flow the same as pickup?"	Discover two order types
"Should we build payment or assume third-party?"	Narrow scope
"User-side only or delivery-agent side too?"	Scope management
"Notification service — build or integrate?"	Scope management

Interviewer's Answers (simulated):

- Two order types: **Delivery** and **Pickup**
- Payment is a **third-party service** — just integrate it
- Focus on **user-side** only
- Notification is a **third-party service**

Key Learning: *Always narrow the scope — you can't build everything in 1 hour.*

Part 4: Design Approach — Bottom-Up

Top-Down vs Bottom-Up

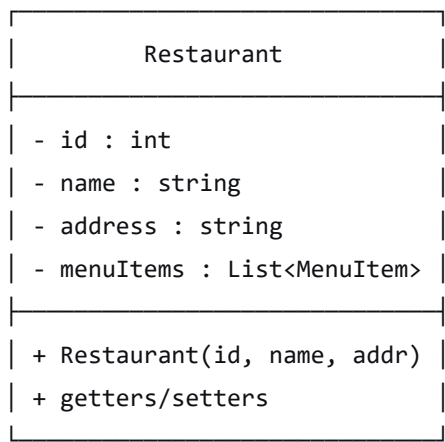
Approach	Description	When to Use
Top-Down	Build the biggest object first, then smaller ones inside	Some specific problems
Bottom-Up	Build smaller objects first, then bigger ones	Most LLD interviews

This case study uses Bottom-Up.

Part 5: UML Design — Class by Class

Entity 1: Restaurant

text

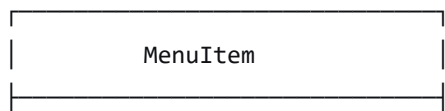


Relationships:

- Restaurant **HAS-A** list of MenuItems → **Composition** (MenuItems can't exist without Restaurant)

Entity 2: MenuItem

text



- code : string	
- name : string	
- price : double	

+ MenuItem(code, name, price)	
+ getters/setters	

Note: Model class — no special methods, just data + getters/setters.

Entity 3: RestaurantManager (Singleton)

text

RestaurantManager	

- restaurants : List<Restaurant>	
- instance : RestaurantManager	

+ getInstance() : RestaurantManager	
+ addRestaurant(r)	
+ searchByLocation(location)	

Why Singleton?

- Only one manager should exist in the app
- Single source of truth for restaurants list

Why Manager class?

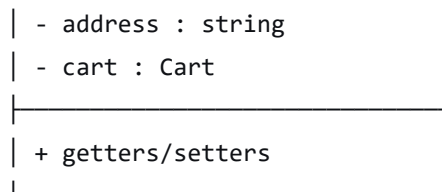
- **Centralized control** over restaurant CRUD
- Client interacts with manager, not directly with restaurants → **loose coupling**

Entity 4: User

text

User	

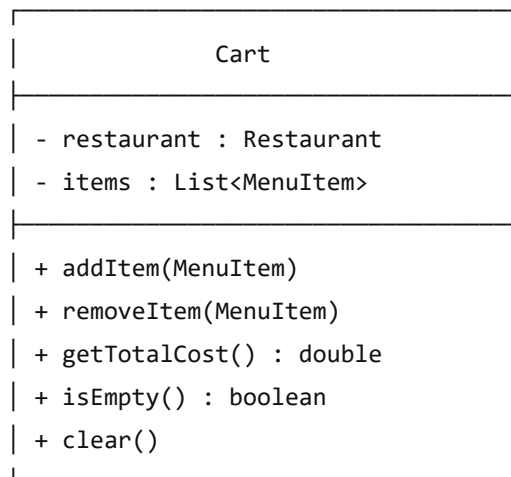
- id : int	
- name : string	



Relationship: User HAS-A Cart → Composition (Cart can't exist without User)

Entity 5: Cart

text

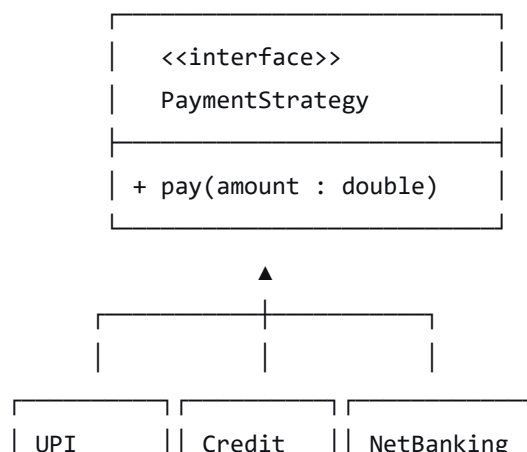


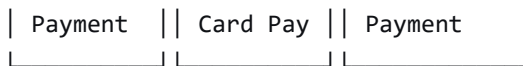
Relationships:

- Cart **HAS-A** Restaurant (Simple association — 1:1)
- Cart **HAS-A** List of MenuItems (Simple association — 1:many)

Entity 6: Payment Strategy (Strategy Pattern)

text



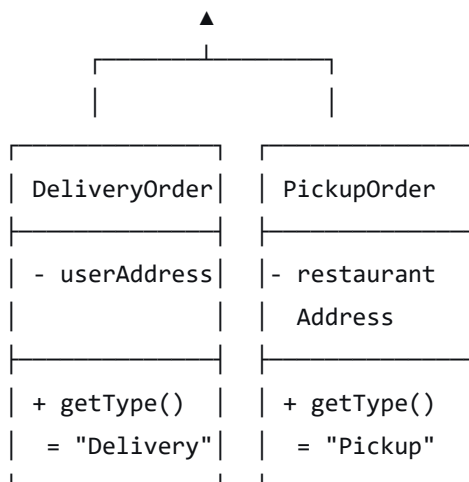
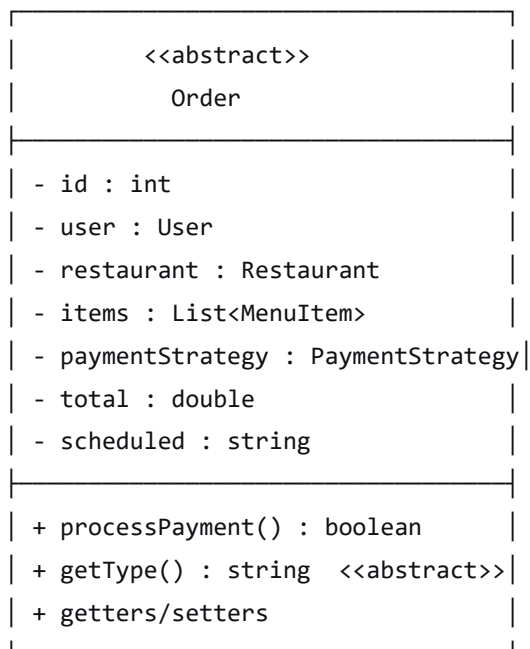


Why Strategy Pattern?

- Multiple payment methods are interchangeable
- Client (Order) doesn't care which strategy is used
- **OCP** — adding new payment method = new strategy class

Entity 7: Order

text

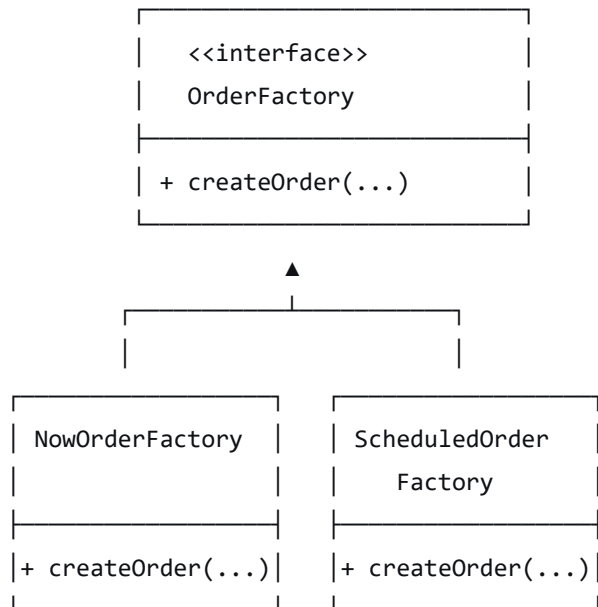


Relationships:

- Order **HAS-A** User, Restaurant, List of MenuItem, PaymentStrategy → **Simple associations**

Entity 8: Order Factories (Factory Method Pattern)

text

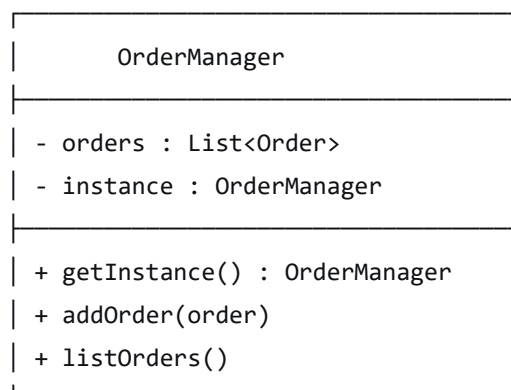


Why Factory Method?

- Order creation logic is complex
- Different factories create **now vs scheduled** orders
- Extensible — add new factory type without changing existing code

Entity 9: OrderManager (Singleton)

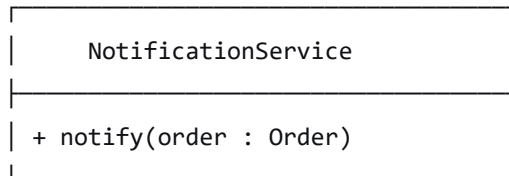
text



Relationship with Order: OrderManager **HAS-A** Order → **Aggregation** (Orders can exist independently)

Entity 10: NotificationService

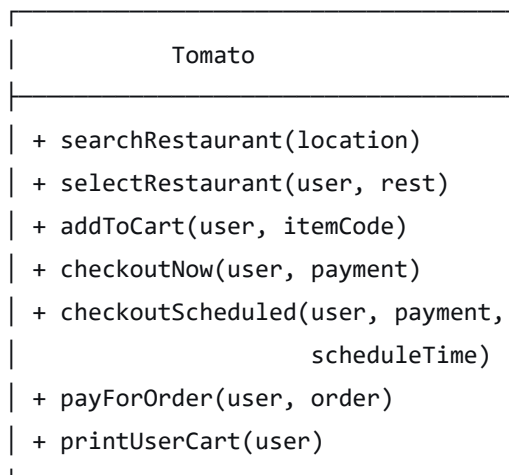
text



Assumption: Third-party service — just structural definition.

Entity 11: Tomato (Orchestrator)

text



Role: Orchestrator / Single point of contact for clients (front-end).

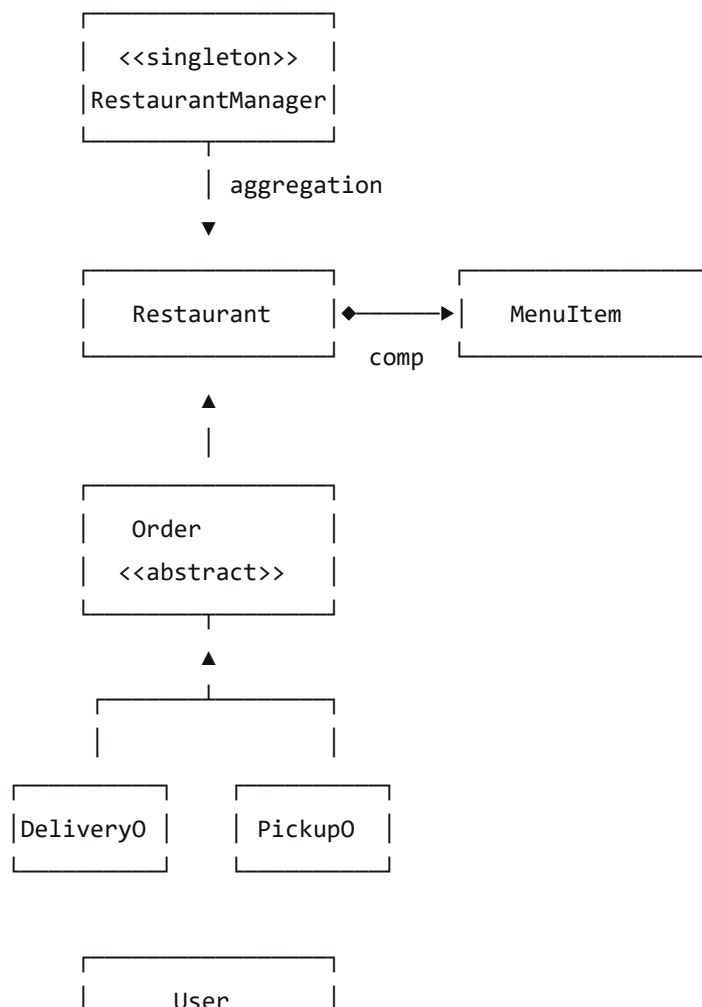
Trade-off: Breaks SRP and Principle of Least Knowledge but provides a clean API for the client. This is a **pragmatic decision**.

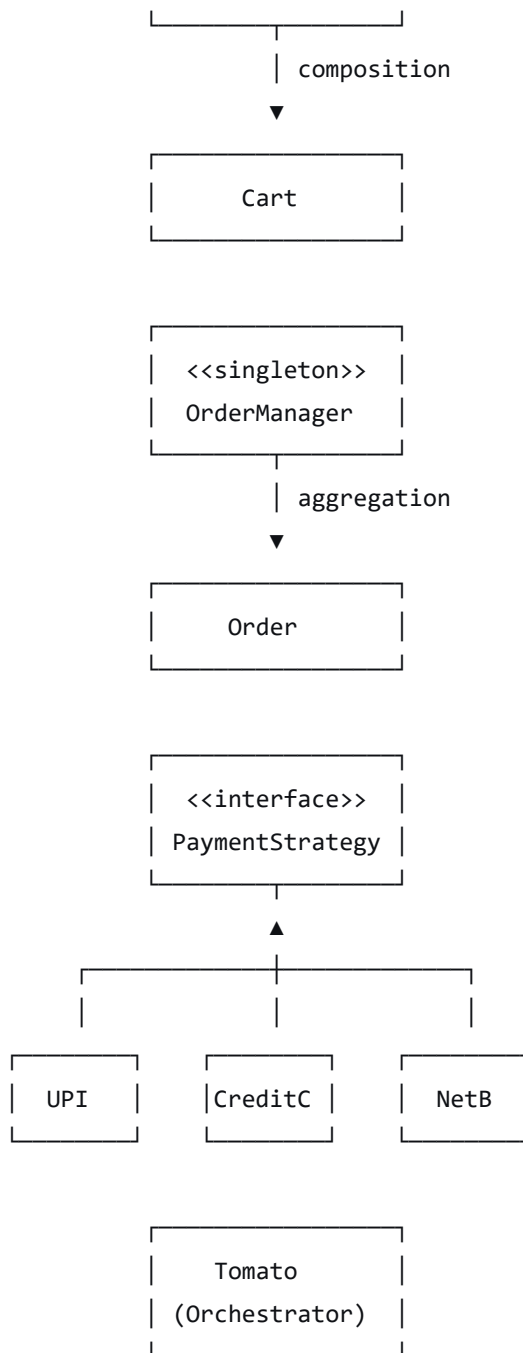
Part 6: Design Patterns Used

Pattern	Where	Why
Singleton	RestaurantManager, OrderManager	Single source of truth
Strategy	PaymentStrategy	Interchangeable payment methods
Factory Method	OrderFactory	Complex order creation logic
Composition	Restaurant→MenuItem, User→Cart	Lifecycle dependency
Aggregation	RestaurantManager→Restaurant, OrderManager→Order	Loose containment

Part 7: Complete Class Diagram (Clean View)

text





Part 8: Java Implementation

Model Classes

java

```
import java.util.*;
```

```
// ===== MENU ITEM =====
```

```
class MenuItem {
    private String code;
```

```

    private String name;
    private double price;

    public MenuItem(String code, String name, double price) {
        this.code = code;
        this.name = name;
        this.price = price;
    }
    public String getCode() { return code; }
    public String getName() { return name; }
    public double getPrice() { return price; }
}

// ===== RESTAURANT =====
class Restaurant {
    private static int nextId = 1;
    private int id;
    private String name;
    private String address;
    private List<MenuItem> menuItems = new ArrayList<>();

    public Restaurant(String name, String address) {
        this.id = nextId++;
        this.name = name;
        this.address = address;
    }

    public void addMenuItem(MenuItem item) { menuItems.add(item); }

    public int getId() { return id; }
    public String getName() { return name; }
    public String getAddress() { return address; }
    public List<MenuItem> getMenuItems() { return menuItems; }

    @Override
    public String toString() {
        return "Restaurant #" + id + " - " + name + " @ " + address;
    }
}

// ===== CART =====
class Cart {
    private Restaurant restaurant;
    private List<MenuItem> items = new ArrayList<>();

    public Restaurant getRestaurant() { return restaurant; }
    public void setRestaurant(Restaurant r) { this.restaurant = r; }
}

```

```

    public void addItem(MenuItem item) {
        if (restaurant == null) throw new RuntimeException("Select a restaurant first");
        items.add(item);
    }
    public void removeItem(MenuItem item) { items.remove(item); }
    public List<MenuItem> getItems() { return items; }
    public boolean isEmpty() { return items.isEmpty(); }
    public void clear() { items.clear(); restaurant = null; }

    public double getTotalCost() {
        double total = 0;
        for (MenuItem i : items) total += i.getPrice();
        return total;
    }
}

// ===== USER =====
class User {
    private int id;
    private String name;
    private String address;
    private Cart cart = new Cart();

    public User(int id, String name, String address) {
        this.id = id;
        this.name = name;
        this.address = address;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public String getAddress() { return address; }
    public Cart getCart() { return cart; }
}

```

Manager Classes (Singleton)

java

```

// ===== RESTAURANT MANAGER =====
class RestaurantManager {
    private static RestaurantManager instance;
    private List<Restaurant> restaurants = new ArrayList<>();

    private RestaurantManager() {}

    public static RestaurantManager getInstance() {
        if (instance == null) {

```



```

        synchronized (RestaurantManager.class) {
            if (instance == null) instance = new RestaurantManager();
        }
    }
    return instance;
}

public void addRestaurant(Restaurant r) { restaurants.add(r); }

public List<Restaurant> searchByLocation(String location) {
    List<Restaurant> result = new ArrayList<>();
    for (Restaurant r : restaurants) {
        if (r.getAddress().equalsIgnoreCase(location)) result.add(r);
    }
    return result;
}
}

// ===== ORDER MANAGER =====
class OrderManager {
    private static OrderManager instance;
    private List<Order> orders = new ArrayList<>();

    private OrderManager() {}

    public static OrderManager getInstance() {
        if (instance == null) {
            synchronized (OrderManager.class) {
                if (instance == null) instance = new OrderManager();
            }
        }
        return instance;
    }

    public void addOrder(Order o) { orders.add(o); }
    public void listOrders() {
        for (Order o : orders)
            System.out.println("Order #" + o.getId() + " Type: " + o.getType());
    }
}
}

```

Strategy Pattern — Payment

java

```

interface PaymentStrategy {
    void pay(double amount);
}

```

```
}
```

```
class UPIPayment implements PaymentStrategy {
    private String mobile;
    public UPIPayment(String mobile) { this.mobile = mobile; }
    public void pay(double amount) {
        System.out.println("Paid ₹" + amount + " using UPI (" + mobile + ")");
    }
}

class CreditCardPayment implements PaymentStrategy {
    private String cardNumber;
    public CreditCardPayment(String cardNumber) { this.cardNumber = cardNumber; }
    public void pay(double amount) {
        System.out.println("Paid ₹" + amount + " using Card (" + cardNumber + ")");
    }
}
```

Order Hierarchy

```
java
```

```
abstract class Order {
    private static int nextId = 1;
    private int id;
    private User user;
    private Restaurant restaurant;
    private List<MenuItem> items;
    private PaymentStrategy paymentStrategy;
    private double total;
    private String scheduled;

    public Order(User user, Restaurant restaurant, List<MenuItem> items,
        PaymentStrategy ps, double total, String scheduled) {
        this.id = nextId++;
        this.user = user;
        this.restaurant = restaurant;
        this.items = items;
        this.paymentStrategy = ps;
        this.total = total;
        this.scheduled = scheduled;
    }

    public int getId() { return id; }
    public User getUser() { return user; }
    public Restaurant getRestaurant() { return restaurant; }
    public List<MenuItem> getItems() { return items; }
```

```

    public double getTotal() { return total; }

    public boolean processPayment() {
        if (paymentStrategy == null) {
            System.out.println("Please choose payment mode first");
            return false;
        }
        paymentStrategy.pay(total);
        return true;
    }

    public abstract String getType();
}

class DeliveryOrder extends Order {
    private String userAddress;
    public DeliveryOrder(User user, Restaurant r, List<MenuItem> items,
        PaymentStrategy ps, double total, String schedule,
        String userAddress) {
        super(user, r, items, ps, total, schedule);
        this.userAddress = userAddress;
    }
    public String getUserAddress() { return userAddress; }
    @Override public String getType() { return "Delivery"; }
}

class PickupOrder extends Order {
    private String restaurantAddress;
    public PickupOrder(User user, Restaurant r, List<MenuItem> items,
        PaymentStrategy ps, double total, String schedule,
        String restaurantAddress) {
        super(user, r, items, ps, total, schedule);
        this.restaurantAddress = restaurantAddress;
    }
    public String getRestaurantAddress() { return restaurantAddress; }
    @Override public String getType() { return "Pickup"; }
}

```

Factory Method — Order Creation

java

```

abstract class OrderFactory {
    public abstract Order createOrder(User user, Cart cart, Restaurant restaurant,
        List<MenuItem> items, PaymentStrategy ps,
        String orderType, double total, String schedule);
}

```

```

class NowOrderFactory extends OrderFactory {
    public Order createOrder(User user, Cart cart, Restaurant restaurant,
                             List<MenuItem> items, PaymentStrategy ps,
                             String orderType, double total, String schedule) {

        Order order;
        if (orderType.equalsIgnoreCase("Delivery")) {
            order = new DeliveryOrder(user, restaurant, items, ps, total,
                                      schedule, user.getAddress());
        } else {
            order = new PickupOrder(user, restaurant, items, ps, total,
                                    schedule, restaurant.getAddress());
        }
        return order;
    }
}

class ScheduledOrderFactory extends OrderFactory {
    private String scheduleTime;
    public ScheduledOrderFactory(String scheduleTime) {
        this.scheduleTime = scheduleTime;
    }
    public Order createOrder(User user, Cart cart, Restaurant restaurant,
                             List<MenuItem> items, PaymentStrategy ps,
                             String orderType, double total, String schedule) {

        Order order;
        if (orderType.equalsIgnoreCase("Delivery")) {
            order = new DeliveryOrder(user, restaurant, items, ps, total,
                                      scheduleTime, user.getAddress());
        } else {
            order = new PickupOrder(user, restaurant, items, ps, total,
                                    scheduleTime, restaurant.getAddress());
        }
        return order;
    }
}

```

Notification Service

java

```

class NotificationService {
    public void notify(Order order) {
        System.out.println("Notification: New " + order.getType()
                           + " order placed! Order ID: " + order.getId());
        System.out.println("User: " + order.getUser().getName());
        System.out.println("Restaurant: " + order.getRestaurant().getName());
    }
}

```

```

        System.out.println("Total: ₹" + order.getTotal());
        System.out.println("Scheduled: " + order.getTotal());
    }
}

```

Tomato (Orchestrator)

java

```

class Tomato {
    private RestaurantManager restaurantManager = RestaurantManager.getInstance();
    private OrderManager orderManager = OrderManager.getInstance();
    private NotificationService notificationService = new NotificationService();

    public List<Restaurant> searchRestaurant(String location) {
        return restaurantManager.searchByLocation(location);
    }

    public void selectRestaurant(User user, Restaurant restaurant) {
        user.getCart().setRestaurant(restaurant);
        System.out.println("Selected restaurant: " + restaurant.getName());
    }

    public void addToCart(User user, String itemCode) {
        Restaurant r = user.getCart().getRestaurant();
        if (r == null) {
            System.out.println("Please select a restaurant first");
            return;
        }
        for (MenuItem item : r.getMenuItems()) {
            if (item.getCode().equals(itemCode)) {
                user.getCart().addItem(item);
                System.out.println("Added " + item.getName() + " to cart");
                return;
            }
        }
        System.out.println("Item not found");
    }

    public Order checkoutNow(User user, PaymentStrategy paymentStrategy, String orderType) {
        Cart cart = user.getCart();
        if (cart.isEmpty()) {
            System.out.println("Cart is empty");
            return null;
        }
        OrderFactory factory = new NowOrderFactory();
    }
}

```

```

    Order order = factory.createOrder(
        user, cart, cart.getRestaurant(), cart.getItems(),
        paymentStrategy, orderType, cart.getTotalCost(),
        java.time.LocalDateTime.now().toString());
    orderManager.addOrder(order);
    return order;
}

public Order checkoutScheduled(User user, PaymentStrategy paymentStrategy,
                               String orderType, String scheduleTime) {
    Cart cart = user.getCart();
    if (cart.isEmpty()) {
        System.out.println("Cart is empty");
        return null;
    }
    OrderFactory factory = new ScheduledOrderFactory(scheduleTime);
    Order order = factory.createOrder(
        user, cart, cart.getRestaurant(), cart.getItems(),
        paymentStrategy, orderType, cart.getTotalCost(), scheduleTime);
    orderManager.addOrder(order);
    return order;
}

public boolean payForOrder(User user, Order order) {
    boolean success = order.processPayment();
    if (success) {
        notificationService.notify(order);
        user.getCart().clear();
    }
    return success;
}

public void printUserCart(User user) {
    Cart c = user.getCart();
    System.out.println("Cart:");
    for (MenuItem item : c.getItems())
        System.out.println("  - " + item.getName() + " : ₹" + item.getPrice());
    System.out.println("Total: ₹" + c.getTotalCost());
}
}

```

Main (Client)

java

```

public class Main {
    public static void main(String[] args) {

```

```

Tomato app = new Tomato();

// Seed data
Restaurant r1 = new Restaurant("Bikaner", "Delhi");
r1.addItem(new MenuItem("M1", "Chole Bhature", 120));
r1.addItem(new MenuItem("M2", "Samosa", 15));
RestaurantManager.getInstance().addRestaurant(r1);

Restaurant r2 = new Restaurant("Domino's", "Mumbai");
r2.addItem(new MenuItem("M3", "Pizza", 300));
RestaurantManager.getInstance().addRestaurant(r2);

// Happy flow
User user = new User(1001, "Aditya", "Delhi");

List<Restaurant> results = app.searchRestaurant("Delhi");
if (results.isEmpty()) {
    System.out.println("No restaurants found");
    return;
}
for (Restaurant r : results) System.out.println(r);

app.selectRestaurant(user, results.get(0));
app.addToCart(user, "M1");
app.addToCart(user, "M2");
app.printUserCart(user);

PaymentStrategy upi = new UPIPayment("9999999999");
Order order = app.checkoutNow(user, upi, "Delivery");

if (order != null) app.payForOrder(user, order);
}
}

```

Output:

text

```

Restaurant #1 - Bikaner @ Delhi
Selected restaurant: Bikaner
Added Chole Bhature to cart
Added Samosa to cart
Cart:
  - Chole Bhature : ₹120.0
  - Samosa : ₹15.0
Total: ₹135.0
Paid ₹135.0 using UPI (9999999999)
Notification: New Delivery order placed! Order ID: 1

```

User: Aditya
Restaurant: Bikaner
Total: ₹135.0

Part 9: Python Implementation

python

```
from abc import ABC, abstractmethod
from datetime import datetime

# ===== MENU ITEM =====
class MenuItem:
    def __init__(self, code, name, price):
        self.code = code
        self.name = name
        self.price = price

# ===== RESTAURANT =====
class Restaurant:
    _next_id = 1
    def __init__(self, name, address):
        self.id = Restaurant._next_id
        Restaurant._next_id += 1
        self.name = name
        self.address = address
        self.menu_items = []

    def add_menu_item(self, item):
        self.menu_items.append(item)

    def __repr__(self):
        return f"Restaurant #{self.id} - {self.name} @ {self.address}"

# ===== CART =====
class Cart:
    def __init__(self):
        self.restaurant = None
        self.items = []

    def add_item(self, item):
        if self.restaurant is None:
```



```

        raise RuntimeError("Select a restaurant first")
    self.items.append(item)

def remove_item(self, item):
    self.items.remove(item)

def total_cost(self):
    return sum(i.price for i in self.items)

def is_empty(self):
    return len(self.items) == 0

def clear(self):
    self.items.clear()
    self.restaurant = None

# ===== USER =====
class User:
    def __init__(self, uid, name, address):
        self.id = uid
        self.name = name
        self.address = address
        self.cart = Cart()

# ===== RESTAURANT MANAGER (SINGLETON) =====
class RestaurantManager:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance.restaurants = []
        return cls._instance

    def add_restaurant(self, r):
        self.restaurants.append(r)

    def search_by_location(self, location):
        return [r for r in self.restaurants
                if r.address.lower() == location.lower()]

# ===== ORDER MANAGER (SINGLETON) =====
class OrderManager:
    _instance = None
    def __new__(cls):
        if cls._instance is None:

```

```

        cls._instance = super().__new__(cls)
        cls._instance.orders = []
    return cls._instance

def add_order(self, o):
    self.orders.append(o)

# ===== PAYMENT STRATEGY =====
class PaymentStrategy(ABC):
    @abstractmethod
    def pay(self, amount): pass

class UPIPayment(PaymentStrategy):
    def __init__(self, mobile):
        self.mobile = mobile
    def pay(self, amount):
        print(f"Paid ₹{amount} using UPI ({self.mobile})")

class CreditCardPayment(PaymentStrategy):
    def __init__(self, card):
        self.card = card
    def pay(self, amount):
        print(f"Paid ₹{amount} using Card ({self.card})")

# ===== ORDER HIERARCHY =====
class Order(ABC):
    _next_id = 1
    def __init__(self, user, restaurant, items, payment_strategy, total, scheduled):
        self.id = Order._next_id
        Order._next_id += 1
        self.user = user
        self.restaurant = restaurant
        self.items = items
        self.payment_strategy = payment_strategy
        self.total = total
        self.scheduled = scheduled

    def process_payment(self):
        if self.payment_strategy is None:
            print("Please choose payment mode first")
            return False
        self.payment_strategy.pay(self.total)
        return True

```

```

@abstractmethod
def get_type(self): pass

class DeliveryOrder(Order):
    def __init__(self, user, restaurant, items, ps, total, schedule, user_address):
        super().__init__(user, restaurant, items, ps, total, schedule)
        self.user_address = user_address
    def get_type(self): return "Delivery"

class PickupOrder(Order):
    def __init__(self, user, restaurant, items, ps, total, schedule, restaurant_address):
        super().__init__(user, restaurant, items, ps, total, schedule)
        self.restaurant_address = restaurant_address
    def get_type(self): return "Pickup"

# ===== ORDER FACTORIES =====
class OrderFactory(ABC):
    @abstractmethod
    def create_order(self, user, cart, restaurant, items, ps, order_type,
                    total, schedule): pass

class NowOrderFactory(OrderFactory):
    def create_order(self, user, cart, restaurant, items, ps, order_type,
                    total, schedule):
        if order_type.lower() == "delivery":
            return DeliveryOrder(user, restaurant, items, ps, total,
                                datetime.now().isoformat(), user.address)
        return PickupOrder(user, restaurant, items, ps, total,
                            datetime.now().isoformat(), restaurant.address)

class ScheduledOrderFactory(OrderFactory):
    def __init__(self, schedule_time):
        self.schedule_time = schedule_time

    def create_order(self, user, cart, restaurant, items, ps, order_type,
                    total, schedule):
        if order_type.lower() == "delivery":
            return DeliveryOrder(user, restaurant, items, ps, total,
                                self.schedule_time, user.address)
        return PickupOrder(user, restaurant, items, ps, total,
                            self.schedule_time, restaurant.address)

```

```
# ===== NOTIFICATION =====
```

```
class NotificationService:
```

```
    def notify(self, order):
```

```
        print(f"Notification: New {order.get_type()} order placed! "
              f"Order ID: {order.id}")
        print(f"User: {order.user.name}")
        print(f"Restaurant: {order.restaurant.name}")
        print(f"Total: ₹{order.total}")
```

```
# ===== TOMATO (ORCHESTRATOR) =====
```

```
class Tomato:
```

```
    def __init__(self):
```

```
        self.restaurant_manager = RestaurantManager()
        self.order_manager = OrderManager()
        self.notification_service = NotificationService()
```

```
    def search_restaurant(self, location):
```

```
        return self.restaurant_manager.search_by_location(location)
```

```
    def select_restaurant(self, user, restaurant):
```

```
        user.cart.restaurant = restaurant
        print(f"Selected restaurant: {restaurant.name}")
```

```
    def add_to_cart(self, user, item_code):
```

```
        r = user.cart.restaurant
        if r is None:
            print("Please select a restaurant first")
            return
        for item in r.menu_items:
            if item.code == item_code:
                user.cart.add_item(item)
                print(f"Added {item.name} to cart")
                return
        print("Item not found")
```

```
    def checkout_now(self, user, payment_strategy, order_type):
```

```
        cart = user.cart
        if cart.is_empty():
            print("Cart is empty")
            return None
        factory = NowOrderFactory()
        order = factory.create_order(user, cart, cart.restaurant, cart.items,
                                     payment_strategy, order_type,
                                     cart.total_cost(), "")
        self.order_manager.add_order(order)
        return order
```

```

def checkout_scheduled(self, user, payment_strategy, order_type, schedule):
    cart = user.cart
    if cart.is_empty():
        print("Cart is empty")
        return None
    factory = ScheduledOrderFactory(schedule)
    order = factory.create_order(user, cart, cart.restaurant, cart.items,
                                payment_strategy, order_type,
                                cart.total_cost(), schedule)
    self.order_manager.add_order(order)
    return order

def pay_for_order(self, user, order):
    success = order.process_payment()
    if success:
        self.notification_service.notify(order)
        user.cart.clear()
    return success

def print_user_cart(self, user):
    print("Cart:")
    for item in user.cart.items:
        print(f" - {item.name} : ₹{item.price}")
    print(f"Total: ₹{user.cart.total_cost()}")

# ===== MAIN =====
if __name__ == "__main__":
    app = Tomato()

    # Seed data
    r1 = Restaurant("Bikaner", "Delhi")
    r1.add_menu_item(MenuItem("M1", "Chole Bhature", 120))
    r1.add_menu_item(MenuItem("M2", "Samosa", 15))
    RestaurantManager().add_restaurant(r1)

    r2 = Restaurant("Dominos", "Mumbai")
    r2.add_menu_item(MenuItem("M3", "Pizza", 300))
    RestaurantManager().add_restaurant(r2)

    # Happy flow
    user = User(1001, "Aditya", "Delhi")

    results = app.search_restaurant("Delhi")
    if not results:
        print("No restaurants found")
    else:
        for r in results:

```

```
print(r)

app.select_restaurant(user, results[0])
app.add_to_cart(user, "M1")
app.add_to_cart(user, "M2")
app.print_user_cart(user)

upi = UPIPayment("9999999999")
order = app.checkout_now(user, upi, "Delivery")
if order:
    app.pay_for_order(user, order)
```

Output:

text

```
Restaurant #1 - Bikaner @ Delhi
Selected restaurant: Bikaner
Added Chole Bhature to cart
Added Samosa to cart
Cart:
  - Chole Bhature : ₹120
  - Samosa : ₹15
Total: ₹135
Paid ₹135 using UPI (9999999999)
Notification: New Delivery order placed! Order ID: 1
User: Aditya
Restaurant: Bikaner
Total: ₹135
```

Part 10: Key Design Decisions & Trade-offs

1. Bottom-Up Approach

Built smaller objects first (MenuItem → Restaurant → Cart → User → Order) then composed them.

2. Managers as Singletons

- Single source of truth
- Central point of contact
- **Trade-off:** Singleton is hard to test, but for managers it's acceptable

3. Tomato as Orchestrator

- **Breaks SRP** (knows about all classes)
- **Breaks Principle of Least Knowledge** (touches nested objects like `user.getCart().getRestaurant()`)
- **BUT** provides a clean single point of contact for clients
- **Trade-off accepted** because real applications need such facades

4. Factory Method for Orders

- Separate **creation logic** from **business logic**
- Two factories: Now and Scheduled
- Extensible for future order types (e.g., Group orders)

5. Strategy Pattern for Payments

- Interchangeable payment methods
- Open/Closed Principle satisfied

6. Composition vs Aggregation

- **Composition:** Restaurant→MenuItem, User→Cart (lifecycle-dependent)
- **Aggregation:** RestaurantManager→Restaurant, OrderManager→Order (independent existence)

Part 11: Possible Extensions (Discussion)

Extension 1: Payment Strategy Factory

Instead of client choosing payment strategy, introduce a **PaymentStrategyFactory**:

java

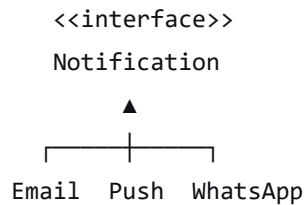
```
class PaymentStrategyFactory {  
    public static PaymentStrategy create(String type) {  
        if (type.equals("upi")) return new UPIPayment(...);  
        if (type.equals("card")) return new CreditCardPayment(...);  
        return null;  
    }  
}
```

```
}  
}
```

Extension 2: Notification Hierarchy

Instead of one `NotificationService` , use hierarchy:

text



Extension 3: API Layer Separation

Split Tomato into:

- **API/Controller Layer** — handles HTTP requests
- **Service Layer** — business logic
- **Repository Layer** — data access

This is the **modern framework approach** (Spring Boot, Django).

Extension 4: Delivery Agent Side

Add delivery agent flow with:

- Order assignment
- Location tracking
- Delivery status updates

Part 12: Master Summary

Entities Built

Entity	Type	Patterns Used
MenuItem	Model	—
Restaurant	Model	Composition (with MenuItem)
RestaurantManager	Manager	Singleton
Cart	Model	Composition (with User)
User	Model	—
Order	Abstract	—
DeliveryOrder, PickupOrder	Concrete	Inheritance
PaymentStrategy	Interface	Strategy
UPI, Card, NetBanking	Concrete	Strategy implementations
OrderFactory	Abstract	Factory Method
NowOrderFactory, ScheduledOrderFactory	Concrete	Factory Method
OrderManager	Manager	Singleton
NotificationService	Service	—
Tomato	Orchestrator	Facade (trade-off)

Design Principles Applied

Principle	How Applied
SRP	Each model/manager has one role (Tomato is intentional exception)
OCP	New payment methods, order factories without modifying existing code
LSP	DeliveryOrder & PickupOrder fully substitutable as Order

Principle	How Applied
ISP	Small interfaces (PaymentStrategy, OrderFactory)
DIP	Order depends on PaymentStrategy abstraction
Composition > Inheritance	Used heavily for relationships
DRY	Factory pattern eliminates duplicate order creation code

Design Patterns Used

1. **Singleton** — RestaurantManager, OrderManager
2. **Strategy** — PaymentStrategy
3. **Factory Method** — OrderFactory
4. **Composition/Aggregation** — Entity relationships

Key Takeaways

1. **LLD interviews are conversations** — ask, discuss, refine
2. **Scope narrows through counter-questioning**
3. **Bottom-Up approach** works for most problems
4. **Managers centralize access** — reduces coupling
5. **Singletons for managers** — single source of truth
6. **Strategy for payments** — interchangeable algorithms
7. **Factory Method for orders** — separated creation logic
8. **Orchestrator (Tomato) is a pragmatic trade-off** — breaks SRP for clean client API
9. **UML first, code later** — saves time in interviews
10. **Design patterns emerge from need**, not forced
11. **Subjectivity is fine** — composition vs aggregation depends on context
12. **Real-world apps are modular** — separate models, managers, factories, strategies, services

Final Note

This case study demonstrates **the entire LLD interview process** for a complex real-world application. The key is to:

- **Think incrementally** — build entities one by one
- **Discuss trade-offs** — especially around SRP violations and design decisions
- **Apply patterns organically** — only when they solve a problem
- **Write clean, extensible code** — that could actually run

The final design is **not perfect** but is **pragmatic, extensible, and interview-ready**.



हां जी करें स्टार्ट। सो वेलकम बै...
TXT 78.26KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also with explanation

Observer Design Pattern — Complete Notes

Part 1: Introduction — What Problem Does It Solve?

Real-World Analogy: YouTube Subscriptions

When you **subscribe** to a YouTube channel and it uploads a new video, **you get notified**. How does this work?

- YouTube channel = **Observable** (Subject)
- Your account = **Observer**
- Subscription = **Registration**
- New video upload = **State change**
- Notification = **Update**

The Core Problem

When one object changes state, multiple other objects need to be notified automatically.

Polling vs Pushing

Approach	How It Works	Problem
Polling	Observer repeatedly asks "did you change?"	Wasteful, timing issues
Pushing	Observable notifies observers when changed	Efficient — Observer Pattern

Example of Polling (Bad):

- You repeatedly ask YouTube: "Any new video? Any new video? Any new video?"
- Wasteful — most answers are "No"
- What frequency? Every second? Millisecond?

Observer Pattern solves this with Pushing:

- YouTube tells you: "Hey, new video uploaded!"

Part 2: Observer Pattern Definition

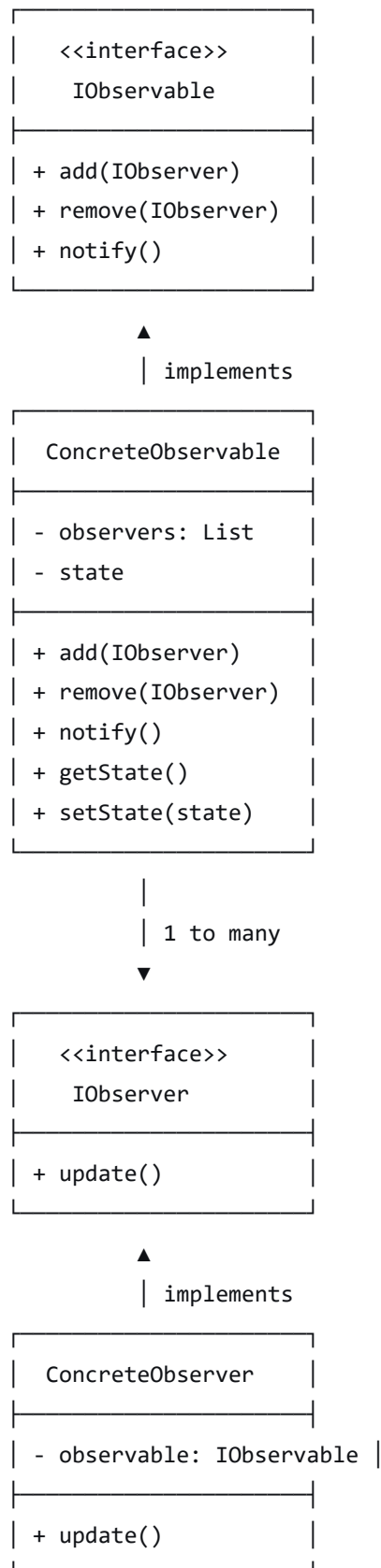
Define a one-to-many relationship between objects so that when one object changes state, all its dependents are notified and updated automatically.

Key Concepts

Term	Role	Example
Observable (Subject)	Object being watched	YouTube channel
Observer	Object watching	Subscriber
subscribe/add	Register an observer	Subscribe button
unsubscribe/remove	Unregister observer	Unsubscribe
notify	Inform all observers	Send notification
update	Observer's response	Read new video

Part 3: UML Diagram

text

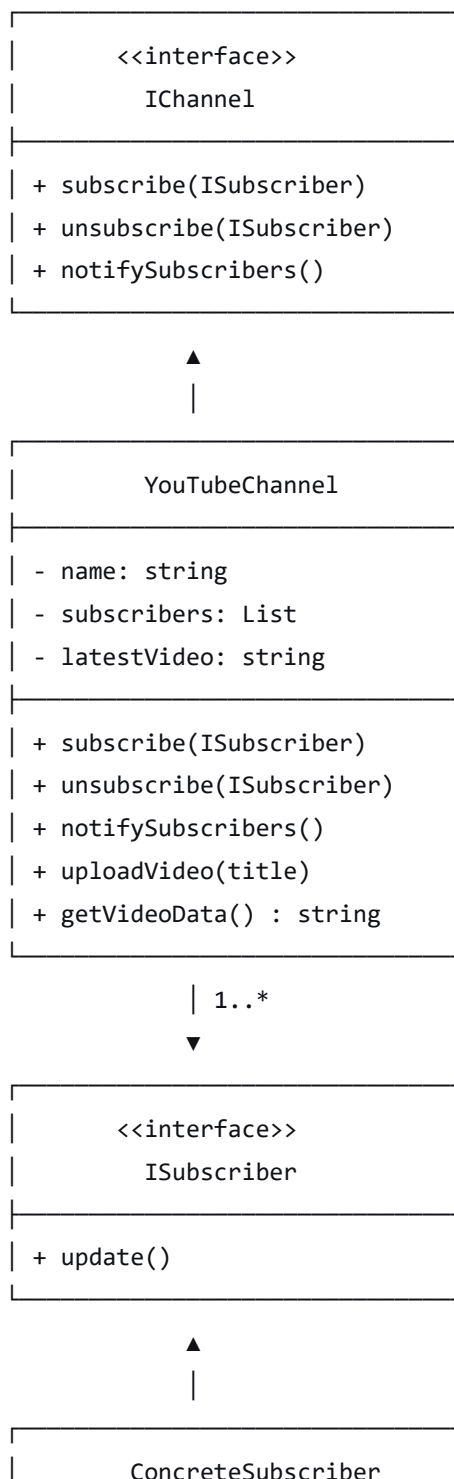


Key Relationship: ConcreteObserver **HAS-A** ConcreteObservable (so it can read state when notified).

Note on Naming Convention: In UML, pure abstract classes (interfaces) are often prefixed with I (e.g., Iobservable , Iobserver).

Part 4: YouTube Example — Class Design

text



- name: string	
- channel: IChannel	
+ update()	

Part 5: Java Implementation

java

```
import java.util.ArrayList;
import java.util.List;

// ===== OBSERVER INTERFACE =====
interface ISubscriber {
    void update();
}

// ===== OBSERVABLE INTERFACE =====
interface IChannel {
    void subscribe(ISubscriber subscriber);
    void unsubscribe(ISubscriber subscriber);
    void notifySubscribers();
}

// ===== CONCRETE OBSERVABLE =====
class YouTubeChannel implements IChannel {
    private String name;
    private List<ISubscriber> subscribers = new ArrayList<>();
    private String latestVideo;

    public YouTubeChannel(String name) {
        this.name = name;
    }

    @Override
    public void subscribe(ISubscriber subscriber) {
        if (!subscribers.contains(subscriber)) {
            subscribers.add(subscriber);
            System.out.println(subscriber + " subscribed to " + name);
        }
    }

    @Override
```

```
public void unsubscribe(ISubscriber subscriber) {
    subscribers.remove(subscriber);
    System.out.println(subscriber + " unsubscribed from " + name);
}

@Override
public void notifySubscribers() {
    for (ISubscriber subscriber : subscribers) {
        subscriber.update();
    }
}

public void uploadVideo(String title) {
    this.latestVideo = title;
    System.out.println(name + " uploaded: " + title);
    notifySubscribers();
}

public String getVideoData() {
    return "Check out our new video: " + latestVideo;
}
}

// ===== CONCRETE OBSERVER =====
class ConcreteSubscriber implements ISubscriber {
    private String name;
    private YouTubeChannel channel;

    public ConcreteSubscriber(String name, YouTubeChannel channel) {
        this.name = name;
        this.channel = channel;
    }

    @Override
    public void update() {
        System.out.println("Hey " + name + ", " + channel.getVideoData());
    }

    @Override
    public String toString() {
        return name;
    }
}

// ===== MAIN =====
public class Main {
    public static void main(String[] args) {
        YouTubeChannel channel = new YouTubeChannel("CodeArmy");
```



```
ConcreteSubscriber varun = new ConcreteSubscriber("Varun", channel);
ConcreteSubscriber tarun = new ConcreteSubscriber("Tarun", channel);

channel.subscribe(varun);
channel.subscribe(tarun);

channel.uploadVideo("Observer Pattern Tutorial");

System.out.println("\n--- Varun unsubscribes ---\n");
channel.unsubscribe(varun);

channel.uploadVideo("Decorator Pattern Tutorial");
    }
}
```

Output:

text

```
Varun subscribed to CodeArmy
Tarun subscribed to CodeArmy
CodeArmy uploaded: Observer Pattern Tutorial
Hey Varun, Check out our new video: Observer Pattern Tutorial
Hey Tarun, Check out our new video: Observer Pattern Tutorial

--- Varun unsubscribes ---

Varun unsubscribed from CodeArmy
CodeArmy uploaded: Decorator Pattern Tutorial
Hey Tarun, Check out our new video: Decorator Pattern Tutorial
```

Code Explanation

Class	Responsibility
ISubscriber	Observer interface with <code>update()</code>
IChannel	Observable interface with <code>subscribe/unsubscribe/notify</code>
YouTubeChannel	Maintains list of subscribers; calls <code>update()</code> on each when new video uploaded

Class	Responsibility
ConcreteSubscriber	Holds reference to channel; reads video data on <code>update()</code>
Main	Client — subscribes, uploads videos, unsubscribes

Key Flow:

1. Subscribers register via `subscribe()`
2. Channel's `uploadVideo()` triggers `notifySubscribers()`
3. Each subscriber's `update()` is called
4. Subscriber reads new data via `channel.getVideoData()`

Part 6: Python Implementation

python

```
from abc import ABC, abstractmethod
from typing import List

# ===== OBSERVER INTERFACE =====
class ISubscriber(ABC):
    @abstractmethod
    def update(self): pass

# ===== OBSERVABLE INTERFACE =====
class IChannel(ABC):
    @abstractmethod
    def subscribe(self, subscriber: ISubscriber): pass

    @abstractmethod
    def unsubscribe(self, subscriber: ISubscriber): pass

    @abstractmethod
    def notify_subscribers(self): pass

# ===== CONCRETE OBSERVABLE =====
class YouTubeChannel(IChannel):
    def __init__(self, name):
```

```

        self.name = name
        self.subscribers: List[ISubscriber] = []
        self.latest_video = None

    def subscribe(self, subscriber):
        if subscriber not in self.subscribers:
            self.subscribers.append(subscriber)
            print(f"{subscriber} subscribed to {self.name}")

    def unsubscribe(self, subscriber):
        if subscriber in self.subscribers:
            self.subscribers.remove(subscriber)
            print(f"{subscriber} unsubscribed from {self.name}")

    def notify_subscribers(self):
        for subscriber in self.subscribers:
            subscriber.update()

    def upload_video(self, title):
        self.latest_video = title
        print(f"{self.name} uploaded: {title}")
        self.notify_subscribers()

    def get_video_data(self):
        return f"Check out our new video: {self.latest_video}"

# ===== CONCRETE OBSERVER =====
class ConcreteSubscriber(ISubscriber):
    def __init__(self, name, channel):
        self.name = name
        self.channel = channel

    def update(self):
        print(f"Hey {self.name}, {self.channel.get_video_data()}")

    def __repr__(self):
        return self.name

# ===== MAIN =====
if __name__ == "__main__":
    channel = YouTubeChannel("CodeArmy")

    varun = ConcreteSubscriber("Varun", channel)
    tarun = ConcreteSubscriber("Tarun", channel)

    channel.subscribe(varun)

```

```
channel.subscribe(tarun)

channel.upload_video("Observer Pattern Tutorial")

print("\n--- Varun unsubscribes ---\n")
channel.unsubscribe(varun)

channel.upload_video("Decorator Pattern Tutorial")
```

Output:

text

```
Varun subscribed to CodeArmy
Tarun subscribed to CodeArmy
CodeArmy uploaded: Observer Pattern Tutorial
Hey Varun, Check out our new video: Observer Pattern Tutorial
Hey Tarun, Check out our new video: Observer Pattern Tutorial

--- Varun unsubscribes ---

Varun unsubscribed from CodeArmy
CodeArmy uploaded: Decorator Pattern Tutorial
Hey Tarun, Check out our new video: Decorator Pattern Tutorial
```

Part 7: Design Decisions & Trade-offs

1. Concrete Observable ↔ Concrete Observer Relationship

Unlike most patterns where we only connect abstract classes, Observer Pattern often has **ConcreteObserver HAS-A ConcreteObservable**.

Why? So the observer can call `observable.getState()` to fetch the new value on update.

2. SRP Violation

`YouTubeChannel` has **two responsibilities**:

1. **Observer pattern logic** — subscribe/unsubscribe/notify
2. **Business logic** — upload video, get video data

Why this is acceptable:

- Observer logic is **stable** — rarely changes
- Business logic is **dynamic** — changes often
- Separating them adds complexity without much benefit
- Google's standard Observer UML also breaks SRP this way

Alternative (if you must follow SRP): Make `IChannel` a **concrete abstract class** with the observer logic in it, and let `YouTubeChannel` only handle business logic. But this is more complex.

3. When to Separate SRP

Follow the principle: *"Extract the changing part from the stable part"*. If your observer logic is stable, keeping it with business logic is a **pragmatic trade-off**.

Part 8: Real-World Use Cases

1. Notification Systems

- New notification arrives → all subscribed users notified

2. Social Media Feeds (Facebook, Instagram)

- New post uploaded → followers get notified
- New comment → post author gets notified

3. Event Handling (Frontend)

- Button click → multiple event listeners triggered
- JavaScript `addEventListener()` is Observer Pattern

4. Stock Market

- Stock price changes → all subscribed traders notified

5. MVC Architecture

- Model changes → View updates automatically

6. Message Queues / Pub-Sub

- Publishers send messages → subscribers receive them

7. Cache Invalidation

- Data updated → all caches invalidated

Part 9: Comparison with Other Patterns

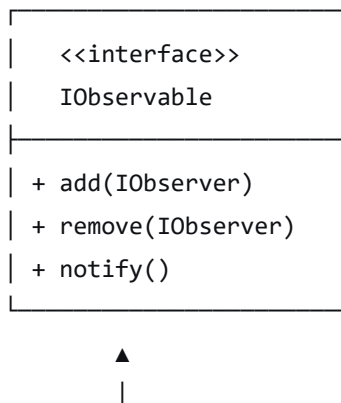
Pattern	Difference
Observer	One-to-many, notify on state change
Strategy	One-to-one, swap algorithms
Factory	Create objects without exposing logic
Pub-Sub	Like Observer, but with a message broker

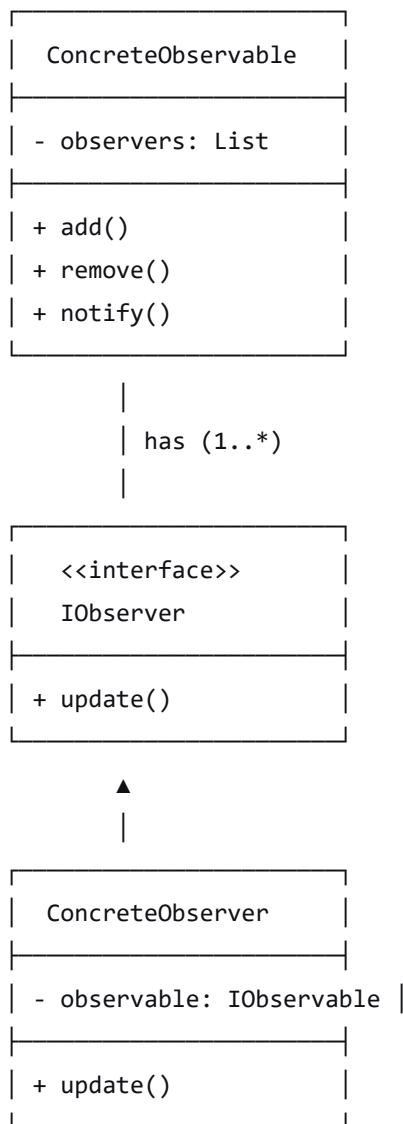
Observer vs Pub-Sub:

- **Observer:** Observable holds direct references to observers
- **Pub-Sub:** Publisher and subscribers don't know each other — a **broker** sits in between

Part 10: Complete UML (Cleaner Version)

text





Part 11: Key Points Summary

Definition

Define a **one-to-many dependency** so that when one object changes state, all its dependents are notified automatically.

Components

- **IObservable / Subject** — interface with subscribe/unsubscribe/notify
- **ConcreteObservable** — implements the interface and holds observer list
- **IObserver** — interface with update method
- **ConcreteObserver** — implements update; holds reference to observable

Benefits

1. **Loose coupling** — Observables don't know concrete observer types
2. **Open/Closed** — Add new observers without modifying observable
3. **Push-based** — Efficient notification instead of wasteful polling
4. **Runtime flexibility** — Subscribe/unsubscribe anytime

Drawbacks

1. **SRP violation** (usually acceptable trade-off)
2. **Notification order** is not guaranteed
3. **Memory leaks** if observers aren't unsubscribed
4. **Cascading updates** can cause performance issues

Design Patterns Tip

- **Observer vs Polling**: Observer uses push, polling uses pull
- **Observer is everywhere**: Event listeners, MVC, pub-sub are all observer-like

Part 12: Exercises / Extensions

Exercise 1: Stock Market Ticker

Design a system where:

- `Stock` is the observable (has price)
- `Trader` is the observer
- When stock price changes, all traders get notified

Exercise 2: Weather Station

Design:

- `WeatherStation` measures temperature, humidity, pressure
- `Display` (mobile app, web app, TV) observes it
- On new measurement, all displays update

Exercise 3: News Feed

Design:

- User (with followers) posts content
- All followers should see new posts
- Unfollows should stop notifications

Exercise 4: Email Notifications

Design:

- New order placed → notify customer, warehouse, delivery partner
- All three are observers of `Order`

Key Takeaways

1. **Observer Pattern** solves the "notify on change" problem elegantly
2. **Push over Pull** — notify automatically instead of polling
3. **One-to-many** relationship between Observable and Observers
4. **Loose coupling** — Subject doesn't know concrete observer types
5. **Four components**: `IObservable`, `ConcreteObservable`, `IObserver`, `ConcreteObserver`
6. **SRP trade-off** — Observable often handles both observer logic and business logic
7. **Real-world** — notification systems, social feeds, event handling, MVC
8. **Python/Java** implementations are nearly identical in structure
9. **Add/Remove/Notify** are the core operations
10. **Update method** in observer fetches state from observable on notification



हां जी करें स्टार्ट। सो वेलकम बै...
TXT 78.46KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also with explanation

Decorator Design Pattern — Complete Notes

Part 1: Introduction — What Problem Does It Solve?

The Core Problem

Add additional responsibilities/functionality to an object dynamically at runtime without using inheritance.

Why Not Just Use Inheritance?

You might think: *"I'll just create child classes to extend functionality."*

But inheritance has a fatal flaw — Class Explosion.

Real-World Example: Mario Game

In Mario, as the game progresses, Mario gets **power-ups**:

- 🍄 Mushroom → Height up
- 🌸 Flower → Gun shooting ability
- ★ Star → Star power (fast + destroy everything)

Now, if we use **inheritance**, we'd need classes for **every combination**:

text

```
Mario
├─ MarioWithHeightUp
├─ MarioWithGun
├─ MarioWithStar
├─ MarioWithHeightUpAndGun
├─ MarioWithHeightUpAndStar
├─ MarioWithGunAndStar
└─ MarioWithHeightUpAndGunAndStar
```

Now add a **Fly power-up**:

text

```
+ MarioWithFly
+ MarioWithFlyAndHeight
+ MarioWithFlyAndGun
+ MarioWithFlyAndStar
+ MarioWithFlyAndHeightAndGun
+ MarioWithFlyAndHeightAndStar
```

- + MarioWithFlyAndGunAndStar
- + MarioWithFlyAndHeightAndGunAndStar

This is **Class Explosion** — every new feature doubles the number of classes.

Decorator Pattern says: Wrap the object with another object that adds functionality — dynamically, not via inheritance.

Part 2: Core Concept — Wrapping Objects

Basic Idea

text

Client → Decorator1 → Decorator2 → BaseObject
 (wraps) (wraps)

Each decorator:

- 1. **Has-a** base object (composition)
- 2. **Is-a** base object (inheritance — to behave like one)
- 3. **Delegates** calls to base object, then **enhances** the result

The Two Relationships

Decorator has **two relationships** with the base class:

Relationship	Purpose
IS-A (inheritance)	So decorator can be used wherever base is expected
HAS-A (composition)	So decorator can call the base object's methods

This is a unique pattern where **both IS-A and HAS-A appear together**.

Example: "Do Something"

text

Original:

BaseObject.doSomething() → "I did something"

Wrapped with Decorator1:

Decorator1.doSomething()

→ calls BaseObject.doSomething() → "I did something"

→ appends " am amazing" → "I did something am amazing"

Wrapped with Decorator1 + Decorator2:

Decorator2.doSomething()

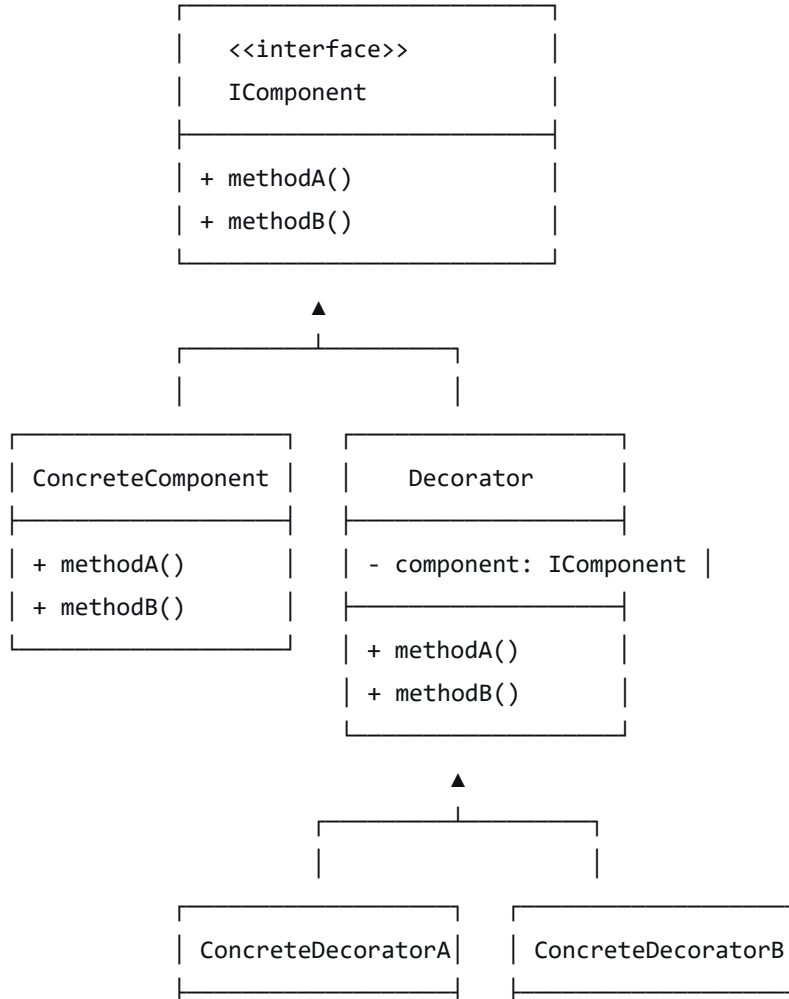
→ calls Decorator1.doSomething() → "I did something am amazing"

→ appends " today" → "I did something am amazing today"

Key Insight: This is **recursion** — the call chain goes down to the base object, then flows back up with each layer adding something.

Part 3: UML Diagram

text



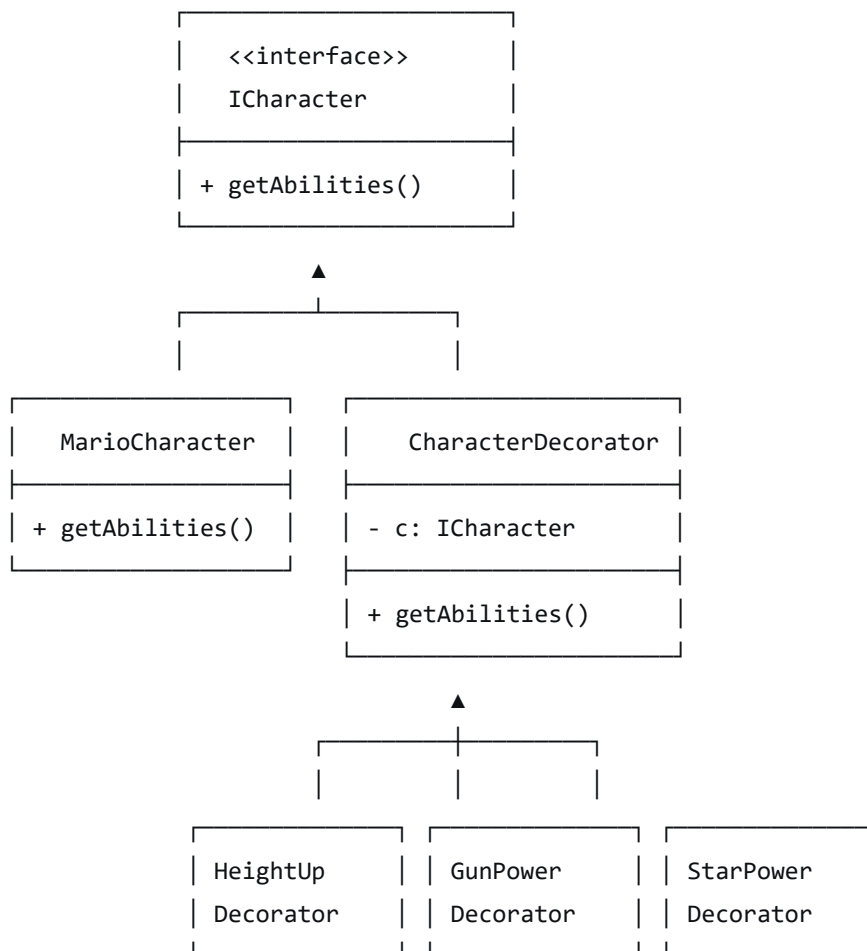


Relationships:

- ConcreteComponent **IS-A** IComponent
- Decorator **IS-A** IComponent (inheritance)
- Decorator **HAS-A** IComponent (composition)
- ConcreteDecorator **IS-A** Decorator

Part 4: Mario Example — UML

text



Part 5: Java Implementation

java

```
// ===== COMPONENT INTERFACE =====
interface ICharacter {
    String getAbilities();
}

// ===== CONCRETE COMPONENT =====
class MarioCharacter implements ICharacter {
    @Override
    public String getAbilities() {
        return "Mario";
    }
}

// ===== ABSTRACT DECORATOR =====
abstract class CharacterDecorator implements ICharacter {
    protected ICharacter character; // HAS-A relationship

    public CharacterDecorator(ICharacter character) {
        this.character = character;
    }
}

// ===== CONCRETE DECORATORS =====
class HeightUpDecorator extends CharacterDecorator {
    public HeightUpDecorator(ICharacter character) {
        super(character);
    }

    @Override
    public String getAbilities() {
        return character.getAbilities() + " with Height Up";
    }
}

class GunPowerDecorator extends CharacterDecorator {
    public GunPowerDecorator(ICharacter character) {
        super(character);
    }

    @Override
    public String getAbilities() {
        return character.getAbilities() + " with Gun";
    }
}

class StarPowerDecorator extends CharacterDecorator {
```

```
public StarPowerDecorator(ICharacter character) {
    super(character);
}

@Override
public String getAbilities() {
    return character.getAbilities() + " with Star Power (Limited Time)";
}
}

// ===== MAIN =====
public class Main {
    public static void main(String[] args) {
        // Base Mario
        ICharacter mario = new MarioCharacter();
        System.out.println(mario.getAbilities());
        // Output: Mario

        // Mario with Height Up
        ICharacter heightMario = new HeightUpDecorator(new MarioCharacter());
        System.out.println(heightMario.getAbilities());
        // Output: Mario with Height Up

        // Mario with Height + Gun
        ICharacter gunMario = new GunPowerDecorator(
            new HeightUpDecorator(new MarioCharacter()));
        System.out.println(gunMario.getAbilities());
        // Output: Mario with Height Up with Gun

        // Mario with Height + Gun + Star
        ICharacter starMario = new StarPowerDecorator(
            new GunPowerDecorator(
                new HeightUpDecorator(
                    new MarioCharacter()))));
        System.out.println(starMario.getAbilities());
        // Output: Mario with Height Up with Gun with Star Power (Limited Time)
    }
}
```

Code Explanation

Class	Role
ICharacter	Component interface with <code>getAbilities()</code>
MarioCharacter	Concrete component — base Mario

Class	Role
CharacterDecorator	Abstract decorator with HAS-A ICharacter
HeightUpDecorator	Adds "with Height Up" to abilities
GunPowerDecorator	Adds "with Gun"
StarPowerDecorator	Adds "with Star Power"

Key Flow:

1. Each decorator wraps another ICharacter
2. getAbilities() calls the wrapped character's method
3. Then appends its own ability
4. Chain flows down to base, then back up adding strings

Part 6: Python Implementation

python

```

from abc import ABC, abstractmethod

# ===== COMPONENT INTERFACE =====
class ICharacter(ABC):
    @abstractmethod
    def get_abilities(self) -> str: pass

# ===== CONCRETE COMPONENT =====
class MarioCharacter(ICharacter):
    def get_abilities(self):
        return "Mario"

# ===== ABSTRACT DECORATOR =====
class CharacterDecorator(ICharacter):
    def __init__(self, character: ICharacter):
        self._character = character # HAS-A relationship

    def get_abilities(self):
        return self._character.get_abilities()

```



```
# ===== CONCRETE DECORATORS =====

class HeightUpDecorator(CharacterDecorator):
    def get_abilities(self):
        return self._character.get_abilities() + " with Height Up"

class GunPowerDecorator(CharacterDecorator):
    def get_abilities(self):
        return self._character.get_abilities() + " with Gun"

class StarPowerDecorator(CharacterDecorator):
    def get_abilities(self):
        return self._character.get_abilities() + " with Star Power (Limited Time)"

# ===== MAIN =====
if __name__ == "__main__":
    # Base Mario
    mario = MarioCharacter()
    print(mario.get_abilities())
    # Output: Mario

    # Mario with Height Up
    height_mario = HeightUpDecorator(MarioCharacter())
    print(height_mario.get_abilities())
    # Output: Mario with Height Up

    # Mario with Height + Gun
    gun_mario = GunPowerDecorator(HeightUpDecorator(MarioCharacter()))
    print(gun_mario.get_abilities())
    # Output: Mario with Height Up with Gun

    # Mario with Height + Gun + Star
    star_mario = StarPowerDecorator(
        GunPowerDecorator(
            HeightUpDecorator(
                MarioCharacter()
            )
        )
    )
    print(star_mario.get_abilities())
    # Output: Mario with Height Up with Gun with Star Power (Limited Time)
```

Output:

text

Mario
Mario with Height Up
Mario with Height Up with Gun
Mario with Height Up with Gun with Star Power (Limited Time)

Part 7: Definition

Decorator Pattern attaches additional responsibilities to an object dynamically.
Decorators provide a flexible alternative to subclassing for extending functionality.

Key Points from Definition

1. **Dynamically** — at runtime, not compile time
2. **Attaches responsibilities** — adds behavior to object
3. **Flexible alternative to subclassing** — avoids class explosion

Part 8: Why Both IS-A and HAS-A?

Relationship	Why Needed
IS-A (inheritance)	So the decorator can be used wherever the base component is expected. Client treats decorator as a component.
HAS-A (composition)	So the decorator can hold a reference to the wrapped object and call its methods.

Both are essential:

- IS-A lets the decorator fit into a component's place
- HAS-A lets the decorator **delegate** and **enhance**

Recursion Effect: When you stack decorators, the call flows down to the base and then back up — this is like recursion through different classes.

Part 9: Runtime Flexibility — Unwrapping

You can build the chain in any order:

```
java

// Order 1: Height → Gun → Star
ICharacter mario1 = new StarPowerDecorator(
    new GunPowerDecorator(
        new HeightUpDecorator(new MarioCharacter())));

// Order 2: Gun → Star → Height
ICharacter mario2 = new HeightUpDecorator(
    new StarPowerDecorator(
        new GunPowerDecorator(new MarioCharacter())));

// Order 3: Star → Height → Gun
ICharacter mario3 = new GunPowerDecorator(
    new HeightUpDecorator(
        new StarPowerDecorator(new MarioCharacter())));
```

No class explosion. You get infinite combinations from just **4 classes** (Mario + 3 decorators).

Adding a new decorator (FlyPowerDecorator):

- Just create 1 new class
- Works with **all existing combinations**
- vs. inheritance where you'd need 2^n classes

Part 10: Real-World Use Cases

1. Text Editor (Google Docs)

- Base: Text
- Decorators: BoldDecorator , ItalicDecorator , UnderlineDecorator , FontSizeDecorator
- Stack them to format text

2. Form Validation

- Base: Form

- Decorators: EmailValidator , SQLInjectionValidator , CSSInjectionValidator
- Chain validators

3. Java I/O Streams (Real Example!)

java

```
BufferedReader reader = new BufferedReader(  
    new InputStreamReader(  
        new FileInputStream("file.txt")));
```

Each wrapper adds functionality:

- FileInputStream — reads bytes
- InputStreamReader — bytes → chars
- BufferedReader — adds buffering

4. Pizza Ordering

- Base: Pizza
- Decorators: CheeseDecorator , PepperoniDecorator , MushroomDecorator

5. Coffee Shop

- Base: Espresso
- Decorators: MilkDecorator , SugarDecorator , WhipDecorator

6. HTTP Request Processing

- Base: HttpRequest
- Decorators: AuthDecorator , LoggingDecorator , RateLimitDecorator

Part 11: Comparison with Other Patterns

Pattern	Purpose
Decorator	Add responsibilities dynamically

Pattern	Purpose
Strategy	Swap algorithms at runtime
Factory	Create objects
Observer	Notify on state change
Adapter	Change interface

Decorator vs Inheritance:

- Inheritance = compile-time, static, class explosion
- Decorator = runtime, dynamic, no explosion

Part 12: Key Takeaways

1. **Decorator Pattern** = Attach responsibilities dynamically without inheritance
2. **Avoids class explosion** — one new decorator vs 2^n subclasses
3. **Uses both IS-A and HAS-A relationships**
4. **Recursive-like call chain** — call flows down and up
5. **Runtime flexible** — stack decorators in any order
6. **Open/Closed Principle** — new decorators don't modify existing code
7. **Alternative to subclassing** — for extending functionality
8. **Real examples:** Text editor, form validation, Java I/O, pizza/coffee ordering
9. **Recursive delegation** — the call chain acts like recursion

Part 13: When to Use Decorator Pattern

✅ Use When:

- Need to add responsibilities dynamically
- Subclassing leads to class explosion
- Need to combine features in any order
- Want to add/remove features at runtime

- Feature combinations are multiplicative

✖ Avoid When:

- Only one or two variations (inheritance is fine)
- Order doesn't matter and combinations are few
- Performance is critical (wrappers add overhead)

Part 14: Summary Table

Aspect	Detail
Intent	Add responsibilities dynamically
Type	Structural pattern
Key Relationship	IS-A + HAS-A
Alternative to	Subclassing
Benefits	Flexible, avoids class explosion, runtime extensible
Drawbacks	Many small classes, harder to debug (deep chains)
Real Usage	Java I/O, GUI components, validation, text formatting

Final Note

"Decorator Pattern wraps objects with additional behavior — like wrapping a gift, each layer adds something but the inner gift remains the same."

Decorator is one of the most elegant patterns because it cleverly combines **inheritance (IS-A)** with **composition (HAS-A)** to achieve runtime flexibility without the explosion of subclasses. It's the pattern behind Java's powerful I/O stream API and many UI frameworks.



हां जी करें स्टार्ट। सो वेलकम बै...
TXT 118.49KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also

LLD Case Study: Notification System — Complete Notes

Part 1: Problem Statement

Design a **Notification Service** that:

1. Is a **plug-and-play** model — integrate with minimal code changes
2. Is **highly extensible** — easily add new notification types (SMS, Email, Popup, WhatsApp, ...)
3. Supports **dynamic enhancement** of notifications (headers, footers, signatures, timestamps)
4. **Stores** all notifications (history)
5. **Logs** all notifications

This is a **classic LLD interview problem** that combines multiple design patterns.

Part 2: Requirements

Functional Requirements

- Send notifications via **SMS, Email, Popup** (extensible to more)
- Dynamically enhance notification content (decorators)
- Store all notifications (history)
- Log notifications

Non-Functional Requirements

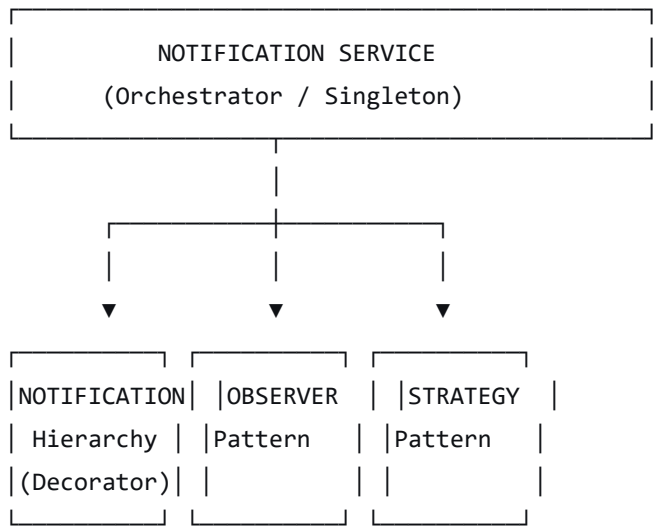
- **Plug-and-play** — minimal integration code
- **Extensible** — add new notification types without modifying existing code
- **Follows SOLID principles**

Part 3: Design Approach — Top-Down

We use **Top-Down**: first build the smaller components (Notification hierarchy, Observer components, Strategy components), then build the big `NotificationService` .

Three Major Components

text



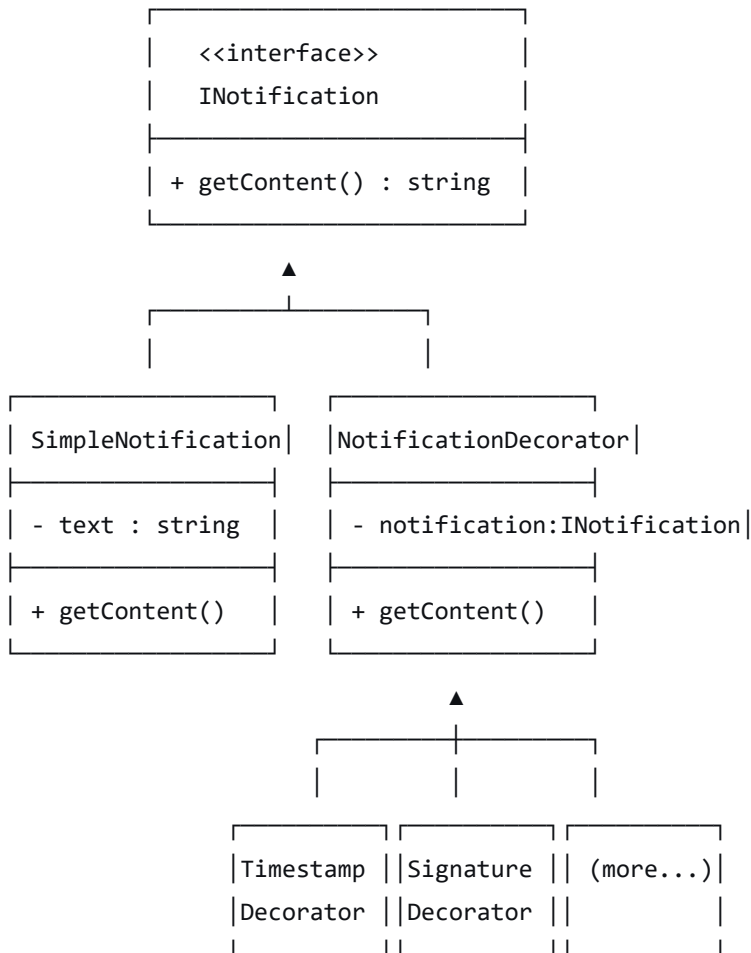
Design Patterns Used

Pattern	Purpose
Decorator	Dynamically enhance notification content
Observer	Notify all interested parties when notification is pushed
Strategy	Different delivery methods (SMS, Email, Popup)
Singleton	Only one NotificationService instance

Part 4: Component 1 — Notification Hierarchy (Decorator)

Class Diagram

text



Explanation

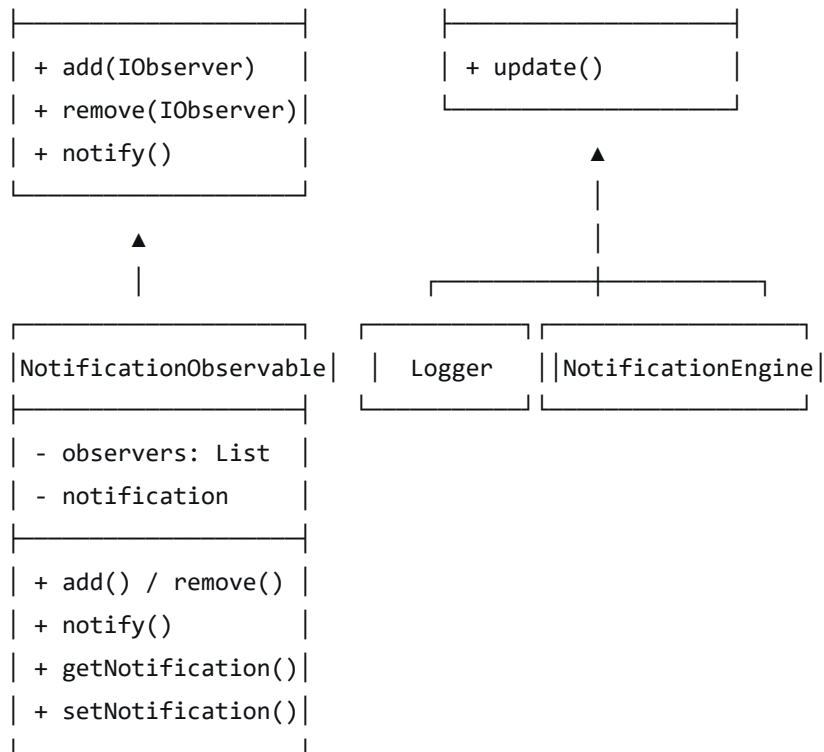
- `SimpleNotification` = the base notification (just a text)
- `NotificationDecorator` = abstract decorator with **IS-A** + **HAS-A** relationships
- `TimestampDecorator` , `SignatureDecorator` = concrete decorators that add timestamp/signature
- Decorators can be **stacked in any order, unlimited times**

Part 5: Component 2 — Observer Pattern

Class Diagram

text





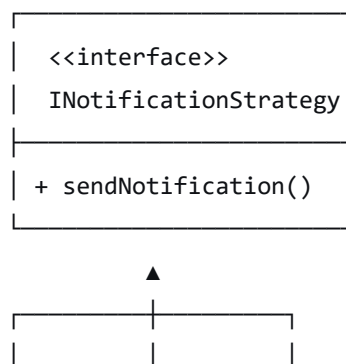
Explanation

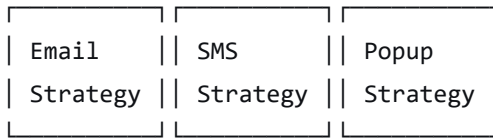
- `NotificationObservable` = the observable that holds the current notification and notifies observers
- `Logger` = observer that logs every notification
- `NotificationEngine` = observer that sends notifications through strategies

Part 6: Component 3 — Strategy Pattern (Notification Delivery)

Class Diagram

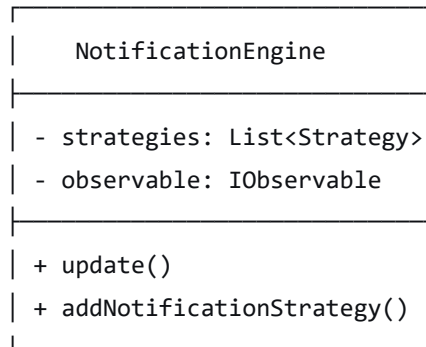
text





NotificationEngine — Uses Multiple Strategies

text

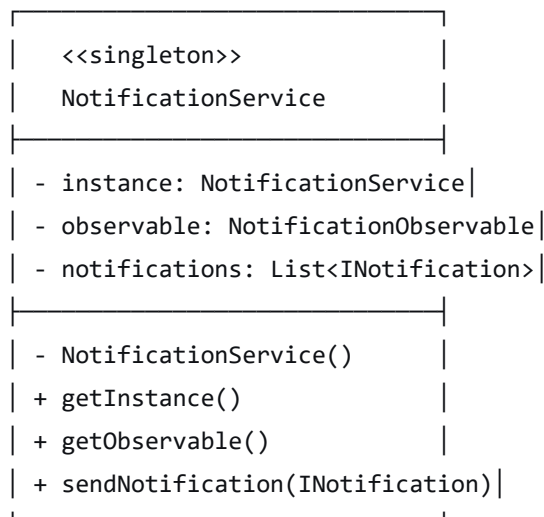


Key Insight: Instead of **HAS-A single strategy**, we have **HAS-A list of strategies** — this lets us send through **multiple channels** at once.

Part 7: The Orchestrator — NotificationService (Singleton)

Class Diagram

text



Flow

1. Client calls `sendNotification(notification)`
2. `NotificationService` adds to **history list**
3. Calls `observable.setNotification(notification)`
4. `Observable`'s `notify()` triggers all observers
5. Each observer's `update()` runs:
 - `Logger` logs the notification
 - `NotificationEngine` reads the notification and sends via all strategies

Part 8: Complete Java Implementation

```

java

import java.util.*;

// ===== NOTIFICATION HIERARCHY (DECORATOR) =====
interface INotification {
    String getContent();
}

class SimpleNotification implements INotification {
    private String text;
    public SimpleNotification(String text) { this.text = text; }
    public String getContent() { return text; }
}

abstract class NotificationDecorator implements INotification {
    protected INotification notification;

    public NotificationDecorator(INotification notification) {
        this.notification = notification;
    }

    @Override
    public String getContent() {
        return notification.getContent();
    }
}

class TimestampDecorator extends NotificationDecorator {
    public TimestampDecorator(INotification notification) {

```

```

        super(notification);
    }

    @Override
    public String getContent() {
        return "[2024-01-01 10:00] " + notification.getContent();
    }
}

class SignatureDecorator extends NotificationDecorator {
    private String signature;

    public SignatureDecorator(INotification notification, String signature) {
        super(notification);
        this.signature = signature;
    }

    @Override
    public String getContent() {
        return notification.getContent() + " -- " + signature;
    }
}

// ===== OBSERVER PATTERN =====
interface IObserver {
    void update();
}

interface IObservable {
    void add(IObserver observer);
    void remove(IObserver observer);
    void notifyObservers();
}

class NotificationObservable implements IObservable {
    private List<IObserver> observers = new ArrayList<>();
    private INotification currentNotification;

    @Override
    public void add(IObserver observer) { observers.add(observer); }

    @Override
    public void remove(IObserver observer) { observers.remove(observer); }

    @Override
    public void notifyObservers() {
        for (IObserver o : observers) o.update();
    }
}

```

```

    public void setNotification(INotification notification) {
        this.currentNotification = notification;
        notifyObservers();
    }

    public INotification getNotification() { return currentNotification; }

    public String getNotificationContent() {
        return currentNotification != null ? currentNotification.getContent() : "";
    }
}

// ===== STRATEGY PATTERN =====
interface INotificationStrategy {
    void sendNotification(String content);
}

class EmailStrategy implements INotificationStrategy {
    private String email;
    public EmailStrategy(String email) { this.email = email; }
    public void sendNotification(String content) {
        System.out.println("Sending Email to " + email + ": " + content);
    }
}

class SMSStrategy implements INotificationStrategy {
    private String mobile;
    public SMSStrategy(String mobile) { this.mobile = mobile; }
    public void sendNotification(String content) {
        System.out.println("Sending SMS to " + mobile + ": " + content);
    }
}

class PopupStrategy implements INotificationStrategy {
    public void sendNotification(String content) {
        System.out.println("Popup: " + content);
    }
}

// ===== OBSERVERS =====
class Logger implements IObservable {
    private NotificationObservable observable;
    public Logger(NotificationObservable observable) {
        this.observable = observable;
    }
    public void update() {
        System.out.println("LOG: " + observable.getNotificationContent());
    }
}

```

```
}
```

```
class NotificationEngine implements IObserver {
    private List<INotificationStrategy> strategies = new ArrayList<>();
    private NotificationObservable observable;

    public NotificationEngine(NotificationObservable observable) {
        this.observable = observable;
    }

    public void addNotificationStrategy(INotificationStrategy strategy) {
        strategies.add(strategy);
    }

    public void update() {
        String content = observable.getNotificationContent();
        for (INotificationStrategy strategy : strategies) {
            strategy.sendNotification(content);
        }
    }
}

// ===== SINGLETON NOTIFICATION SERVICE =====
class NotificationService {
    private static NotificationService instance;
    private NotificationObservable observable;
    private List<INotification> notifications = new ArrayList<>();

    private NotificationService() {
        this.observable = new NotificationObservable();
    }

    public static NotificationService getInstance() {
        if (instance == null) {
            synchronized (NotificationService.class) {
                if (instance == null) instance = new NotificationService();
            }
        }
        return instance;
    }

    public NotificationObservable getObservable() {
        return observable;
    }

    public void sendNotification(INotification notification) {
        notifications.add(notification); // history
        observable.setNotification(notification);
    }
}
```

```

    }
}

// ===== CLIENT (MAIN) =====
public class Main {
    public static void main(String[] args) {
        NotificationService service = NotificationService.getInstance();

        // Create observers that auto-subscribe to the observable
        new Logger(service.getObservable());

        NotificationEngine engine = new NotificationEngine(service.getObservable());
        engine.addNotificationStrategy(new EmailStrategy("user@example.com"));
        engine.addNotificationStrategy(new SMSStrategy("+911234567890"));
        engine.addNotificationStrategy(new PopupStrategy());

        // Build a notification with decorators
        INotification notification = new SignatureDecorator(
            new TimestampDecorator(
                new SimpleNotification("Your order has been shipped!")),
            "Customer Care");

        // Send it
        service.sendNotification(notification);
    }
}

```

Output:

text

```

LOG: [2024-01-01 10:00] Your order has been shipped! -- Customer Care
Sending Email to user@example.com: [2024-01-01 10:00] Your order has been shipped! -- Customer Care
Sending SMS to +911234567890: [2024-01-01 10:00] Your order has been shipped! -- Customer Care
Popup: [2024-01-01 10:00] Your order has been shipped! -- Customer Care

```

Part 9: Python Implementation

python

```

from abc import ABC, abstractmethod
from datetime import datetime
import threading

```



```
# ===== NOTIFICATION HIERARCHY (DECORATOR) =====

class INotification(ABC):
    @abstractmethod
    def get_content(self) -> str: pass


class SimpleNotification(INotification):
    def __init__(self, text):
        self.text = text
    def get_content(self):
        return self.text


class NotificationDecorator(INotification):
    def __init__(self, notification: INotification):
        self._notification = notification
    def get_content(self):
        return self._notification.get_content()


class TimestampDecorator(NotificationDecorator):
    def get_content(self):
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M")
        return f"[{timestamp}] " + self._notification.get_content()


class SignatureDecorator(NotificationDecorator):
    def __init__(self, notification, signature):
        super().__init__(notification)
        self.signature = signature
    def get_content(self):
        return self._notification.get_content() + f" -- {self.signature}"


# ===== OBSERVER PATTERN =====

class IObserver(ABC):
    @abstractmethod
    def update(self): pass


class IObservable(ABC):
    @abstractmethod
    def add(self, observer: IObserver): pass
    @abstractmethod
    def remove(self, observer: IObserver): pass
    @abstractmethod
```

```

def notify_observers(self): pass

class NotificationObservable(IObservable):
    def __init__(self):
        self._observers = []
        self._current_notification = None

    def add(self, observer):
        self._observers.append(observer)

    def remove(self, observer):
        if observer in self._observers:
            self._observers.remove(observer)

    def notify_observers(self):
        for observer in self._observers:
            observer.update()

    def set_notification(self, notification):
        self._current_notification = notification
        self.notify_observers()

    def get_notification(self):
        return self._current_notification

    def get_notification_content(self):
        return self._current_notification.get_content() if self._current_notification else ""

e """

# ===== STRATEGY PATTERN =====
class INotificationStrategy(ABC):
    @abstractmethod
    def send_notification(self, content): pass

class EmailStrategy(INotificationStrategy):
    def __init__(self, email):
        self.email = email
    def send_notification(self, content):
        print(f"Sending Email to {self.email}: {content}")

class SMSStrategy(INotificationStrategy):
    def __init__(self, mobile):
        self.mobile = mobile
    def send_notification(self, content):

```

```

        print(f"Sending SMS to {self.mobile}: {content}")

class PopupStrategy(INotificationStrategy):
    def send_notification(self, content):
        print(f"Popup: {content}")

# ===== OBSERVERS =====
class Logger(IObserver):
    def __init__(self):
        self.observable = NotificationService().get_instance().get_observable()
        self.observable.add(self)

    def update(self):
        print(f"LOG: {self.observable.get_notification_content()}")

class NotificationEngine(IObserver):
    def __init__(self):
        self.strategies = []
        self.observable = NotificationService().get_instance().get_observable()
        self.observable.add(self)

    def add_strategy(self, strategy: INotificationStrategy):
        self.strategies.append(strategy)

    def update(self):
        content = self.observable.get_notification_content()
        for strategy in self.strategies:
            strategy.send_notification(content)

# ===== SINGLETON NOTIFICATION SERVICE =====
class NotificationService:
    _instance = None
    _lock = threading.Lock()

    def __new__(cls):
        if cls._instance is None:
            with cls._lock:
                if cls._instance is None:
                    cls._instance = super().__new__(cls)
                    cls._instance._observable = NotificationObservable()
                    cls._instance._notifications = []
        return cls._instance

    def get_instance(self):

```

```

        return self

    def get_observable(self):
        return self._observable

    def send_notification(self, notification: INotification):
        self._notifications.append(notification) # history
        self._observable.set_notification(notification)

# ===== CLIENT =====
if __name__ == "__main__":
    service = NotificationService()
    obs = service.get_observable()

    # Logger auto-subscribes on creation
    logger = Logger()

    engine = NotificationEngine()
    engine.add_strategy(EmailStrategy("user@example.com"))
    engine.add_strategy(SMSStrategy("+911234567890"))
    engine.add_strategy(PopupStrategy())

    # Build a decorated notification
    notification = SignatureDecorator(
        TimestampDecorator(
            SimpleNotification("Your order has been shipped!")),
        "Customer Care")

    service.send_notification(notification)

```

Output:

text

```

LOG: [2024-01-01 10:00] Your order has been shipped! -- Customer Care
Sending Email to user@example.com: [2024-01-01 10:00] Your order has been shipped! -- Customer Care
Sending SMS to +911234567890: [2024-01-01 10:00] Your order has been shipped! -- Customer Care
Popup: [2024-01-01 10:00] Your order has been shipped! -- Customer Care

```

Part 10: How the Design Patterns Interact

The Complete Flow

text

1. Client builds notification:

SimpleNotification → TimestampDecorator → SignatureDecorator

2. Client calls NotificationService.sendNotification(notification):

↓

3. NotificationService:

- Stores in history list
- Calls observable.setNotification(notification)

↓

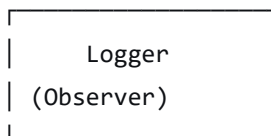
4. Observable:

- Sets current notification
- Calls notifyObservers()

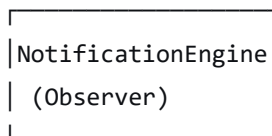
↓

5. For each observer:

↓



Logs to console



Calls each strategy:

- EmailStrategy
- SMSStrategy
- PopupStrategy

Responsibilities

Component	Pattern	Responsibility
SimpleNotification	—	Holds base text
TimestampDecorator	Decorator	Adds timestamp
SignatureDecorator	Decorator	Adds signature
NotificationObservable	Observer	Notifies observers
Logger	Observer	Logs notification
NotificationEngine	Observer	Sends via strategies

Component	Pattern	Responsibility
EmailStrategy , SMSStrategy , PopupStrategy	Strategy	Actual delivery
NotificationService	Singleton	Orchestrator

Part 11: Plug-and-Play Model

Why It's Plug-and-Play

A client only needs to:

```
java
```

```
NotificationService service = NotificationService.getInstance();  
// Optionally: create observers  
service.sendNotification(myNotification);
```

What the client doesn't need to know:

- How observers are created
- How the observable is set up
- How strategies work
- What decorators exist

How to Extend (Without Modifying Existing Code)

Extension	How to Add
New notification type	Add new <code>INotification</code> implementation
New decorator	Add new <code>NotificationDecorator</code> subclass
New observer	Add new <code>IObserver</code> implementation
New delivery method	Add new <code>INotificationStrategy</code> implementation

All 4 extensions follow Open/Closed Principle.

Part 12: Key Concepts Recap

Combining Multiple Patterns — The Real Power

Pattern	Role
Decorator	Enriches notification content dynamically
Observer	Notifies multiple interested parties
Strategy	Swappable delivery mechanisms
Singleton	Single service instance (centralized history)

The "HAS-A list of strategies" Twist

Standard Strategy Pattern uses HAS-A Strategy .

Here we use HAS-A List<Strategy> — allowing **fan-out** delivery.

Why Singleton for NotificationService?

- Must have **one central history** of notifications
- Multiple instances → multiple lists → inconsistent history

Key Takeaways

1. **Notification System** is a classic LLD problem combining **4 design patterns**
2. **Decorator Pattern** adds dynamic enhancement without class explosion
3. **Observer Pattern** decouples notification pushing from receiving
4. **Strategy Pattern** enables plug-and-play delivery methods
5. **Singleton** ensures single source of truth for history
6. **Plug-and-play** = client doesn't need to know internal complexity
7. **Multiple patterns interacting** = powerful, extensible design
8. **Top-Down approach** used (build small components, then combine)
9. **HAS-A list of strategies** for fan-out delivery

10. **Extensibility at every level** — new notification types, decorators, observers, strategies all added without modifying existing code

Final Note

"Real-world LLD rarely uses one pattern — it composes multiple patterns to solve different concerns. This Notification System demonstrates how Decorator, Observer, Strategy, and Singleton all work together to build a robust, extensible solution."



हां जी करें स्टार्ट। सो वेलकम बै...
TXT 79.28KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also

Command Design Pattern — Complete Notes

Part 1: Introduction

What Problem Does Command Pattern Solve?

The **Command Pattern** encapsulates a **request as an object**, letting you:

- Parameterize clients with different requests
- Queue or log requests
- Support **undoable operations**

The Core Idea

Instead of a **source** directly calling a **receiver's** method, we introduce a **command object** in between:

text

Without Command Pattern:

Source —————> Receiver
(tightly coupled)

With Command Pattern:

Source —→ Command —→ Receiver
(loosely coupled)

Why the extra layer?

- **Decoupling:** Source doesn't know which receiver it's talking to
- **Flexibility:** Command can be swapped at runtime
- **Undo/Redo:** Commands can be stored in a stack and undone
- **Logging/Queuing:** Commands are objects — can be serialized, queued

Part 2: Real-World Example — Smart Home Automation

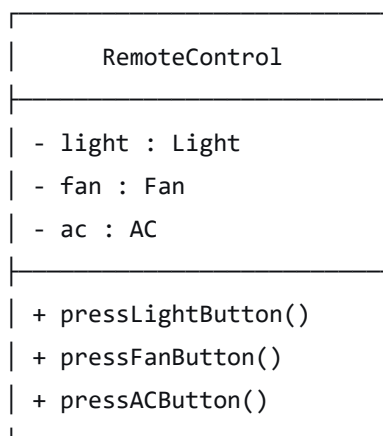
Scenario

You're building a **Smart Home Automation System**:

- A **Remote Control** with buttons
- Each button controls a **Smart Device** (Light, Fan, AC, etc.)
- Buttons can be **reassigned dynamically** to different devices

Naive (Bad) Design

text



Problems:

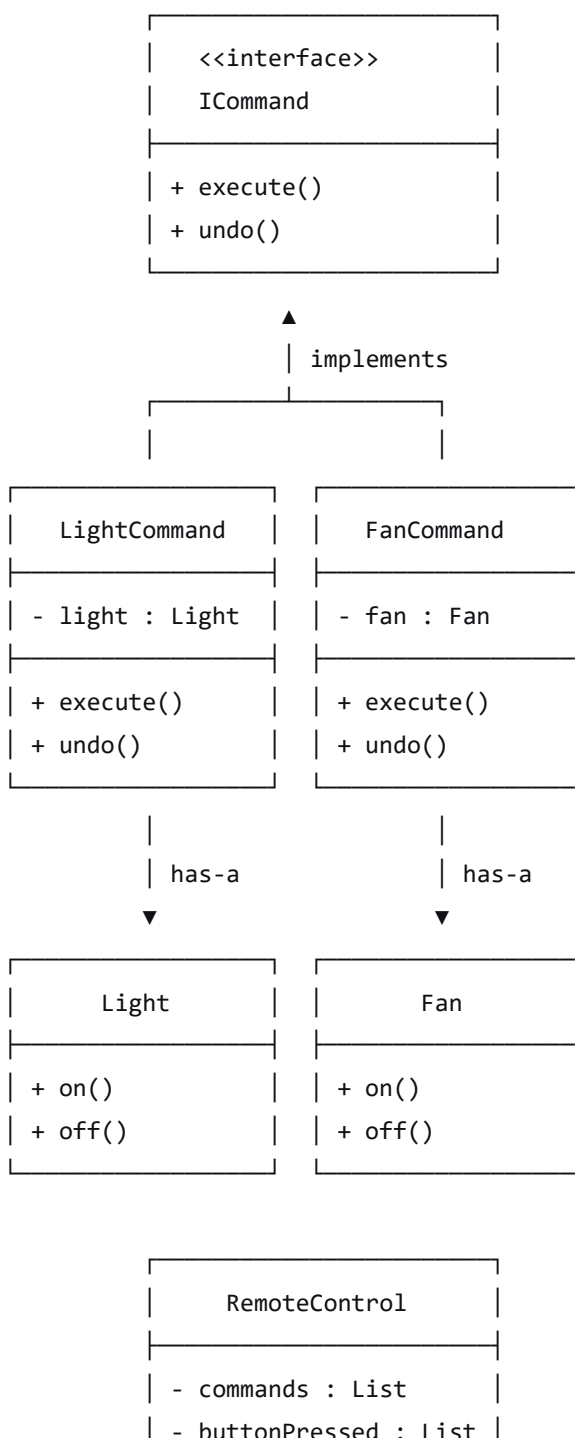
- Remote is **tightly coupled** to specific devices

- Adding a new device (Heater) requires modifying RemoteControl
- Cannot reassign buttons at runtime
- Violates **Open/Closed Principle**

Part 3: Command Pattern Solution

Class Diagram (Single Button)

text



<pre> + setCommand(idx, cmd) + pressButton(idx) </pre>
--

Key Relationships

- RemoteControl **HAS-A** ICommand
- ConcreteCommand **HAS-A** Receiver
- ConcreteCommand **IS-A** ICommand

Why the has-a relationship is on concrete command (not abstract):

- The **abstract** ICommand doesn't know what it will execute
- The **concrete** command knows **which receiver** it talks to
- This is an **intent-based design decision** — the abstract command has no reason to hold a receiver reference

Part 4: Java Implementation

```
java
```

```

import java.util.ArrayList;
import java.util.List;

// ===== COMMAND INTERFACE =====
interface ICommand {
    void execute();
    void undo();
}

// ===== RECEIVERS =====
class Light {
    private String location;
    public Light(String location) { this.location = location; }

    public void on() {
        System.out.println(location + " light is ON");
    }

    public void off() {
        System.out.println(location + " light is OFF");
    }
}

```

```

    }
}

class Fan {
    private String location;
    public Fan(String location) { this.location = location; }

    public void on() {
        System.out.println(location + " fan is ON");
    }
    public void off() {
        System.out.println(location + " fan is OFF");
    }
}

// ===== CONCRETE COMMANDS =====
class LightCommand implements ICommand {
    private Light light;    // HAS-A receiver

    public LightCommand(Light light) { this.light = light; }

    @Override
    public void execute() { light.on(); }

    @Override
    public void undo() { light.off(); }
}

class FanCommand implements ICommand {
    private Fan fan;    // HAS-A receiver

    public FanCommand(Fan fan) { this.fan = fan; }

    @Override
    public void execute() { fan.on(); }

    @Override
    public void undo() { fan.off(); }
}

// ===== INVOKER (Remote Control) =====
class RemoteControl {
    private ICommand[] commands;
    private boolean[] buttonPressed;

    public RemoteControl(int numButtons) {
        commands = new ICommand[numButtons];
        buttonPressed = new boolean[numButtons];
    }
}

```

```

        // Default: all buttons empty (null command) and not pressed
    }

    public void setCommand(int index, ICommand command) {
        if (index < 0 || index >= commands.length) {
            System.out.println("Invalid button index");
            return;
        }
        if (commands[index] != null) {
            System.out.println("Replacing existing command on button " + index);
        }
        commands[index] = command;
        buttonPressed[index] = false;
    }

    public void pressButton(int index) {
        if (index < 0 || index >= commands.length) {
            System.out.println("Invalid button index");
            return;
        }
        if (commands[index] == null) {
            System.out.println("No command assigned at button " + index);
            return;
        }

        if (!buttonPressed[index]) {
            commands[index].execute();
            buttonPressed[index] = true;
        } else {
            commands[index].undo();
            buttonPressed[index] = false;
        }
    }
}

// ===== CLIENT =====
public class Main {
    public static void main(String[] args) {
        // Create receivers
        Light livingRoomLight = new Light("Living Room");
        Fan ceilingFan = new Fan("Ceiling");

        // Create remote with 4 buttons
        RemoteControl remote = new RemoteControl(4);

        // Assign commands to buttons
        remote.setCommand(0, new LightCommand(livingRoomLight));
        remote.setCommand(1, new FanCommand(ceilingFan));
    }
}

```

```
// Simulate button presses (toggle behavior)
remote.pressButton(0); // Light ON
remote.pressButton(0); // Light OFF
remote.pressButton(1); // Fan ON
remote.pressButton(1); // Fan OFF
remote.pressButton(2); // No command assigned
}
}
```

Output:

```
text

Living Room light is ON
Living Room light is OFF
Ceiling fan is ON
Ceiling fan is OFF
No command assigned at button 2
```

Code Walkthrough

Component	Role
ICommand	Interface with execute() and undo()
Light , Fan	Receivers — know how to actually do the work
LightCommand , FanCommand	Concrete commands — bridge between invoker and receiver
RemoteControl	Invoker — calls execute() / undo() on command; doesn't know receiver
Main	Client — wires everything up

Key Insight: The RemoteControl **doesn't know** about Light or Fan . It only calls command.execute() . The command translates that call into light.on() .

Part 5: Python Implementation

```
python
```

```

from abc import ABC, abstractmethod

# ===== COMMAND INTERFACE =====
class ICommand(ABC):
    @abstractmethod
    def execute(self): pass

    @abstractmethod
    def undo(self): pass

# ===== RECEIVERS =====
class Light:
    def __init__(self, location):
        self.location = location

    def on(self):
        print(f"{self.location} light is ON")

    def off(self):
        print(f"{self.location} light is OFF")

class Fan:
    def __init__(self, location):
        self.location = location

    def on(self):
        print(f"{self.location} fan is ON")

    def off(self):
        print(f"{self.location} fan is OFF")

# ===== CONCRETE COMMANDS =====
class LightCommand(ICommand):
    def __init__(self, light: Light):
        self.light = light

    def execute(self):
        self.light.on()

    def undo(self):
        self.light.off()

class FanCommand(ICommand):

```

```

def __init__(self, fan: Fan):
    self.fan = fan

def execute(self):
    self.fan.on()

def undo(self):
    self.fan.off()

# ===== INVOKER (Remote Control) =====
class RemoteControl:
    def __init__(self, num_buttons):
        self.commands = [None] * num_buttons
        self.button_pressed = [False] * num_buttons

    def set_command(self, index, command: ICommand):
        if index < 0 or index >= len(self.commands):
            print("Invalid button index")
            return
        if self.commands[index] is not None:
            print(f"Replacing existing command on button {index}")
        self.commands[index] = command
        self.button_pressed[index] = False

    def press_button(self, index):
        if index < 0 or index >= len(self.commands):
            print("Invalid button index")
            return
        if self.commands[index] is None:
            print(f"No command assigned at button {index}")
            return

        if not self.button_pressed[index]:
            self.commands[index].execute()
            self.button_pressed[index] = True
        else:
            self.commands[index].undo()
            self.button_pressed[index] = False

# ===== CLIENT =====
if __name__ == "__main__":
    # Create receivers
    living_room_light = Light("Living Room")
    ceiling_fan = Fan("Ceiling")

    # Create remote with 4 buttons

```



```
remote = RemoteControl(4)

# Assign commands to buttons
remote.set_command(0, LightCommand(living_room_light))
remote.set_command(1, FanCommand(ceiling_fan))

# Simulate button presses
remote.press_button(0) # Light ON
remote.press_button(0) # Light OFF
remote.press_button(1) # Fan ON
remote.press_button(1) # Fan OFF
remote.press_button(2) # No command assigned
```

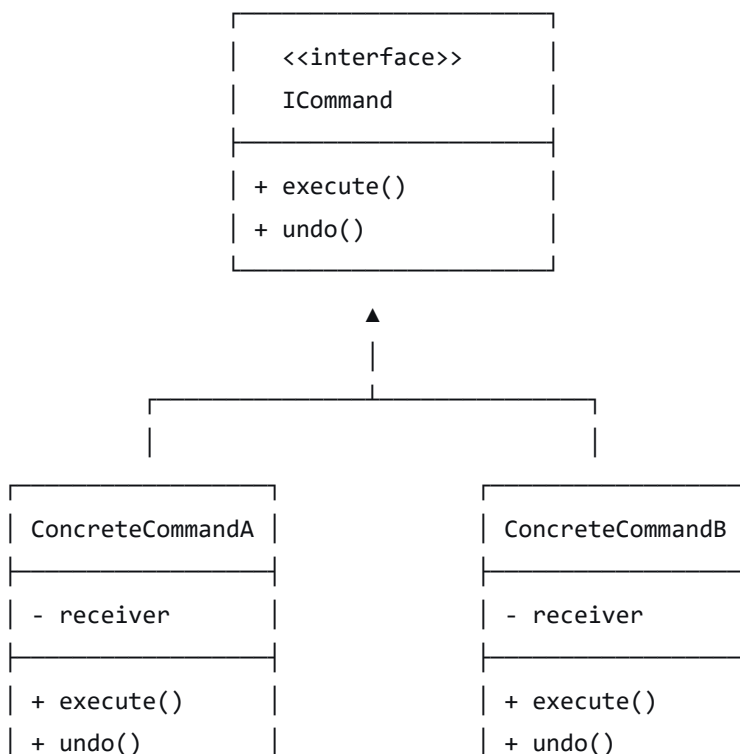
Output:

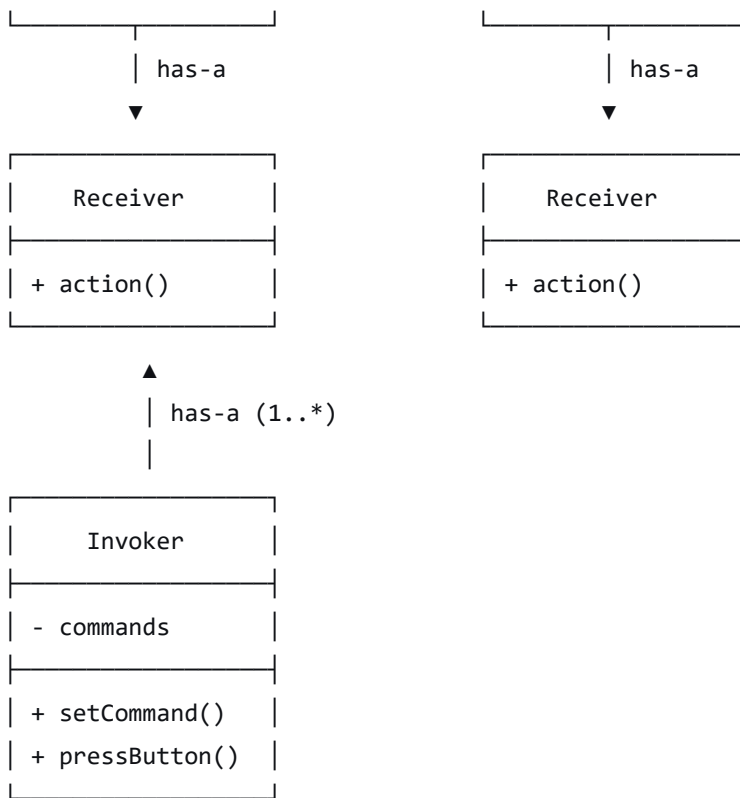
text

```
Living Room light is ON
Living Room light is OFF
Ceiling fan is ON
Ceiling fan is OFF
No command assigned at button 2
```

Part 6: Standard UML Diagram

text





Part 7: Real-World Use Cases

1. Undo/Redo in Text Editors (Ctrl+Z / Ctrl+Y)

Every action (typing, formatting, deleting) is a **Command** stored in a **stack**:

- Ctrl+Z → pop and undo() the last command
- Ctrl+Y → execute() again

2. Keyboard Shortcuts

- Ctrl+S → SaveCommand.execute()
- Ctrl+P → PrintCommand.execute()
- Keys are mapped to commands; can be reassigned

3. GUI Buttons and Menu Items

- Each button holds an ICommand
- Click → command.execute()
- Different buttons can share or have unique commands

4. Transaction Systems

- Each transaction is a Command
- Can be committed or rolled back (`execute()` / `undo()`)

5. Task Queues / Job Schedulers

- Commands queued and executed asynchronously
- Useful in multi-threaded systems

6. Macro Recording

- Record a sequence of commands, replay them later

7. Smart Home (our example)

- Buttons map to commands; commands map to devices

Part 8: Why NOT Just Use an Interface on the Abstract Command?

The Common Confusion

Why is `HAS-A Receiver` on the **Concrete Command** and not on the **Abstract Command**?

Answer: Different commands handle different receivers:

- `LightCommand` needs a `Light`
- `FanCommand` needs a `Fan`
- `ACCommand` needs an `AC`

There's **no common receiver** the abstract command could hold. Hence, the `has-a` relationship belongs only on the concrete command.

Why Not Make All Devices Inherit from `Appliance` ?

Consider:

- `Light` has `on()` / `off()`

- Fan has `on()` / `off()` and **`speed()`**
- AC has `on()` / `off()` , **`temperature`**, **`timer`**

If we force all devices into one abstract class, we violate **LSP** (Liskov Substitution Principle) — not all devices share the same features.

Therefore: Each concrete command holds a concrete receiver directly.

Part 9: Key Concepts Summary

Definition

Command Pattern encapsulates a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations.

Components

Component	Role	Example
Command	Interface for executing operations	<code>ICommand</code>
ConcreteCommand	Implements command; holds receiver	<code>LightCommand</code> , <code>FanCommand</code>
Receiver	Knows how to perform the work	<code>Light</code> , <code>Fan</code>
Invoker	Asks command to execute	<code>RemoteControl</code>
Client	Creates commands and assigns receivers	<code>Main</code>

Benefits

1. **Loose coupling** — Invoker doesn't know receiver
2. **Open/Closed** — New commands without changing invoker
3. **Undo/Redo support** — Commands have `undo()`
4. **Logging/Queueing** — Commands are objects

5. Dynamic assignment — Buttons can be reassigned at runtime

Trade-offs

1. More classes — one command per action
2. Slight overhead from indirection

Part 10: When to Use Command Pattern

✅ Use When:

- Need **undo/redo** functionality
- Want to **parameterize** objects with operations
- Operations need to be **queued, logged, or scheduled**
- Want to **decouple** invoker from receiver
- Need **dynamic button/shortcut** assignment
- Building transactional systems (commit/rollback)

❌ Avoid When:

- Simple direct method calls suffice
- No undo/queue/log requirements
- Action count is small and static

Key Takeaways

1. **Command Pattern** encapsulates a request as an object
2. Introduces **three objects**: Source (Invoker), Command, Receiver
3. **Decouples** invoker from receiver — looser coupling
4. **Undoable operations** — key feature
5. **Dynamic assignment** — swap commands at runtime
6. **Uses 4 roles**: Command, ConcreteCommand, Receiver, Invoker
7. **HAS-A** relationship is on **concrete command**, not abstract
8. **Real-world uses**: Ctrl+Z, keyboard shortcuts, GUI buttons, transactions, smart home

9. **Satisfies SOLID** — especially Open/Closed and LSP

10. **Foundation** for macro recording, task queues, and undo stacks

"When you want to treat a request as a first-class object — to queue, log, undo, or reassign — reach for Command Pattern."



हां जी तो करें स्टार्ट। सो वेलक...
TXT 57.69KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also

Adapter Design Pattern — Complete Notes

Part 1: Introduction — Real-World Analogy

Real-Life Adapters

Example 1: International Travel

- You're in a US hotel room with US-type sockets
- Your charger is India-based
- You use an **international adapter** to bridge them

Example 2: USB-C to USB-A

- Your phone uses Type-C cable
- Your laptop only accepts Type-B
- You need a Type-C → Type-A adapter

Programming Analogy

The Adapter Pattern helps **two incompatible interfaces communicate**:

text

Interface A (Existing Code) ↔ Adapter ↔ Interface B (Third-Party Library)

Both interfaces are incompatible (different method names, data formats, etc.)
Adapter bridges them so they can work together.

Part 2: The Problem Adapter Solves

Common Scenario

You're building an application. You need a **third-party library** (payment gateway, notification service, data provider, etc.).

Problem: The third-party library's interface is **completely different** from your application's interface.

Naive Solution (Bad)

Directly call the third-party library from your existing code:

```
java

// In your application
thirdPartyProvider.getDataInItsOwnFormat();
```

Problems:

- Your application is **tightly coupled** to that specific library
- Switching libraries requires modifying your application code
- Testing becomes hard (library becomes a hidden dependency)

Adapter Pattern Solution

Introduce an **Adapter** between your code and the third party:

```
text

Your Code ↔ Adapter ↔ Third-Party Library
```

Now your code knows **only about the Adapter**. The Adapter knows how to talk to the third party.

Part 3: Puzzle Piece Analogy

Imagine two puzzle pieces that **don't fit** because their connectors are different shapes.

text

```
[Your Existing Code]  X  [Third-Party Library]
                    (incompatible shapes)
```

Introduce an adapter that has **both connector shapes**:

text

$$[\text{Your Existing Code}] \leftarrow [\text{Adapter}] \rightarrow [\text{Third-Party Library}]$$

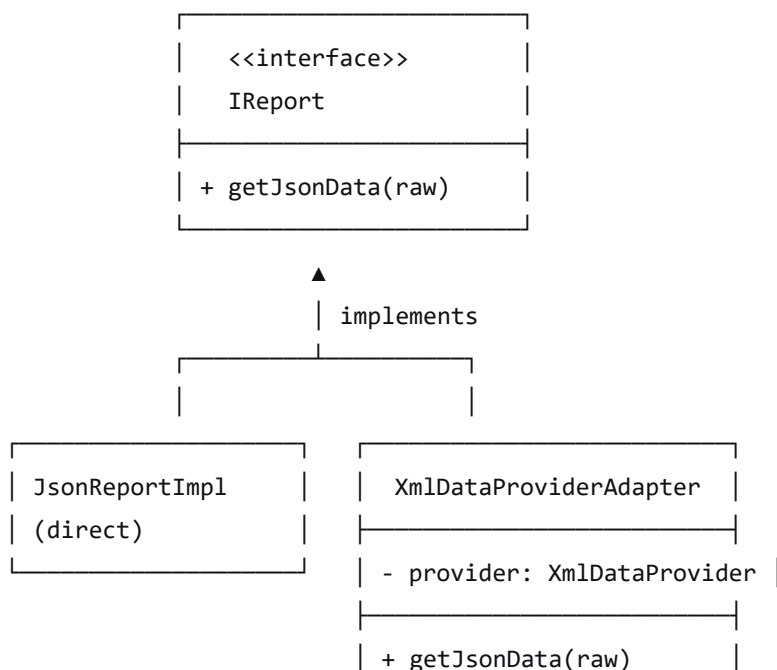
The adapter connects to both sides.

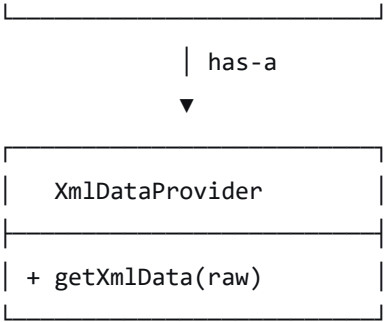
Part 4: Class Diagram (Object Adapter)

Real Example: Report Provider

Scenario: Client wants JSON reports. But the third-party library provides XML only.
Introduce an adapter.

text





Key Relationships

- Adapter **IS-A** Target interface (so client can use it as IReport)
- Adapter **HAS-A** Adaptee (holds a reference to the third-party library)

Two Types of Adapters

Type	Relationship with Adaptee	Preference
Object Adapter	HAS-A (composition)	✅ Preferred
Class Adapter	IS-A (multiple inheritance)	Rarely used

Part 5: Java Implementation

Scenario

- Client wants **JSON data**
- Third-party provides **XML data**
- Adapter converts XML → JSON

```
java

// ===== TARGET INTERFACE (What client expects) =====
interface IReport {
    String getJsonData(String rawData);
}

// ===== ADAPTEE (Third-party Library) =====
class XmlDataProvider {
    public String getXmlData(String rawData) {
```

```

        // Pretend this fetches XML data
        return "<report><name>Alice</name><id>42</id></report>";
    }
}

// ===== OBJECT ADAPTER =====
class XmlDataProviderAdapter implements IReport {
    private XmlDataProvider provider;    // HAS-A adaptee

    public XmlDataProviderAdapter(XmlDataProvider provider) {
        this.provider = provider;
    }

    @Override
    public String getJsonData(String rawData) {
        // 1. Get XML from adaptee
        String xml = provider.getXmlData(rawData);

        // 2. Convert XML → JSON (simplified)
        String name = extractTag(xml, "name");
        String id = extractTag(xml, "id");

        return "{\"name\":\"" + name + "\", \"id\":\"" + id + "\"}";
    }

    private String extractTag(String xml, String tag) {
        int start = xml.indexOf("<" + tag + ">") + tag.length() + 2;
        int end = xml.indexOf("</" + tag + ">");
        return xml.substring(start, end);
    }
}

// ===== CLIENT =====
class Client {
    public void getReport(IReport report, String rawData) {
        String json = report.getJsonData(rawData);
        System.out.println("Report: " + json);
    }
}

// ===== MAIN =====
public class Main {
    public static void main(String[] args) {
        XmlDataProvider xmlProvider = new XmlDataProvider();
        IReport adapter = new XmlDataProviderAdapter(xmlProvider);

        Client client = new Client();
        client.getReport(adapter, "raw data");
    }
}

```

```
}  
}
```

Output:

```
text  
  
Report: {"name": "Alice", "id": 42}
```

Code Walkthrough

Component	Role
IReport	Target interface — client expects this
XmlDataProvider	Adaptee — third-party with incompatible interface
XmlDataProviderAdapter	Adapter — bridges both
Client	Uses only the IReport interface

Key Insight: The `Client` doesn't know XML even exists. It just calls `getJsonData()` on the adapter.

Part 6: Python Implementation

```
python  
  
from abc import ABC, abstractmethod  
  
# ===== TARGET INTERFACE =====  
class IReport(ABC):  
    @abstractmethod  
    def get_json_data(self, raw_data: str) -> str: pass  
  
# ===== ADAPTEE (Third-party Library) =====  
class XmlDataProvider:  
    def get_xml_data(self, raw_data: str) -> str:  
        # Pretend this fetches XML data  
        return "<report><name>Alice</name><id>42</id></report>"
```

```
# ===== OBJECT ADAPTER =====
class XmlDataProviderAdapter(IReport):
    def __init__(self, provider: XmlDataProvider):
        self._provider = provider    # HAS-A adaptee

    def get_json_data(self, raw_data: str) -> str:
        # 1. Get XML from adaptee
        xml = self._provider.get_xml_data(raw_data)

        # 2. Extract fields (simplified)
        name = self._extract_tag(xml, "name")
        id_val = self._extract_tag(xml, "id")

        return f'{{"name": "{name}", "id": {id_val}}}'

    def _extract_tag(self, xml, tag):
        start = xml.index(f"<{tag}>") + len(tag) + 2
        end = xml.index(f"</{tag}>")
        return xml[start:end]

# ===== CLIENT =====
class Client:
    def get_report(self, report: IReport, raw_data: str):
        json_data = report.get_json_data(raw_data)
        print(f"Report: {json_data}")

# ===== MAIN =====
if __name__ == "__main__":
    xml_provider = XmlDataProvider()
    adapter = XmlDataProviderAdapter(xml_provider)

    client = Client()
    client.get_report(adapter, "raw data")
```

Output:

text

Report: {"name": "Alice", "id": 42}

Part 7: Class Adapter vs Object Adapter

Object Adapter (Preferred)

text

Adapter IS-A Target (inheritance)

Adapter HAS-A Adaptee (composition)

Class Adapter

text

Adapter IS-A Target (inheritance)

Adapter IS-A Adaptee (inheritance)

Uses **multiple inheritance** — C++ supports this, Java doesn't.

UML Comparison

Object Adapter:

text

Target ← Adapter —has-a→ Adaptee

Class Adapter:

text

Target ← Adapter —is-a→ Adaptee
(Multiple Inheritance)

Why Object Adapter is Preferred

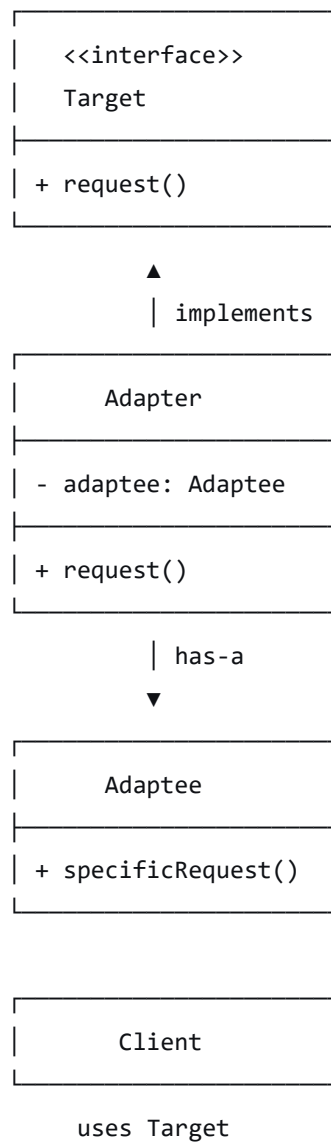
1. **Composition over inheritance** — general OO principle
2. **Java doesn't support multiple inheritance** — Class Adapter can't be done in Java
3. **More flexible** — can adapt multiple adaptees, change at runtime
4. **Single responsibility** — cleaner class hierarchy

Part 8: Definition

Adapter Pattern converts the interface of a class into another interface that clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces.

Part 9: Standard UML Diagram

text



Flow:

1. Client calls Target.request()
2. Adapter implements request() by calling adaptee.specificRequest()
3. Adaptee does the actual work

Part 10: Real-World Use Cases

1. Third-Party Library Integration

- Payment gateways (Razorpay, Stripe)
- Notification services (Twilio, Firebase)
- Data providers

Why: Client code stays clean; adapter hides third-party details.

2. Legacy Code Integration

- Old Java 6 code being used by modern Java 21 application
- Adapter bridges old and new APIs

3. Data Format Conversion

- XML → JSON (as in example)
- CSV → JSON
- SOAP → REST

4. Database Drivers

- JDBC driver acts as an adapter between Java app and database-specific protocols

5. GUI Components

- Old GUI widget adapted to new interface

6. Media Players

- An audio player that can play MP3
- Adapts VLC, MP4, etc. to the same interface

7. Testing / Mocking

- Adapt third-party interface to a mockable interface for unit testing

Part 11: Key Points Summary

Components

Component	Role
Target	Interface client expects
Client	Uses Target interface
Adaptee	Existing/incompatible class (often third-party)
Adapter	Bridges Target and Adaptee

Benefits

1. **Loose coupling** — client doesn't depend on third-party directly
2. **Single Responsibility** — conversion logic is isolated
3. **Open/Closed** — add new adapters without changing client
4. **Reusability** — adapters can be reused across projects
5. **Testability** — easy to mock adapters

Drawbacks

1. Extra layer of indirection (minor overhead)
2. Can lead to many small classes

Part 12: When to Use Adapter Pattern

✅ Use When:

- Integrating third-party libraries
- Working with legacy code
- Converting between data formats
- Two interfaces are incompatible but both must work together
- You want to decouple from a specific library

✗ Avoid When:

- Interfaces are already compatible
- You control both sides and can just modify them
- Only one small usage (sometimes direct call is fine)

Part 13: Adapter vs Other Patterns

Pattern	Purpose
Adapter	Convert one interface to another
Decorator	Add responsibilities to object
Facade	Simplify a complex subsystem
Proxy	Control access to object
Bridge	Separate abstraction from implementation

Adapter vs Decorator:

- Adapter **changes interface**
- Decorator **enhances behavior** without changing interface

Adapter vs Facade:

- Adapter works on **one class**
- Facade works on **many classes**

Key Takeaways

1. **Adapter Pattern** converts incompatible interfaces
2. **Three actors:** Target, Adaptee, Adapter
3. **Object Adapter** (composition) is preferred over **Class Adapter** (multiple inheritance)
4. Uses **IS-A** (with Target) + **HAS-A** (with Adaptee)
5. **Real-world uses:** third-party integration, legacy code, data conversion

6. **Client only knows** the Target interface — decoupled from Adaptee
7. **Foundation** for clean architecture — allows swapping implementations
8. **Common in interviews** — anytime you integrate external systems
9. **Simple but powerful** — very intuitive once understood
10. **Follows SOLID** — especially Open/Closed and Dependency Inversion

"Adapter Pattern: Wrap an incompatible interface so it fits where you need it — like a travel plug adapter, but for code."



हां जी, करें स्टार्ट। सो, वेलकम ...
TXT 49.76KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also

Facade Design Pattern — Complete Notes

Part 1: Introduction — What Is Facade?

The Core Problem

Imagine you have a **complex subsystem** with many classes (A, B, C, D, E) that interact with each other through a web of dependencies:

text

```
A depends on B and C
B depends on C and D
D depends on E
...
```

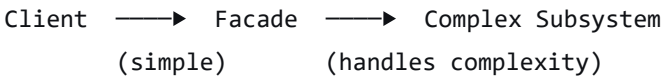
A **client** wants to use this subsystem to accomplish **one task**. But to do so, the client would have to interact with all these classes in the correct order — that's painful and fragile.

The Facade Solution

Introduce a **Facade class** — a single gateway that:

- Interacts with the complex subsystem on behalf of the client
- Exposes a **simplified, unified interface** to the client
- Hides all internal complexity

text



Definition:

Facade Pattern provides a simplified, unified interface to a set of interfaces in a subsystem. It hides the complexity of the subsystem and exposes only what is necessary to the user.

Part 2: Key Benefits

Benefit	Explanation
Simplified Interface	Client deals with one method, not many
Decoupling	Client is decoupled from subsystem internals
Hides Complexity	Client doesn't need to know about internal classes
Follows Principle of Least Knowledge	Client talks only to Facade

Part 3: Real-World Example — Computer Boot Up

The Scenario

When you press the **Start** button on your computer:

- You (the client) press **one button**
- Behind the scenes, **many subsystems** initialize

Subsystems Involved

Subsystem	Role
PowerSupply	Provides power
CoolingSystem	Starts fans
CPU	Initializes
Memory	Performs self-test
HardDrive	Spins up
BIOS	Boots using CPU + Memory
OperatingSystem	Loads

Without Facade

The client would have to:

1. Call `PowerSupply.providePower()`
2. Call `CoolingSystem.startFans()`
3. Call `CPU.initialize()`
4. Call `Memory.selfTest()`
5. Call `HardDrive.spinUp()`
6. Call `BIOS.boot()`
7. Call `OperatingSystem.load()`

...in the correct order. That's fragile and full of coupling.

With Facade

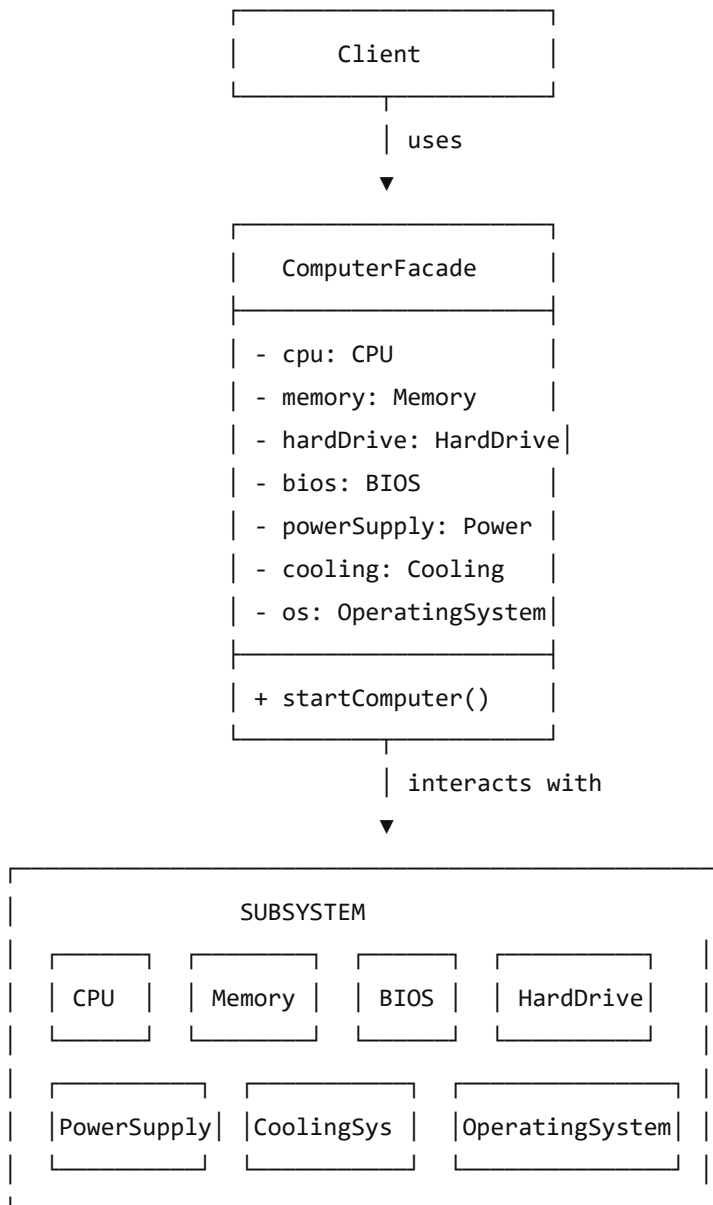
The client just calls:

```
java  
  
computerFacade.startComputer();
```

Everything else happens internally.

Part 4: Class Diagram

text



Key Points:

- Client only knows `ComputerFacade`
- Facade holds references to all subsystem classes
- Facade exposes only `startComputer()`

Part 5: Java Implementation

java

```
// ===== SUBSYSTEM CLASSES =====  
class PowerSupply {  
    public void providePower() {  
        System.out.println("Power supply: providing power");  
    }  
}  
  
class CoolingSystem {  
    public void startFans() {  
        System.out.println("Cooling system: fans started");  
    }  
}  
  
class CPU {  
    public void initialize() {  
        System.out.println("CPU: initialized");  
    }  
}  
  
class Memory {  
    public void selfTest() {  
        System.out.println("Memory: self test passed");  
    }  
}  
  
class HardDrive {  
    public void spinUp() {  
        System.out.println("Hard Drive: spinning up");  
    }  
}  
  
class BIOS {  
    private CPU cpu;  
    private Memory memory;  
  
    public BIOS(CPU cpu, Memory memory) {  
        this.cpu = cpu;  
        this.memory = memory;  
    }  
  
    public void boot() {  
        cpu.initialize();  
        memory.selfTest();  
        System.out.println("BIOS: booted");  
    }  
}
```

```

class OperatingSystem {
    public void load() {
        System.out.println("Operating System: loaded");
    }
}

// ===== FACADE =====
class ComputerFacade {
    private PowerSupply powerSupply;
    private CoolingSystem coolingSystem;
    private CPU cpu;
    private Memory memory;
    private HardDrive hardDrive;
    private BIOS bios;
    private OperatingSystem operatingSystem;

    public ComputerFacade() {
        this.powerSupply = new PowerSupply();
        this.coolingSystem = new CoolingSystem();
        this.cpu = new CPU();
        this.memory = new Memory();
        this.hardDrive = new HardDrive();
        this.bios = new BIOS(cpu, memory);
        this.operatingSystem = new OperatingSystem();
    }

    public void startComputer() {
        System.out.println("Starting computer...");
        powerSupply.providePower();
        coolingSystem.startFans();
        hardDrive.spinUp();
        bios.boot();
        operatingSystem.load();
        System.out.println("Computer booted successfully.");
    }
}

// ===== CLIENT =====
public class Main {
    public static void main(String[] args) {
        ComputerFacade computer = new ComputerFacade();
        computer.startComputer();
    }
}

```

Output:

```
text

Starting computer...
Power supply: providing power
Cooling system: fans started
Hard Drive: spinning up
CPU: initialized
Memory: self test passed
BIOS: booted
Operating System: loaded
Computer booted successfully.
```

Walkthrough

Component	Role
PowerSupply , CoolingSystem , CPU , Memory , HardDrive , BIOS , OperatingSystem	Complex subsystem classes
ComputerFacade	Exposes startComputer() — orchestrates everything
Main	Client — just calls one method

Part 6: Python Implementation

```
python

# ===== SUBSYSTEM CLASSES =====
class PowerSupply:
    def provide_power(self):
        print("Power supply: providing power")

class CoolingSystem:
    def start_fans(self):
        print("Cooling system: fans started")

class CPU:
    def initialize(self):
        print("CPU: initialized")
```



```
class Memory:
    def self_test(self):
        print("Memory: self test passed")

class HardDrive:
    def spin_up(self):
        print("Hard Drive: spinning up")

class BIOS:
    def __init__(self, cpu: CPU, memory: Memory):
        self.cpu = cpu
        self.memory = memory

    def boot(self):
        self.cpu.initialize()
        self.memory.self_test()
        print("BIOS: booted")

class OperatingSystem:
    def load(self):
        print("Operating System: loaded")

# ===== FACADE =====
class ComputerFacade:
    def __init__(self):
        self.power_supply = PowerSupply()
        self.cooling_system = CoolingSystem()
        self.cpu = CPU()
        self.memory = Memory()
        self.hard_drive = HardDrive()
        self.bios = BIOS(self.cpu, self.memory)
        self.operating_system = OperatingSystem()

    def start_computer(self):
        print("Starting computer...")
        self.power_supply.provide_power()
        self.cooling_system.start_fans()
        self.hard_drive.spin_up()
        self.bios.boot()
        self.operating_system.load()
        print("Computer booted successfully.")
```

```
# ===== CLIENT =====  
if __name__ == "__main__":  
    computer = ComputerFacade()  
    computer.start_computer()
```

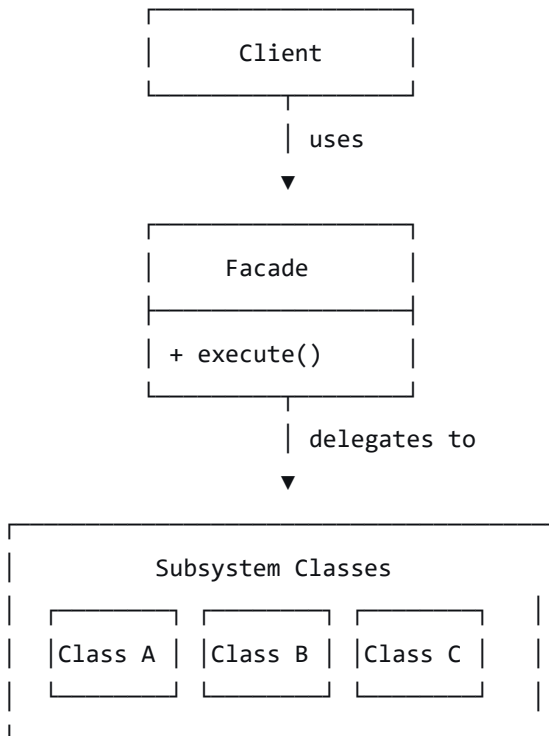
Output:

text

```
Starting computer...  
Power supply: providing power  
Cooling system: fans started  
Hard Drive: spinning up  
CPU: initialized  
Memory: self test passed  
BIOS: booted  
Operating System: loaded  
Computer booted successfully.
```

Part 7: Standard UML Diagram

text



Client → Facade → Subsystem

Part 8: Principle of Least Knowledge (Law of Demeter)

What It Says

"Talk only to your immediate friends."

A class should only call methods on:

1. **Itself** (`this.method()`)
2. **Its own fields** (objects it `has-a`)
3. **Objects passed as parameters** to the method
4. **Objects it creates itself**

The Four Rules

Rule	Example (Valid)
Call your own methods	<code>this.m1()</code>
Call methods of your fields	<code>this.field.method()</code>
Call methods of parameters	<code>method(param) → param.m()</code>
Call methods of objects you create	<code>Object obj = new Object(); obj.m()</code>

What's FORBIDDEN

✗ "Friends of friends" — calling methods on objects returned by a friend

Bad Example

text

A has-a B, B has-a C

A.m1() calls b.getC().m3() ← VIOLATION

Why bad? A now knows about B's internals — coupling increases.

Good Example

text

A has-a B, B has-a C

A.m1() calls b.doWork() ← B internally calls C

A only talks to B, not B's friends.

How Facade Helps

Facade **hides the subsystem** from the client. The client only talks to the facade — never to A, B, C, D directly. This is essentially enforcing LoD at the architecture level.

Key Principle

| The less a class knows about other classes, the better the design.

This creates **loosely coupled** designs — the primary goal of LLD.

Part 9: Real-World Use Cases

1. Computer Boot Up (as in example)

- Client presses **one button** → Facade orchestrates CPU, Memory, BIOS, OS

2. Gaming Engine (Unity, Unreal)

- Client clicks "**Load Game**"
- Facade loads assets, initializes physics, sets up memory, spawns UI, etc.

3. Payment Gateway

- Client calls `makePayment(amount)`
- Facade internally:
 - Checks balance
 - Validates PIN
 - Detects fraud
 - Sends notification

- Updates records

4. JDBC API (Java)

- `DriverManager.getConnection()` is a facade over complex driver loading and connection pooling

5. Spring Framework

- `JdbcTemplate` is a facade over JDBC's complex API

6. Compilers

- `compile()` method is a facade over lexical analysis, parsing, code generation

7. Home Theater System

- Facade `watchMovie()` calls: turn on TV, dim lights, start DVD, set audio, etc.

Part 10: Facade vs Adapter

Common Confusion: Both patterns introduce an intermediary class.

The Key Difference: Intent

Aspect	Facade	Adapter
Intent	Simplify complex subsystem	Bridge incompatible interfaces
Goal	Hide complexity	Convert interface
Number of classes	Works with many	Works with one (usually)
Client knows	Only facade	Only target interface
When to use	Complex subsystem	Third-party / legacy integration

How to Decide

Ask: "Am I hiding complexity or converting an interface?"

- Hiding → **Facade**
- Converting → **Adapter**

UML Looks Similar

Both use a HAS-A relationship between the mediator class and the wrapped class(es).
The **intent** is what distinguishes them.

Part 11: Advantages and Drawbacks

Advantages

1. **Simplified interface** — easier for clients
2. **Loose coupling** — client isn't tied to subsystem internals
3. **Improved readability** — subsystem complexity is isolated
4. **Easier to test** — mock the facade
5. **Reduces dependencies** — change subsystem without touching client

Drawbacks

1. **Potential god-object** — if facade grows too big
2. **Hides flexibility** — client can't fine-tune subsystem behavior
3. **Extra indirection** — minor overhead
4. **Risk of over-simplification** — client may need more control

Part 12: When to Use Facade Pattern

✅ Use When:

- Subsystem is complex with many classes
- Client needs only a simple interface
- You want to layer your architecture
- Client shouldn't be coupled to subsystem internals

- Following the Principle of Least Knowledge

✗ Avoid When:

- Subsystem is already simple
- Client needs fine-grained control
- Facade becomes a god-class doing everything

Part 13: Summary Table

Aspect	Detail
Intent	Provide a simplified, unified interface to a subsystem
Type	Structural pattern
Key Relation	Facade HAS-A subsystem classes
Client sees	Only the Facade
Related Principle	Principle of Least Knowledge (Law of Demeter)
Helps with	Decoupling, hiding complexity
Common mistake	Confusing with Adapter

Key Takeaways

1. **Facade = single gateway to a complex subsystem**
2. **Hides complexity** — client doesn't need to know internal classes
3. **Simplified interface** — usually just one or two methods
4. **Promotes loose coupling** — client is decoupled from subsystem
5. **Implements Principle of Least Knowledge** — "talk only to immediate friends"
6. **4 LoD rules**: call self, fields, params, objects created
7. **Client talks only to Facade** — not to subsystem classes
8. **Facade vs Adapter** — intent is the difference (hide vs convert)

9. **Real examples:** computer boot, gaming engine, payment gateway, JDBC

10. **Common in architecture** — layered designs, service wrappers

"Facade is your app's front desk: one number to call, and everything gets handled internally."



हां जी करें स्टार्ट। सो वेलकम बै...
TXT 208.14KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also

LLD Case Study: Music Player Application — Complete Notes

Part 1: Problem Statement & Requirements

Functional Requirements

1. **Play / Pause songs**
2. **Create playlists**, add songs, play entire playlist
3. **Multiple playback modes:**
 - **Sequential** — play in order
 - **Random** — shuffle
 - **Custom** — user queues songs
 - Extensible for future modes
4. **Support multiple output devices:**
 - Bluetooth speaker
 - Wired speaker
 - Headphones
 - Extensible for new devices

Non-Functional Requirements

- Scalable, maintainable, extensible

- Follow SOLID principles
- Clean code

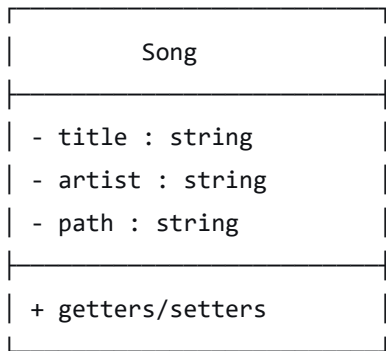
Part 2: Design Patterns Used

Pattern	Where Used
Strategy	Playlist playback modes (sequential/random/custom)
Singleton	Managers (DeviceManager, PlaylistManager, StrategyManager, Facade)
Adapter	Bridging third-party device APIs to our interface
Factory	Creating devices
Facade	MusicPlayerFacade — single point of contact

Part 3: Entity Model

Song

text



Playlist

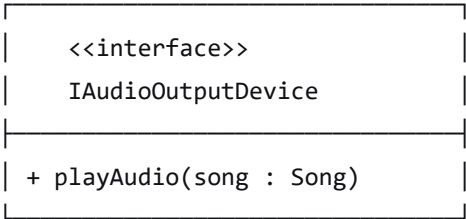
text



Part 4: Device Hierarchy (Adapter Pattern)

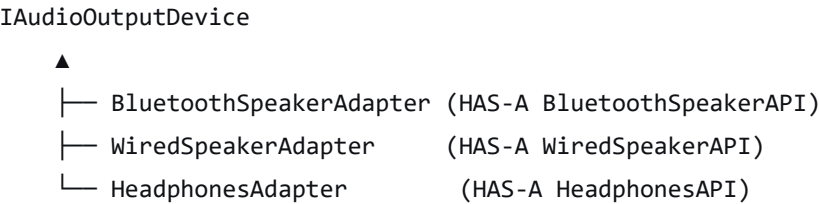
Interface

text



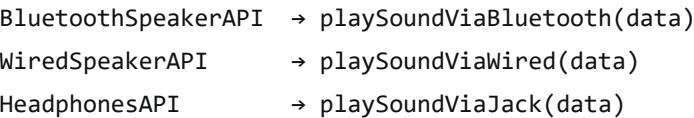
Concrete Adapters

text



External APIs (Third-Party)

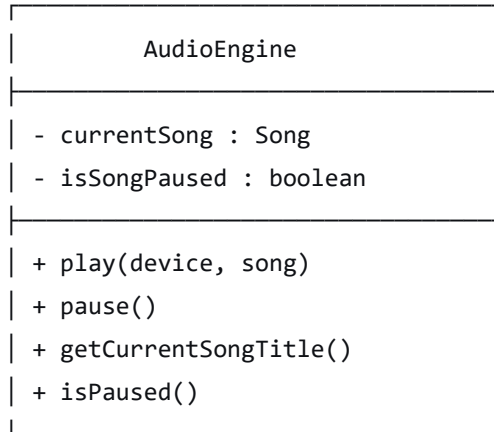
text



Adapter bridges: `playAudio(song)` → calls respective external API.

Part 5: Audio Engine

text

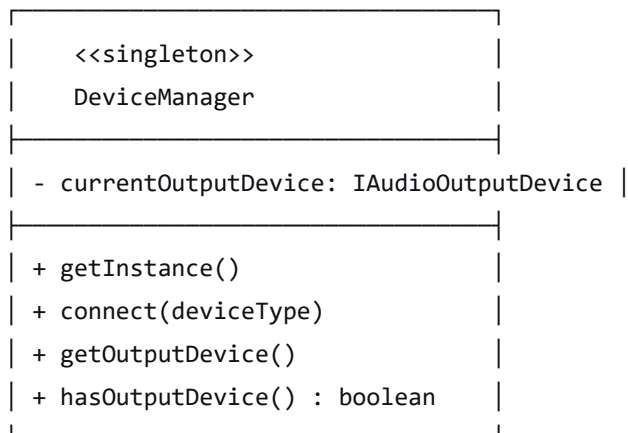


Responsibility: Actual orchestrator of "playing" and "pausing" a song on a given device.

Part 6: Managers (Singleton)

DeviceManager

text



Uses DeviceFactory to create devices.

PlaylistManager

text

```
<<singleton>>
PlaylistManager

- playlists : Map<string, Playlist>

+ getInstance()
+ createPlaylist(name)
+ addSongToPlaylist(name, song)
+ getPlaylist(name) : Playlist
```

Uses Map for O(1) lookup by name.

PlayingStrategyManager

text

```
<<singleton>>
PlayingStrategyManager

- sequential, random, custom

+ getInstance()
+ createStrategy(type) : Strategy
```

Part 7: Playback Strategy (Strategy Pattern)

Interface

text

```
<<interface>>
IPlayingStrategy

+ setPlaylist(playlist)
```

```
| + next() : Song |
| + previous() : Song |
| + hasNext() : boolean |
| + hasPrevious() : boolean |
| + addToNext(song) {empty} |
```

Concrete Strategies

Strategy	How It Works
SequentialPlayingStrategy	Maintains current index, moves forward/backward
RandomPlayingStrategy	Picks random song, maintains history stack for "previous"
CustomQueueStrategy	Maintains a queue + stack (user adds to queue)

Note on addToNext

Empty implementation in the interface (trade-off for ISP). Only Custom strategy uses it.

Part 8: Facade — MusicPlayerFacade

text

```
| <<singleton>> |
| MusicPlayerFacade |
|-----|
| - audioEngine : AudioEngine |
| - strategy : IPlayingStrategy |
| - currentPlaylist : Playlist |
|-----|
| + getInstance() |
| + connectDevice(deviceType) |
| + setStrategy(strategyType) |
| + loadPlaylist(playlistName) |
| + playSong(song) |
| + pauseSong() |
| + playAllTracks() |
```

```

| + playNextTrack()
| + playPreviousTrack()
| + enqueueNext(song)
|

```

Responsibility: All client-facing operations. Hides the entire subsystem.

Part 9: Application (Orchestrator)

text

```

class MusicPlayerApp {
- allSongs : List<Song>

+ createSong(...)
+ findSongByTitle(title)
+ createPlaylist(name)
+ addSongToPlaylist(name, song)
+ connectAudioDevice(type)
+ selectPlayStrategy(type)
+ loadPlaylist(name)
+ playSong(song) / pause()
+ playAllTracks()
+ playNextTrack()
+ playPreviousTrack()
+ enqueueNext(song)
}

```

Purpose: Single entry point for the client. Delegates to either `PlaylistManager` or `MusicPlayerFacade`.

Part 10: Complete Java Implementation

Enums

java

```

enum DeviceType {
    BLUETOOTH, WIRED, HEADPHONES
}

```

```
}

enum PlayingStrategyType {
    SEQUENTIAL, RANDOM, CUSTOM
}
```

Models

java

```
class Song {
    private String title;
    private String artist;
    private String path;

    public Song(String title, String artist, String path) {
        this.title = title;
        this.artist = artist;
        this.path = path;
    }

    public String getTitle() { return title; }
    public String getArtist() { return artist; }
    public String getPath() { return path; }
}

class Playlist {
    private String name;
    private List<Song> songs = new ArrayList<>();

    public Playlist(String name) { this.name = name; }
    public void addSong(Song song) { songs.add(song); }
    public String getName() { return name; }
    public List<Song> getSongs() { return songs; }
}
```

External Device APIs (3rd Party Simulation)

java

```
class BluetoothSpeakerAPI {
    public void playSoundViaBluetooth(String data) {
        System.out.println("Playing via Bluetooth: " + data);
    }
}
```

```

class WiredSpeakerAPI {
    public void playSoundViaWired(String data) {
        System.out.println("Playing via Wired Speaker: " + data);
    }
}

class HeadphonesAPI {
    public void playSoundViaJack(String data) {
        System.out.println("Playing via Headphones: " + data);
    }
}

```

Adapters

java

```

interface IAudioOutputDevice {
    void playAudio(Song song);
}

class BluetoothSpeakerAdapter implements IAudioOutputDevice {
    private BluetoothSpeakerAPI api;
    public BluetoothSpeakerAdapter(BluetoothSpeakerAPI api) { this.api = api; }
    public void playAudio(Song song) {
        api.playSoundViaBluetooth(song.getTitle() + " by " + song.getArtist());
    }
}

class WiredSpeakerAdapter implements IAudioOutputDevice {
    private WiredSpeakerAPI api;
    public WiredSpeakerAdapter(WiredSpeakerAPI api) { this.api = api; }
    public void playAudio(Song song) {
        api.playSoundViaWired(song.getTitle() + " by " + song.getArtist());
    }
}

class HeadphonesAdapter implements IAudioOutputDevice {
    private HeadphonesAPI api;
    public HeadphonesAdapter(HeadphonesAPI api) { this.api = api; }
    public void playAudio(Song song) {
        api.playSoundViaJack(song.getTitle() + " by " + song.getArtist());
    }
}

```

Device Factory

java

```
class DeviceFactory {
    public static IAudioOutputDevice createDevice(DeviceType type) {
        switch (type) {
            case BLUETOOTH: return new BluetoothSpeakerAdapter(new BluetoothSpeakerAPI
());
            case WIRED:      return new WiredSpeakerAdapter(new WiredSpeakerAPI());
            case HEADPHONES: return new HeadphonesAdapter(new HeadphonesAPI());
        }
        return null;
    }
}
```

Audio Engine

java

```
class AudioEngine {
    private Song currentSong;
    private boolean isSongPaused = false;

    public void play(IAudioOutputDevice device, Song song) {
        if (song == null) throw new RuntimeException("Song cannot be null");
        if (isSongPaused && currentSong != null && currentSong.getTitle().equals(song.get
Title())) {
            isSongPaused = false;
            System.out.println("Resuming song: " + song.getTitle());
        } else {
            currentSong = song;
            isSongPaused = false;
        }
        device.playAudio(song);
    }

    public void pause() {
        if (currentSong == null) throw new RuntimeException("No song is playing");
        if (isSongPaused) throw new RuntimeException("Song already paused");
        isSongPaused = true;
        System.out.println("Paused: " + currentSong.getTitle());
    }

    public String getCurrentSongTitle() {
        return currentSong != null ? currentSong.getTitle() : "";
    }
}
```

```

    public boolean isPaused() { return isSongPaused; }
}

```

Managers

java

```

class DeviceManager {
    private static DeviceManager instance;
    private IAudioOutputDevice currentOutputDevice;

    private DeviceManager() {}

    public static DeviceManager getInstance() {
        if (instance == null) {
            synchronized (DeviceManager.class) {
                if (instance == null) instance = new DeviceManager();
            }
        }
        return instance;
    }

    public void connect(DeviceType type) {
        if (currentOutputDevice != null) currentOutputDevice = null;
        currentOutputDevice = DeviceFactory.createDevice(type);
        System.out.println(type + " device connected");
    }

    public IAudioOutputDevice getOutputDevice() { return currentOutputDevice; }
    public boolean hasOutputDevice() { return currentOutputDevice != null; }
}

```

java

```

class PlaylistManager {
    private static PlaylistManager instance;
    private Map<String, Playlist> playlists = new HashMap<>();

    private PlaylistManager() {}

    public static PlaylistManager getInstance() {
        if (instance == null) {
            synchronized (PlaylistManager.class) {
                if (instance == null) instance = new PlaylistManager();
            }
        }
    }
}

```

```

        return instance;
    }

    public void createPlaylist(String name) {
        if (playlists.containsKey(name))
            throw new RuntimeException("Playlist already exists: " + name);
        playlists.put(name, new Playlist(name));
    }

    public void addSongToPlaylist(String playlistName, Song song) {
        if (!playlists.containsKey(playlistName))
            throw new RuntimeException("Playlist not found: " + playlistName);
        playlists.get(playlistName).addSong(song);
    }

    public Playlist getPlaylist(String name) {
        if (!playlists.containsKey(name))
            throw new RuntimeException("Playlist not found: " + name);
        return playlists.get(name);
    }
}

```

Strategy

java

```

interface IPlayingStrategy {
    void setPlaylist(Playlist playlist);
    Song next();
    Song previous();
    boolean hasNext();
    boolean hasPrevious();
    default void addToNext(Song song) {} // only used by Custom
}

class SequentialPlayingStrategy implements IPlayingStrategy {
    private Playlist playlist;
    private int currentIndex = -1;

    public void setPlaylist(Playlist playlist) {
        this.playlist = playlist;
        this.currentIndex = -1;
    }

    public boolean hasNext() {
        return playlist != null && !playlist.getSongs().isEmpty()
            && currentIndex + 1 < playlist.getSongs().size();
    }
}

```

```

    }

    public boolean hasPrevious() {
        return playlist != null && currentIndex - 1 >= 0;
    }

    public Song next() {
        if (!hasNext()) throw new RuntimeException("No next song");
        return playlist.getSongs().get(++currentIndex);
    }

    public Song previous() {
        if (!hasPrevious()) throw new RuntimeException("No previous song");
        return playlist.getSongs().get(--currentIndex);
    }
}

class RandomPlayingStrategy implements IPlayingStrategy {
    private Playlist playlist;
    private List<Song> remainingSongs = new ArrayList<>();
    private Deque<Song> history = new ArrayDeque<>();

    public void setPlaylist(Playlist playlist) {
        this.playlist = playlist;
        this.remainingSongs = new ArrayList<>(playlist.getSongs());
        this.history.clear();
    }

    public boolean hasNext() { return !remainingSongs.isEmpty(); }
    public boolean hasPrevious() { return !history.isEmpty(); }

    public Song next() {
        if (!hasNext()) throw new RuntimeException("No next song");
        int idx = (int) (Math.random() * remainingSongs.size());
        Song song = remainingSongs.remove(idx);
        history.push(song);
        return song;
    }

    public Song previous() {
        if (!hasPrevious()) throw new RuntimeException("No previous song");
        Song song = history.pop();
        remainingSongs.add(song);
        return song;
    }
}

class CustomQueueStrategy implements IPlayingStrategy {

```

```

private Playlist playlist;
private Queue<Song> nextQueue = new LinkedList<>();
private Deque<Song> history = new ArrayDeque<>();

public void setPlaylist(Playlist playlist) {
    this.playlist = playlist;
    nextQueue.clear();
    history.clear();
}

public void addToNext(Song song) { nextQueue.offer(song); }

public boolean hasNext() {
    return !nextQueue.isEmpty()
        || (playlist != null && !playlist.getSongs().isEmpty());
}

public boolean hasPrevious() { return !history.isEmpty(); }

public Song next() {
    Song song;
    if (!nextQueue.isEmpty()) {
        song = nextQueue.poll();
    } else {
        // fallback: sequential
        song = playlist.getSongs().get(0);
    }
    history.push(song);
    return song;
}

public Song previous() {
    if (!hasPrevious()) throw new RuntimeException("No previous song");
    return history.pop();
}
}

```

Strategy Manager

java

```

class PlayingStrategyManager {
    private static PlayingStrategyManager instance;

    private PlayingStrategyManager() {}

    public static PlayingStrategyManager getInstance() {

```

```

        if (instance == null) {
            synchronized (PlayingStrategyManager.class) {
                if (instance == null) instance = new PlayingStrategyManager();
            }
        }
        return instance;
    }

    public IPlayingStrategy createStrategy(PlayingStrategyType type) {
        switch (type) {
            case SEQUENTIAL: return new SequentialPlayingStrategy();
            case RANDOM:      return new RandomPlayingStrategy();
            case CUSTOM:      return new CustomQueueStrategy();
        }
        return null;
    }
}

```

Facade

java

```

class MusicPlayerFacade {
    private static MusicPlayerFacade instance;
    private AudioEngine audioEngine = new AudioEngine();
    private IPlayingStrategy strategy;
    private Playlist currentPlaylist;

    private MusicPlayerFacade() {}

    public static MusicPlayerFacade getInstance() {
        if (instance == null) {
            synchronized (MusicPlayerFacade.class) {
                if (instance == null) instance = new MusicPlayerFacade();
            }
        }
        return instance;
    }

    public void connectDevice(DeviceType type) {
        DeviceManager.getInstance().connect(type);
    }

    public void setStrategy(PlayingStrategyType type) {
        strategy = PlayingStrategyManager.getInstance().createStrategy(type);
    }
}

```

```

public void loadPlaylist(String name) {
    currentPlaylist = PlaylistManager.getInstance().getPlaylist(name);
    if (strategy == null) setStrategy(PlayingStrategyType.SEQUENTIAL);
    strategy.setPlaylist(currentPlaylist);
    System.out.println("Playlist loaded: " + name);
}

public void playSong(Song song) {
    if (!DeviceManager.getInstance().hasOutputDevice())
        throw new RuntimeException("No output device connected");
    audioEngine.play(DeviceManager.getInstance().getOutputDevice(), song);
}

public void pauseSong() { audioEngine.pause(); }

public void playAllTracks() {
    while (strategy.hasNext()) {
        playSong(strategy.next());
    }
}

public void playNextTrack() {
    if (strategy.hasNext()) playSong(strategy.next());
    else System.out.println("End of playlist");
}

public void playPreviousTrack() {
    if (strategy.hasPrevious()) playSong(strategy.previous());
    else System.out.println("No previous song");
}

public void enqueueNext(Song song) { strategy.addToNext(song); }
}

```

Application (Orchestrator)

java

```

class MusicPlayerApp {
    private List<Song> allSongs = new ArrayList<>();

    public Song createSong(String title, String artist, String path) {
        Song s = new Song(title, artist, path);
        allSongs.add(s);
        return s;
    }
}

```

```

    public Song findSongByTitle(String title) {
        return allSongs.stream().filter(s -> s.getTitle().equals(title)).findFirst().orElse(null);
    }

    public void createPlaylist(String name) { PlaylistManager.getInstance().createPlaylist(name); }

    public void addSongToPlaylist(String name, Song s) { PlaylistManager.getInstance().addSongToPlaylist(name, s); }

    public void connectAudioDevice(DeviceType type) { MusicPlayerFacade.getInstance().connectDevice(type); }

    public void selectPlayStrategy(PlayingStrategyType type) { MusicPlayerFacade.getInstance().setStrategy(type); }

    public void loadPlaylist(String name) { MusicPlayerFacade.getInstance().loadPlaylist(name); }

    public void playSong(Song s) { MusicPlayerFacade.getInstance().playSong(s); }
    public void pause() { MusicPlayerFacade.getInstance().pauseSong(); }
    public void playAllTracks() { MusicPlayerFacade.getInstance().playAllTracks(); }
    public void playNextTrack() { MusicPlayerFacade.getInstance().playNextTrack(); }
    public void playPreviousTrack() { MusicPlayerFacade.getInstance().playPreviousTrack(); }

    public void enqueueNext(Song s) { MusicPlayerFacade.getInstance().enqueueNext(s); }
}

```

Client (Main)

java

```

public class Main {
    public static void main(String[] args) {
        MusicPlayerApp app = new MusicPlayerApp();

        Song s1 = app.createSong("Kesariya", "Arijit Singh", "/songs/kesariya.mp3");
        Song s2 = app.createSong("Chaiyya Chaiyya", "Sukhwinder", "/songs/chaiyya.mp3");
        Song s3 = app.createSong("Tum Hi Ho", "Arijit Singh", "/songs/tumhiho.mp3");
        Song s4 = app.createSong("Jai Ho", "A.R. Rahman", "/songs/jaiho.mp3");
        Song s5 = app.createSong("Zinda", "Siddharth", "/songs/zinda.mp3");

        app.createPlaylist("Bollywood Vibes");
        app.addSongToPlaylist("Bollywood Vibes", s1);
        app.addSongToPlaylist("Bollywood Vibes", s2);
        app.addSongToPlaylist("Bollywood Vibes", s3);
        app.addSongToPlaylist("Bollywood Vibes", s4);

        app.connectAudioDevice(DeviceType.BLUETOOTH);
        app.playSong(s1);
        app.pause();
    }
}

```



```

app.playSong(s1);

app.selectPlayStrategy(PlayingStrategyType.SEQUENTIAL);
app.loadPlaylist("Bollywood Vibes");
app.playAllTracks();

app.selectPlayStrategy(PlayingStrategyType.RANDOM);
app.loadPlaylist("Bollywood Vibes");
app.playAllTracks();

app.selectPlayStrategy(PlayingStrategyType.CUSTOM);
app.loadPlaylist("Bollywood Vibes");
app.enqueueNext(s1);
app.enqueueNext(s3);
app.playNextTrack();
app.playNextTrack();
app.playNextTrack();
    }
}

```

Sample Output:

text

```

BLUETOOTH device connected
Playing via Bluetooth: Kesariya by Arijit Singh
Paused: Kesariya
Playing via Bluetooth: Kesariya by Arijit Singh
Playlist loaded: Bollywood Vibes
Playing via Bluetooth: Kesariya by Arijit Singh
Playing via Bluetooth: Chaiyya Chaiyya by Sukhwinder
Playing via Bluetooth: Tum Hi Ho by Arijit Singh
Playing via Bluetooth: Jai Ho by A.R. Rahman
Playlist loaded: Bollywood Vibes
Playing via Bluetooth: Tum Hi Ho by Arijit Singh
Playing via Bluetooth: Kesariya by Arijit Singh
Playing via Bluetooth: Chaiyya Chaiyya by Sukhwinder
Playing via Bluetooth: Jai Ho by A.R. Rahman
...

```

Part 11: Python Implementation (Key Sections)

python

```
from abc import ABC, abstractmethod
from enum import Enum
from collections import deque
import random
import threading
```

```
class DeviceType(Enum):
    BLUETOOTH = 1
    WIRED = 2
    HEADPHONES = 3
```

```
class PlayingStrategyType(Enum):
    SEQUENTIAL = 1
    RANDOM = 2
    CUSTOM = 3
```

```
# ===== MODELS =====
```

```
class Song:
    def __init__(self, title, artist, path):
        self.title = title
        self.artist = artist
        self.path = path
```

```
class Playlist:
    def __init__(self, name):
        self.name = name
        self.songs = []

    def add_song(self, song):
        self.songs.append(song)
```

```
# ===== EXTERNAL APIS =====
```

```
class BluetoothSpeakerAPI:
    def play_sound_via_bluetooth(self, data):
        print(f"Playing via Bluetooth: {data}")
```

```
class WiredSpeakerAPI:
    def play_sound_via_wired(self, data):
        print(f"Playing via Wired Speaker: {data}")
```

```
class HeadphonesAPI:
    def play_sound_via_jack(self, data):
        print(f"Playing via Headphones: {data}")
```

```
# ===== ADAPTERS =====

class IAudioOutputDevice(ABC):
    @abstractmethod
    def play_audio(self, song): pass

class BluetoothSpeakerAdapter(IAudioOutputDevice):
    def __init__(self, api): self.api = api
    def play_audio(self, song):
        self.api.play_sound_via_bluetooth(f"{song.title} by {song.artist}")

class WiredSpeakerAdapter(IAudioOutputDevice):
    def __init__(self, api): self.api = api
    def play_audio(self, song):
        self.api.play_sound_via_wired(f"{song.title} by {song.artist}")

class HeadphonesAdapter(IAudioOutputDevice):
    def __init__(self, api): self.api = api
    def play_audio(self, song):
        self.api.play_sound_via_jack(f"{song.title} by {song.artist}")

# ===== FACTORY =====

class DeviceFactory:
    @staticmethod
    def create_device(device_type):
        if device_type == DeviceType.BLUETOOTH:
            return BluetoothSpeakerAdapter(BluetoothSpeakerAPI())
        elif device_type == DeviceType.WIRED:
            return WiredSpeakerAdapter(WiredSpeakerAPI())
        elif device_type == DeviceType.HEADPHONES:
            return HeadphonesAdapter(HeadphonesAPI())
        return None

# ===== AUDIO ENGINE =====

class AudioEngine:
    def __init__(self):
        self.current_song = None
        self.is_song_paused = False

    def play(self, device, song):
        if song is None:
            raise RuntimeError("Song cannot be null")
        if self.is_song_paused and self.current_song and \
```

```

        self.current_song.title == song.title:
            self.is_song_paused = False
            print(f"Resuming: {song.title}")
        else:
            self.current_song = song
            self.is_song_paused = False
        device.play_audio(song)

    def pause(self):
        if self.current_song is None:
            raise RuntimeError("No song playing")
        self.is_song_paused = True
        print(f"Paused: {self.current_song.title}")

# ===== SINGLETON MANAGERS =====
class DeviceManager:
    _instance = None
    _lock = threading.Lock()

    def __new__(cls):
        with cls._lock:
            if cls._instance is None:
                cls._instance = super().__new__(cls)
                cls._instance.current_device = None
            return cls._instance

    def connect(self, device_type):
        self.current_device = DeviceFactory.create_device(device_type)
        print(f"{device_type.name} device connected")

    def get_output_device(self): return self.current_device
    def has_output_device(self): return self.current_device is not None

class PlaylistManager:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance.playlists = {}
        return cls._instance

    def create_playlist(self, name):
        if name in self.playlists:
            raise RuntimeError(f"Playlist exists: {name}")
        self.playlists[name] = Playlist(name)

```

```

def add_song(self, playlist_name, song):
    if playlist_name not in self.playlists:
        raise RuntimeError(f"Playlist not found: {playlist_name}")
    self.playlists[playlist_name].add_song(song)

def get_playlist(self, name):
    return self.playlists[name]

# ===== STRATEGIES =====
class IPlayingStrategy(ABC):
    @abstractmethod
    def set_playlist(self, playlist): pass
    @abstractmethod
    def next(self): pass
    @abstractmethod
    def previous(self): pass
    @abstractmethod
    def has_next(self): pass
    @abstractmethod
    def has_previous(self): pass
    def add_to_next(self, song): pass

class SequentialPlayingStrategy(IPlayingStrategy):
    def __init__(self):
        self.playlist = None
        self.current_index = -1

    def set_playlist(self, playlist):
        self.playlist = playlist
        self.current_index = -1

    def has_next(self):
        return self.playlist and self.current_index + 1 < len(self.playlist.songs)

    def has_previous(self):
        return self.playlist and self.current_index - 1 >= 0

    def next(self):
        if not self.has_next(): raise RuntimeError("No next")
        self.current_index += 1
        return self.playlist.songs[self.current_index]

    def previous(self):
        if not self.has_previous(): raise RuntimeError("No previous")
        self.current_index -= 1

```

```
        return self.playlist.songs[self.current_index]

class RandomPlayingStrategy(IPlayingStrategy):
    def __init__(self):
        self.playlist = None
        self.remaining = []
        self.history = []

    def set_playlist(self, playlist):
        self.playlist = playlist
        self.remaining = list(playlist.songs)
        self.history = []

    def has_next(self): return len(self.remaining) > 0
    def has_previous(self): return len(self.history) > 0

    def next(self):
        if not self.has_next(): raise RuntimeError("No next")
        idx = random.randint(0, len(self.remaining) - 1)
        song = self.remaining.pop(idx)
        self.history.append(song)
        return song

    def previous(self):
        if not self.has_previous(): raise RuntimeError("No previous")
        song = self.history.pop()
        self.remaining.append(song)
        return song

class CustomQueueStrategy(IPlayingStrategy):
    def __init__(self):
        self.playlist = None
        self.next_queue = deque()
        self.history = []

    def set_playlist(self, playlist):
        self.playlist = playlist
        self.next_queue.clear()
        self.history.clear()

    def add_to_next(self, song): self.next_queue.append(song)

    def has_next(self):
        return len(self.next_queue) > 0 or (self.playlist and self.playlist.songs)

    def has_previous(self): return len(self.history) > 0
```

```

def next(self):
    if self.next_queue:
        song = self.next_queue.popleft()
    else:
        song = self.playlist.songs[0]
    self.history.append(song)
    return song

def previous(self):
    if not self.has_previous(): raise RuntimeError("No previous")
    return self.history.pop()

class PlayingStrategyManager:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    def create_strategy(self, type_):
        if type_ == PlayingStrategyType.SEQUENTIAL:
            return SequentialPlayingStrategy()
        if type_ == PlayingStrategyType.RANDOM:
            return RandomPlayingStrategy()
        if type_ == PlayingStrategyType.CUSTOM:
            return CustomQueueStrategy()
        return None

# ===== FACADE =====
class MusicPlayerFacade:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance.audio_engine = AudioEngine()
            cls._instance.strategy = None
            cls._instance.current_playlist = None
        return cls._instance

    def connect_device(self, type_):
        DeviceManager().connect(type_)

    def set_strategy(self, type_):

```

```

self.strategy = PlayingStrategyManager().create_strategy(type_)

def load_playlist(self, name):
    self.current_playlist = PlaylistManager().get_playlist(name)
    if self.strategy is None:
        self.set_strategy(PlayingStrategyType.SEQUENTIAL)
    self.strategy.set_playlist(self.current_playlist)
    print(f"Playlist loaded: {name}")

def play_song(self, song):
    if not DeviceManager().has_output_device():
        raise RuntimeError("No device connected")
    self.audio_engine.play(DeviceManager().get_output_device(), song)

def pause_song(self):
    self.audio_engine.pause()

def play_all_tracks(self):
    while self.strategy.has_next():
        self.play_song(self.strategy.next())

def play_next_track(self):
    if self.strategy.has_next():
        self.play_song(self.strategy.next())

def play_previous_track(self):
    if self.strategy.has_previous():
        self.play_song(self.strategy.previous())

def enqueue_next(self, song):
    self.strategy.add_to_next(song)

# ===== APPLICATION =====
class MusicPlayerApp:
    def __init__(self):
        self.all_songs = []

    def create_song(self, title, artist, path):
        s = Song(title, artist, path)
        self.all_songs.append(s)
        return s

    def find_song_by_title(self, title):
        for s in self.all_songs:
            if s.title == title: return s
        return None

```



```

def create_playlist(self, name): PlaylistManager().create_playlist(name)
def add_song_to_playlist(self, name, song): PlaylistManager().add_song(name, song)
def connect_audio_device(self, t): MusicPlayerFacade().connect_device(t)
def select_play_strategy(self, t): MusicPlayerFacade().set_strategy(t)
def load_playlist(self, name): MusicPlayerFacade().load_playlist(name)
def play_song(self, s): MusicPlayerFacade().play_song(s)
def pause(self): MusicPlayerFacade().pause_song()
def play_all_tracks(self): MusicPlayerFacade().play_all_tracks()
def play_next_track(self): MusicPlayerFacade().play_next_track()
def play_previous_track(self): MusicPlayerFacade().play_previous_track()
def enqueue_next(self, s): MusicPlayerFacade().enqueue_next(s)

if __name__ == "__main__":
    app = MusicPlayerApp()
    s1 = app.create_song("Kesariya", "Arijit Singh", "/songs/kesariya.mp3")
    s2 = app.create_song("Chaiyya Chaiyya", "Sukhwinder", "/songs/chaiyya.mp3")
    s3 = app.create_song("Tum Hi Ho", "Arijit Singh", "/songs/tumhiho.mp3")
    s4 = app.create_song("Jai Ho", "A.R. Rahman", "/songs/jaiho.mp3")

    app.create_playlist("Bollywood Vibes")
    app.add_song_to_playlist("Bollywood Vibes", s1)
    app.add_song_to_playlist("Bollywood Vibes", s2)
    app.add_song_to_playlist("Bollywood Vibes", s3)
    app.add_song_to_playlist("Bollywood Vibes", s4)

    app.connect_audio_device(DeviceType.BLUETOOTH)
    app.play_song(s1)
    app.pause()

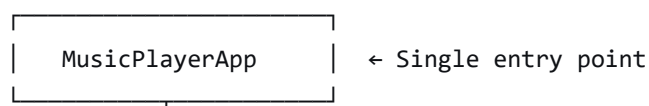
    app.select_play_strategy(PlayingStrategyType.SEQUENTIAL)
    app.load_playlist("Bollywood Vibes")
    app.play_all_tracks()

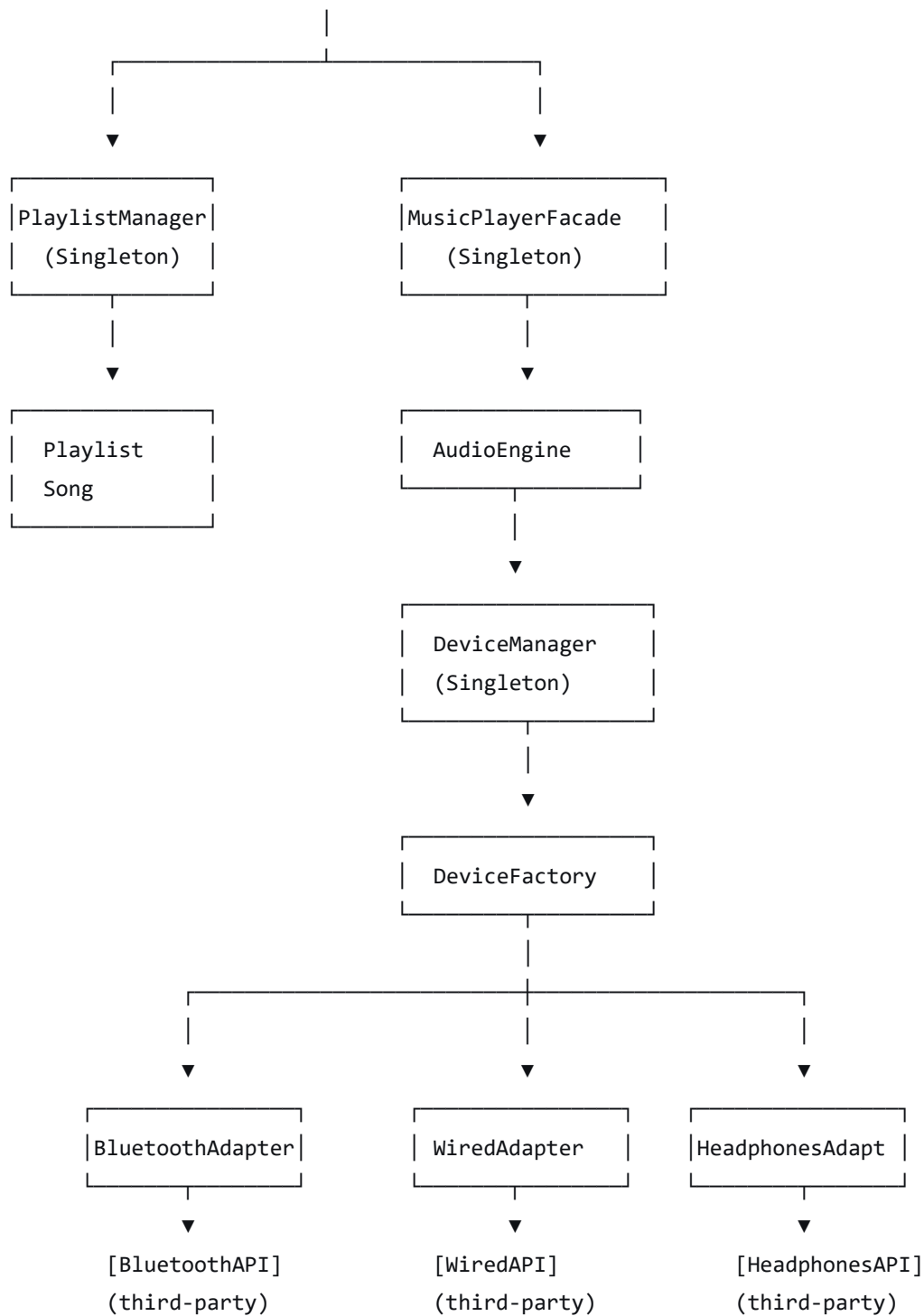
    app.select_play_strategy(PlayingStrategyType.RANDOM)
    app.load_playlist("Bollywood Vibes")
    app.play_all_tracks()

```

Part 12: Architecture Recap

text





Plus PlayingStrategyManager + IPlayingStrategy hierarchy for playback modes.

Part 13: SOLID Principles Applied

Principle

How Applied

SRP

Each class has one responsibility (AudioEngine plays, DeviceManager manages devices, etc.)

Principle	How Applied
OCP	New devices, strategies added without modifying existing code
LSP	All strategies fully substitutable; all adapters substitutable as <code>IAudioOutputDevice</code>
ISP	Strategy interface has only relevant methods; empty <code>addToNext</code> is a trade-off
DIP	Facade depends on abstractions (<code>IAudioOutputDevice</code> , <code>IPlayingStrategy</code>)

Key Takeaways

1. **Music Player** is a great interview problem showcasing **multiple design patterns working together**
2. **5 design patterns used** — Strategy, Singleton, Adapter, Factory, Facade
3. **Managers** are central points of control — always Singleton
4. **Facade** hides entire subsystem behind one simple interface
5. **Adapter** bridges third-party device APIs to our interface
6. **Strategy** enables runtime selection of playback mode
7. **Bottom-up approach** — build smaller components first, compose them
8. **Application = single entry point** — client only knows `MusicPlayerApp`
9. **Playlist Manager uses Map** for O(1) playlist lookup
10. **Extensible at every level** — new devices, strategies, and features

"Real-world LLD combines many patterns. Music Player demonstrates how Strategy, Singleton, Adapter, Factory, and Facade work together to build a clean, extensible application."



हां जी, करें स्टार्ट। सो, वेलकम ...
TXT 102.03KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python

also

Composite Design Pattern — Complete Notes

Part 1: Introduction

What Is Composite Pattern?

The **Composite Design Pattern** composes objects into **tree-like structures** to represent **part-whole hierarchies**. It lets clients treat **individual objects** (leaves) and **compositions** (containers) **uniformly**.

When to Use It

Whenever a problem can be expressed as a **tree with two node types**:

- **Leaf nodes** — end points (no children)
- **Composite nodes** — containers (can hold leaves or other composites)

Classic Example: File System — folders contain files and other folders.

Connection to DSA

If you've studied trees in DSA, you already understand the structure:

- **Root node** → Composite
- **Internal nodes** → Composite
- **Leaf nodes** → Leaf

Part 2: The Problem — File System Design

Without Composite Pattern

You'd create:

- **File class** — has name, size, content, open()
- **Folder class** — has name, list of files, list of folders

Problem: To iterate through everything, you'd need:

java

```
if (element instanceof File) {
    // print file name
} else if (element instanceof Folder) {
    // recurse into folder
}
```

- Multiple lists (files and folders) → can't preserve order
- Type checking everywhere → ugly code
- New methods (e.g., `cd`, `getSize`) require more `if-else` chains

The Core Problem

File and Folder are treated differently, but they should be treated **uniformly**.

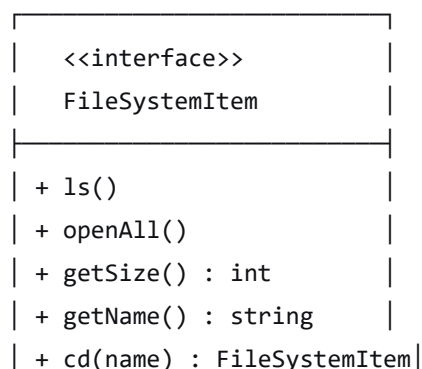
Part 3: The Composite Solution

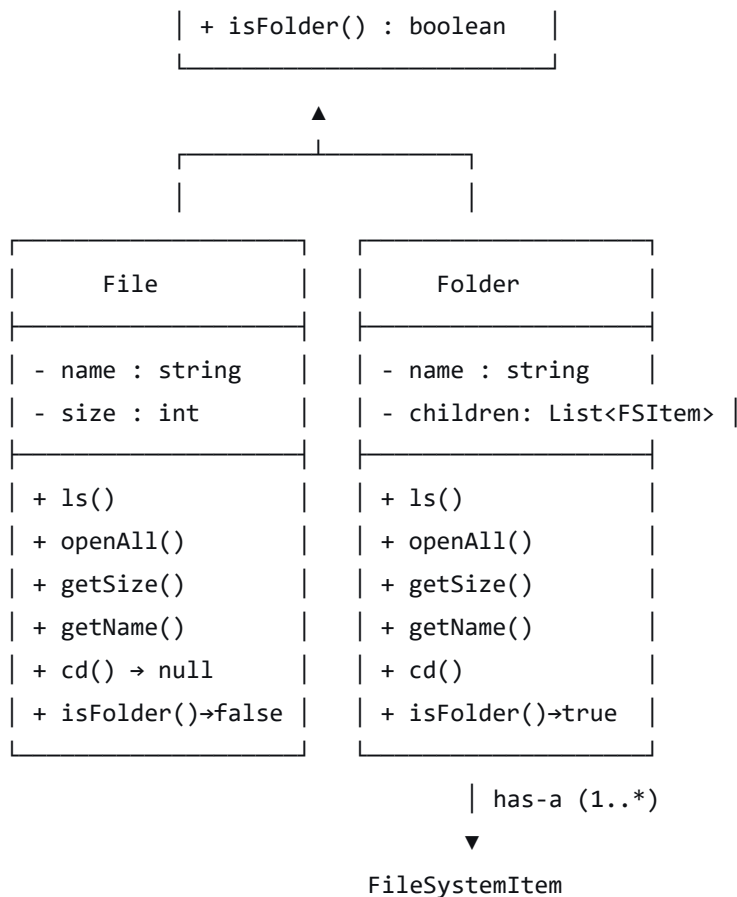
Key Idea

1. Introduce a **common interface** (or abstract class) for both File and Folder
2. Both "are-a" this common interface
3. The **Folder** additionally "has-a" list of the common interface
4. Client treats both uniformly — no `if-else`, no type checking

Class Diagram

text





Key Relationships:

- File **IS-A** FileSystemItem
- Folder **IS-A** FileSystemItem
- Folder **HAS-A** List of FileSystemItem (1-to-many)

This dual IS-A + HAS-A is similar to **Decorator Pattern**.

Part 4: How Recursion Works

Imagine this hierarchy:

text

Root/

├─ File1.txt

├─ File2.txt

├─ Docs/

│ └─ Resume.pdf

│ └─ Notes.txt

```
└─ Images/
  └─ Photo.jpg
```

openAll() on Root — Execution Trace

text

```
Root.openAll() → prints "Root"
  → for each child: child.openAll()
    └─ File1.openAll() → prints "File1.txt" (leaf)
    └─ File2.openAll() → prints "File2.txt" (leaf)
    └─ Docs.openAll() → prints "Docs"
      |   → Resume.openAll() → prints "Resume.pdf"
      |   → Notes.openAll() → prints "Notes.txt"
    └─ Images.openAll() → prints "Images"
      |   → Photo.openAll() → prints "Photo.jpg"
```

No if-else needed! Each node knows how to handle itself.

getSize() on Root — Execution Trace

text

```
Root.getSize() → sums sizes of children
  └─ File1.getSize() → 1
  └─ File2.getSize() → 1
  └─ Docs.getSize()
    |   └─ Resume.getSize() → 1
    |   └─ Notes.getSize() → 1
    |   = 2
  └─ Images.getSize()
    |   └─ Photo.getSize() → 1
    |   = 1
Total: 1 + 1 + 2 + 1 = 5
```

Part 5: Java Implementation

java

```
import java.util.ArrayList;
import java.util.List;
```

```
// ===== COMPONENT INTERFACE =====  
interface FileSystemItem {  
    void ls();  
    void openAll();  
    int getSize();  
    String getName();  
    FileSystemItem cd(String name);  
    boolean isFolder();  
}  
  
// ===== LEAF: FILE =====  
class File implements FileSystemItem {  
    private String name;  
    private int size;  
  
    public File(String name, int size) {  
        this.name = name;  
        this.size = size;  
    }  
  
    @Override  
    public void ls() {  
        System.out.println(name);  
    }  
  
    @Override  
    public void openAll() {  
        System.out.println(name);  
    }  
  
    @Override  
    public int getSize() { return size; }  
  
    @Override  
    public String getName() { return name; }  
  
    @Override  
    public FileSystemItem cd(String target) {  
        return null;    // cannot cd into a file  
    }  
  
    @Override  
    public boolean isFolder() { return false; }  
}  
  
// ===== COMPOSITE: FOLDER =====  
class Folder implements FileSystemItem {  
    private String name;
```



```
private List<FileSystemItem> children = new ArrayList<>();

public Folder(String name) {
    this.name = name;
}

public void add(FileSystemItem item) {
    children.add(item);
}

@Override
public void ls() {
    for (FileSystemItem child : children) {
        System.out.println(child.getName());
    }
}

@Override
public void openAll() {
    System.out.println(name);
    for (FileSystemItem child : children) {
        child.openAll();
    }
}

@Override
public int getSize() {
    int total = 0;
    for (FileSystemItem child : children) {
        total += child.getSize();
    }
    return total;
}

@Override
public String getName() { return name; }

@Override
public FileSystemItem cd(String target) {
    for (FileSystemItem child : children) {
        if (child.isFolder() && child.getName().equals(target)) {
            return child;
        }
    }
    return null;
}

@Override
```

```

    public boolean isFolder() { return true; }
}

// ===== CLIENT =====
public class Main {
    public static void main(String[] args) {
        Folder root = new Folder("Root");
        root.add(new File("File1.txt", 1));
        root.add(new File("File2.txt", 1));

        Folder docs = new Folder("Docs");
        docs.add(new File("Resume.pdf", 1));
        docs.add(new File("Notes.txt", 1));
        root.add(docs);

        Folder images = new Folder("Images");
        images.add(new File("Photo.jpg", 1));
        root.add(images);

        System.out.println("=== openAll() on Root ===");
        root.openAll();

        System.out.println("\n=== ls() on Root ===");
        root.ls();

        System.out.println("\n=== getSize() on Root ===");
        System.out.println("Total Size: " + root.getSize());

        System.out.println("\n=== cd('Docs') on Root ===");
        FileSystemItem cwd = root.cd("Docs");
        if (cwd != null) cwd.ls();
    }
}

```

Output:

text

```

=== openAll() on Root ===
Root
File1.txt
File2.txt
Docs
Resume.pdf
Notes.txt
Images
Photo.jpg

```

```
=== ls() on Root ===
File1.txt
File2.txt
Docs
Images

=== getSize() on Root ===
Total Size: 5

=== cd('Docs') on Root ===
Resume.pdf
Notes.txt
```

Code Walkthrough

Method	File Behavior	Folder Behavior
ls()	Prints own name	Prints children names
openAll()	Prints own name	Prints own name, then recurses into all children
getSize()	Returns own size	Returns sum of children's sizes
cd(name)	Returns null (not applicable)	Finds folder child by name
isFolder()	false	true

Key Insight: Client just calls `item.openAll()` — no type checks needed. Polymorphism handles it.

Part 6: Python Implementation

```
python

from abc import ABC, abstractmethod

# ===== COMPONENT INTERFACE =====
class FileSystemItem(ABC):
    @abstractmethod
    def ls(self): pass
```

```
@abstractmethod
def open_all(self): pass

@abstractmethod
def get_size(self): pass

@abstractmethod
def get_name(self): pass

@abstractmethod
def cd(self, target): pass

@abstractmethod
def is_folder(self): pass

# ===== LEAF: FILE =====
class File(FileSystemItem):
    def __init__(self, name, size):
        self.name = name
        self.size = size

    def ls(self):
        print(self.name)

    def open_all(self):
        print(self.name)

    def get_size(self):
        return self.size

    def get_name(self):
        return self.name

    def cd(self, target):
        return None    # cannot cd into a file

    def is_folder(self):
        return False

# ===== COMPOSITE: FOLDER =====
class Folder(FileSystemItem):
    def __init__(self, name):
        self.name = name
        self.children = []

    def add(self, item):
```

```

        self.children.append(item)

    def ls(self):
        for child in self.children:
            print(child.get_name())

    def open_all(self):
        print(self.name)
        for child in self.children:
            child.open_all()

    def get_size(self):
        return sum(child.get_size() for child in self.children)

    def get_name(self):
        return self.name

    def cd(self, target):
        for child in self.children:
            if child.is_folder() and child.get_name() == target:
                return child
        return None

    def is_folder(self):
        return True

# ===== CLIENT =====
if __name__ == "__main__":
    root = Folder("Root")
    root.add(File("File1.txt", 1))
    root.add(File("File2.txt", 1))

    docs = Folder("Docs")
    docs.add(File("Resume.pdf", 1))
    docs.add(File("Notes.txt", 1))
    root.add(docs)

    images = Folder("Images")
    images.add(File("Photo.jpg", 1))
    root.add(images)

    print("=== open_all() on Root ===")
    root.open_all()

    print("\n=== ls() on Root ===")
    root.ls()

```

```

print("\n=== get_size() on Root ===")
print(f"Total Size: {root.get_size()}")

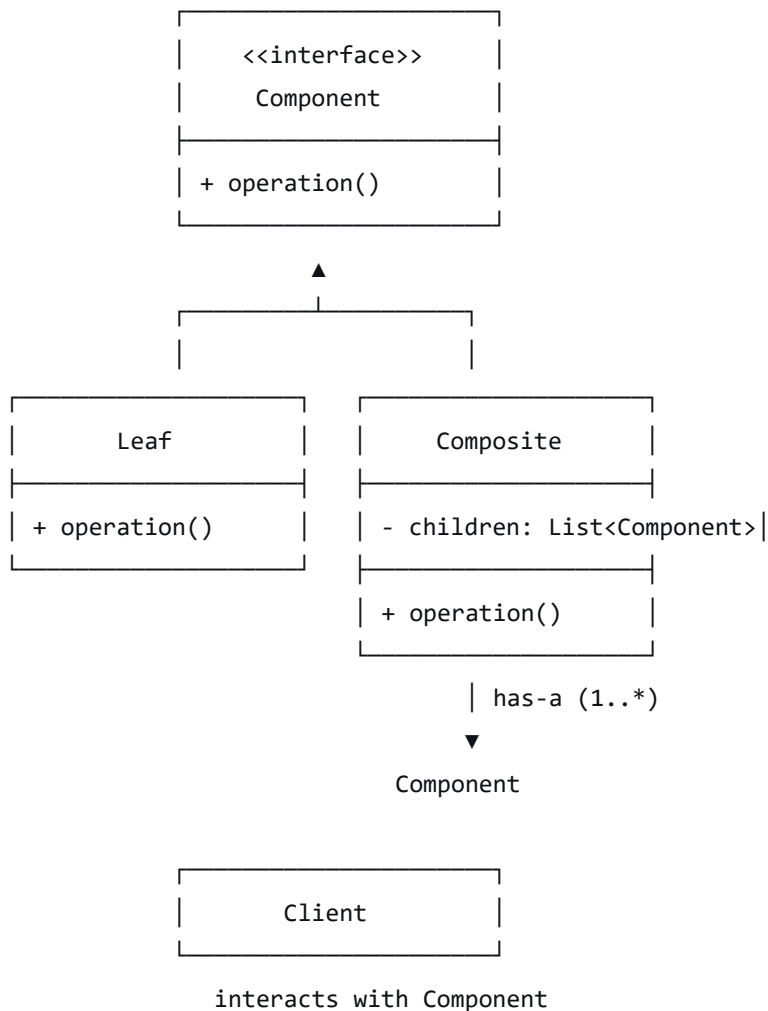
print("\n=== cd('Docs') on Root ===")
cwd = root.cd("Docs")
if cwd:
    cwd.ls()

```

Output: Same as Java.

Part 7: Standard UML Diagram

text



Key Points:

- Leaf IS-A Component
- Composite IS-A Component

- Composite HAS-A List of Component
- Client only knows Component

Part 8: Definition

Composite Pattern composes objects into tree structures to represent part-whole hierarchies. Composite lets clients treat individual objects and compositions of objects uniformly.

Key Phrases

- **Tree structures** — hierarchical data
- **Part-whole hierarchy** — composite (whole) contains parts (leaves or other composites)
- **Treat uniformly** — client doesn't distinguish between leaf and composite

Part 9: Real-World Use Cases

1. File System

- Folders and files (our example)

2. GUI Component Hierarchy

- Window contains Panels, Panels contain Buttons/TextFields
- Render all → recurse

3. Menu / Dropdown

- Menu has submenus and items
- File → Save As → PDF / Text

4. Organization Chart

- CEO → Managers → Employees

- Compute total salary

5. HTML/XML DOM

- Elements contain other elements and text nodes

6. Arithmetic Expressions

- $(a + b) * c$ — Expression tree

7. Shopping Cart

- Individual products and product bundles

Part 10: Advantages & Trade-offs

Advantages

1. **Uniform treatment** — client treats leaves and composites the same
2. **Recursive structure** — natural fit for tree problems
3. **Open/Closed** — new component types without changing client
4. **Simplified client code** — no `if-else` type checking
5. **Flexible hierarchy** — any depth of nesting

Trade-offs

1. **Over-generalization** — some methods don't make sense for leaves (e.g., `cd` on a file)
2. **Type safety loss** — client may call methods that return null or do nothing
3. **Hard to restrict children types** — a folder can contain any `FileSystemItem`

Part 11: When to Use Composite Pattern

✓ Use When:

- Data is hierarchical/tree-like

- Need part-whole hierarchies
- Want uniform treatment of leaves and composites
- Operations naturally recurse (size, print, render, validate)
- Client shouldn't care about node types

✗ Avoid When:

- Data is not hierarchical
- Leaves and composites have fundamentally different interfaces
- Type checking is unavoidable

Part 12: Composite vs Decorator

Aspect	Composite	Decorator
Intent	Part-whole hierarchy	Add responsibilities
Structure	Tree	Chain
Relationships	IS-A + HAS-A (1..*)	IS-A + HAS-A (1)
Children	Multiple	One wrapped object
Example	File system	Java I/O streams

Both use **IS-A + HAS-A** — but Composite wraps **many**, Decorator wraps **one**.

Key Takeaways

1. **Composite Pattern** = uniform treatment of tree-structured data
2. **Two node types**: Leaf (end point) and Composite (container)
3. **Common interface** — both implement same methods
4. **Recursion** — composites recurse into children
5. **Client doesn't distinguish** — just calls interface methods
6. **Classic example**: File System (folders and files)
7. **Uses IS-A + HAS-A** relationships

8. **Eliminates** `if-else` **type checks** in client
9. **Common in real world:** GUI, menus, XML, expressions
10. **Tree-thinking essential** — DSA trees background helps enormously

"Composite Pattern: treat leaves and branches the same way, and let recursion handle the rest."



हां जी करें स्टार्ट। सो वेलकम बै...
TXT 54.42KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also

Template Method Design Pattern — Complete Notes

Part 1: Introduction

What is Template Method Pattern?

The **Template Method Pattern** defines the **skeleton of an algorithm** in a base class, deferring some steps to subclasses. It lets subclasses **redefine certain steps** of an algorithm **without changing its structure**.

Core Idea

- **Base class** defines the **order (skeleton)** of algorithm steps
- **Subclasses** implement/override the **individual steps**
- The **overall algorithm structure** remains unchanged
- The **implementation of each step** varies

Real-World Analogy: ML Pipeline

Every machine learning model follows the **same pipeline**:

1. **Load Data**
2. **Preprocess Data**
3. **Train Model**

4. Evaluate Model

5. Save Model

Whether you use Neural Network, Decision Tree, Random Forest, or SVM — the **order never changes**. Only the implementation differs.

Template Method enforces this order.

Part 2: The Problem — ML Training Pipeline

The Pipeline (Fixed Order)

text

1. Load Data
2. Preprocess Data
3. Train Model
4. Evaluate Model
5. Save Model

Every ML model must follow this order — you can't train before loading, can't save before evaluating, etc.

The Naive Approach (Without Template Method)

Each model class (NeuralNetwork, DecisionTree) would have its own training method that manually calls each step.

Problems:

- Developer can accidentally **reorder steps** (e.g., train before preprocess)
- **No enforced structure** — fragile
- **Duplicated code** across every model
- No **guarantee** of correct order

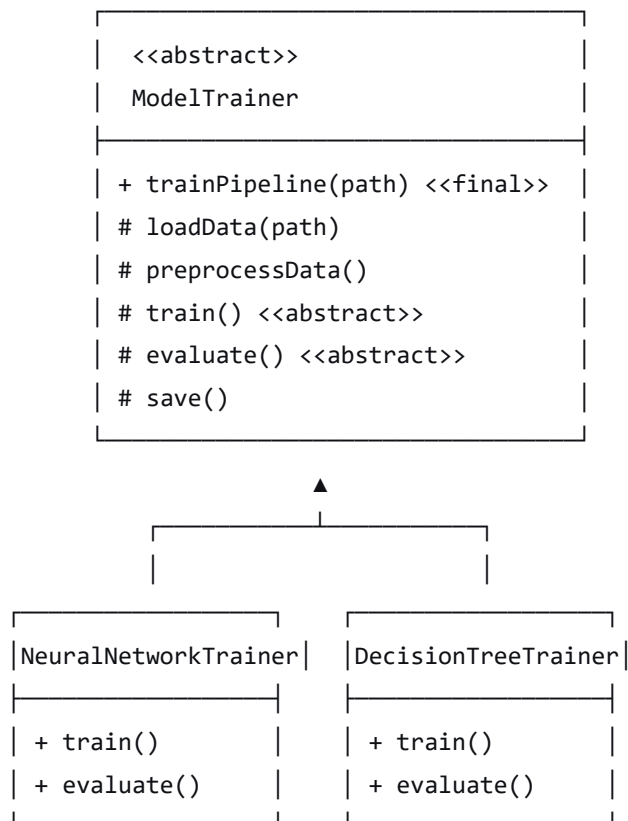
The Template Method Solution

1. Base class `ModelTrainer` defines the **template method** `trainPipeline()`
2. Template method calls steps in **fixed order**

3. Steps like `train()` , `evaluate()` are **abstract** — subclasses implement
4. Steps like `loadData()` , `preprocess()` , `save()` can be **default** — subclasses optionally override
5. `trainPipeline()` is **final** — cannot be overridden

Part 3: Class Diagram

text



Key Points:

- `trainPipeline()` is the **template method** (final)
- Some methods are **abstract** (`train` , `evaluate`) — must be overridden
- Some methods have **default implementations** (`loadData` , `preprocess` , `save`) — optionally overridden
- Subclasses **must** implement abstract methods
- Subclasses **can** override default methods

Part 4: How It Works

Template Method Flow

text

Client calls: `modelTrainer.trainPipeline("data.csv")`

Inside `trainPipeline()` – FIXED ORDER:

- 1. `loadData("data.csv")`
- 2. `preprocessData()`
- 3. `train()` ← subclass-specific
- 4. `evaluate()` ← subclass-specific
- 5. `save()`

The order is guaranteed by the base class. Subclasses cannot change it.

What Subclasses Control

Step	Type	Subclass Control
<code>loadData()</code>	Default	Optional override
<code>preprocessData()</code>	Default	Optional override
<code>train()</code>	Abstract	Must override
<code>evaluate()</code>	Abstract	Must override
<code>save()</code>	Default	Optional override

Part 5: Java Implementation

java

```
// ===== ABSTRACT CLASS WITH TEMPLATE METHOD =====
abstract class ModelTrainer {

    // Template method – the SKELETON. Cannot be overridden.
    public final void trainPipeline(String dataPath) {
        loadData(dataPath);
        preprocessData();
    }
}
```

```

        train();
        evaluate();
        save();
    }

    // Hook methods with DEFAULT implementations (can be overridden)
    protected void loadData(String path) {
        System.out.println("Loading data from: " + path);
    }

    protected void preprocessData() {
        System.out.println("Splitting into train/test, normalizing...");
    }

    // Abstract methods – MUST be implemented by subclasses
    protected abstract void train();
    protected abstract void evaluate();

    protected void save() {
        System.out.println("Saving model to disk.");
    }
}

// ===== CONCRETE CLASS: NEURAL NETWORK =====
class NeuralNetworkTrainer extends ModelTrainer {

    @Override
    protected void train() {
        System.out.println("Training Neural Network with 100 epochs.");
    }

    @Override
    protected void evaluate() {
        System.out.println("Evaluating Neural Network accuracy and loss.");
    }

    @Override
    protected void save() {
        System.out.println("Saving Neural Network model in .h5 format.");
    }
}

// ===== CONCRETE CLASS: DECISION TREE =====
class DecisionTreeTrainer extends ModelTrainer {

    @Override
    protected void train() {
        System.out.println("Training Decision Tree using Information Gain.");
    }
}

```

```
}

@Override
protected void evaluate() {
    System.out.println("Evaluating Decision Tree on validation set.");
}
// Uses default save() and loadData() and preprocessData()
}

// ===== CLIENT =====
public class Main {
    public static void main(String[] args) {
        ModelTrainer nn = new NeuralNetworkTrainer();
        nn.trainPipeline("data/nn_dataset.csv");

        System.out.println("\n-----\n");

        ModelTrainer dt = new DecisionTreeTrainer();
        dt.trainPipeline("data/dt_dataset.csv");
    }
}
```

Output:

text

```
Loading data from: data/nn_dataset.csv
Splitting into train/test, normalizing...
Training Neural Network with 100 epochs.
Evaluating Neural Network accuracy and loss.
Saving Neural Network model in .h5 format.
```

```
Loading data from: data/dt_dataset.csv
Splitting into train/test, normalizing...
Training Decision Tree using Information Gain.
Evaluating Decision Tree on validation set.
Saving model to disk.
```

Code Walkthrough

Method	Type	Overridden By
trainPipeline	Template (final)	Nobody

Method	Type	Overridden By
loadData	Default	Nobody (optional)
preprocessData	Default	Nobody (optional)
train	Abstract	Both subclasses
evaluate	Abstract	Both subclasses
save	Default	Only NeuralNetworkTrainer

Part 6: Python Implementation

python

```

from abc import ABC, abstractmethod

# ===== ABSTRACT CLASS WITH TEMPLATE METHOD =====
class ModelTrainer(ABC):

    # Template method – FIXED ORDER
    def train_pipeline(self, data_path: str):
        self.load_data(data_path)
        self.preprocess_data()
        self.train()
        self.evaluate()
        self.save()

    # Default hooks (can be overridden)
    def load_data(self, path: str):
        print(f"Loading data from: {path}")

    def preprocess_data(self):
        print("Splitting into train/test, normalizing...")

    # Abstract methods – MUST be implemented
    @abstractmethod
    def train(self): pass

    @abstractmethod
    def evaluate(self): pass

```



```

def save(self):
    print("Saving model to disk.")

# ===== CONCRETE CLASS: NEURAL NETWORK =====
class NeuralNetworkTrainer(ModelTrainer):
    def train(self):
        print("Training Neural Network with 100 epochs.")

    def evaluate(self):
        print("Evaluating Neural Network accuracy and loss.")

    def save(self):
        print("Saving Neural Network model in .h5 format.")

# ===== CONCRETE CLASS: DECISION TREE =====
class DecisionTreeTrainer(ModelTrainer):
    def train(self):
        print("Training Decision Tree using Information Gain.")

    def evaluate(self):
        print("Evaluating Decision Tree on validation set.")

# Uses default load_data, preprocess_data, save

# ===== CLIENT =====
if __name__ == "__main__":
    nn = NeuralNetworkTrainer()
    nn.train_pipeline("data/nn_dataset.csv")

    print("\n-----\n")

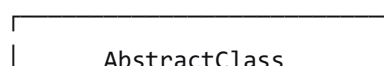
    dt = DecisionTreeTrainer()
    dt.train_pipeline("data/dt_dataset.csv")

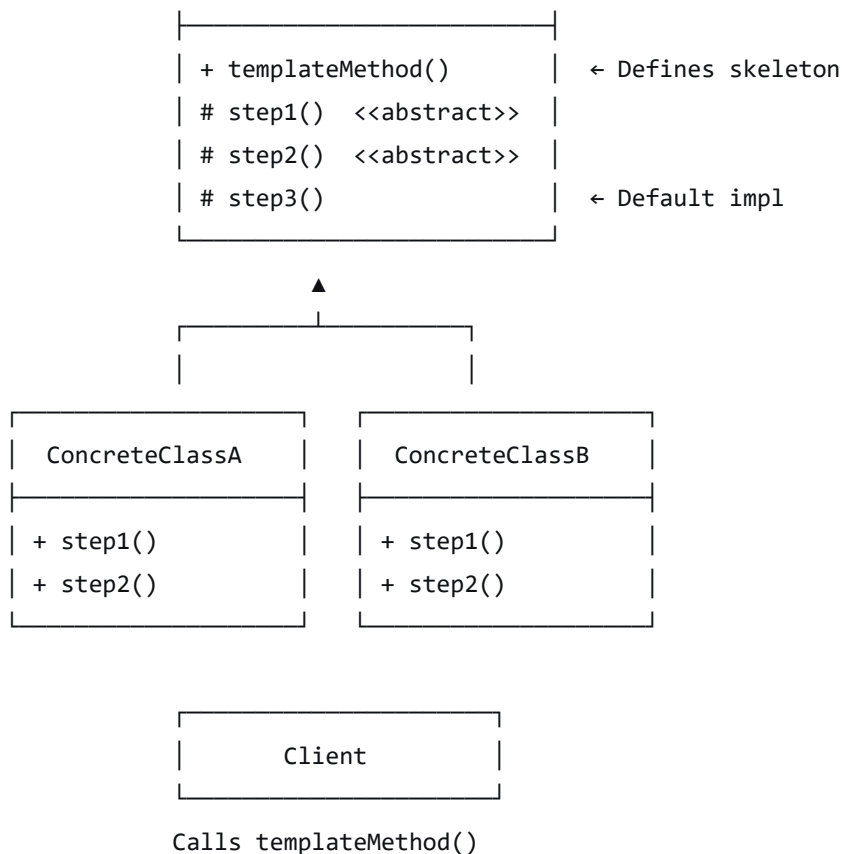
```

Output: Same as Java.

Part 7: Standard UML Diagram

text





Key Points:

- `templateMethod()` in **abstract class** defines the algorithm skeleton
- `step1()` and `step2()` are **abstract** — subclass-specific
- `step3()` may have **default implementation**
- Client only calls `templateMethod()` — never individual steps

Part 8: Definition

Template Method Pattern defines the skeleton of an algorithm in a method, deferring some steps to subclasses. Template Method lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.

Key Points

1. **Skeleton** = fixed sequence of steps
2. **Defer** certain steps = abstract/hook methods
3. **Redefine steps** = subclasses implement them
4. **Without changing structure** = order stays the same

Part 9: Template Method vs Similar Patterns

Pattern	Difference
Template Method	Defines algorithm skeleton, subclasses fill steps
Strategy	Interchangeable algorithms (composition)
Factory	Object creation
Observer	Notification on change

Template Method uses inheritance, Strategy uses **composition**.

Part 10: Hooks vs Abstract Methods

Abstract Methods

- **Must** be overridden by subclasses
- **No** default implementation
- Used when **every subclass must define its own version**

Hook Methods

- **Optional** to override
- Have **default implementation**
- Used when **common behavior exists but override is optional**

Example from ML Pipeline

Method	Type	Why
train()	Abstract	Every model trains differently
evaluate()	Abstract	Every model evaluates differently

Method	Type	Why
loadData()	Hook	Most models load similarly; optional override
preprocessData()	Hook	Common preprocessing; optional override
save()	Hook	Default save; NeuralNetwork overrides for .h5

Part 11: Real-World Use Cases

1. ML Training Pipelines

- Load → Preprocess → Train → Evaluate → Save

2. Payment Processing

- Validate → Debit → Credit → Confirm → Complete
- UPI, Credit Card, Net Banking — same pipeline, different step implementations

3. Data Migration

- Extract → Transform → Load (ETL)

4. Compilation Pipeline

- Lex → Parse → Semantic Analysis → Optimize → Codegen

5. Web Request Handling

- Authenticate → Authorize → Validate → Process → Respond

6. Game Loop

- Input → Update → Render → Repeat

7. File Parsing

- Open → Read Header → Parse Body → Validate → Close

8. Testing Framework

- Setup → Run Tests → Teardown

9. HTTP Request Pipeline

- Middleware chain: Auth → Rate Limit → Logging → Handler

10. Video Encoding

- Decode → Resize → Encode → Package

Part 12: Advantages & Trade-offs

Advantages

1. **Enforces algorithm structure** — clients can't reorder steps
2. **Code reuse** — common steps in base class
3. **Open/Closed** — new subclasses without modifying base
4. **Consistent behavior** — always the same pipeline
5. **Eliminates duplication** — common code lives in base
6. **Hollywood Principle** — "Don't call us, we'll call you"

Trade-offs

1. **Rigid structure** — hard to change step order
2. **Inheritance-bound** — subclasses tightly coupled to base
3. **Limited flexibility** — strategy might be better for fully interchangeable algorithms
4. **Abstract method overhead** — every subclass must implement all abstract methods

Part 13: When to Use Template Method

✅ Use When:

- Multiple classes follow the **same algorithm structure**
- Only **individual steps vary** between classes
- Need to **enforce execution order**
- Want to **avoid duplication** of common code
- Pipeline patterns (ETL, ML, payments)

❌ Avoid When:

- Classes have **completely different algorithms**
- Need **runtime algorithm swapping** (use Strategy)
- Don't want **inheritance coupling**

Part 14: Key Insights

Hollywood Principle

"Don't call us, we'll call you."

The base class calls the subclass's methods — not the other way around.

Final vs Overridable

- Template method is typically `final` / `const` / non-overridable
- Prevents subclasses from breaking the algorithm structure

Abstract vs Hook

- **Abstract methods** — required
- **Hook methods** — optional with default behavior

Key Takeaways

1. **Template Method** = skeleton of algorithm in base class
2. **Fixed order** — subclasses fill in the details

3. **Abstract methods** — must be overridden
4. **Hook methods** — optional overrides
5. **Template method** — usually final, cannot be overridden
6. **Hollywood Principle** — base class calls subclass methods
7. **Classic example** — ML training pipeline
8. **Real-world uses** — payment, ETL, compilation, HTTP middleware
9. **Uses inheritance** — unlike Strategy (composition)
10. **Enforces structure** — prevents accidental step reordering

"Template Method: fix the skeleton, vary the flesh."



हां जी करें स्टार्ट। सो वेलकम बै...
TXT 95.94KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also

Proxy Design Pattern — Complete Notes

Part 1: Introduction

What is Proxy Pattern?

Proxy Pattern provides a **surrogate or placeholder** for another object to **control access** to it. The proxy acts as a **representative** of the real object — clients don't know they're talking to a proxy rather than the real thing.

Core Concept

text

Client → Proxy → Real Object
(acts as a representative)

The **client cannot differentiate** between the proxy and the real object. It calls the same methods, but the proxy intervenes — validating, deferring, or routing the request.

Why Introduce a Proxy?

Reason	Description
Security	Validate/authenticate before forwarding
Performance (Lazy Loading)	Defer expensive operations until needed
Remote Access	Hide network communication from client
Logging	Log all interactions transparently
Caching	Cache results transparently

Part 2: Three Types of Proxy

Type	Purpose
Virtual Proxy	Lazy initialization of expensive resources
Protection Proxy	Access control / authentication
Remote Proxy	Local representative of a remote object

The structure is the **SAME** in all three. Only the **intent** differs.

Part 3: Virtual Proxy — Example: Image Display

The Problem

An `ImageDisplay` class:

- Constructor loads image from disk, compresses it, applies filters — **expensive!**
- `display()` method just shows the already-loaded image

Issue: If the client never calls `display()`, the expensive constructor still runs — **wasting memory**.

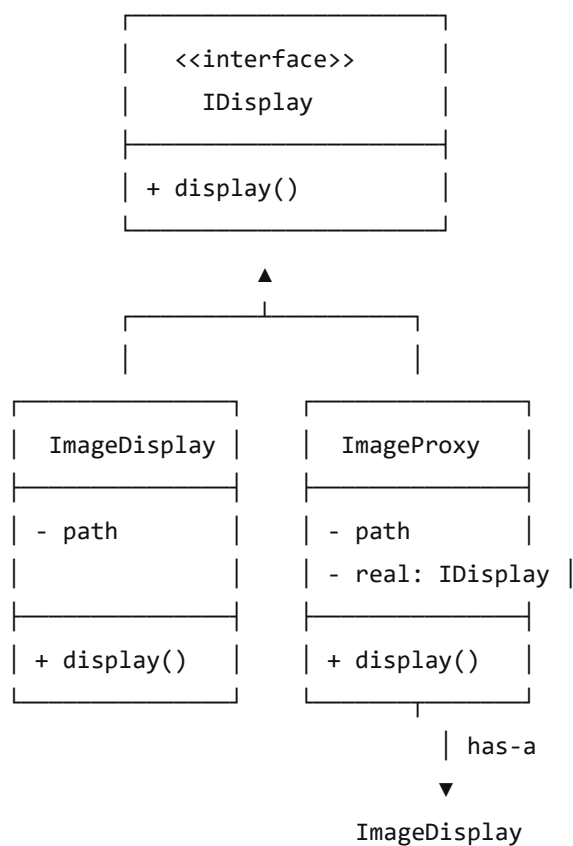
The Solution: Virtual Proxy

Introduce ImageProxy :

- Constructor **doesn't create** the real ImageDisplay
- Holds a reference (initially null)
- `display()` checks if the real object exists; if not, creates it lazily
- Then forwards the call

Class Diagram

text



Java Implementation

java

```

// ===== INTERFACE =====
interface IDisplay {
    void display();
}

// ===== REAL SUBJECT (Expensive) =====

```

```

class ImageDisplay implements IDisplay {
    private String path;

    public ImageDisplay(String path) {
        this.path = path;
        // Expensive operations (simulated)
        System.out.println("Loading image from disk: " + path);
        System.out.println("Compressing image...");
        System.out.println("Applying filters...");
    }

    @Override
    public void display() {
        System.out.println("Displaying image: " + path);
    }
}

// ===== PROXY (Lazy Loading) =====
class ImageProxy implements IDisplay {
    private String path;
    private ImageDisplay realImage; // null until needed

    public ImageProxy(String path) {
        this.path = path;
        this.realImage = null; // Expensive op deferred
    }

    @Override
    public void display() {
        if (realImage == null) {
            realImage = new ImageDisplay(path); // Lazy init
        }
        realImage.display();
    }
}

// ===== CLIENT =====
public class Main {
    public static void main(String[] args) {
        IDisplay img = new ImageProxy("sample.jpg");
        System.out.println("--- Proxy created (no expensive work done yet) ---");
        img.display(); // Expensive work happens HERE
        img.display(); // Cached - no re-load
    }
}

```

Output:

text

```

--- Proxy created (no expensive work done yet) ---
Loading image from disk: sample.jpg
Compressing image...
Applying filters...
Displaying image: sample.jpg
Displaying image: sample.jpg

```

Python Implementation

python

```

from abc import ABC, abstractmethod

class IDisplay(ABC):
    @abstractmethod
    def display(self): pass

class ImageDisplay(IDisplay):
    def __init__(self, path):
        self.path = path
        print(f"Loading image from disk: {path}")
        print("Compressing image...")
        print("Applying filters...")

    def display(self):
        print(f"Displaying image: {self.path}")

class ImageProxy(IDisplay):
    def __init__(self, path):
        self.path = path
        self.real_image = None  # Lazy

    def display(self):
        if self.real_image is None:
            self.real_image = ImageDisplay(self.path)
        self.real_image.display()

# Usage
if __name__ == "__main__":
    img = ImageProxy("sample.jpg")
    print("--- Proxy created (no expensive work done yet) ---")

```

```
img.display()
img.display()
```

Part 4: Protection Proxy — Example: Premium PDF Reader

The Problem

A `RealDocumentReader` can unlock PDFs. But **only premium users** should be able to unlock. How to enforce?

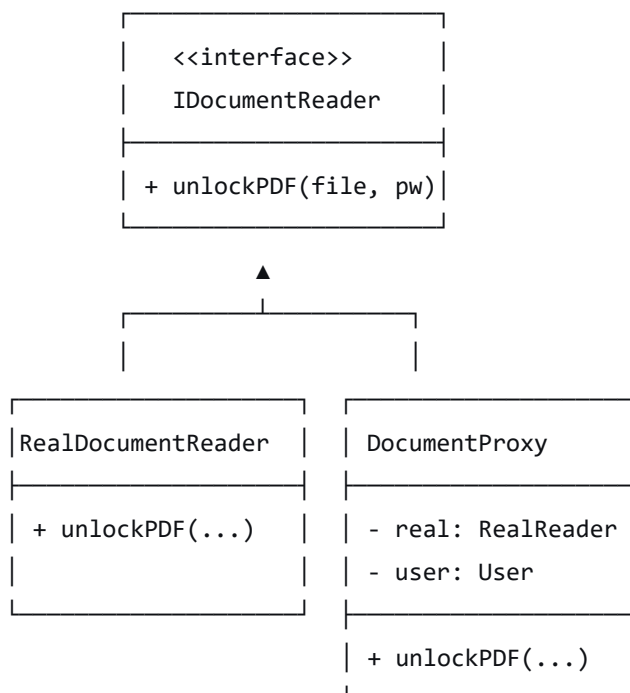
The Solution: Protection Proxy

Introduce `DocumentProxy` :

- Holds a reference to `RealDocumentReader`
- Holds a reference to the `User` trying to access
- `unlockPDF()` checks if `user.isPremium` before forwarding

Class Diagram

text



Java Implementation

java

```
// ===== INTERFACE =====
interface IDocumentReader {
    void unlockPDF(String file, String password);
}

// ===== USER =====
class User {
    private String name;
    private boolean premium;

    public User(String name, boolean premium) {
        this.name = name;
        this.premium = premium;
    }

    public String getName() { return name; }
    public boolean isPremium() { return premium; }
}

// ===== REAL SUBJECT =====
class RealDocumentReader implements IDocumentReader {
    @Override
    public void unlockPDF(String file, String password) {
        System.out.println("Unlocking PDF: " + file + " with password");
    }
}

// ===== PROXY (Protection) =====
class DocumentProxy implements IDocumentReader {
    private User user;
    private RealDocumentReader realReader;

    public DocumentProxy(User user) {
        this.user = user;
        this.realReader = new RealDocumentReader();
    }

    @Override
    public void unlockPDF(String file, String password) {
        if (!user.isPremium()) {
            throw new RuntimeException(
                "Access denied. Only premium users can unlock PDFs.");
        }
        realReader.unlockPDF(file, password);
    }
}
```

```

    }

    // ===== CLIENT =====
    public class Main {
        public static void main(String[] args) {
            User premiumUser = new User("Alice", true);
            User freeUser = new User("Bob", false);

            IDocumentReader premiumReader = new DocumentProxy(premiumUser);
            IDocumentReader freeReader = new DocumentProxy(freeUser);

            premiumReader.unlockPDF("book.pdf", "secret123");
            try {
                freeReader.unlockPDF("book.pdf", "secret123");
            } catch (RuntimeException e) {
                System.out.println("Error: " + e.getMessage());
            }
        }
    }
}

```

Output:

text

Unlocking PDF: book.pdf with password

Error: Access denied. Only premium users can unlock PDFs.

Python Implementation

python

```

class IDocumentReader(ABC):
    @abstractmethod
    def unlock_pdf(self, file, password): pass

class User:
    def __init__(self, name, premium):
        self.name = name
        self.premium = premium

class RealDocumentReader(IDocumentReader):
    def unlock_pdf(self, file, password):
        print(f"Unlocking PDF: {file} with password")

```

```

class DocumentProxy(IDocumentReader):
    def __init__(self, user: User):
        self.user = user
        self.real_reader = RealDocumentReader()

    def unlock_pdf(self, file, password):
        if not self.user.premium:
            raise RuntimeError("Access denied. Premium users only.")
        self.real_reader.unlock_pdf(file, password)

# Usage
if __name__ == "__main__":
    premium = DocumentProxy(User("Alice", True))
    free = DocumentProxy(User("Bob", False))

    premium.unlock_pdf("book.pdf", "secret123")
    try:
        free.unlock_pdf("book.pdf", "secret123")
    except RuntimeError as e:
        print(f"Error: {e}")

```

Part 5: Remote Proxy — Example: Remote Data Service

The Problem

A `DataService` exists on a **remote server**. Client wants to call `fetchData()` but shouldn't handle network logic.

The Solution: Remote Proxy

`DataServiceProxy` :

- Exists **locally**
- Establishes network connection on first call
- Forwards calls to real remote service
- Returns result to client as if it were local

Java Implementation

java

```
// ===== INTERFACE =====
interface IDataService {
    String fetchData();
}

// ===== REAL SUBJECT (on remote server) =====
class DataService implements IDataService {
    @Override
    public String fetchData() {
        // Imagine this does database lookup on remote server
        return "Data from remote server";
    }
}

// ===== REMOTE PROXY (Local) =====
class DataServiceProxy implements IDataService {
    private DataService realService; // null until connected

    @Override
    public String fetchData() {
        if (realService == null) {
            // Simulate establishing internet connection
            System.out.println("Establishing connection to remote server...");
            realService = new DataService();
        }
        return realService.fetchData();
    }
}

// ===== CLIENT =====
public class Main {
    public static void main(String[] args) {
        IDataService service = new DataServiceProxy();
        System.out.println("--- Client calls fetchData ---");
        System.out.println("Result: " + service.fetchData());
        System.out.println("--- Client calls fetchData again ---");
        System.out.println("Result: " + service.fetchData());
    }
}
```

Output:

text

```
--- Client calls fetchData ---
Establishing connection to remote server...
```



```

Result: Data from remote server
--- Client calls fetchData again ---
Result: Data from remote server

```

Python Implementation

python

```

class IDataService(ABC):
    @abstractmethod
    def fetch_data(self): pass

class DataService(IDataService):
    def fetch_data(self):
        return "Data from remote server"

class DataServiceProxy(IDataService):
    def __init__(self):
        self._real_service = None

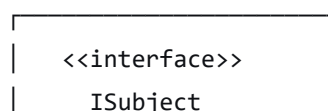
    def fetch_data(self):
        if self._real_service is None:
            print("Establishing connection to remote server...")
            self._real_service = DataService()
        return self._real_service.fetch_data()

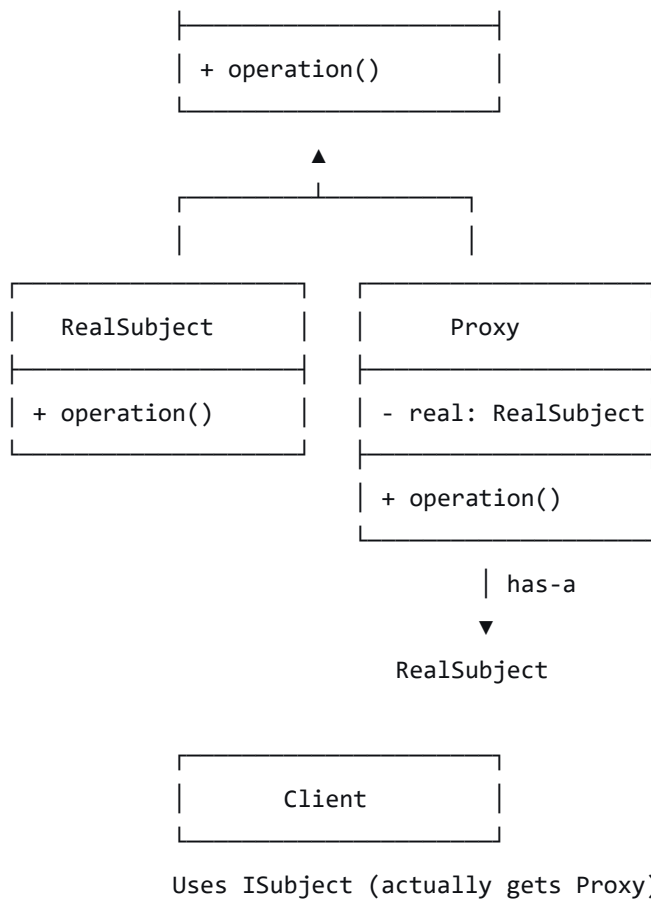
# Usage
if __name__ == "__main__":
    service = DataServiceProxy()
    print("--- Client calls fetch_data ---")
    print(f"Result: {service.fetch_data()}")
    print("--- Client calls fetch_data again ---")
    print(f"Result: {service.fetch_data()}")

```

Part 6: Standard UML Diagram

text





Key Relationships:

- **RealSubject IS-A ISubject**
- **Proxy IS-A ISubject**
- **Proxy HAS-A RealSubject**
- **Client always gets the Proxy reference**

Part 7: Definition

Proxy Pattern provides a surrogate or placeholder for another object to control access to it.

Key Points

- **Surrogate/Placeholder** — represents another object
- **Control access** — intervene in requests
- **Transparent** — client doesn't know it's talking to a proxy

Part 8: Lazy Loading vs Eager Loading

In proxies, you can choose when to create the real object:

Type	When Real Object Created	Trade-off
Lazy Loading	On first method call	Slower first call, less memory
Eager Loading	In proxy constructor	Faster first call, more memory

Example — Lazy loading:

java

```
public void display() {  
    if (realImage == null) realImage = new ImageDisplay(path);  
    realImage.display();  
}
```

Example — Eager loading:

java

```
public DocumentProxy(User user) {  
    this.user = user;  
    this.realReader = new RealDocumentReader(); // Created immediately  
}
```

Part 9: Proxy vs Related Patterns

Pattern	Difference
Proxy	Controls access; same interface as real object
Decorator	Adds responsibilities; same interface
Adapter	Converts interface to another
Facade	Simplifies complex subsystem

Proxy vs Decorator: Both wrap an object with the same interface.

- **Decorator:** adds behavior
- **Proxy:** controls access

Proxy vs Adapter:

- **Adapter:** converts interface
- **Proxy:** same interface, different behavior on access

Part 10: Real-World Use Cases

1. Database Connection Pool

- Proxy manages connection lifecycle

2. Spring AOP

- Proxies intercept method calls for logging, transactions, security

3. Hibernate Lazy Loading

- Proxy represents unloaded entity; loads on first access

4. API Gateway

- Remote proxy for microservices (cart service → user service proxy)

5. Caching

- Proxy caches expensive results

6. Image/Video Viewers

- Virtual proxy defers loading until scroll/click

7. Virtual Private Network (VPN)

- Remote proxy hides actual server

8. Access Control

- Premium features, admin panels, etc.

9. Spring Cloud Feign

- Client-side proxies for REST calls

10. Firewall

- Proxy filters requests

Part 11: Advantages & Trade-offs

Advantages

1. **Access control** — enforce security transparently
2. **Lazy loading** — save memory with expensive resources
3. **Remote transparency** — hide network complexity
4. **Caching** — improve performance
5. **Logging/Monitoring** — intercept requests
6. **Open/Closed** — add concerns without modifying real object

Trade-offs

1. **Extra indirection** — slight overhead per call
2. **Code complexity** — more classes
3. **First-call latency** (lazy loading)
4. **Debugging harder** — you're not calling the real object directly

Part 12: When to Use Proxy Pattern

✅ Use When:

- Need to control access to an object

- Want to defer expensive object creation (virtual proxy)
- Need authentication/authorization (protection proxy)
- Object is on a remote server (remote proxy)
- Want to add logging/caching transparently
- Following Spring AOP-style concerns

✗ Avoid When:

- No access control needed
- Object is cheap to create
- Overhead of indirection isn't worth it

Part 13: Summary Table

Aspect	Details
Intent	Provide a surrogate for another object to control access
Type	Structural pattern
Key Relationships	Proxy IS-A Subject + HAS-A RealSubject
Client sees	Only <code>ISubject</code> (gets Proxy instance)
Variants	Virtual, Protection, Remote
Common uses	Lazy loading, auth, remote calls, caching

Key Takeaways

1. **Proxy Pattern** = surrogate/placeholder for another object
2. **Three types**: Virtual, Protection, Remote
3. **Structure is the same** — only the intent differs
4. **Virtual Proxy** — lazy loading of expensive resources
5. **Protection Proxy** — access control
6. **Remote Proxy** — hide network complexity

7. **Transparent to client** — same interface
8. **Uses IS-A + HAS-A** relationships
9. **Real-world:** Spring AOP, Hibernate, API gateways, image viewers
10. **Lazy vs Eager loading** — trade-off between first-call latency and memory

"Proxy Pattern: stand in front of an object, control who gets in and when."

This response is AI-generated, for reference only.



हां जी करें स्टार्ट। सो वेलकम बै...
TXT 78.06KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also

Chain of Responsibility Design Pattern — Complete Notes

Part 1: Introduction

What is Chain of Responsibility?

Chain of Responsibility (CoR) lets you pass a request along a **chain of handlers**. Each handler decides whether to:

- **Process** the request, or
- **Pass it** to the next handler

Core Concept

text

Client → H1 → H2 → H3 → (end)
(each decides: handle or forward)

Each handler holds a reference to the **next handler** — like a linked list.

Structure Resembles Linked List

But there's a key difference:

- **Linked List** — data structure for **storage**
- **CoR** — behavioral pattern for **processing requests**

Part 2: Real-World Example — ATM Cash Dispenser

The Scenario

You go to an ATM and request ₹3000. How does the ATM dispense it?

Internally: The ATM has **different handlers** for different denominations:

- ₹1000 handler
- ₹500 handler
- ₹200 handler
- ₹100 handler

Each handler:

1. Checks how many notes it can dispense
2. Dispenses what it can
3. Passes the remaining amount to the next handler

Example Flow (₹6000)

Step	Handler	Action	Remaining
1	₹1000 (5 notes)	Dispense $5 \times ₹1000 = ₹5000$	₹1000
2	₹500 (10 notes)	Dispense $2 \times ₹500 = ₹1000$	₹0
3	Result	₹6000 dispensed	Done

Example Flow (₹300)

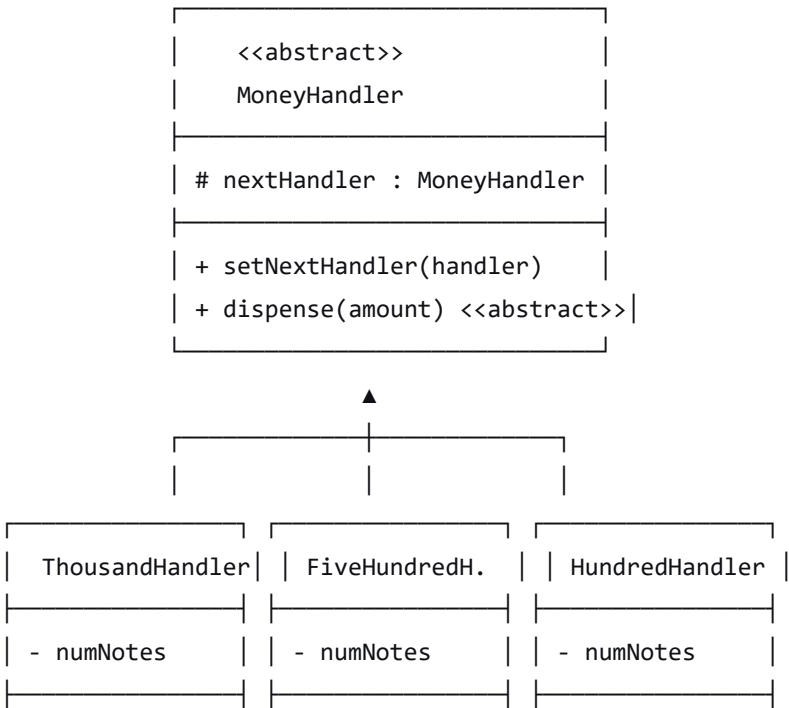
Step	Handler	Action	Remaining
1	₹1000	Can't handle, forward	₹300
2	₹500	Can't handle, forward	₹300
3	₹200	Dispense 1 × ₹200	₹100
4	₹100	Dispense 1 × ₹100	₹0
5	Result	₹300 dispensed	Done

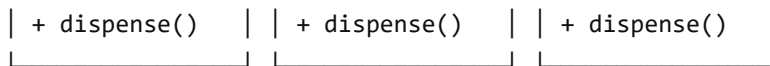
Example Flow (₹150) — Unfulfillable

Step	Handler	Action
1-3	₹1000, ₹500, ₹200	Forward
4	₹100 (only ₹100 notes)	Dispense ₹100, remaining ₹50
5	End of chain	Fail — can't dispense ₹50

Part 3: Class Diagram

text





Key Relationships:

- All handlers **IS-A** MoneyHandler
- MoneyHandler **HAS-A** another MoneyHandler (its next) — **self-referential**

Why Use Interface Self-Reference (Not Concrete-to-Concrete)?

If ThousandHandler had a hardcoded reference to FiveHundredHandler :

- **Tight coupling** — changing denominations requires code changes
- New handlers require modifying existing handlers
- Violates **Open/Closed Principle**

Using abstract MoneyHandler reference:

- Handlers know nothing about each other
- **Dynamic chain configuration** at runtime
- New handlers added without modifying existing code

Part 4: Java Implementation

java

```
// ===== ABSTRACT HANDLER =====
abstract class MoneyHandler {
    protected MoneyHandler nextHandler;

    public MoneyHandler() {
        this.nextHandler = null;
    }

    public void setNextHandler(MoneyHandler next) {
        this.nextHandler = next;
    }

    // Abstract method – each handler implements its own logic
    public abstract void dispense(int amount);
}
```

```
// ===== CONCRETE HANDLER: ₹1000 =====  
class ThousandHandler extends MoneyHandler {  
    private int numNotes;  
  
    public ThousandHandler(int numNotes) {  
        this.numNotes = numNotes;  
    }  
  
    @Override  
    public void dispense(int amount) {  
        int notesNeeded = amount / 1000;  
        if (notesNeeded > numNotes) {  
            notesNeeded = numNotes;  
            numNotes = 0;  
        } else {  
            numNotes -= notesNeeded;  
        }  
  
        if (notesNeeded > 0) {  
            System.out.println("Dispensing " + notesNeeded + " x ₹1000 notes");  
        }  
  
        int remainingAmount = amount - (notesNeeded * 1000);  
        if (remainingAmount > 0) {  
            if (nextHandler != null) {  
                nextHandler.dispense(remainingAmount);  
            } else {  
                System.out.println("Remaining amount ₹" + remainingAmount  
                    + " cannot be fulfilled (insufficient funds)");  
            }  
        }  
    }  
}  
  
// ===== CONCRETE HANDLER: ₹500 =====  
class FiveHundredHandler extends MoneyHandler {  
    private int numNotes;  
  
    public FiveHundredHandler(int numNotes) {  
        this.numNotes = numNotes;  
    }  
  
    @Override  
    public void dispense(int amount) {  
        int notesNeeded = amount / 500;  
        if (notesNeeded > numNotes) {  
            notesNeeded = numNotes;  
            numNotes = 0;  
        }  
    }  
}
```

```

    } else {
        numNotes -= notesNeeded;
    }

    if (notesNeeded > 0) {
        System.out.println("Dispensing " + notesNeeded + " x ₹500 notes");
    }

    int remainingAmount = amount - (notesNeeded * 500);
    if (remainingAmount > 0) {
        if (nextHandler != null) {
            nextHandler.dispense(remainingAmount);
        } else {
            System.out.println("Remaining amount ₹" + remainingAmount
                + " cannot be fulfilled (insufficient funds)");
        }
    }
}

// ===== CONCRETE HANDLER: ₹200 =====
class TwoHundredHandler extends MoneyHandler {
    private int numNotes;

    public TwoHundredHandler(int numNotes) {
        this.numNotes = numNotes;
    }

    @Override
    public void dispense(int amount) {
        int notesNeeded = amount / 200;
        if (notesNeeded > numNotes) {
            notesNeeded = numNotes;
            numNotes = 0;
        } else {
            numNotes -= notesNeeded;
        }

        if (notesNeeded > 0) {
            System.out.println("Dispensing " + notesNeeded + " x ₹200 notes");
        }

        int remainingAmount = amount - (notesNeeded * 200);
        if (remainingAmount > 0) {
            if (nextHandler != null) {
                nextHandler.dispense(remainingAmount);
            } else {
                System.out.println("Remaining amount ₹" + remainingAmount

```

```

        + " cannot be fulfilled (insufficient funds)");
    }
}

// ===== CONCRETE HANDLER: ₹100 =====
class HundredHandler extends MoneyHandler {
    private int numNotes;

    public HundredHandler(int numNotes) {
        this.numNotes = numNotes;
    }

    @Override
    public void dispense(int amount) {
        int notesNeeded = amount / 100;
        if (notesNeeded > numNotes) {
            notesNeeded = numNotes;
            numNotes = 0;
        } else {
            numNotes -= notesNeeded;
        }

        if (notesNeeded > 0) {
            System.out.println("Dispensing " + notesNeeded + " x ₹100 notes");
        }

        int remainingAmount = amount - (notesNeeded * 100);
        if (remainingAmount > 0) {
            // Last handler - no next
            System.out.println("Remaining amount ₹" + remainingAmount
                + " cannot be fulfilled (insufficient funds)");
        }
    }
}

// ===== CLIENT =====
public class Main {
    public static void main(String[] args) {
        // Create handlers
        MoneyHandler thousand = new ThousandHandler(3);
        MoneyHandler fiveHundred = new FiveHundredHandler(5);
        MoneyHandler twoHundred = new TwoHundredHandler(10);
        MoneyHandler hundred = new HundredHandler(20);

        // Set up the chain
        thousand.setNextHandler(fiveHundred);
    }
}

```

```

        fiveHundred.setNextHandler(twoHundred);
        twoHundred.setNextHandler(hundred);

        // Test
        System.out.println("=== Dispensing ₹300 ===");
        thousand.dispense(300);

        System.out.println("\n=== Dispensing ₹3000 ===");
        thousand.dispense(3000);

        System.out.println("\n=== Dispensing ₹4000 ===");
        thousand.dispense(4000);

        System.out.println("\n=== Dispensing ₹150 (should fail) ===");
        thousand.dispense(150);
    }
}

```

Output:

text

```

=== Dispensing ₹300 ===
Dispensing 1 x ₹200 notes
Dispensing 1 x ₹100 notes

=== Dispensing ₹3000 ===
Dispensing 3 x ₹1000 notes

=== Dispensing ₹4000 ===
Dispensing 3 x ₹1000 notes
Dispensing 2 x ₹500 notes

=== Dispensing ₹150 (should fail) ===
Dispensing 1 x ₹100 notes
Remaining amount ₹50 cannot be fulfilled (insufficient funds)

```

Part 5: Python Implementation

python

```
from abc import ABC, abstractmethod
```

```
# ===== ABSTRACT HANDLER =====
```

```

class MoneyHandler(ABC):
    def __init__(self):
        self.next_handler = None

    def set_next_handler(self, next_handler):
        self.next_handler = next_handler

    @abstractmethod
    def dispense(self, amount):
        pass

# ===== CONCRETE HANDLERS =====
class ThousandHandler(MoneyHandler):
    def __init__(self, num_notes):
        super().__init__()
        self.num_notes = num_notes

    def dispense(self, amount):
        notes_needed = amount // 1000
        if notes_needed > self.num_notes:
            notes_needed = self.num_notes
            self.num_notes = 0
        else:
            self.num_notes -= notes_needed

        if notes_needed > 0:
            print(f"Dispensing {notes_needed} x ₹1000 notes")

        remaining = amount - (notes_needed * 1000)
        if remaining > 0:
            if self.next_handler:
                self.next_handler.dispense(remaining)
            else:
                print(f"Remaining ₹{remaining} cannot be fulfilled")

class FiveHundredHandler(MoneyHandler):
    def __init__(self, num_notes):
        super().__init__()
        self.num_notes = num_notes

    def dispense(self, amount):
        notes_needed = amount // 500
        if notes_needed > self.num_notes:
            notes_needed = self.num_notes
            self.num_notes = 0
        else:

```

```
self.num_notes -= notes_needed

if notes_needed > 0:
    print(f"Dispensing {notes_needed} x ₹500 notes")

remaining = amount - (notes_needed * 500)
if remaining > 0:
    if self.next_handler:
        self.next_handler.dispense(remaining)
    else:
        print(f"Remaining ₹{remaining} cannot be fulfilled")

class TwoHundredHandler(MoneyHandler):
    def __init__(self, num_notes):
        super().__init__()
        self.num_notes = num_notes

    def dispense(self, amount):
        notes_needed = amount // 200
        if notes_needed > self.num_notes:
            notes_needed = self.num_notes
            self.num_notes = 0
        else:
            self.num_notes -= notes_needed

        if notes_needed > 0:
            print(f"Dispensing {notes_needed} x ₹200 notes")

        remaining = amount - (notes_needed * 200)
        if remaining > 0:
            if self.next_handler:
                self.next_handler.dispense(remaining)
            else:
                print(f"Remaining ₹{remaining} cannot be fulfilled")

class HundredHandler(MoneyHandler):
    def __init__(self, num_notes):
        super().__init__()
        self.num_notes = num_notes

    def dispense(self, amount):
        notes_needed = amount // 100
        if notes_needed > self.num_notes:
            notes_needed = self.num_notes
            self.num_notes = 0
        else:
```



```

        self.num_notes -= notes_needed

    if notes_needed > 0:
        print(f"Dispensing {notes_needed} x ₹100 notes")

    remaining = amount - (notes_needed * 100)
    if remaining > 0:
        print(f"Remaining ₹{remaining} cannot be fulfilled")

# ===== CLIENT =====
if __name__ == "__main__":
    thousand = ThousandHandler(3)
    five_hundred = FiveHundredHandler(5)
    two_hundred = TwoHundredHandler(10)
    hundred = HundredHandler(20)

    # Build chain
    thousand.set_next_handler(five_hundred)
    five_hundred.set_next_handler(two_hundred)
    two_hundred.set_next_handler(hundred)

    print("=== Dispensing ₹300 ===")
    thousand.dispense(300)

    print("\n=== Dispensing ₹3000 ===")
    thousand.dispense(3000)

    print("\n=== Dispensing ₹4000 ===")
    thousand.dispense(4000)

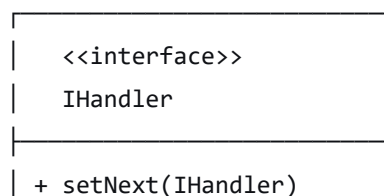
    print("\n=== Dispensing ₹150 (should fail) ===")
    thousand.dispense(150)

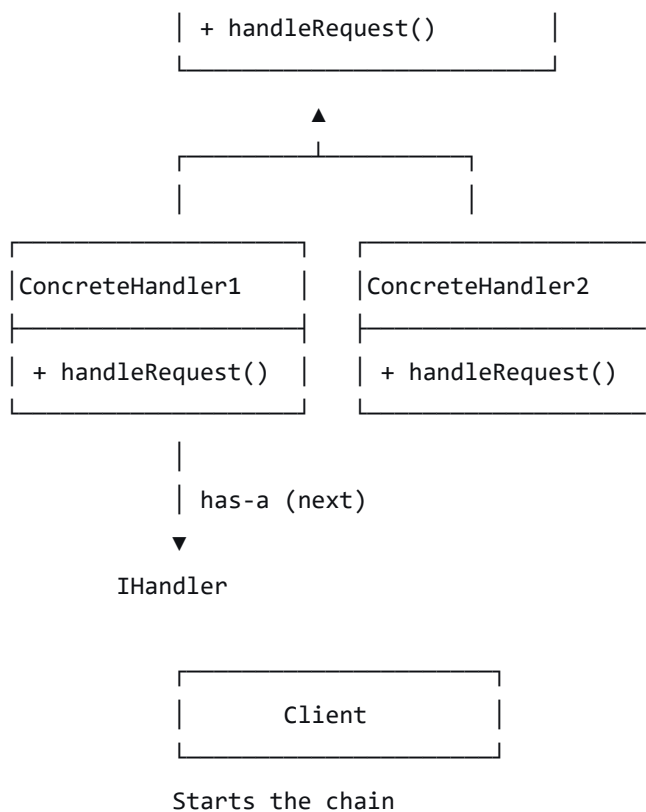
```

Output: Same as Java.

Part 6: Standard UML Diagram

text





Key Points:

- **IHandler HAS-A IHandler** — self-referential
- Concrete handlers **IS-A** IHandler
- Client holds reference to **first** handler only
- Chain is built dynamically

Part 7: Definition

Chain of Responsibility allows an object to pass a request along a chain of potential handlers. Each handler decides either to process the request or pass it to the next handler in the chain.

Key Phrases

- **Chain of handlers** — multiple receivers
- **Pass the request** — decision made by each handler
- **Process or forward** — handler's choice

Part 8: Chain of Responsibility vs Linked List

Aspect	Linked List	Chain of Responsibility
Intent	Data storage	Request processing
Structure	Nodes with data + next	Handlers with handle logic + next
Self-reference	Points to same-type node	Points to interface
Objects	All nodes from same class	Different concrete handlers
Use	DSA operations	Design pattern

Key Difference: In CoR, handlers are **different classes** implementing the same interface.

Part 9: CoR vs Composite

Aspect	Composite	CoR
Structure	Tree	Chain (line)
Traversal	Recursive (tree)	Sequential (linked-list)
Children	Many	One (next)
Intent	Part-whole hierarchy	Pass request along

Both feel similar because both involve recursion-like traversal.

Part 10: Real-World Use Cases

1. Logger System

- Info → Debug → Error handlers
- Each level decides whether to log

2. Leave Request System (Interview Classic)

- Employee requests leave
- ≤ 2 days $\rightarrow$ Team Lead
- 2-5 days $\rightarrow$ Manager
- | 5 days $\rightarrow$ Director

3. ATM Cash Dispenser

- ₹1000 $\rightarrow$ ₹500 $\rightarrow$ ₹200 $\rightarrow$ ₹100 handlers

4. HTTP Request Middleware

- Auth $\rightarrow$ Rate Limit $\rightarrow$ Logging $\rightarrow$ Handler

5. Spring Security Filter Chain

- Multiple filters process request in sequence

6. GUI Event Handling

- Event bubbles up the component hierarchy

7. Vending Machine

- Different handlers for different denominations

8. Customer Support Escalation

- L1 $\rightarrow$ L2 $\rightarrow$ L3 support

9. Approval Workflows

- Purchase approval: Manager $\rightarrow$ Director $\rightarrow$ VP $\rightarrow$ CEO

10. Compiler Error Handlers

- Warning $\rightarrow$ Info $\rightarrow$ Error recovery

Part 11: Advantages & Trade-offs

Advantages

1. **Loose coupling** — sender doesn't know which handler processes
2. **Open/Closed** — new handlers added without modifying existing
3. **Single Responsibility** — each handler has one job
4. **Dynamic chain** — can reconfigure at runtime
5. **Flexibility** — request can be processed at any point

Trade-offs

1. **No guarantee** — request may go unhandled
2. **Debugging complexity** — trace through chain
3. **Performance** — long chains = slow
4. **Circular chains** — possible if not careful

Part 12: When to Use CoR

✅ Use When:

- Multiple objects may handle a request
- Handler isn't known in advance
- Need dynamic chain configuration
- Want to decouple sender from receivers
- Workflow/approval/escalation scenarios
- Middleware/pipeline processing

❌ Avoid When:

- Request has exactly one handler
- Chain is fixed and simple
- Performance is critical
- No need for dynamic configuration

Key Takeaways

1. **CoR** = pass request along chain of handlers
2. **Each handler decides**: process or forward
3. **Self-referential interface** — handlers hold reference to next
4. **Linked-list-like structure** but different intent
5. **Open/Closed Principle** — new handlers without modifications
6. **Single Responsibility** — each handler does one thing
7. **Classic examples** — ATM cash dispenser, logger, leave approval
8. **Real-world**: Spring Security, HTTP middleware, GUI events
9. **Dynamic chain** — configure at runtime
10. **Trade-off**: no guarantee of handling + potential for circular chains

"Chain of Responsibility: pass the buck until someone handles it."



हां जी करें स्टार्ट। सो वेलकम बै...
TXT 162.55KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also

Payment Gateway System — Complete LLD Notes

Part 1: Introduction

What is a Payment Gateway?

A **Payment Gateway** is a mediator between **client applications** (Amazon, Swiggy, Zomato, etc.) and **banking systems** (Paytm, Razorpay, GPay, etc.). It handles the complete payment flow — validation, initiation, confirmation — while hiding the complexity of integrating multiple payment providers.

Two Different Concepts (Not to Be Confused)

Concept	What It Is
Payment Gateway System	Pluggable module used by apps to accept payments
Banking System Architecture	Internal system (Paytm/GPay backend) that moves money

This case study focuses on the **Payment Gateway System** — the plug-and-play module that applications integrate.

Part 2: Functional Requirements

Requirement	Description
Support Multiple Providers	Paytm, Razorpay, more can be added
Easily Add New Gateways	Open/Closed Principle — new gateways without modifying existing
Standard Payment Flow	Validate → Initiate → Confirm (fixed order)
Required Validations	Sufficient funds, valid currency, valid request
Error Handling	Log errors
Retry Mechanism	Retry failed payments up to threshold

Part 3: Standard Payment Flow

Every payment must follow this **fixed sequence**:

text

1. Validate → Check funds, currency, valid request
2. Initiate → Actually call the provider (Paytm/Razorpay)
3. Confirm → Verify the payment succeeded

Order must never change. You can't confirm before initiating.

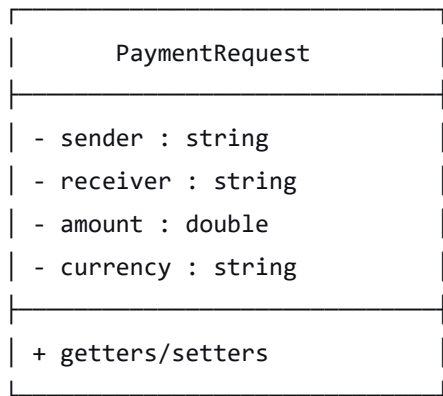
Part 4: Design Patterns Used

Pattern	Purpose
Template Method	Enforce validate → initiate → confirm order
Factory	Create gateway objects
Singleton	Factory, Service, Controller
Proxy (Protection)	Add retry mechanism, protect real gateway
Composition	Gateway owns its banking system

Part 5: Entity Model

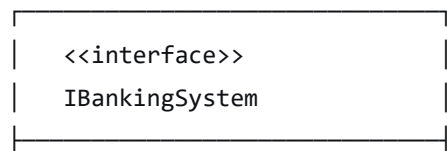
PaymentRequest (Model)

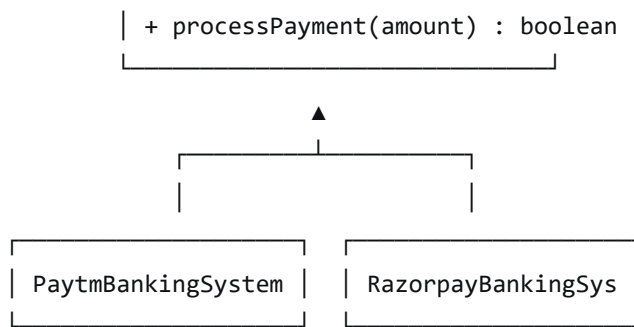
text



Part 6: Banking System Hierarchy

text

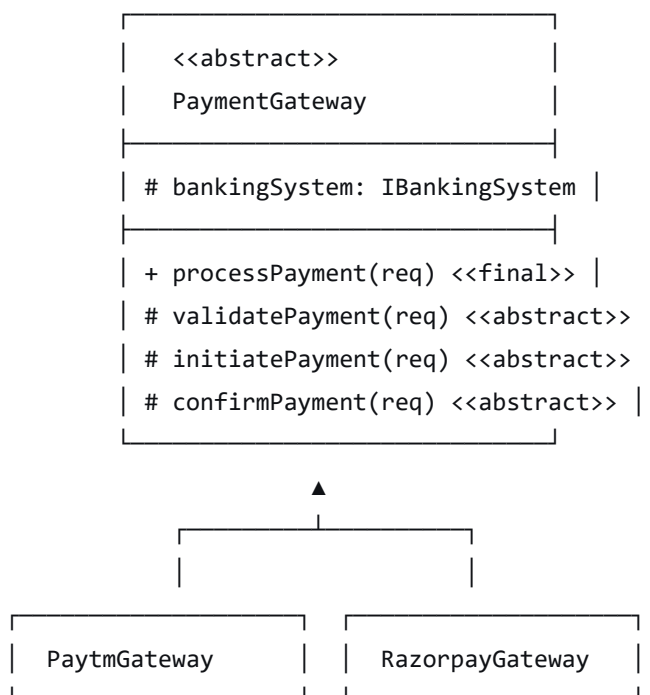




Note: In the real world, these are remote services. Here they're treated as **plug-and-play** (they exist; we just use them).

Part 7: Payment Gateway Hierarchy (Template Method)

text



Template Method in Action

text

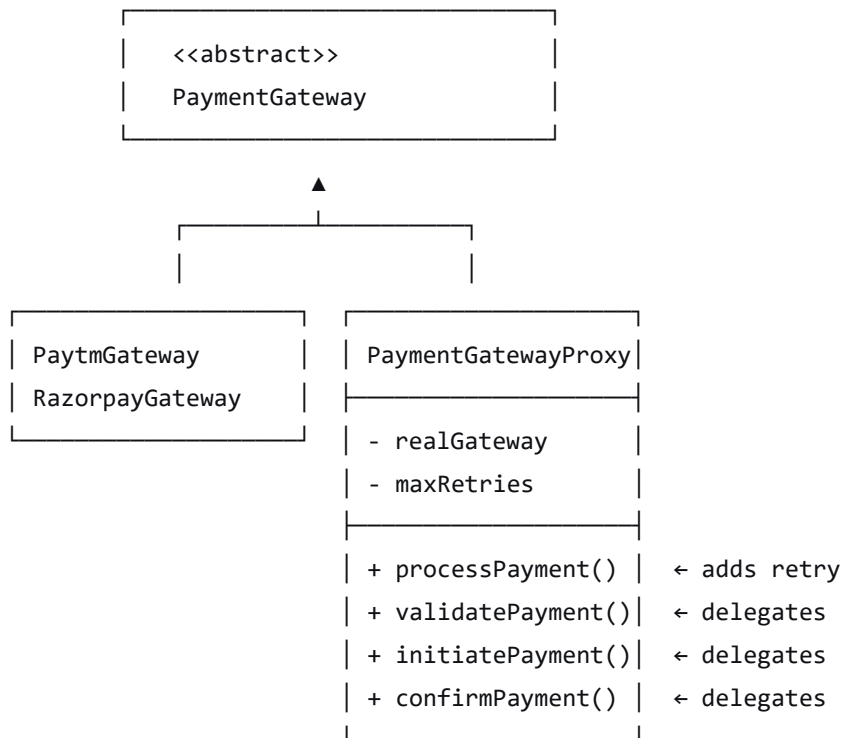
```

processPayment(req):
    if (!validatePayment(req)) throw error
    if (!initiatePayment(req)) throw error
    if (!confirmPayment(req)) throw error
    return true
  
```

Subclasses override `validatePayment` , `initiatePayment` , `confirmPayment` . **Order is fixed** by the base class.

Part 8: Payment Gateway Proxy (Protection Proxy)

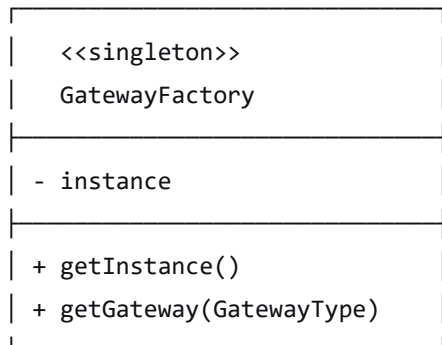
text



Key: `processPayment()` in the proxy implements **retry logic**. Other methods just delegate.

Part 9: Gateway Factory (Singleton)

text



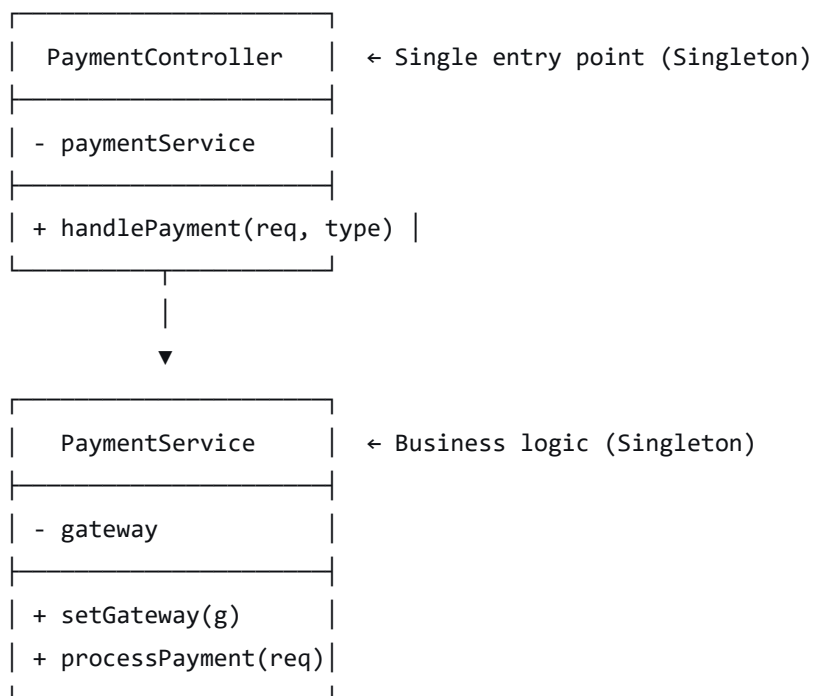
getGateway(type) returns:

- A **PaymentGatewayProxy** wrapping a real gateway
- The proxy holds `PaytmGateway` OR `RazorpayGateway`

Why proxy? Client always goes through proxy → retry logic applies.

Part 10: Service & Controller Layers

text



Controller's handlePayment() Flow

text

1. Get gateway from GatewayFactory (based on type)
2. Set it on PaymentService
3. Call `PaymentService.processPayment(request)`
4. Return result

Part 11: Java Implementation

Enums

java

```
enum GatewayType {  
    PAYTM, RAZORPAY  
}
```

Model — PaymentRequest

java

```
class PaymentRequest {  
    private String sender;  
    private String receiver;  
    private double amount;  
    private String currency;  
  
    public PaymentRequest(String sender, String receiver, double amount, String currency)  
{  
        this.sender = sender;  
        this.receiver = receiver;  
        this.amount = amount;  
        this.currency = currency;  
    }  
    public String getSender() { return sender; }  
    public String getReceiver() { return receiver; }  
    public double getAmount() { return amount; }  
    public String getCurrency() { return currency; }  
}
```

Banking System Hierarchy

java

```
interface IBankingSystem {  
    boolean processPayment(double amount);  
}  
  
class PaytmBankingSystem implements IBankingSystem {  
    @Override  
    public boolean processPayment(double amount) {  
        System.out.println("Processing payment of " + amount + " via Paytm Banking");  
        // Simulate 20% success rate  
        return Math.random() < 0.20;  
    }  
}
```

```

    }
}

class RazorpayBankingSystem implements IBankingSystem {
    @Override
    public boolean processPayment(double amount) {
        System.out.println("Processing payment of " + amount + " via Razorpay Banking");
        // Simulate 90% success rate
        return Math.random() < 0.90;
    }
}

```

Abstract Gateway (Template Method)

java

```

abstract class PaymentGateway {
    protected IBankingSystem bankingSystem;

    public PaymentGateway() {
        this.bankingSystem = null;
    }

    // Template method – FIXED ORDER
    public final boolean processPayment(PaymentRequest req) {
        if (!validatePayment(req)) {
            throw new RuntimeException("Validation failed");
        }
        if (!initiatePayment(req)) {
            throw new RuntimeException("Initiation failed");
        }
        if (!confirmPayment(req)) {
            throw new RuntimeException("Confirmation failed");
        }
        return true;
    }

    protected abstract boolean validatePayment(PaymentRequest req);
    protected abstract boolean initiatePayment(PaymentRequest req);
    protected abstract boolean confirmPayment(PaymentRequest req);
}

```

Concrete Gateways

java

```
class PaytmGateway extends PaymentGateway {
    public PaytmGateway() {
        this.bankingSystem = new PaytmBankingSystem();
    }

    @Override
    protected boolean validatePayment(PaymentRequest req) {
        System.out.println("Validating payment for " + req.getSender());
        return req.getAmount() > 0 && "INR".equals(req.getCurrency());
    }

    @Override
    protected boolean initiatePayment(PaymentRequest req) {
        System.out.println("Initiating payment of " + req.getAmount() + " INR");
        return bankingSystem.processPayment(req.getAmount());
    }

    @Override
    protected boolean confirmPayment(PaymentRequest req) {
        System.out.println("Confirming payment");
        return true;
    }
}

class RazorpayGateway extends PaymentGateway {
    public RazorpayGateway() {
        this.bankingSystem = new RazorpayBankingSystem();
    }

    @Override
    protected boolean validatePayment(PaymentRequest req) {
        System.out.println("Razorpay validating payment");
        return req.getAmount() > 0;    // accepts any currency
    }

    @Override
    protected boolean initiatePayment(PaymentRequest req) {
        System.out.println("Razorpay initiating ₹" + req.getAmount());
        return bankingSystem.processPayment(req.getAmount());
    }

    @Override
    protected boolean confirmPayment(PaymentRequest req) {
        System.out.println("Razorpay confirming payment");
        return true;
    }
}
```

Protection Proxy with Retry

java

```
class PaymentGatewayProxy extends PaymentGateway {
    private PaymentGateway realGateway;
    private int maxRetries;

    public PaymentGatewayProxy(PaymentGateway realGateway, int maxRetries) {
        this.realGateway = realGateway;
        this.maxRetries = maxRetries;
    }

    @Override
    public boolean processPayment(PaymentRequest req) {
        for (int attempt = 1; attempt <= maxRetries; attempt++) {
            try {
                boolean success = realGateway.processPayment(req);
                if (success) return true;
            } catch (Exception e) {
                System.out.println("Attempt " + attempt + " failed: " + e.getMessage());
            }
        }
        throw new RuntimeException("Payment failed after " + maxRetries + " retries for "
+ req.getSender());
    }

    @Override
    protected boolean validatePayment(PaymentRequest req) {
        return realGateway.validatePayment(req);
    }

    @Override
    protected boolean initiatePayment(PaymentRequest req) {
        return realGateway.initiatePayment(req);
    }

    @Override
    protected boolean confirmPayment(PaymentRequest req) {
        return realGateway.confirmPayment(req);
    }
}
```

Gateway Factory (Singleton)

java

```

class GatewayFactory {
    private static GatewayFactory instance;

    private GatewayFactory() {}

    public static GatewayFactory getInstance() {
        if (instance == null) {
            synchronized (GatewayFactory.class) {
                if (instance == null) instance = new GatewayFactory();
            }
        }
        return instance;
    }

    public PaymentGateway getGateway(GatewayType type) {
        PaymentGateway real;
        int maxRetries;
        if (type == GatewayType.PAYTM) {
            real = new PaytmGateway();
            maxRetries = 3;
        } else {
            real = new RazorpayGateway();
            maxRetries = 1;
        }
        return new PaymentGatewayProxy(real, maxRetries);
    }
}

```

Service (Singleton)

java

```

class PaymentService {
    private static PaymentService instance;
    private PaymentGateway gateway;

    private PaymentService() {}

    public static PaymentService getInstance() {
        if (instance == null) {
            synchronized (PaymentService.class) {
                if (instance == null) instance = new PaymentService();
            }
        }
        return instance;
    }
}

```



```

    public void setGateway(PaymentGateway gateway) {
        this.gateway = gateway;
    }

    public boolean processPayment(PaymentRequest req) {
        if (gateway == null) throw new RuntimeException("No gateway set");
        return gateway.processPayment(req);
    }
}

```

Controller (Singleton)

java

```

class PaymentController {
    private static PaymentController instance;
    private PaymentService service;

    private PaymentController() {
        this.service = PaymentService.getInstance();
    }

    public static PaymentController getInstance() {
        if (instance == null) {
            synchronized (PaymentController.class) {
                if (instance == null) instance = new PaymentController();
            }
        }
        return instance;
    }

    public boolean handlePayment(PaymentRequest req, GatewayType type) {
        PaymentGateway gateway = GatewayFactory.getInstance().getGateway(type);
        service.setGateway(gateway);
        return service.processPayment(req);
    }
}

```

Client

java

```

public class Main {
    public static void main(String[] args) {

```

```

PaymentRequest req1 = new PaymentRequest("Aditya", "Shubham", 1000, "INR");
boolean r1 = PaymentController.getInstance()
    .handlePayment(req1, GatewayType.PAYTM);
System.out.println("Paytm Result: " + (r1 ? "SUCCESS" : "FAILURE"));

System.out.println("-----");

PaymentRequest req2 = new PaymentRequest("Shubham", "Aditya", 500, "USD");
boolean r2 = PaymentController.getInstance()
    .handlePayment(req2, GatewayType.RAZORPAY);
System.out.println("Razorpay Result: " + (r2 ? "SUCCESS" : "FAILURE"));
    }
}

```

Sample Output:

text

```

Validating payment for Aditya
Initiating payment of 1000.0 INR
Processing payment of 1000.0 via Paytm Banking
Attempt 1 failed: Initiation failed
Validating payment for Aditya
Initiating payment of 1000.0 INR
Processing payment of 1000.0 via Paytm Banking
Attempt 2 failed: Initiation failed
Validating payment for Aditya
Initiating payment of 1000.0 INR
Processing payment of 1000.0 via Paytm Banking
Confirming payment
Paytm Result: SUCCESS

```

Part 12: Python Implementation

python

```

import random
from enum import Enum
from abc import ABC, abstractmethod

class GatewayType(Enum):
    PAYTM = 1
    RAZORPAY = 2

```

```

# ===== MODEL =====
class PaymentRequest:
    def __init__(self, sender, receiver, amount, currency):
        self.sender = sender
        self.receiver = receiver
        self.amount = amount
        self.currency = currency

# ===== BANKING SYSTEM =====
class IBankingSystem(ABC):
    @abstractmethod
    def process_payment(self, amount): pass

class PaytmBankingSystem(IBankingSystem):
    def process_payment(self, amount):
        print(f"Paytm processing {amount}")
        return random.random() < 0.20

class RazorpayBankingSystem(IBankingSystem):
    def process_payment(self, amount):
        print(f"Razorpay processing {amount}")
        return random.random() < 0.90

# ===== TEMPLATE METHOD GATEWAY =====
class PaymentGateway(ABC):
    def __init__(self):
        self.banking_system = None

    # Template method – fixed order
    def process_payment(self, req: PaymentRequest) -> bool:
        if not self.validate_payment(req):
            raise RuntimeError("Validation failed")
        if not self.initiate_payment(req):
            raise RuntimeError("Initiation failed")
        if not self.confirm_payment(req):
            raise RuntimeError("Confirmation failed")
        return True

    @abstractmethod
    def validate_payment(self, req): pass

    @abstractmethod
    def initiate_payment(self, req): pass

```

```

@abstractmethod
def confirm_payment(self, req): pass

class PaytmGateway(PaymentGateway):
    def __init__(self):
        super().__init__()
        self.banking_system = PaytmBankingSystem()

    def validate_payment(self, req):
        print(f"Paytm validating {req.sender}")
        return req.amount > 0 and req.currency == "INR"

    def initiate_payment(self, req):
        print(f"Paytm initiating {req.amount} INR")
        return self.banking_system.process_payment(req.amount)

    def confirm_payment(self, req):
        print("Paytm confirming")
        return True

class RazorpayGateway(PaymentGateway):
    def __init__(self):
        super().__init__()
        self.banking_system = RazorpayBankingSystem()

    def validate_payment(self, req):
        print("Razorpay validating")
        return req.amount > 0

    def initiate_payment(self, req):
        print(f"Razorpay initiating {req.amount}")
        return self.banking_system.process_payment(req.amount)

    def confirm_payment(self, req):
        print("Razorpay confirming")
        return True

# ===== PROXY WITH RETRY =====
class PaymentGatewayProxy(PaymentGateway):
    def __init__(self, real_gateway, max_retries):
        super().__init__()
        self.real_gateway = real_gateway
        self.max_retries = max_retries

```

```

def process_payment(self, req: PaymentRequest) -> bool:
    for attempt in range(1, self.max_retries + 1):
        try:
            if self.real_gateway.process_payment(req):
                return True
        except Exception as e:
            print(f"Attempt {attempt} failed: {e}")
    raise RuntimeError(
        f"Payment failed after {self.max_retries} retries for {req.sender}"
    )

def validate_payment(self, req): return self.real_gateway.validate_payment(req)
def initiate_payment(self, req): return self.real_gateway.initiate_payment(req)
def confirm_payment(self, req): return self.real_gateway.confirm_payment(req)

# ===== FACTORY (SINGLETON) =====
class GatewayFactory:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    def get_gateway(self, type_: GatewayType) -> PaymentGateway:
        if type_ == GatewayType.PAYTM:
            return PaymentGatewayProxy(PaytmGateway(), 3)
        return PaymentGatewayProxy(RazorpayGateway(), 1)

# ===== SERVICE (SINGLETON) =====
class PaymentService:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance.gateway = None
        return cls._instance

    def set_gateway(self, gateway):
        self.gateway = gateway

    def process_payment(self, req):
        if self.gateway is None:
            raise RuntimeError("No gateway set")
        return self.gateway.process_payment(req)

```

```
# ===== CONTROLLER (SINGLETON) =====
class PaymentController:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance.service = PaymentService()
        return cls._instance

    def handle_payment(self, req: PaymentRequest, type_: GatewayType) -> bool:
        gateway = GatewayFactory().get_gateway(type_)
        self.service.set_gateway(gateway)
        return self.service.process_payment(req)

# ===== CLIENT =====
if __name__ == "__main__":
    req1 = PaymentRequest("Aditya", "Shubham", 1000, "INR")
    try:
        r1 = PaymentController().handle_payment(req1, GatewayType.PAYTM)
        print(f"Paytm Result: {'SUCCESS' if r1 else 'FAILURE'}")
    except RuntimeError as e:
        print(f"Paytm Error: {e}")

    print("-----")

    req2 = PaymentRequest("Shubham", "Aditya", 500, "USD")
    try:
        r2 = PaymentController().handle_payment(req2, GatewayType.RAZORPAY)
        print(f"Razorpay Result: {'SUCCESS' if r2 else 'FAILURE'}")
    except RuntimeError as e:
        print(f"Razorpay Error: {e}")
```

Part 13: Architecture Summary

text

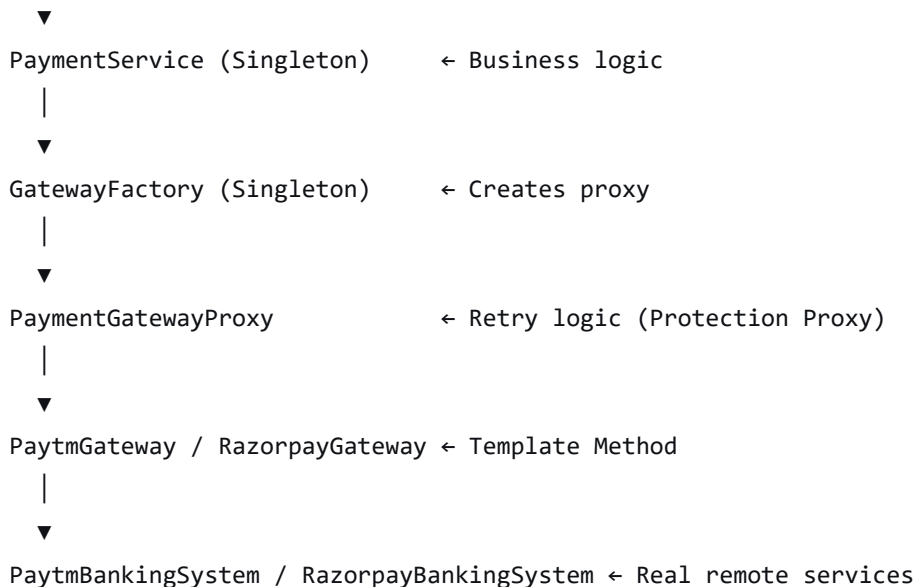
Client

|

▼

PaymentController (Singleton) ← Entry point

|



Part 14: SOLID Principles Applied

Principle	How Applied
SRP	Each class has one job (gateway=process, proxy=retry, factory=create)
OCP	New gateways via new subclasses; no modification to existing
LSP	All gateways substitutable as PaymentGateway
ISP	Small interfaces (IBankingSystem has one method)
DIP	Gateway depends on IBankingSystem abstraction

Part 15: Design Patterns Used

Pattern	Where
Template Method	PaymentGateway.processPayment() — fixed flow
Protection Proxy	PaymentGatewayProxy — retry logic
Factory	GatewayFactory — creates proxies

Pattern	Where
Singleton	Factory, Service, Controller
Composition	Gateway HAS-A banking system

Part 16: Homework Extensions

1. Multiple Retry Mechanisms

Extend the proxy to support:

- **Linear Retry** — fixed delay between retries
- **Exponential Backoff** — delay doubles each retry

Make retry strategy swappable (Strategy Pattern).

2. Recurring Payments

Add subscription-based flow:

- **Normal Payment** — one-time
- **Recurring Payment** — periodic (YouTube Premium, Netflix)

Use **Template Method** to standardize both flows.

Key Takeaways

1. **Payment Gateway** = mediator between app and banking systems
2. **Three-step payment flow**: Validate → Initiate → Confirm
3. **Template Method** enforces fixed order
4. **Protection Proxy** adds retry logic transparently
5. **Factory + Singleton** for clean creation & single instance
6. **Controller → Service → Gateway** layered architecture
7. **Plug-and-play design** — client uses single controller
8. **Extensible** — new gateways via new subclasses

9. **Composition** — gateway owns banking system

10. **Real-world** — used by Amazon, Swiggy, Zomato, etc.

"Payment Gateway: hide the complexity of multiple providers behind a single, clean interface — with retries, validations, and a fixed flow."



हां जी तो करें स्टार्ट। सो वेलक...
TXT 234.98KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also

Discount Coupon Application — Complete LLD Notes

Part 1: Introduction

What is a Discount Coupon System?

A **plug-and-play module** that any e-commerce/inventory/food-delivery application can integrate to:

- Manage multiple types of coupons
- Apply discounts at cart level or product level
- Allow stacking coupons (or restrict stacking)
- Calculate final payable amount after all discounts

Real-world examples: Swiggy 60% OFF coupons, Zomato Pro discounts, Amazon bank offers, Blinkit category discounts

Part 2: Functional Requirements

#	Requirement
1	Add new coupons at runtime (Banking, Seasonal, Loyalty, Bulk Purchase)
2	Support cart-level AND product-level discounts

#	Requirement
3	Multiple discount strategies : Flat, Percentage, Percentage-with-Cap
4	Coupons can be combined (or restricted from combining)
5	Thread-safety — limited coupons must be claimed safely
6	Plug-and-play — integrate anywhere with minimal changes
7	Scalable — new coupons/strategies added without modifying existing code

Part 3: Happy Flow

text

User adds products to Cart

↓

Cart maintains list of CartItems (product + quantity)

↓

Coupon Manager registers various coupons

↓

Coupons applied on cart (some stackable, some independent)

↓

Each coupon calculates its discount (via strategy)

↓

Cart's final total = original total - all applicable discounts

↓

User pays final amount

Note: No `User` class needed — this is a plug-and-play module. Assumptions like "isLoyaltyMember" are kept on Cart (to be fetched from user in real app).

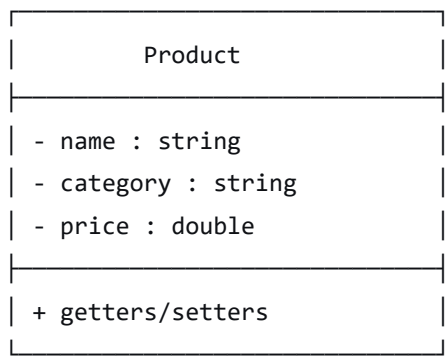
Part 4: Design Patterns Used

Pattern	Purpose
Strategy	Different discount calculations (Flat, Percentage, Percentage-with-Cap)
Chain of Responsibility	Coupon chain — apply coupon, pass to next
Template Method	applyDiscount() defines fixed flow: isApplicable → getDiscount → isCombinable
Composition	Cart has CartItems; CouponManager has Coupons
Singleton	CouponManager, DiscountStrategyManager
Factory	DiscountStrategyManager creates strategies

Part 5: Entity Model

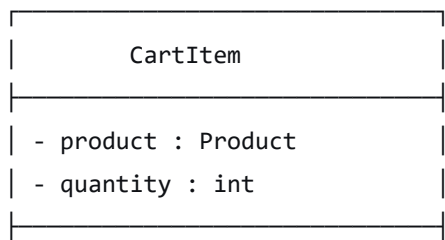
Product (Model)

text



CartItem

text



```

| + getProduct() |
| + getItemTotal() : double | // product.price × quantity
|_____|

```

Cart

text

```

|_____|
|      Cart      |
|_____|
| - items : List<CartItem> |
| - originalTotal : double |
| - currentTotal : double  |
| - isLoyaltyMember : boolean |
| - paymentBank : string   |
|_____|
| + addProduct(product, quantity) |
| + applyDiscount(amount)         |
| + getOriginalTotal() : double   |
| + getCurrentTotal() : double    |
|_____|

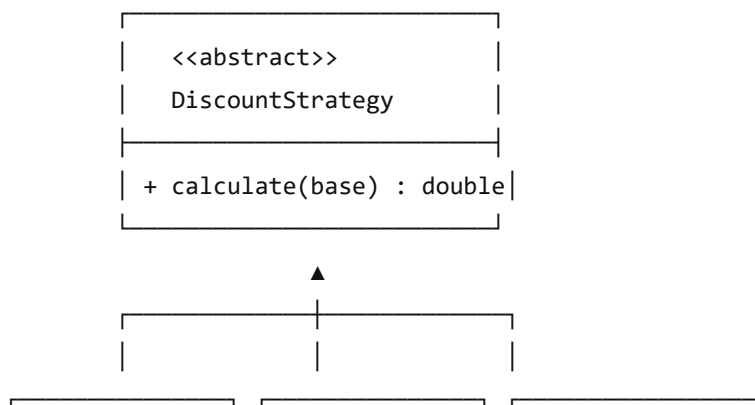
```

Relationships

- Cart **COMPOSED-OF** CartItem (CartItem cannot exist without Cart)
- CartItem **AGGREGATED-WITH** Product (Product exists independently)

Part 6: Discount Strategy Hierarchy

text



FlatDiscount	PercentDiscount	PercentWithCap
- amount	- percent	- percent
		- cap

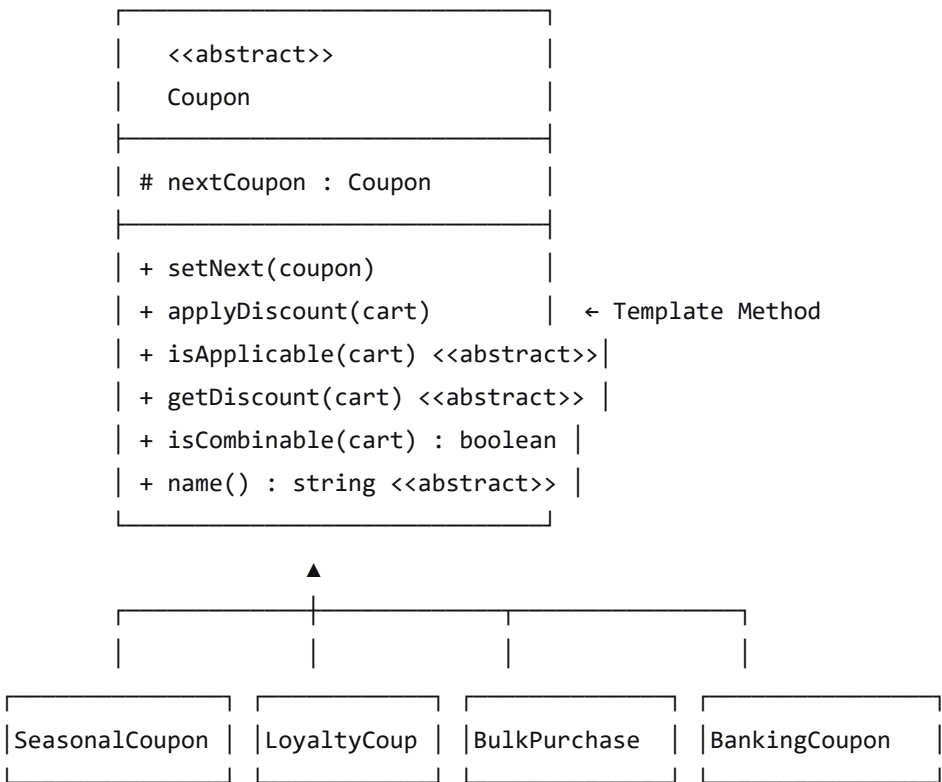
- **Flat** — subtract fixed amount (min of amount and base)
- **Percentage** — $\text{base} \times \text{percent} / 100$
- **Percentage-with-Cap** — $\min(\text{percent-based}, \text{cap})$

DiscountStrategyManager (Singleton)

Creates strategies by `StrategyType` enum (FLAT, PERCENT, PERCENT_WITH_CAP).

Part 7: Coupon Hierarchy (Chain of Responsibility + Template Method)

text



The Template Method: `applyDiscount(cart)`

text

```
applyDiscount(cart):
    if (isApplicable(cart)):
        discount = getDiscount(cart)
        cart.applyDiscount(discount)
        print("Applied coupon, discount = " + discount)

    if (!isCombinable(cart)):
        return // stop chain

    if (nextCoupon != null):
        nextCoupon.applyDiscount(cart) // recurse
```

Fixed flow:

- 1. Check isApplicable
- 2. If yes → getDiscount → apply on cart
- 3. Check isCombinable
- 4. If yes → recurse to nextCoupon

Concrete Coupons

Coupon	Applicability	Strategy
SeasonalCoupon	Category match (e.g., "Clothing")	Percent
LoyaltyCoupon	cart.isLoyaltyMember == true	Percent
BulkPurchaseCoupon	cart.total >= threshold	Flat
BankingCoupon	cart.paymentBank == bankName && cart.total >= minSpend	Percent-with-Cap

Part 8: CouponManager (Singleton)

text

```
<<singleton>>
CouponManager
```

- head : Coupon		
- mutex : Mutex		← Thread safety

+ getInstance()		
+ registerCoupon(coupon)		
+ applyAllCoupons(cart) : double		
+ getApplicableCoupons(cart)		

- Uses a **linked list** structure (head pointer to first coupon)
- **Register** → append to end of chain (thread-safe with mutex)
- **Apply All** → Call head.applyDiscount(cart)
- **Get Applicable** → return list of coupon names applicable to the cart

Part 9: Complete Java Implementation

Enums

java

```
enum StrategyType { FLAT, PERCENT, PERCENT_WITH_CAP }
```

Strategy Classes

java

```
abstract class DiscountStrategy {
    public abstract double calculate(double baseAmount);
}

class FlatDiscountStrategy extends DiscountStrategy {
    private double flatAmount;
    public FlatDiscountStrategy(double flatAmount) { this.flatAmount = flatAmount; }
    public double calculate(double baseAmount) {
        return Math.min(flatAmount, baseAmount);
    }
}

class PercentageDiscountStrategy extends DiscountStrategy {
    private double percent;
```

```

    public PercentageDiscountStrategy(double percent) { this.percent = percent; }
    public double calculate(double baseAmount) {
        return (percent / 100.0) * baseAmount;
    }
}

class PercentageWithCapStrategy extends DiscountStrategy {
    private double percent, cap;
    public PercentageWithCapStrategy(double percent, double cap) {
        this.percent = percent;
        this.cap = cap;
    }
    public double calculate(double baseAmount) {
        double discount = (percent / 100.0) * baseAmount;
        return Math.min(discount, cap);
    }
}

```

Strategy Manager (Singleton)

java

```

class DiscountStrategyManager {
    private static DiscountStrategyManager instance;

    private DiscountStrategyManager() {}

    public static DiscountStrategyManager getInstance() {
        if (instance == null) {
            synchronized (DiscountStrategyManager.class) {
                if (instance == null) instance = new DiscountStrategyManager();
            }
        }
        return instance;
    }

    public DiscountStrategy getStrategy(StrategyType type, double p1, Double p2) {
        switch (type) {
            case FLAT: return new FlatDiscountStrategy(p1);
            case PERCENT: return new PercentageDiscountStrategy(p1);
            case PERCENT_WITH_CAP: return new PercentageWithCapStrategy(p1, p2);
        }
        return null;
    }
}

```


Model Classes

java

```
class Product {
    private String name, category;
    private double price;
    public Product(String name, String category, double price) {
        this.name = name;
        this.category = category;
        this.price = price;
    }
    public String getName() { return name; }
    public String getCategory() { return category; }
    public double getPrice() { return price; }
}

class CartItem {
    private Product product;
    private int quantity;
    public CartItem(Product p, int qty) { this.product = p; this.quantity = qty; }
    public Product getProduct() { return product; }
    public double getItemTotal() { return product.getPrice() * quantity; }
}
```

Cart

java

```
import java.util.*;

class Cart {
    private List<CartItem> items = new ArrayList<>();
    private double originalTotal = 0;
    private double currentTotal = 0;
    private boolean isLoyaltyMember = false;
    private String paymentBank = "";

    public void addProduct(Product p, int qty) {
        CartItem item = new CartItem(p, qty);
        items.add(item);
        originalTotal += item.getItemTotal();
        currentTotal += item.getItemTotal();
    }

    public void applyDiscount(double amount) {
        currentTotal -= amount;
    }
}
```

```

        if (currentTotal < 0) currentTotal = 0;
    }

    public List<CartItem> getItems() { return items; }
    public double getOriginalTotal() { return originalTotal; }
    public double getCurrentTotal() { return currentTotal; }
    public void setLoyaltyMember(boolean b) { this.isLoyaltyMember = b; }
    public boolean isLoyaltyMember() { return isLoyaltyMember; }
    public void setPaymentBank(String bank) { this.paymentBank = bank; }
    public String getPaymentBank() { return paymentBank; }
}

```

Coupon Hierarchy (Abstract)

java

```

abstract class Coupon {
    protected Coupon nextCoupon = null;

    public void setNext(Coupon next) { this.nextCoupon = next; }

    // Template Method
    public void applyDiscount(Cart cart) {
        if (isApplicable(cart)) {
            double discount = getDiscount(cart);
            cart.applyDiscount(discount);
            System.out.println("Applied " + name() + ": ₹" + discount);
        }
        if (!isCombinable(cart)) return;
        if (nextCoupon != null) nextCoupon.applyDiscount(cart);
    }

    public abstract boolean isApplicable(Cart cart);
    public abstract double getDiscount(Cart cart);
    public boolean isCombinable(Cart cart) { return true; }
    public abstract String name();
}

```

Concrete Coupons

java

```

class SeasonalCoupon extends Coupon {
    private String category;
    private DiscountStrategy strategy;
}

```

```

    public SeasonalCoupon(String category, double percent) {
        this.category = category;
        this.strategy = DiscountStrategyManager.getInstance()
            .getStrategy(StrategyType.PERCENT, percent, null);
    }

    public boolean isApplicable(Cart cart) {
        for (CartItem i : cart.getItems())
            if (i.getProduct().getCategory().equals(category)) return true;
        return false;
    }

    public double getDiscount(Cart cart) {
        double subtotal = 0;
        for (CartItem i : cart.getItems())
            if (i.getProduct().getCategory().equals(category))
                subtotal += i.getItemTotal();
        return strategy.calculate(subtotal);
    }

    public String name() { return "Seasonal (" + category + ")"; }
}

class LoyaltyCoupon extends Coupon {
    private DiscountStrategy strategy;
    public LoyaltyCoupon(double percent) {
        this.strategy = DiscountStrategyManager.getInstance()
            .getStrategy(StrategyType.PERCENT, percent, null);
    }
    public boolean isApplicable(Cart cart) { return cart.isLoyaltyMember(); }
    public double getDiscount(Cart cart) {
        return strategy.calculate(cart.getCurrentTotal());
    }
    public String name() { return "Loyalty Discount"; }
}

class BulkPurchaseCoupon extends Coupon {
    private double threshold;
    private DiscountStrategy strategy;
    public BulkPurchaseCoupon(double threshold, double flatAmount) {
        this.threshold = threshold;
        this.strategy = DiscountStrategyManager.getInstance()
            .getStrategy(StrategyType.FLAT, flatAmount, null);
    }
    public boolean isApplicable(Cart cart) { return cart.getCurrentTotal() >= threshold; }
    public double getDiscount(Cart cart) { return strategy.calculate(cart.getCurrentTotal()

```

```

    ()); }
    public String name() { return "Bulk Purchase"; }
}

class BankingCoupon extends Coupon {
    private String bank;
    private double minSpend;
    private DiscountStrategy strategy;

    public BankingCoupon(String bank, double minSpend, double percent, double cap) {
        this.bank = bank;
        this.minSpend = minSpend;
        this.strategy = DiscountStrategyManager.getInstance()
            .getStrategy(StrategyType.PERCENT_WITH_CAP, percent, cap);
    }

    public boolean isApplicable(Cart cart) {
        return cart.getPaymentBank().equals(bank) && cart.getCurrentTotal() >= minSpend;
    }

    public double getDiscount(Cart cart) { return strategy.calculate(cart.getCurrentTotal
    ()); }

    public String name() { return "Banking (" + bank + ")"; }
}

```

CouponManager (Singleton)

java

```

import java.util.concurrent.locks.ReentrantLock;

class CouponManager {
    private static CouponManager instance;
    private Coupon head = null;
    private final ReentrantLock lock = new ReentrantLock();

    private CouponManager() {}

    public static CouponManager getInstance() {
        if (instance == null) {
            synchronized (CouponManager.class) {
                if (instance == null) instance = new CouponManager();
            }
        }
        return instance;
    }

    public void registerCoupon(Coupon coupon) {

```

```

        lock.lock();
        try {
            if (head == null) {
                head = coupon;
            } else {
                Coupon curr = head;
                while (curr.nextCoupon != null) curr = curr.nextCoupon;
                curr.nextCoupon = coupon;
            }
        } finally {
            lock.unlock();
        }
    }

    public double applyAllCoupons(Cart cart) {
        lock.lock();
        try {
            if (head != null) head.applyDiscount(cart);
            return cart.getCurrentTotal();
        } finally {
            lock.unlock();
        }
    }

    public List<String> getApplicableCoupons(Cart cart) {
        List<String> result = new ArrayList<>();
        lock.lock();
        try {
            Coupon curr = head;
            while (curr != null) {
                if (curr.isApplicable(cart)) result.add(curr.name());
                curr = curr.nextCoupon;
            }
        } finally {
            lock.unlock();
        }
        return result;
    }
}

```

Client

java

```

public class Main {
    public static void main(String[] args) {
        Product p1 = new Product("Jacket", "Clothing", 1000);
    }
}

```

```

Product p2 = new Product("Smartphone", "Electronics", 2000);
Product p3 = new Product("Jeans", "Clothing", 1000);
Product p4 = new Product("Headphones", "Electronics", 2000);

Cart cart = new Cart();
cart.addProduct(p1, 1);
cart.addProduct(p2, 1);
cart.addProduct(p3, 2);
cart.addProduct(p4, 1);

cart.setLoyaltyMember(true);
cart.setPaymentBank("ABC");

System.out.println("Original Total: ₹" + cart.getOriginalTotal());

CouponManager mgr = CouponManager.getInstance();
mgr.registerCoupon(new SeasonalCoupon("Clothing", 10));
mgr.registerCoupon(new LoyaltyCoupon(5));
mgr.registerCoupon(new BulkPurchaseCoupon(1000, 100));
mgr.registerCoupon(new BankingCoupon("ABC", 2000, 15, 500));

System.out.println("Applicable: " + mgr.getApplicableCoupons(cart));

double finalTotal = mgr.applyAllCoupons(cart);
System.out.println("Final Total: ₹" + finalTotal);
}
}

```

Sample Output:

text

```

Original Total: ₹7000.0
Applicable: [Seasonal (Clothing), Loyalty Discount, Bulk Purchase, Banking (ABC)]
Applied Seasonal (Clothing): ₹300.0
Applied Loyalty Discount: ₹335.0
Applied Bulk Purchase: ₹100.0
Applied Banking (ABC): ₹500.0
Final Total: ₹5765.0

```

Part 10: Python Implementation

python

```

import threading
from enum import Enum
from abc import ABC, abstractmethod
from typing import List

class StrategyType(Enum):
    FLAT = 1
    PERCENT = 2
    PERCENT_WITH_CAP = 3

# ===== STRATEGIES =====
class DiscountStrategy(ABC):
    @abstractmethod
    def calculate(self, base: float) -> float: pass

class FlatDiscount(DiscountStrategy):
    def __init__(self, amount): self.amount = amount
    def calculate(self, base): return min(self.amount, base)

class PercentageDiscount(DiscountStrategy):
    def __init__(self, percent): self.percent = percent
    def calculate(self, base): return (self.percent / 100.0) * base

class PercentageWithCap(DiscountStrategy):
    def __init__(self, percent, cap):
        self.percent = percent
        self.cap = cap
    def calculate(self, base):
        return min((self.percent / 100.0) * base, self.cap)

class DiscountStrategyManager:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    def get_strategy(self, type_, p1, p2=None):
        if type_ == StrategyType.FLAT: return FlatDiscount(p1)
        if type_ == StrategyType.PERCENT: return PercentageDiscount(p1)
        if type_ == StrategyType.PERCENT_WITH_CAP: return PercentageWithCap(p1, p2)

```

```
# ===== MODELS =====

class Product:
    def __init__(self, name, category, price):
        self.name = name
        self.category = category
        self.price = price

class CartItem:
    def __init__(self, product: Product, quantity: int):
        self.product = product
        self.quantity = quantity
    def item_total(self):
        return self.product.price * self.quantity

class Cart:
    def __init__(self):
        self.items: List[CartItem] = []
        self.original_total = 0.0
        self.current_total = 0.0
        self.is_loyalty_member = False
        self.payment_bank = ""

    def add_product(self, product, qty=1):
        item = CartItem(product, qty)
        self.items.append(item)
        self.original_total += item.item_total()
        self.current_total += item.item_total()

    def apply_discount(self, amount):
        self.current_total = max(0, self.current_total - amount)

# ===== COUPON (Chain + Template) =====
class Coupon(ABC):
    def __init__(self):
        self.next_coupon = None

    def set_next(self, coupon):
        self.next_coupon = coupon

    # Template Method
    def apply_discount(self, cart: Cart):
        if self.is_applicable(cart):
            discount = self.get_discount(cart)
            cart.apply_discount(discount)
```



```

        print(f"Applied {self.name()}: ₹{discount:.2f}")

    if not self.is_combinable(cart):
        return

    if self.next_coupon:
        self.next_coupon.apply_discount(cart)

    @abstractmethod
    def is_applicable(self, cart): pass

    @abstractmethod
    def get_discount(self, cart): pass

    def is_combinable(self, cart): return True

    @abstractmethod
    def name(self) -> str: pass


class SeasonalCoupon(Coupon):
    def __init__(self, category, percent):
        super().__init__()
        self.category = category
        self.strategy = DiscountStrategyManager().get_strategy(
            StrategyType.PERCENT, percent)

    def is_applicable(self, cart):
        return any(i.product.category == self.category for i in cart.items)

    def get_discount(self, cart):
        subtotal = sum(i.item_total() for i in cart.items
                        if i.product.category == self.category)
        return self.strategy.calculate(subtotal)

    def name(self): return f"Seasonal ({self.category})"


class LoyaltyCoupon(Coupon):
    def __init__(self, percent):
        super().__init__()
        self.strategy = DiscountStrategyManager().get_strategy(
            StrategyType.PERCENT, percent)

    def is_applicable(self, cart): return cart.is_loyalty_member
    def get_discount(self, cart): return self.strategy.calculate(cart.current_total)
    def name(self): return "Loyalty Discount"

```

```

class BulkPurchaseCoupon(Coupon):
    def __init__(self, threshold, flat_amount):
        super().__init__()
        self.threshold = threshold
        self.strategy = DiscountStrategyManager().get_strategy(
            StrategyType.FLAT, flat_amount)

    def is_applicable(self, cart): return cart.current_total >= self.threshold
    def get_discount(self, cart): return self.strategy.calculate(cart.current_total)
    def name(self): return "Bulk Purchase"

class BankingCoupon(Coupon):
    def __init__(self, bank, min_spend, percent, cap):
        super().__init__()
        self.bank = bank
        self.min_spend = min_spend
        self.strategy = DiscountStrategyManager().get_strategy(
            StrategyType.PERCENT_WITH_CAP, percent, cap)

    def is_applicable(self, cart):
        return cart.payment_bank == self.bank and cart.current_total >= self.min_spend

    def get_discount(self, cart): return self.strategy.calculate(cart.current_total)
    def name(self): return f"Banking ({self.bank})"

# ===== COUPON MANAGER (Singleton + Mutex) =====
class CouponManager:
    _instance = None
    _lock = threading.Lock()

    def __new__(cls):
        if cls._instance is None:
            with cls._lock:
                if cls._instance is None:
                    cls._instance = super().__new__(cls)
                    cls._instance.head = None
                    cls._instance.mtx = threading.Lock()
        return cls._instance

    def register_coupon(self, coupon):
        with self.mtx:
            if self.head is None:
                self.head = coupon
            else:
                curr = self.head

```

```

        while curr.next_coupon is not None:
            curr = curr.next_coupon
        curr.next_coupon = coupon

def apply_all_coupons(self, cart):
    with self.mtx:
        if self.head:
            self.head.apply_discount(cart)
        return cart.current_total

def get_applicable_coupons(self, cart):
    result = []
    with self.mtx:
        curr = self.head
        while curr:
            if curr.is_applicable(cart):
                result.append(curr.name())
            curr = curr.next_coupon
    return result

# ===== CLIENT =====
if __name__ == "__main__":
    p1 = Product("Jacket", "Clothing", 1000)
    p2 = Product("Smartphone", "Electronics", 2000)
    p3 = Product("Jeans", "Clothing", 1000)
    p4 = Product("Headphones", "Electronics", 2000)

    cart = Cart()
    cart.add_product(p1, 1)
    cart.add_product(p2, 1)
    cart.add_product(p3, 2)
    cart.add_product(p4, 1)

    cart.is_loyalty_member = True
    cart.payment_bank = "ABC"

    print(f"Original Total: ₹{cart.original_total}")

    mgr = CouponManager()
    mgr.register_coupon(SeasonalCoupon("Clothing", 10))
    mgr.register_coupon(LoyaltyCoupon(5))
    mgr.register_coupon(BulkPurchaseCoupon(1000, 100))
    mgr.register_coupon(BankingCoupon("ABC", 2000, 15, 500))

    print(f"Applicable: {mgr.get_applicable_coupons(cart)}")
    final = mgr.apply_all_coupons(cart)
    print(f"Final Total: ₹{final:.2f}")

```

Part 11: Architecture Summary

text

Client (Main)

|

▼

CouponManager (Singleton) → Chain of Coupons → Cart

|

▼

Coupon (Template Method: isApplicable → getDiscount → applyDiscount → isCombinable → next)

|

├ SeasonalCoupon → Strategy (Percent)

├ LoyaltyCoupon → Strategy (Percent)

├ BulkPurchase → Strategy (Flat)

└ BankingCoupon → Strategy (Percent-with-Cap)

Cart

├ List<CartItem>

└ ┬ Product (name, category, price)

├ originalTotal, currentTotal

├ isLoyaltyMember

└ paymentBank

DiscountStrategyManager (Singleton) – creates strategies

Part 12: Design Principles Applied

Principle	Application
SRP	Each class has one job (strategy calculates, coupon applies, manager orchestrates)
OCP	New coupons/strategies without modifying existing (except factory enum)
LSP	All strategies/coupons substitutable for their base classes
ISP	Small interfaces

Principle	Application
DIP	Coupon depends on DiscountStrategy abstraction
Chain of Responsibility	Coupons chained — pass along if not combinable
Template Method	Coupon's <code>applyDiscount</code> fixes the flow

Part 13: Key Design Decisions

Why Chain of Responsibility over Decorator?

- Decorator wraps a single object with behavior
- CoR chains multiple handlers and stops at any point (via `isCombinable`)
- Better matches "apply coupon → pass to next (or stop)" semantics

Why is CouponManager Singleton + Thread-Safe?

- Single source of truth for coupon chain
- Mutex protects registration & application (concurrent users)

Why CartItem exists separately?

- Groups product + quantity
- Hides internal structure from clients (clients only see Product)
- Enables per-item calculations (`itemTotal = price × qty`)

Why is `applyDiscount` a Template Method?

- Fixes the order: `isApplicable` → `getDiscount` → `applyDiscount` → `isCombinable` → `next`
- Subclasses only define "what" (is applicable? how much discount?); parent defines "how" (in what order)

Trade-off: OCP Break in Factory

- `DiscountStrategyManager.getStrategy()` uses `if/else` on `StrategyType`

- Adding a new strategy requires modifying this method (minor OCP break)
- **Acceptable** because it's a low-level factory (not business logic)

Part 14: Interview Tips

1. **Start with happy flow** — quickly show the flow without writing code
2. **Draw UML first** — get alignment with interviewer before coding
3. **Discuss multiple patterns** — say "I could use Decorator or CoR; I'll go with CoR because..."
4. **Don't write entire code** — focus on structure; console prints are fine
5. **Mention SOLID** — explain how each principle applies
6. **Handle edge cases** — thread-safety, negative totals, empty cart
7. **Extensibility discussion** — how to add new coupons/strategies

Key Takeaways

1. **Discount Coupon System** = plug-and-play module for e-commerce
2. **Strategy Pattern** for interchangeable discount calculations
3. **Chain of Responsibility** for applying multiple coupons
4. **Template Method** for fixed coupon flow
5. **Singleton + Mutex** for thread-safe coupon manager
6. **Composition** between Cart, CartItem, Product
7. **Cart has two totals** — original (before discounts) and current/final (after discounts)
8. **Combinability** stops chain — some coupons can't stack
9. **Extensible** — new coupons & strategies added without changing existing code
10. **Real-world relevance** — used by Swiggy, Zomato, Amazon, Blinkit for coupon discounts

"A coupon manager is a chain of coupons, each with a strategy — apply, chain, or stop, and the cart's final total tells the story."



हां जी तो करें स्टार्ट। सो वेलक...
TXT 81.24KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also

Bridge Design Pattern — Complete Notes

Part 1: Introduction

What is Bridge Pattern?

The **Bridge Pattern** decouples an **abstraction** (high-level layer) from its **implementation** (low-level layer) so that **both can vary independently**.

Key Insight: Instead of creating $m \times n$ classes for every permutation of two orthogonal dimensions, we create $m + n$ classes and connect them via a "bridge" (composition).

The Problem It Solves: Class Explosion

Consider:

- **Car Types:** Sedan, Hatchback, SUV (m types)
- **Engines:** Petrol, Diesel, Electric (n types)

Without Bridge: Every car can pair with every engine $\rightarrow m \times n$ classes

- SedanWithPetrol, SedanWithDiesel, SedanWithElectric
- SUVWithPetrol, SUVWithDiesel, SUVWithElectric
- HatchbackWithPetrol, HatchbackWithDiesel, HatchbackWithElectric
- **Total: 9 classes** (and grows exponentially)

Add **CNG** engine $\rightarrow$ 3 more classes. Add **Coupe** car $\rightarrow$ 3 more.

Class Explosion = many classes, tight coupling, DRY violations, hard to maintain.

With Bridge: Separate the two dimensions $\rightarrow m + n$ classes = $3 + 3 = 6$ classes (better still, reuses subclasses)

Part 2: The Bridge Solution

Core Concept

Divide any concept into **two layers**:

Layer	Meaning	Example
Abstraction (High-Level)	What it looks like from outside	Car Type (Sedan, SUV)
Implementation (Low-Level)	How it actually works inside	Engine (Petrol, Diesel)

Note: This is different from OOP's abstraction vs concrete classes. Here:

- **Abstraction** = high-level conceptual layer
- **Implementation** = low-level operational layer

The Bridge

The high-level layer holds a **reference** to the low-level layer:

- Car HAS-A Engine
- Car delegates engine-related work to the engine
- Car can be paired with any engine independently

Class Count Reduction

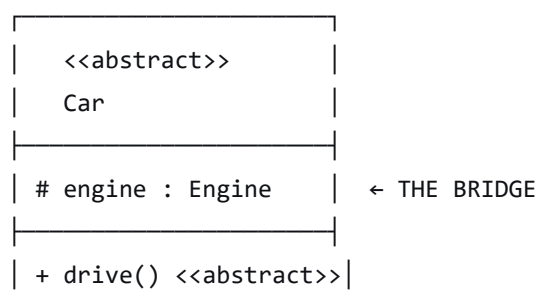
Without Bridge: $m \times n$ classes

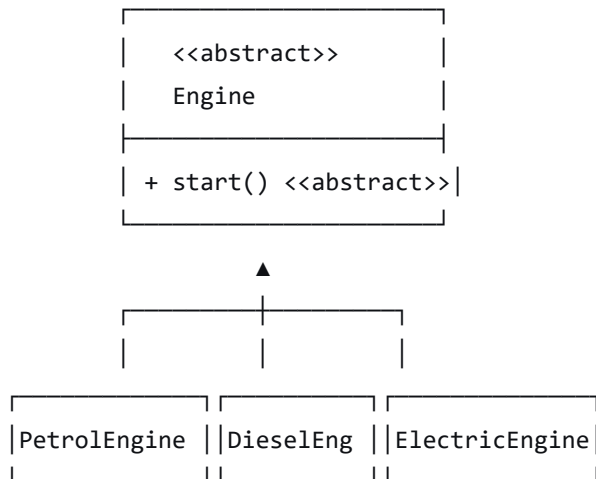
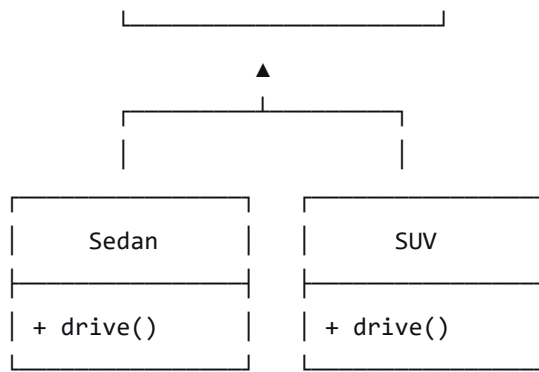
With Bridge: $m + n$ classes

3 car types + 3 engines = 6 classes instead of 9

Part 3: Class Diagram

text





Key Relationship

- Car (abstraction) **HAS-A** Engine (implementation)
- This composition is **the bridge**

How It Works

text

```

Sedan.drive():
    engine.start()      ← delegates to engine
    print("Driving sedan")
  
```

The Sedan doesn't know **which** engine it has — it just calls `engine.start()` .

Part 4: Java Implementation

java

```
// ===== IMPLEMENTATION HIERARCHY (Low-Level) =====
abstract class Engine {
    public abstract void start();
}

class PetrolEngine extends Engine {
    @Override
    public void start() {
        System.out.println("Petrol engine starting with ignition");
    }
}

class DieselEngine extends Engine {
    @Override
    public void start() {
        System.out.println("Diesel engine starting with compression");
    }
}

class ElectricEngine extends Engine {
    @Override
    public void start() {
        System.out.println("Electric engine powering up silently");
    }
}

// ===== ABSTRACTION HIERARCHY (High-Level) =====
abstract class Car {
    protected Engine engine;    // THE BRIDGE

    public Car(Engine engine) {
        this.engine = engine;
    }

    public abstract void drive();
}

class Sedan extends Car {
    public Sedan(Engine engine) { super(engine); }

    @Override
    public void drive() {
        engine.start();
        System.out.println("Driving a Sedan on the highway");
    }
}

class SUV extends Car {
```

```

    public SUV(Engine engine) { super(engine); }

    @Override
    public void drive() {
        engine.start();
        System.out.println("Driving an SUV on rough terrain");
    }
}

// ===== CLIENT =====
public class Main {
    public static void main(String[] args) {
        Engine petrol = new PetrolEngine();
        Engine diesel = new DieselEngine();
        Engine electric = new ElectricEngine();

        Car mySedan = new Sedan(petrol);
        Car mySUV = new SUV(electric);
        Car yourSUV = new SUV(diesel);

        mySedan.drive();
        System.out.println("---");
        mySUV.drive();
        System.out.println("---");
        yourSUV.drive();
    }
}

```

Output:

text

```

Petrol engine starting with ignition
Driving a Sedan on the highway
---
Electric engine powering up silently
Driving an SUV on rough terrain
---
Diesel engine starting with compression
Driving an SUV on rough terrain

```

Code Walkthrough

Class	Role
Engine	Low-level interface with <code>start()</code>
PetrolEngine , DieselEngine , ElectricEngine	Concrete implementations
Car	High-level abstract with <code>engine</code> reference
Sedan , SUV	Concrete abstractions
Main	Client — pairs abstractions with implementations

Part 5: Python Implementation

python

```
from abc import ABC, abstractmethod

# ===== IMPLEMENTATION HIERARCHY =====
class Engine(ABC):
    @abstractmethod
    def start(self): pass

class PetrolEngine(Engine):
    def start(self):
        print("Petrol engine starting with ignition")

class DieselEngine(Engine):
    def start(self):
        print("Diesel engine starting with compression")

class ElectricEngine(Engine):
    def start(self):
        print("Electric engine powering up silently")

# ===== ABSTRACTION HIERARCHY =====
class Car(ABC):
```

```

def __init__(self, engine: Engine):
    self.engine = engine    # THE BRIDGE

@abstractmethod
def drive(self): pass

class Sedan(Car):
    def drive(self):
        self.engine.start()
        print("Driving a Sedan on the highway")

class SUV(Car):
    def drive(self):
        self.engine.start()
        print("Driving an SUV on rough terrain")

# ===== CLIENT =====
if __name__ == "__main__":
    petrol = PetrolEngine()
    diesel = DieselEngine()
    electric = ElectricEngine()

    my_sedan = Sedan(petrol)
    my_suv = SUV(electric)
    your_suv = SUV(diesel)

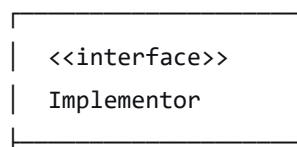
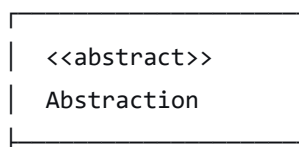
    my_sedan.drive()
    print("---")
    my_suv.drive()
    print("---")
    your_suv.drive()

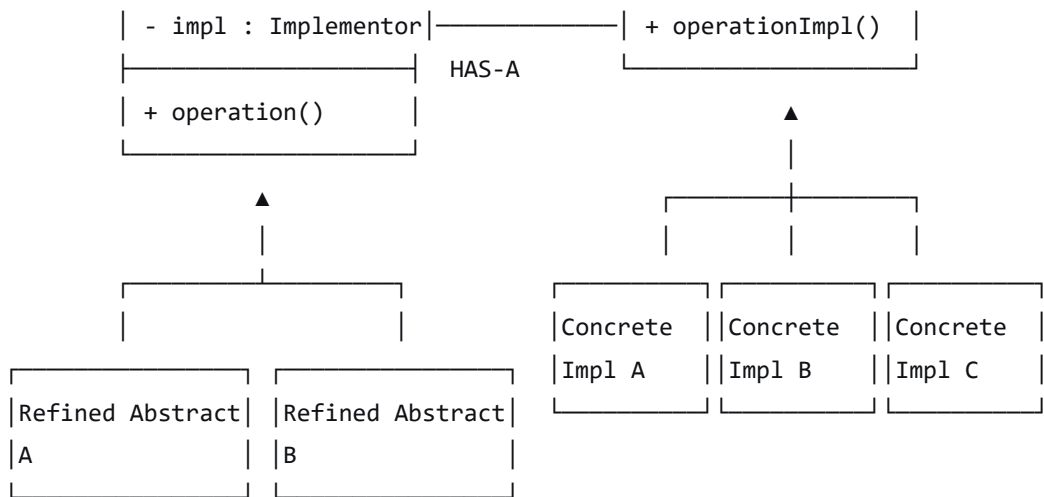
```

Output: Same as Java.

Part 6: Standard UML Diagram

text





- **Abstraction HAS-A Implementor** (the bridge)
- Both hierarchies can grow independently

Part 7: Definition

Bridge Pattern decouples an abstraction from its implementation so that the two can vary independently.

Key Points

- **Decouples** — separates two dimensions
- **Abstraction** — high-level layer
- **Implementation** — low-level layer
- **Vary independently** — add to either side without affecting the other

Part 8: Bridge vs Strategy

Both patterns look nearly identical in UML — they both use composition where a class holds a reference to an interface. The **intent** differs.

Aspect	Strategy	Bridge
Intent	Interchangeable algorithms at runtime	Separate two independent dimensions

Aspect	Strategy	Bridge
Focus	Multiple algorithms for the same task	Two hierarchies that vary independently
Runtime Swap	Yes — core feature	Not intended — pair is set once
Example	Robot with flyable behavior (swap at runtime)	SUV with petrol engine (fixed pairing)
Client	Single client with many algorithms	Two hierarchies growing independently

How to Decide

Ask: "Do I need to swap this at runtime?"

- Yes → Strategy
- No, but I need both hierarchies to vary independently → Bridge

Rule of thumb:

- Strategy = Interchangeable **behavior**
- Bridge = Independent **variation** of two dimensions

Part 9: Real-World Use Cases

1. TV & Remote Control

- **Abstraction:** Remote (basic, touch, voice)
- **Implementation:** TV (LCD, LED, OLED)
- Any remote pairs with any TV

2. GUI Toolkit

- **Abstraction:** Widget (TextBox, Dropdown, RadioButton)
- **Implementation:** OS (Windows, macOS, Linux)
- Widgets vary by OS independently

3. Database Drivers

- **Abstraction:** Database API (query, insert)
- **Implementation:** MySQL, PostgreSQL, MongoDB drivers

4. Graphics Rendering

- **Abstraction:** Shape (Circle, Square)
- **Implementation:** Renderer (OpenGL, DirectX, Vulkan)

5. Cross-Platform Apps

- **Abstraction:** UI components
- **Implementation:** Native platform APIs (iOS, Android)

6. Payment Processing

- **Abstraction:** Payment (one-time, recurring)
- **Implementation:** PaymentProvider (Stripe, PayPal, Razorpay)

7. File Format Handlers

- **Abstraction:** Document (Resume, Report)
- **Implementation:** Format (PDF, DOCX, HTML)

8. Notification Systems

- **Abstraction:** Notification (Alert, Reminder, Promotion)
- **Implementation:** Channel (Email, SMS, Push)

9. Persistence Layer

- **Abstraction:** Repository (UserRepo, OrderRepo)
- **Implementation:** Storage (SQL, NoSQL, File)

10. Message Queue

- **Abstraction:** MessageType (Order, Notification)

- **Implementation:** Broker (RabbitMQ, Kafka, SQS)

Part 10: Advantages & Trade-offs

Advantages

1. **Reduced class count** — $m + n$ instead of $m \times n$
2. **Independent variation** — both hierarchies grow freely
3. **Loose coupling** — abstraction doesn't know concrete implementations
4. **Open/Closed** — new abstractions/implementations without modification
5. **Single Responsibility** — high-level and low-level separated
6. **DRY** — no duplicate code for each combination

Trade-offs

1. **Indirection overhead** — slight performance cost
2. **More abstraction upfront** — need to identify two dimensions
3. **Can be overkill** — simple problems don't need it
4. **Harder to understand initially** — more layers

Part 11: When to Use Bridge Pattern

✓ Use When:

- Two independent dimensions of variation
- Want to avoid $m \times n$ class explosion
- Abstraction and implementation should grow independently
- Need runtime choice of implementation
- Both sides have their own inheritance hierarchies

✗ Avoid When:

- Only one dimension of variation
- Simple cases where inheritance suffices

- No need for independent growth
- Fixed one-to-one pairing

Part 12: Relationship to Other Patterns

Pattern	Relationship
Strategy	Similar structure, different intent (see Part 8)
Adapter	Adapter makes incompatible things work; Bridge separates dimensions
Abstract Factory	Often used with Bridge to create implementation families
Decorator	Adds behavior; Bridge separates dimensions

Key Takeaways

1. **Bridge Pattern** separates abstraction from implementation
2. **Reduces $m \times n$ to $m + n$ classes** — avoids class explosion
3. **Uses composition** — abstraction holds a reference to implementation
4. **Two hierarchies vary independently**
5. **Bridge vs Strategy** — intent differs (independence vs interchangeability)
6. **Real-world uses:** TV/Remote, GUI/OS, DB drivers, cross-platform apps
7. **Interview tip:** Use "abstraction" and "implementation" terms for impression
8. **Trade-off:** More upfront complexity for long-term flexibility
9. **Common confusion** with Strategy — resolve via "intent" discussion
10. **Follows SOLID** — SRP, OCP, DIP all apply naturally

"Bridge Pattern: build a bridge between what things look like and how they work, so each can change without affecting the other."



हां जी तो करें स्टार्ट। वेलकम बै...
TXT 266.48KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also

10-Minute Delivery App (Quick Commerce / Dark Store Inventory Management) — Complete LLD Notes

Part 1: Introduction

What Are We Building?

A 10-minute delivery app (like Zepto or Blinkit) which internally is essentially an inventory management system for quick commerce.

Why This Problem Matters

This is a very common LLD interview question — it tests:

- Inventory management (add/remove/check stock)
- Multi-dark-store handling (order splitting)
- Replenishment strategies (threshold vs weekly)
- Rider assignment (delivery partners)
- Database persistence (as extension)
- Use of multiple design patterns together

What's Different From Zomato/Amazon?

Aspect	Food Delivery (Zomato)	Quick Commerce (Zepto)
Delivery time	30-60 min	10 min
Sources	Restaurants	Dark stores
Order fulfillment	Usually one restaurant	May split across multiple dark stores
Warehouse model	Restaurant = kitchen	Dark store = micro-warehouse
Delivery partner	One picker (mostly)	Multiple pickers if split

Key Concept: Dark Store

A **dark store** is a small warehouse (not open to customers) located near users. It stores a subset of inventory for fast delivery. Multiple dark stores together cover a city.

Part 2: Functional Requirements

1. **Manage Inventory** — add, remove, check stock, list items
2. **Replenishment Strategies** — Threshold-based, Weekly (extensible)
3. **Multiple Inventory Stores** — DB-backed (extensible to File-based)
4. **User sees items** from dark stores closest to them (within 5 km radius)
5. **Order splitting** — If one dark store can't fulfill the entire order, it must be split across multiple dark stores, each assigned to a **separate delivery agent**

Non-Functional Requirements (implicit, same as always): scalable, extensible, SOLID-compliant, clean code.

Part 3: Happy Flow

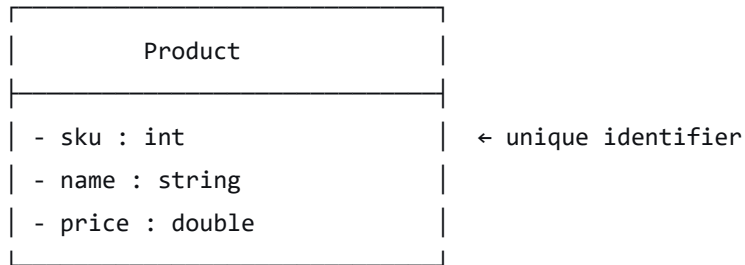
text

```
User opens app → Views items from nearby dark stores
  → Adds items to cart
  → Places order
  → Order Manager checks fulfillment:
    - Try 1 dark store first (optimized path)
    - If not possible, split across multiple
  → Assign delivery partner(s)
  → Delivery partners pick from their dark stores
  → Delivered to user
```

Part 4: Entity Model

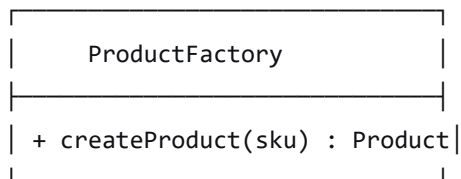
4.1 Product (Model class)

text



4.2 Product Factory

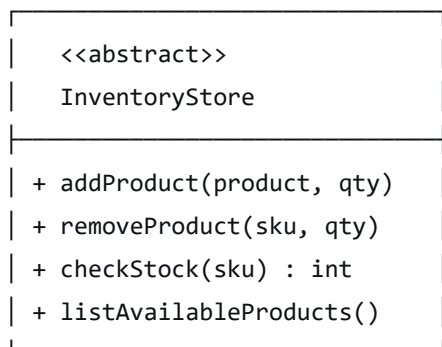
text



- Creates products by SKU (in real world, fetched from DB)
- Simplifies client code (no `new Product(...)` everywhere)

4.3 Inventory Store (Abstract)

text

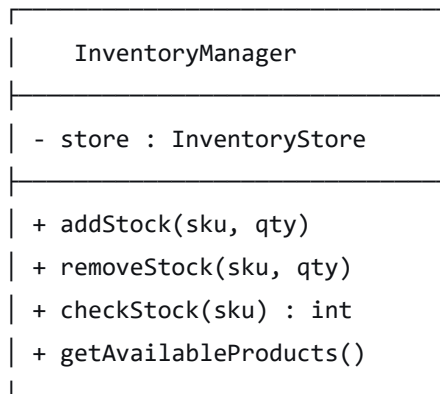


Concrete: `DBInventoryStore` — holds stock in **Map<sku, qty>** and **Map<sku, Product>**

- **Map for stock** (sku → qty): O(1) lookup/update
- **Map for products** (sku → Product): Avoid rebuilding product from factory on each access

4.4 Inventory Manager

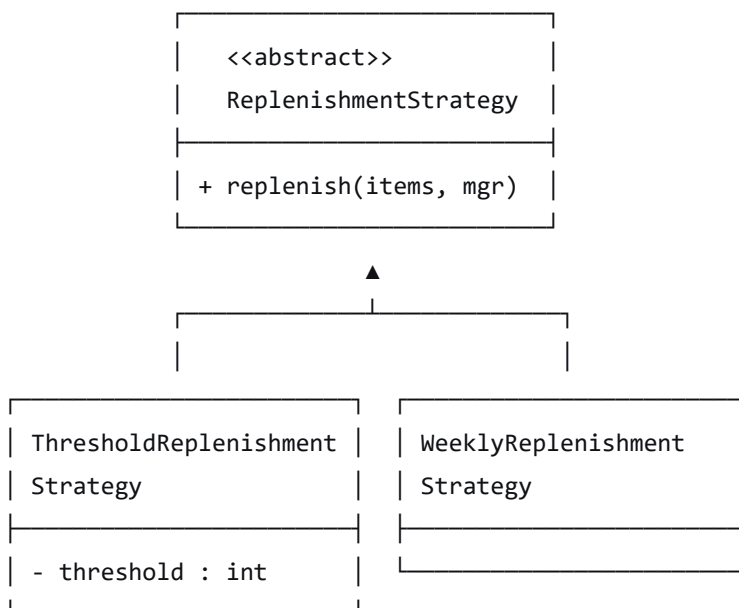
text



Delegation-only — Manager knows "what" (add stock), not "how" (store's internals).

4.5 Replenishment Strategy (Strategy Pattern)

text



- **Threshold:** Replenish only if current stock < threshold
- **Weekly:** Replenish every week (simulated in code)

4.6 Dark Store

text



```

| - name : string          |
| - x, y : double          | ← coordinates
| - inventoryManager : InventoryManager |
| - replenishStrategy : ReplenishmentStrategy |
|
| + distanceTo(ux, uy) : double
| + runReplenishment(items)
| + getAvailableProducts()
| + checkStock(sku) : int
| + addStock(sku, qty)
| + removeStock(sku, qty)

```

4.7 Dark Store Manager (Singleton)

text

```

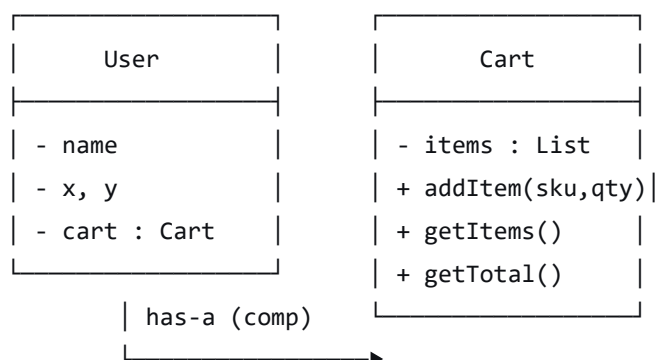
| <<singleton>>
| DarkStoreManager
|
| - stores : List<DarkStore>
|
| + getInstance()
| + registerStore(store)
| + getNearbyStores(ux, uy, maxDist) : List<DarkStore>

```

- **NOT singletons for InventoryManager:** each store has its own manager
- **IS singleton** because one DarkStoreManager handles all stores in the app

4.8 User & Cart

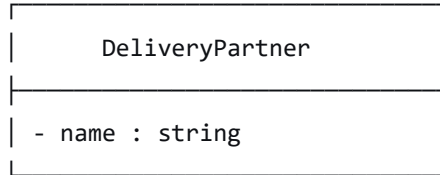
text



Cart → User is pure composition (cart dies with user).

4.9 DeliveryPartner

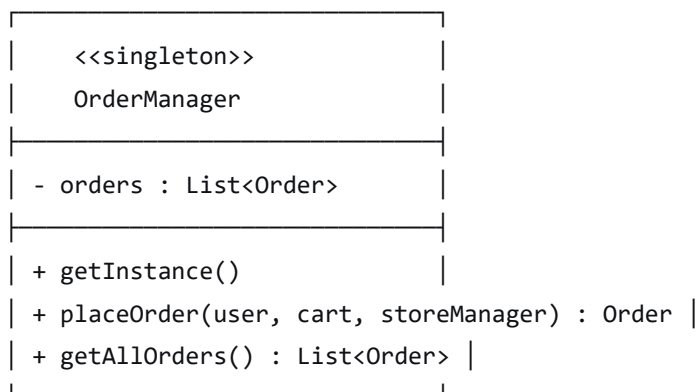
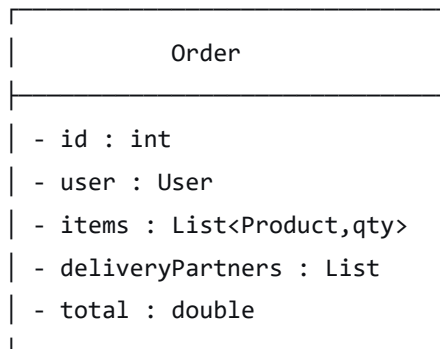
text



Simple prototype (in real app, has location, rating, availability, etc.)

4.10 Order & Order Manager

text



4.11 Zepto (Orchestrator)

Single entry point (like Tomato in Zomato example). All client requests go through Zepto.

Part 5: Design Patterns Used

Pattern	Where
Strategy	Replenishment (Threshold, Weekly)
Composition	DarkStore → InventoryManager (pure composition)
Singleton	DarkStoreManager, OrderManager
Factory	ProductFactory (creates products)
Facade / Orchestrator	Zepto class — single entry point
Delegation	InventoryManager delegates to InventoryStore; DarkStore delegates to InventoryManager
Bridge (modified)	InventoryManager (abstraction) ↔ InventoryStore (implementation)

Part 6: Order Placement Algorithm (Core Logic)

Case 1: One Dark Store Can Fulfill Entire Order

text

1. Get nearby stores (within 5 km)
2. Pick first (closest) store
3. Check if it can fulfill ALL items
4. If yes:
 - Remove items from that store's stock
 - Assign ONE delivery partner
 - Create order

Case 2: Multiple Dark Stores Required

text

1. Get nearby stores
2. Build a map of ALL requested items: {sku → qty}
3. For each store (in order):
 - For each item in needed map:

- Check stock
 - Take min(stock_available, qty_needed)
 - Remove from store's stock
 - Update needed map (or remove if fully served)
 - Add to order items
- If any item was taken from this store:
- Assign a new delivery partner
4. After all stores:
- If any items unfulfilled → error message
 - Recompute order total
 - Finalize order

Key Implementation Detail: The "ToErase" Vector

When iterating over a store's items while also modifying the needed-map, we **cannot modify while iterating**. So:

- Collect SKUs that are fully satisfied into a "toErase" vector
- After the loop over items, erase them from the map

Part 7: Java Implementation

Enums/Models

java

```
class Product {
    private int sku;
    private String name;
    private double price;

    public Product(int sku, String name, double price) {
        this.sku = sku;
        this.name = name;
        this.price = price;
    }

    public int getSku() { return sku; }
    public String getName() { return name; }
    public double getPrice() { return price; }
}
```

```
class ProductFactory {
    public static Product createProduct(int sku) {
```

```

// In real world: fetch from DB. Here: hardcoded mapping
switch (sku) {
    case 1001: return new Product(sku, "Apple", 20);
    case 1002: return new Product(sku, "Banana", 10);
    case 1003: return new Product(sku, "Chocolate", 50);
    case 2001: return new Product(sku, "T-Shirt", 500);
    case 2002: return new Product(sku, "Jeans", 1000);
    default: return new Product(sku, "Item-" + sku, 100);
}
}
}

```

Inventory Store

java

```

import java.util.*;

abstract class InventoryStore {
    public abstract void addProduct(Product p, int qty);
    public abstract void removeProduct(int sku, int qty);
    public abstract int checkStock(int sku);
    public abstract List<Product> listAvailableProducts();
}

class DBInventoryStore extends InventoryStore {
    private Map<Integer, Integer> stocks = new HashMap<>(); // sku → qty
    private Map<Integer, Product> products = new HashMap<>(); // sku → Product

    @Override
    public void addProduct(Product p, int qty) {
        int sku = p.getSku();
        if (!products.containsKey(sku)) {
            products.put(sku, p);
            stocks.put(sku, qty);
        } else {
            stocks.put(sku, stocks.get(sku) + qty);
        }
    }

    @Override
    public void removeProduct(int sku, int qty) {
        if (!stocks.containsKey(sku)) return;
        int remaining = stocks.get(sku) - qty;
        if (remaining > 0) stocks.put(sku, remaining);
        else stocks.remove(sku);
    }
}

```

```

@Override
public int checkStock(int sku) {
    return stocks.getDefault(sku, 0);
}

@Override
public List<Product> listAvailableProducts() {
    List<Product> result = new ArrayList<>();
    for (Map.Entry<Integer, Integer> e : stocks.entrySet()) {
        if (e.getValue() > 0 && products.containsKey(e.getKey())) {
            result.add(products.get(e.getKey()));
        }
    }
    return result;
}
}

```

Inventory Manager

java

```

class InventoryManager {
    private InventoryStore store;

    public InventoryManager(InventoryStore store) { this.store = store; }

    public void addStock(int sku, int qty) {
        Product p = ProductFactory.createProduct(sku);
        store.addProduct(p, qty);
        System.out.println("Added " + qty + " of SKU " + sku);
    }

    public void removeStock(int sku, int qty) {
        store.removeProduct(sku, qty);
    }

    public int checkStock(int sku) { return store.checkStock(sku); }

    public List<Product> getAvailableProducts() {
        return store.listAvailableProducts();
    }
}

```

Replenishment Strategies

```
java
```

```
interface ReplenishmentStrategy {
    void replenish(Map<Integer, Integer> itemsToReplenish, InventoryManager mgr);
}

class ThresholdReplenishmentStrategy implements ReplenishmentStrategy {
    private int threshold;
    public ThresholdReplenishmentStrategy(int threshold) { this.threshold = threshold; }

    @Override
    public void replenish(Map<Integer, Integer> items, InventoryManager mgr) {
        for (Map.Entry<Integer, Integer> e : items.entrySet()) {
            int sku = e.getKey();
            int qtyToAdd = e.getValue();
            if (mgr.checkStock(sku) < threshold) {
                mgr.addStock(sku, qtyToAdd);
            }
        }
    }
}

class WeeklyReplenishmentStrategy implements ReplenishmentStrategy {
    @Override
    public void replenish(Map<Integer, Integer> items, InventoryManager mgr) {
        for (Map.Entry<Integer, Integer> e : items.entrySet()) {
            mgr.addStock(e.getKey(), e.getValue());
        }
    }
}
```

Dark Store

```
java
```

```
class DarkStore {
    private String name;
    private double x, y;
    private InventoryManager inventoryManager;
    private ReplenishmentStrategy replenishStrategy;

    public DarkStore(String name, double x, double y) {
        this.name = name;
        this.x = x;
        this.y = y;
        // Pure composition: dark store creates its own manager
        this.inventoryManager = new InventoryManager(new DBInventoryStore());
    }
}
```

```

    }

    public double distanceTo(double ux, double uy) {
        return Math.sqrt(Math.pow(x - ux, 2) + Math.pow(y - uy, 2));
    }

    public void setReplenishStrategy(ReplenishmentStrategy s) { this.replenishStrategy =
s; }

    public void runReplenishment(Map<Integer, Integer> items) {
        if (replenishStrategy != null) replenishStrategy.replenish(items, inventoryManage
r);
    }

    // Delegation methods
    public List<Product> getAvailableProducts() { return inventoryManager.getAvailablePro
ducts(); }
    public int checkStock(int sku) { return inventoryManager.checkStock(sku); }
    public void addStock(int sku, int qty) { inventoryManager.addStock(sku, qty); }
    public void removeStock(int sku, int qty) { inventoryManager.removeStock(sku, qty); }
    public String getName() { return name; }
}

```

Dark Store Manager

java

```

class DarkStoreManager {
    private static DarkStoreManager instance;
    private List<DarkStore> stores = new ArrayList<>();

    private DarkStoreManager() {}

    public static DarkStoreManager getInstance() {
        if (instance == null) {
            synchronized (DarkStoreManager.class) {
                if (instance == null) instance = new DarkStoreManager();
            }
        }
        return instance;
    }

    public void registerStore(DarkStore store) { stores.add(store); }

    public List<DarkStore> getNearbyStores(double ux, double uy, double maxDist) {
        List<DarkStore> result = new ArrayList<>();
        for (DarkStore s : stores) {

```

```

        if (s.distanceTo(ux, uy) <= maxDist) result.add(s);
    }
    return result;
}
}

```

User & Cart

java

```

class Cart {
    private List<int[]> items = new ArrayList<>(); // {sku, qty}
    private double total = 0;

    public void addItem(int sku, int qty) {
        items.add(new int[]{sku, qty});
        Product p = ProductFactory.createProduct(sku);
        total += p.getPrice() * qty;
    }

    public List<int[]> getItems() { return items; }
    public double getTotal() { return total; }
}

class User {
    private String name;
    private double x, y;
    private Cart cart = new Cart();

    public User(String name, double x, double y) {
        this.name = name; this.x = x; this.y = y;
    }

    public String getName() { return name; }
    public double getX() { return x; }
    public double getY() { return y; }
    public Cart getCart() { return cart; }
}

```

Delivery Partner & Order

java

```

class DeliveryPartner {
    private String name;
    public DeliveryPartner(String name) { this.name = name; }
}

```

```

    public String getName() { return name; }
}

class Order {
    private static int nextId = 1;
    private int id;
    private User user;
    private List<int[]> items = new ArrayList<>();
    private List<DeliveryPartner> deliveryPartners = new ArrayList<>();
    private double total = 0;

    public Order(User user) {
        this.id = nextId++;
        this.user = user;
    }

    public int getId() { return id; }
    public User getUser() { return user; }
    public List<int[]> getItems() { return items; }
    public List<DeliveryPartner> getPartners() { return deliveryPartners; }
    public void setTotal(double t) { this.total = t; }
    public double getTotal() { return total; }
    public void addItem(int sku, int qty) { items.add(new int[]{sku, qty}); }
    public void addPartner(DeliveryPartner p) { deliveryPartners.add(p); }
}

```

Order Manager — Core Logic

java

```

class OrderManager {
    private static OrderManager instance;
    private List<Order> orders = new ArrayList<>();

    private OrderManager() {}

    public static OrderManager getInstance() {
        if (instance == null) {
            synchronized (OrderManager.class) {
                if (instance == null) instance = new OrderManager();
            }
        }
        return instance;
    }

    public Order placeOrder(User user, Cart cart, DarkStoreManager dsMgr) {
        List<DarkStore> nearby = dsMgr.getNearbyStores(user.getX(), user.getY(), 5.0);
        if (nearby.isEmpty()) {

```



```
        System.out.println("No dark stores within 5 km");
        return null;
    }

    List<int[]> requested = cart.getItems();
    Order order = new Order(user);

    // Case 1: Try to fulfill entire order from first store
    DarkStore first = nearby.get(0);
    boolean allInFirst = true;
    for (int[] item : requested) {
        if (first.checkStock(item[0]) < item[1]) {
            allInFirst = false;
            break;
        }
    }

    if (allInFirst) {
        for (int[] item : requested) {
            first.removeStock(item[0], item[1]);
            order.addItem(item[0], item[1]);
        }
        order.addPartner(new DeliveryPartner("Partner-1"));
        order.setTotal(cart.getTotal());
    } else {
        // Case 2: Split across multiple stores
        Map<Integer, Integer> needed = new HashMap<>();
        for (int[] item : requested) needed.put(item[0], item[1]);

        int partnerId = 1;
        for (DarkStore store : nearby) {
            if (needed.isEmpty()) break;

            boolean assigned = false;
            List<Integer> toErase = new ArrayList<>();

            for (Map.Entry<Integer, Integer> e : needed.entrySet()) {
                int sku = e.getKey(), qtyNeeded = e.getValue();
                int available = store.checkStock(sku);
                if (available <= 0) continue;

                int taken = Math.min(available, qtyNeeded);
                store.removeStock(sku, taken);
                order.addItem(sku, taken);
                assigned = true;

                if (taken < qtyNeeded) {
                    e.setValue(qtyNeeded - taken);
                }
            }
        }
    }
}
```

```

        } else {
            toErase.add(sku);
        }
    }

    for (int sku : toErase) needed.remove(sku);

    if (assigned) {
        order.addPartner(new DeliveryPartner("Partner-" + partnerId++));
    }
}

if (!needed.isEmpty()) {
    System.out.println("Could not fulfill: " + needed);
}

// Recompute total
double total = 0;
for (int[] item : order.getItems()) {
    Product p = ProductFactory.createProduct(item[0]);
    total += p.getPrice() * item[1];
}
order.setTotal(total);
}

orders.add(order);
printSummary(order);
return order;
}

private void printSummary(Order order) {
    System.out.println("Order #" + order.getId() + " by " + order.getUser().getName
());
    for (int[] item : order.getItems()) {
        System.out.println("  " + ProductFactory.createProduct(item[0]).getName()
            + " x " + item[1]);
    }
    System.out.println("Total: ₹" + order.getTotal());
    System.out.print("Partners: ");
    for (DeliveryPartner p : order.getPartners()) System.out.print(p.getName() + "
");
    System.out.println();
}

public List<Order> getAllOrders() { return orders; }
}

```

Zepto (Orchestrator)

java

```
class Zepto {
    public static void initialize() {
        DarkStore a = new DarkStore("DarkStore-A", 0, 0);
        a.setReplenishStrategy(new ThresholdReplenishmentStrategy(5));
        a.addStock(1001, 5);    // Apple
        a.addStock(1002, 2);    // Banana

        DarkStore b = new DarkStore("DarkStore-B", 2, 2);
        b.addStock(1001, 3);
        b.addStock(1003, 10);   // Chocolate

        DarkStore c = new DarkStore("DarkStore-C", 4, 4);
        c.addStock(1002, 5);
        c.addStock(2001, 10);   // T-Shirt

        DarkStoreManager mgr = DarkStoreManager.getInstance();
        mgr.registerStore(a);
        mgr.registerStore(b);
        mgr.registerStore(c);
    }

    public static void showAllItems(User user) {
        List<DarkStore> nearby = DarkStoreManager.getInstance()
            .getNearbyStores(user.getX(), user.getY(), 5.0);
        for (DarkStore s : nearby) {
            System.out.println("From " + s.getName() + ":");
            for (Product p : s.getAvailableProducts()) {
                System.out.println("  " + p.getSku() + " - " + p.getName()
                    + " ₹" + p.getPrice());
            }
        }
    }
}
```

Client (Main)

java

```
public class Main {
    public static void main(String[] args) {
        Zepto.initialize();

        User aditya = new User("Aditya", 1, 1);
```

```

System.out.println("User " + aditya.getName() + " came on platform");

Zepto.showAllItems(aditya);

// Add to cart
aditya.getCart().addItem(1001, 4); // Apple x4
aditya.getCart().addItem(1002, 3); // Banana x3
aditya.getCart().addItem(1003, 2); // Chocolate x2

System.out.println("Placing order...");
OrderManager.getInstance().placeOrder(
    aditya, aditya.getCart(), DarkStoreManager.getInstance());
}
}

```

Part 8: Python Implementation (Essentials)

python

```

import math
from abc import ABC, abstractmethod
from enum import Enum

class Product:
    def __init__(self, sku, name, price):
        self.sku = sku
        self.name = name
        self.price = price

class ProductFactory:
    _catalog = {
        1001: ("Apple", 20),
        1002: ("Banana", 10),
        1003: ("Chocolate", 50),
        2001: ("T-Shirt", 500),
        2002: ("Jeans", 1000),
    }

    @staticmethod
    def create_product(sku):
        name, price = ProductFactory._catalog.get(sku, (f"Item-{sku}", 100))
        return Product(sku, name, price)

```

```
# ----- Inventory -----
class InventoryStore(ABC):
    @abstractmethod
    def add_product(self, product, qty): pass
    @abstractmethod
    def remove_product(self, sku, qty): pass
    @abstractmethod
    def check_stock(self, sku): pass
    @abstractmethod
    def list_available_products(self): pass

class DBInventoryStore(InventoryStore):
    def __init__(self):
        self._stocks = {}      # sku -> qty
        self._products = {}    # sku -> Product

    def add_product(self, p, qty):
        self._products[p.sku] = p
        self._stocks[p.sku] = self._stocks.get(p.sku, 0) + qty

    def remove_product(self, sku, qty):
        if sku not in self._stocks: return
        remaining = self._stocks[sku] - qty
        if remaining > 0:
            self._stocks[sku] = remaining
        else:
            self._stocks.pop(sku, None)

    def check_stock(self, sku):
        return self._stocks.get(sku, 0)

    def list_available_products(self):
        return [self._products[sku] for sku, q in self._stocks.items()
                if q > 0 and sku in self._products]

class InventoryManager:
    def __init__(self, store):
        self._store = store

    def add_stock(self, sku, qty):
        self._store.add_product(ProductFactory.create_product(sku), qty)

    def remove_stock(self, sku, qty):
        self._store.remove_product(sku, qty)
```

```

def check_stock(self, sku):
    return self._store.check_stock(sku)

def get_available_products(self):
    return self._store.list_available_products()

# ----- Replenishment Strategy -----
class ReplenishmentStrategy(ABC):
    @abstractmethod
    def replenish(self, items, mgr): pass

class ThresholdReplenishmentStrategy(ReplenishmentStrategy):
    def __init__(self, threshold):
        self._threshold = threshold

    def replenish(self, items, mgr):
        for sku, qty in items.items():
            if mgr.check_stock(sku) < self._threshold:
                mgr.add_stock(sku, qty)

class WeeklyReplenishmentStrategy(ReplenishmentStrategy):
    def replenish(self, items, mgr):
        for sku, qty in items.items():
            mgr.add_stock(sku, qty)

# ----- Dark Store -----
class DarkStore:
    def __init__(self, name, x, y):
        self.name = name
        self.x, self.y = x, y
        self.inventory_manager = InventoryManager(DBInventoryStore())
        self.replenish_strategy = None

    def distance_to(self, ux, uy):
        return math.sqrt((self.x - ux) ** 2 + (self.y - uy) ** 2)

    def set_replenish_strategy(self, s):
        self.replenish_strategy = s

    def run_replenishment(self, items):
        if self.replenish_strategy:
            self.replenish_strategy.replenish(items, self.inventory_manager)

    def get_available_products(self):

```

```

        return self.inventory_manager.get_available_products()

    def check_stock(self, sku):
        return self.inventory_manager.check_stock(sku)

    def add_stock(self, sku, qty):
        self.inventory_manager.add_stock(sku, qty)

    def remove_stock(self, sku, qty):
        self.inventory_manager.remove_stock(sku, qty)

class DarkStoreManager:
    _instance = None
    _lock = None

    def __new__(cls):
        import threading
        if cls._lock is None: cls._lock = threading.Lock()
        if cls._instance is None:
            with cls._lock:
                if cls._instance is None:
                    cls._instance = super().__new__(cls)
                    cls._instance.stores = []
        return cls._instance

    def register_store(self, store):
        self.stores.append(store)

    def get_nearby_stores(self, ux, uy, max_dist):
        return [s for s in self.stores if s.distance_to(ux, uy) <= max_dist]

# ----- User & Cart -----
class Cart:
    def __init__(self):
        self.items = []    # list of (sku, qty)
        self.total = 0

    def add_item(self, sku, qty):
        self.items.append((sku, qty))
        p = ProductFactory.create_product(sku)
        self.total += p.price * qty

class User:
    def __init__(self, name, x, y):
        self.name = name

```

```

self.x, self.y = x, y
self.cart = Cart()

```

```

class DeliveryPartner:
    def __init__(self, name):
        self.name = name

```

```

class Order:
    _next_id = 1
    def __init__(self, user):
        self.id = Order._next_id
        Order._next_id += 1
        self.user = user
        self.items = []
        self.delivery_partners = []
        self.total = 0

```

```

class OrderManager:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance.orders = []
        return cls._instance

    def place_order(self, user, cart, ds_manager):
        nearby = ds_manager.get_nearby_stores(user.x, user.y, 5.0)
        if not nearby:
            print("No dark stores nearby")
            return None

        order = Order(user)
        requested = cart.items

        # Case 1: try to fulfill from first store
        first = nearby[0]
        all_in_first = all(first.check_stock(sku) >= qty for sku, qty in requested)

        if all_in_first:
            for sku, qty in requested:
                first.remove_stock(sku, qty)
                order.items.append((sku, qty))
            order.delivery_partners.append(DeliveryPartner("Partner-1"))
            order.total = cart.total
        else:

```



```

needed = {sku: qty for sku, qty in requested}
partner_id = 1
for store in nearby:
    if not needed: break
    assigned = False
    to_erase = []
    for sku in list(needed.keys()):
        qty_needed = needed[sku]
        available = store.check_stock(sku)
        if available <= 0: continue
        taken = min(available, qty_needed)
        store.remove_stock(sku, taken)
        order.items.append((sku, taken))
        assigned = True
        if taken < qty_needed:
            needed[sku] = qty_needed - taken
        else:
            to_erase.append(sku)
    for sku in to_erase:
        needed.pop(sku, None)
    if assigned:
        order.delivery_partners.append(
            DeliveryPartner(f"Partner-{partner_id}")
        )
        partner_id += 1

if needed:
    print(f"Unfulfilled: {needed}")

# recompute total
order.total = sum(
    ProductFactory.create_product(sku).price * qty
    for sku, qty in order.items
)

self.orders.append(order)
print(f"Order #{order.id} Total ₹{order.total}, "
      f"Partners: {[p.name for p in order.delivery_partners]}")
return order

# ----- Zepto (Orchestrator) -----
class Zepto:
    @staticmethod
    def initialize():
        a = DarkStore("DarkStore-A", 0, 0)
        a.set_replenish_strategy(ThresholdReplenishmentStrategy(5))
        a.add_stock(1001, 5); a.add_stock(1002, 2)

```

```

b = DarkStore("DarkStore-B", 2, 2)
b.add_stock(1001, 3); b.add_stock(1003, 10)

c = DarkStore("DarkStore-C", 4, 4)
c.add_stock(1002, 5); c.add_stock(2001, 10)

mgr = DarkStoreManager()
mgr.register_store(a); mgr.register_store(b); mgr.register_store(c)

@staticmethod
def show_all_items(user):
    for s in DarkStoreManager().get_nearby_stores(user.x, user.y, 5.0):
        print(f"From {s.name}:")
        for p in s.get_available_products():
            print(f"  {p.sku} - {p.name} ₹{p.price}")

# ----- Client -----
if __name__ == "__main__":
    Zepto.initialize()

    aditya = User("Aditya", 1, 1)
    print(f"User {aditya.name} came on platform")

    Zepto.show_all_items(aditya)

    aditya.cart.add_item(1001, 4)
    aditya.cart.add_item(1002, 3)
    aditya.cart.add_item(1003, 2)

    OrderManager().place_order(aditya, aditya.cart, DarkStoreManager())

```

Sample Output:

text

Order #1 Total ₹210.0, Partners: ['Partner-1', 'Partner-2', 'Partner-3']

Part 9: Key Design Decisions

9.1 Why InventoryManager is NOT a Singleton?

In most LLD examples, managers are singletons. But here:

- **One InventoryManager per DarkStore**
- If singleton, all dark stores would share one manager → wrong!

Rule: Manager is singleton only if **one instance makes sense for the entire application**.

9.2 Why Pure Composition Between DarkStore and InventoryManager?

If DarkStore is destroyed, its InventoryManager has no purpose. Cart's items belong to a specific dark store's stock — manager lifecycle follows store.

9.3 Why Two Maps in DBInventoryStore?

- **Map<sku, qty>** — $O(1)$ stock lookup/update
- **Map<sku, Product>** — avoid rebuilding Product on every access

Using only a list would give $O(n)$ search → bad for millions of stock checks.

9.4 Why "ToErase" Vector?

Cannot modify a map while iterating over it (ConcurrentModificationException in Java; RuntimeError in Python). Solution: collect removals and apply after loop.

9.5 Why Order Manager Takes Cart as Parameter (Not Fetched from User)?

Principle of Least Knowledge (Law of Demeter):

- OrderManager shouldn't know `user.getCart()` chain
- Client passes the cart directly

9.6 Bridge Pattern (Modified) Between InventoryManager and InventoryStore

- **Abstraction:** InventoryManager (high-level operations)
- **Implementation:** InventoryStore (actual storage)
- Both can vary — DB store, File store, etc.

Note: Not "pure" Bridge because InventoryManager doesn't currently have its own hierarchy, but the intent matches.

Part 10: SOLID Principles Applied

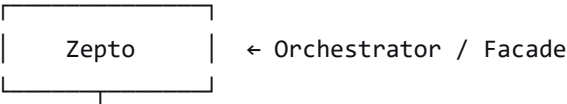
Principle	How
SRP	Product = model, InventoryStore = storage, Manager = delegation, DarkStore = one store's data
OCP	New replenishment strategies / inventory stores via new subclasses
LSP	All stores substitutable as InventoryStore
ISP	Small interfaces
DIP	InventoryManager depends on InventoryStore abstraction
LoD	DarkStore delegates, DarkStoreManager talks only to DarkStore (not its internal manager)

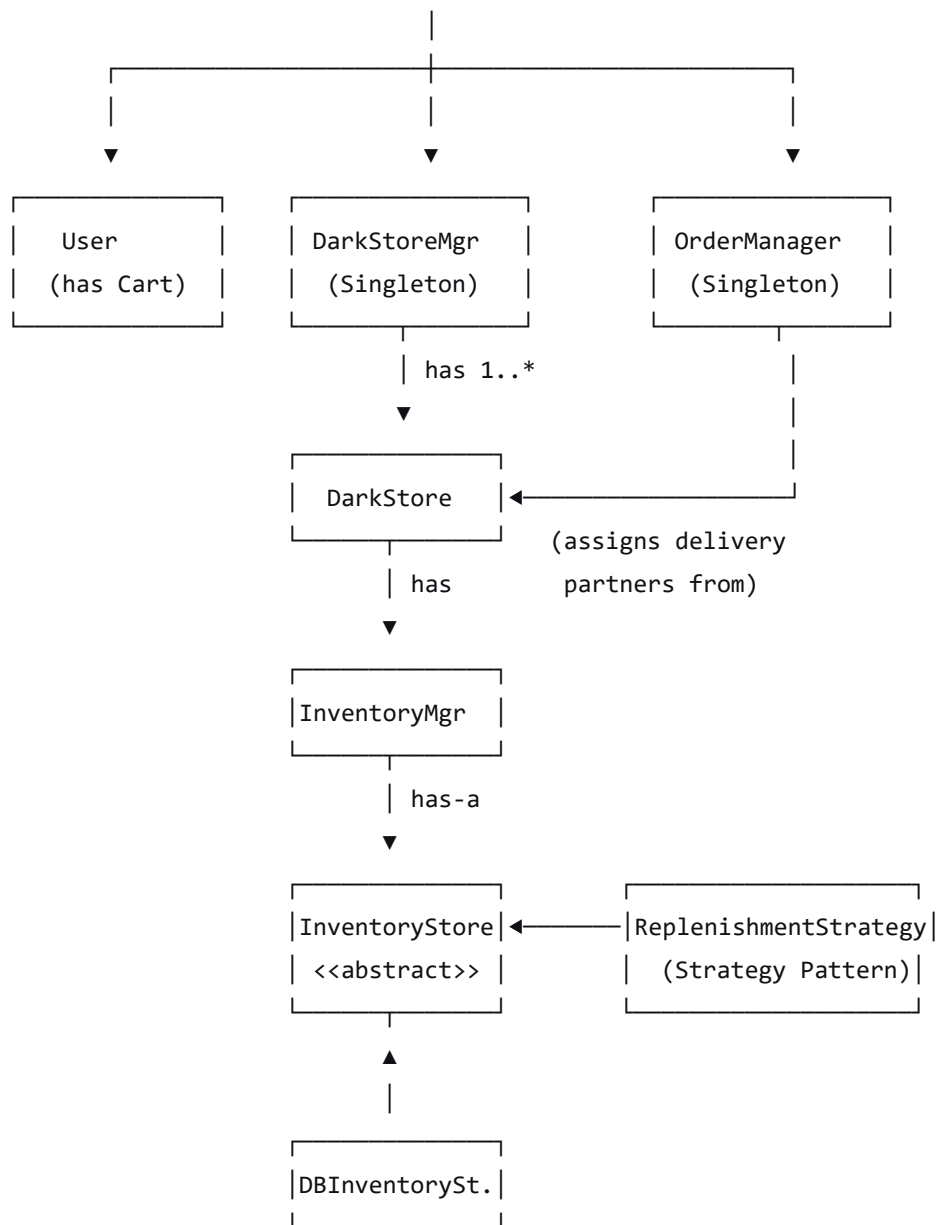
Part 11: Real-World Extensions (Homework)

- 1. Rider Mapping Algorithm — assign closest / highest-rated delivery partner
- 2. Payment Gateway Integration — plug-and-play payment module
- 3. Coupon Mechanism Integration — reuse the coupon system (from earlier lecture)
- 4. Persistency Layer — add Persistence abstraction for DB / Mongo / File
- 5. Modular Code — split into folders (models/, managers/, strategies/)

Part 12: Summary

text





Key Takeaways

1. **Quick commerce = inventory management at its core** with a fast-delivery twist
2. **Dark stores** are micro-warehouses; multiple cover a city
3. **Order splitting** is essential — one order, multiple stores, multiple riders
4. **Replenishment strategies** are pluggable via Strategy pattern
5. **Inventory manager is NOT a singleton** — one per dark store
6. **Map over list** for stock lookup — $O(1)$ vs $O(n)$
7. **Composition over inheritance** — **DarkStore** owns **InventoryManager**
8. **Delegation** — **DarkStore** → **InventoryManager** → **InventoryStore**

9. **Principle of Least Knowledge** — DarkStoreManager doesn't poke into DarkStore's internals
10. **Real-world extensions:** rider mapping, payment, coupons, persistence
11. **Multiple patterns combined:** Strategy, Singleton, Factory, Facade, Composition, Bridge

"Quick commerce LLD = inventory management + order splitting + real-time assignment, glued together by design patterns."